

计算机视觉

第06章 神经网络

软件学院：刘春

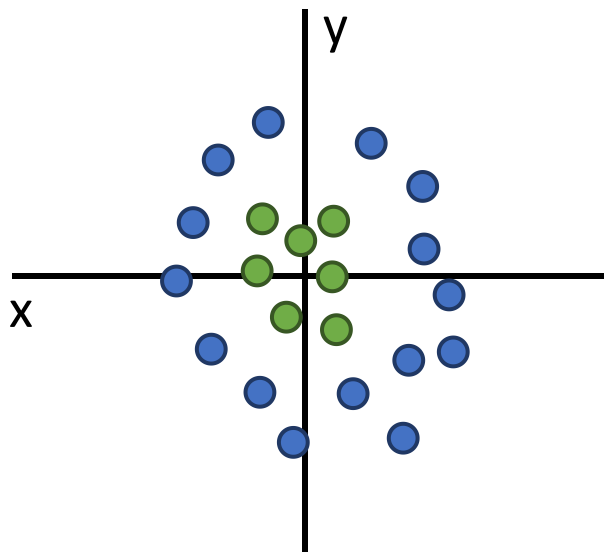
Ref: EECS 498-007, Deep Learning for Computer Vision
Prof. Justin Johnson

本章内容

- 6.1.神经网络
- 6.2.卷积神经网络
- 6.3.神经网络图像分类

问题: 线性分类器无法工作

几何角度

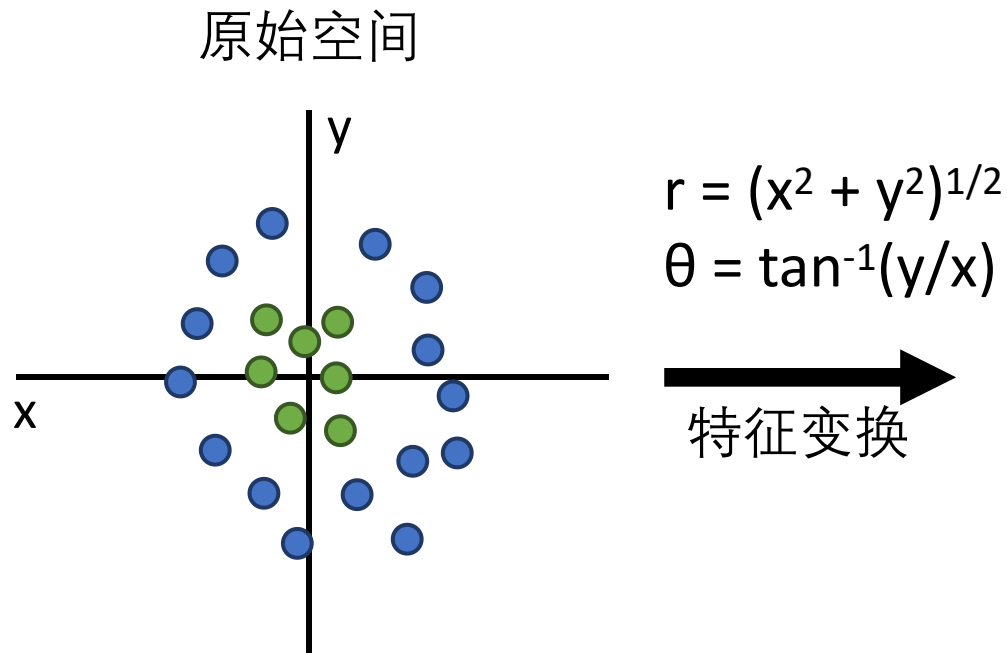


可视化角度

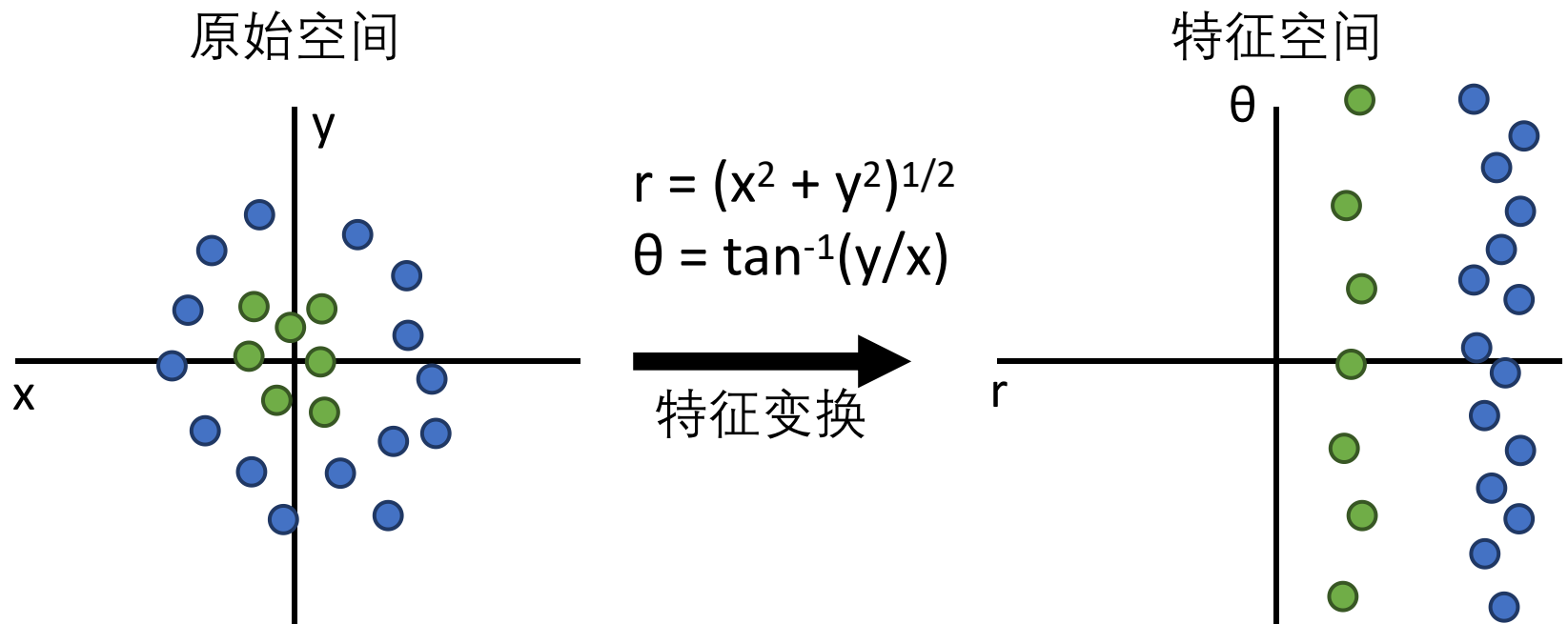
每类一个模板:
不能识别一个类别的多种
模式



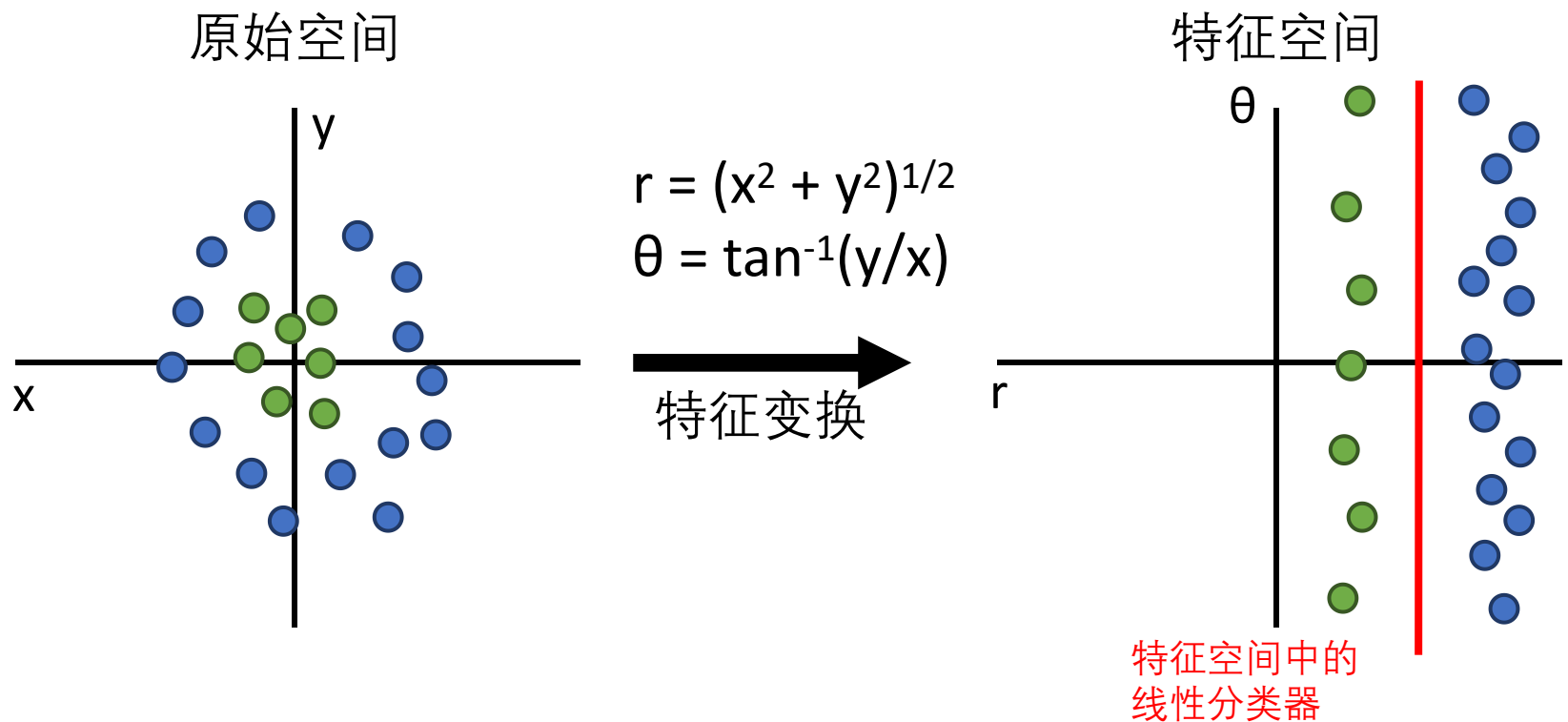
一种解决方案: 特征变换



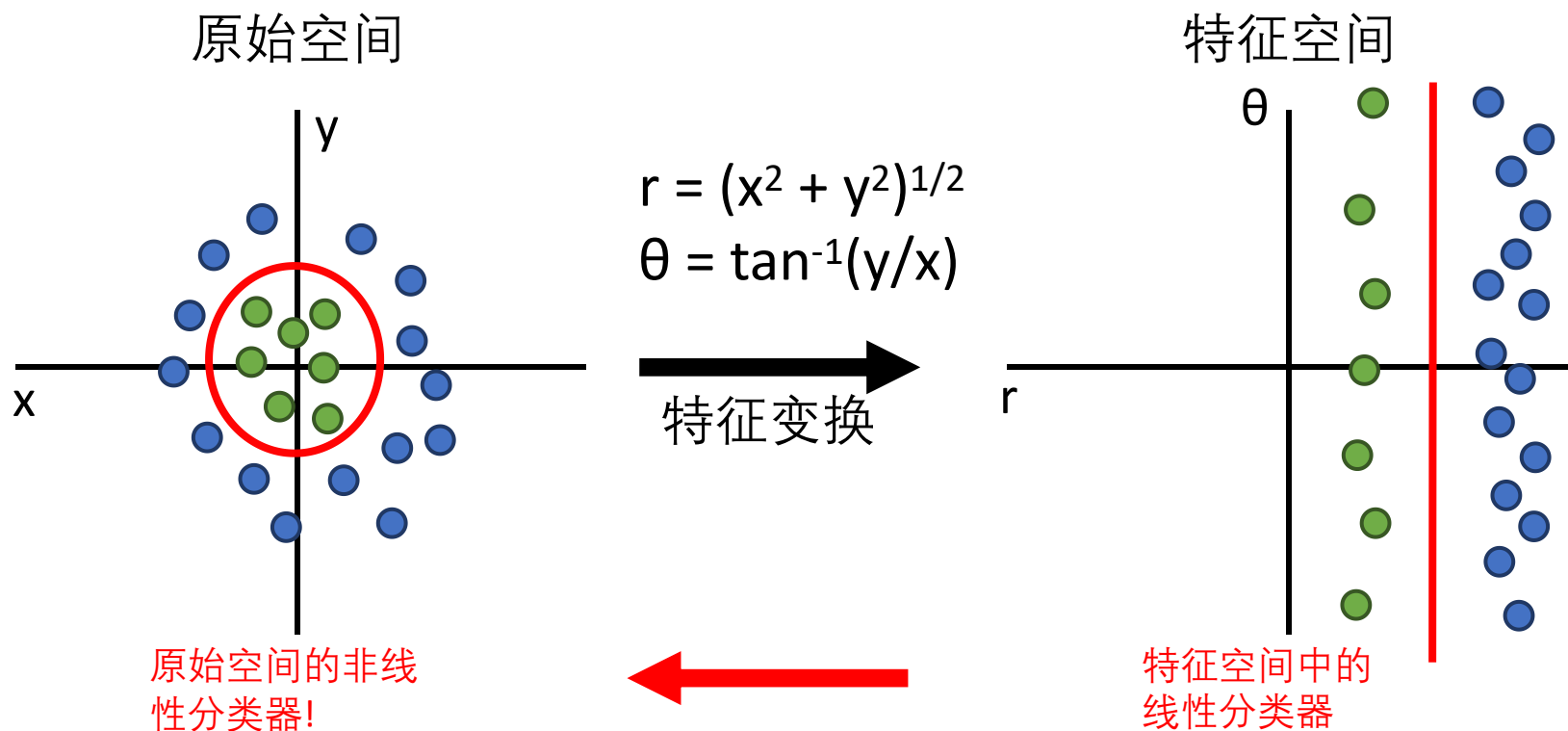
一种解决方案: 特征变换



一种解决方案: 特征变换



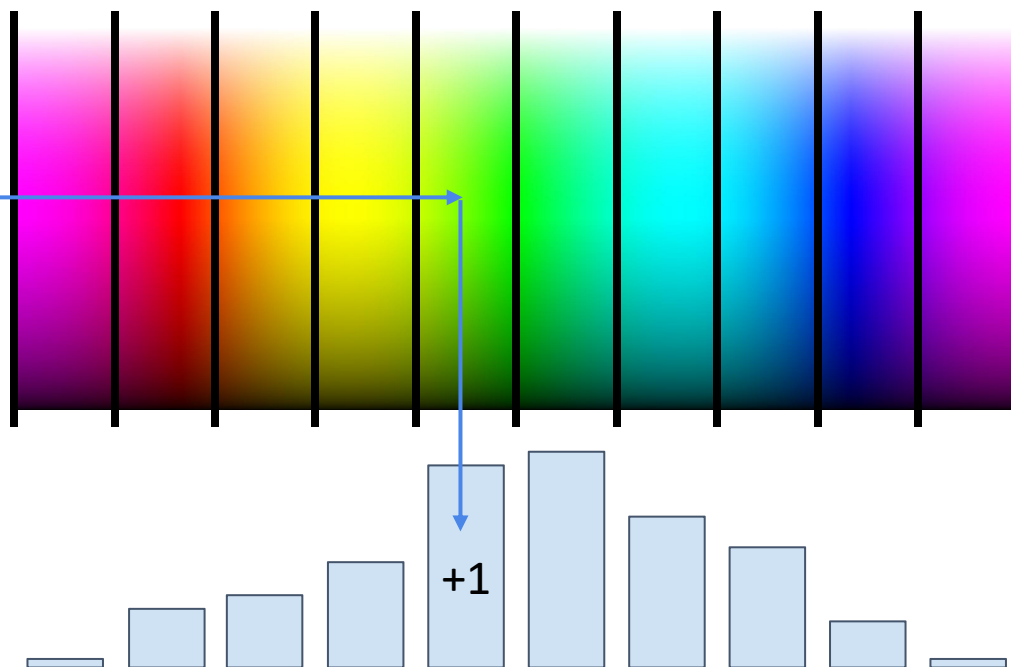
一种解决方案: 特征变换



图像特征: 彩色直方图



忽略纹理, 空域
位置信息



[Frog image](#) is in the public domain

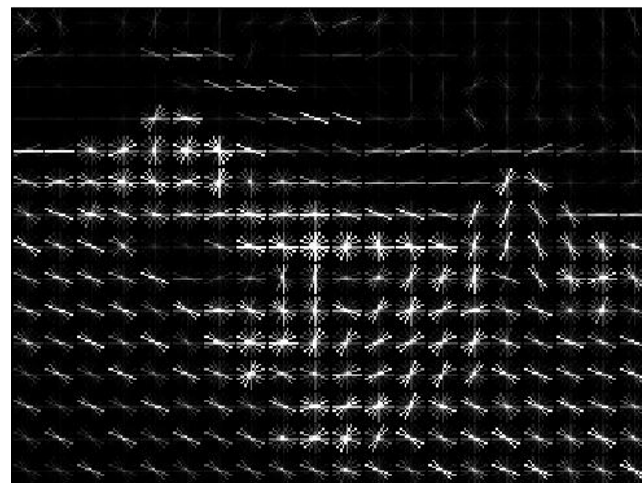
图像特征: 方向梯度直方图(Histogram of Oriented Gradients, HoG)



1. 计算各像素点边缘方向/强度
2. 将图像分为 8×8 子区域
3. 在每个区域计算边缘方向直方图，边缘强度作为权重

Lowe, "Object recognition from local scale-invariant features", ICCV 1999
Dalal and Triggs, "Histograms of oriented gradients for human detection," CVPR 2005

图像特征: 方向梯度直方图(Histogram of Oriented Gradients, HoG)

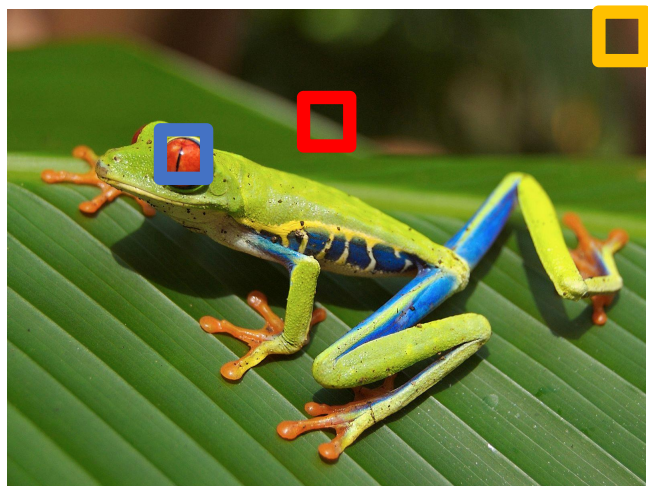


1. 计算各像素点边缘方向/强度
2. 将图像分为 8×8 子区域
3. 在每个区域计算边缘方向直方图, 边缘强度作为权重

举例: 320×240 图像分成 40×30 块; 每块 8 方向; 特征向量维度 $30 \times 40 \times 9 = 10,800$

Lowe, "Object recognition from local scale-invariant features", ICCV 1999
Dalal and Triggs, "Histograms of oriented gradients for human detection," CVPR 2005

图像特征: 方向梯度直方图(Histogram of Oriented Gradients, HoG)



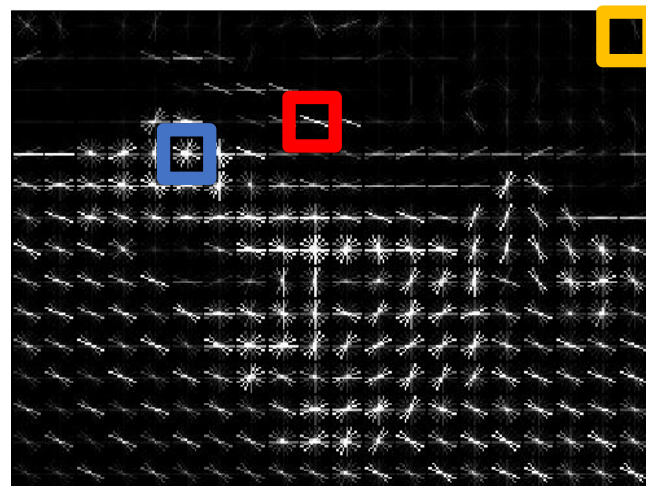
弱边缘

强对角边缘



各个方向
边缘

捕获了纹理
和位置信息,
对小图像变
化稳健



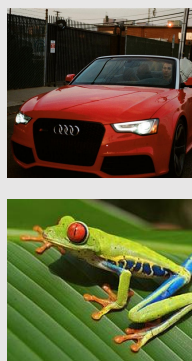
1. 计算各像素点边缘方向/强度
2. 将图像分为 8×8 子区域
3. 在每个区域计算边缘方向直方图, 边缘强度作为权重

举例: 320×240 图像分成 40×30 块; 每块 8 方向; 特征向量维度 $30 \times 40 \times 9 = 10,800$

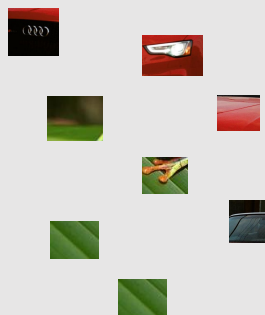
Lowe, "Object recognition from local scale-invariant features", ICCV 1999
Dalal and Triggs, "Histograms of oriented gradients for human detection," CVPR 2005

图像特征: 词袋(数据驱动!)

Step 1: 建立码书



提取随机块



图像块聚类形成
可视化块的码书

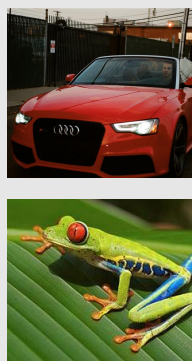


Fei-Fei and Perona, "A bayesian hierarchical model for learning natural scene categories", CVPR 2005

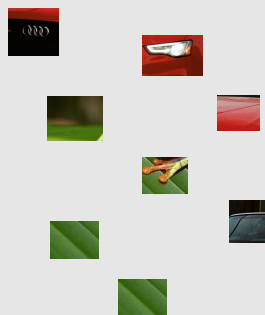
Car image is [CC0 1.0](#) public domain

图像特征: 词袋(数据驱动!)

Step 1: 建立码书



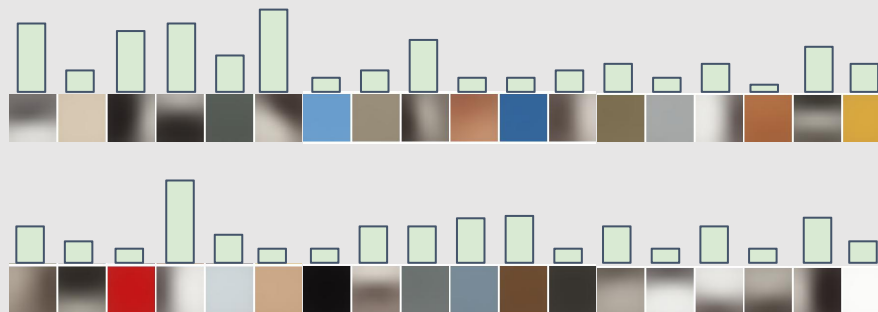
提取随机块



图像块聚类形成
可视化块的码书

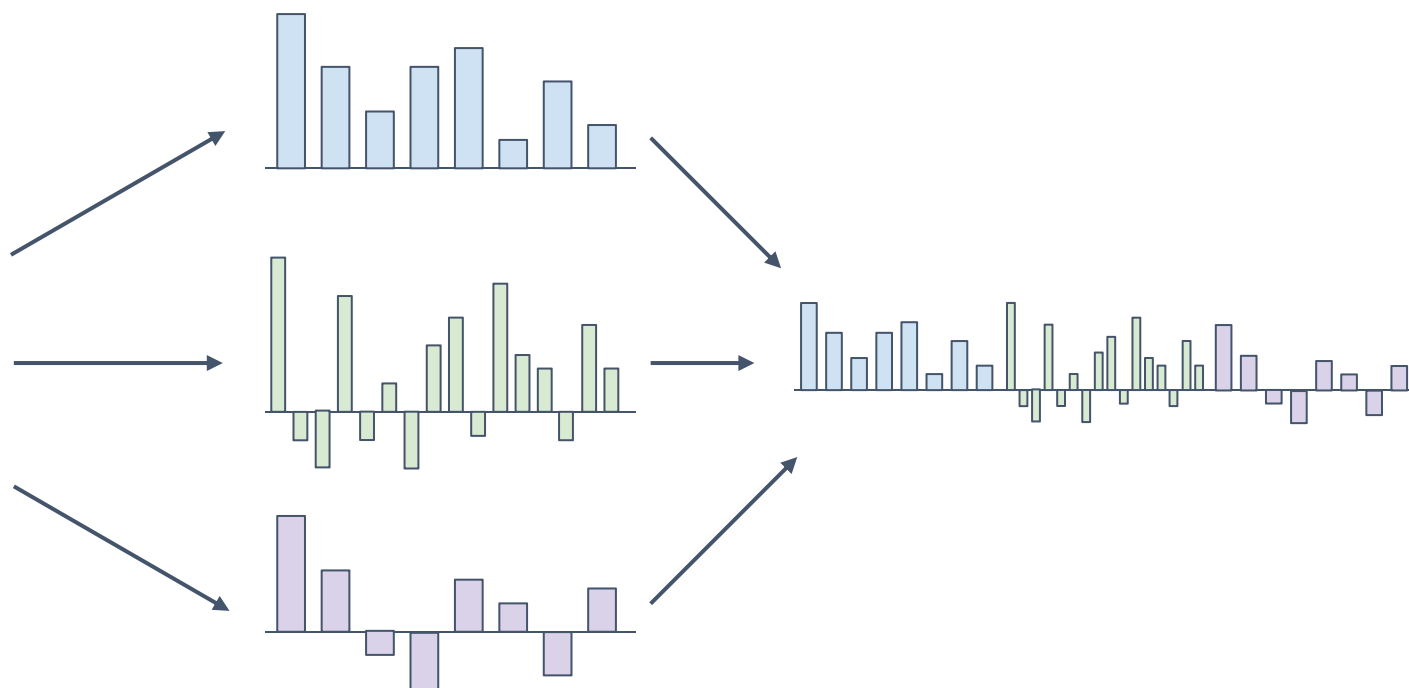


Step 2: 图像编码



Fei-Fei and Perona, "A bayesian hierarchical model for learning natural scene categories", CVPR 2005

图像特征



举例: 2011 ImageNet分类挑战赛获胜者

Low-level feature extraction $\approx$ 10k patches per image

- SIFT: 128-dim
 - color: 96-dim
- } reduced to 64-dim with PCA

FV extraction and compression:

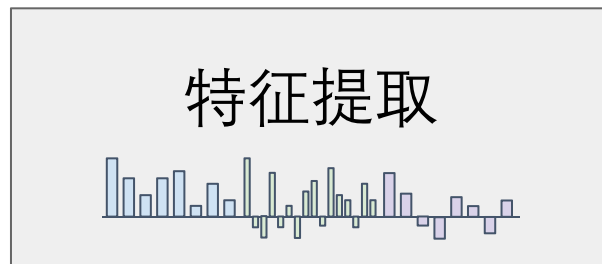
- $N=1,024$ Gaussians, $R=4$ regions $\Rightarrow$ 520K dim x 2
- compression: $G=8$, $b=1$ bit per dimension

One-vs-all SVM learning with SGD

Late fusion of SIFT and color systems

F. Perronnin, J. Sánchez, "Compressed Fisher vectors for LSVRC", PASCAL VOC / ImageNet workshop, ICCV, 2011.

图像特征

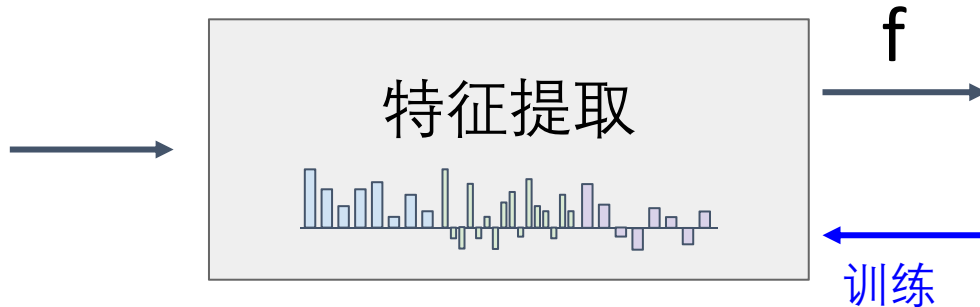


f

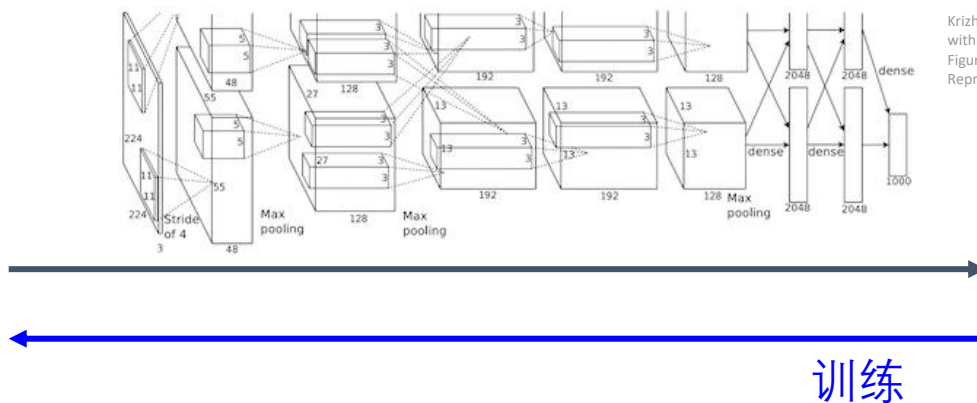
10 个数字给出各
类别的响应分值

训练

图像特征 vs 神经网络



10 个数字给出各
类别的响应分值



Krizhevsky, Sutskever, and Hinton, "Imagenet classification with deep convolutional neural networks", NIPS 2012.
Figure copyright Krizhevsky, Sutskever, and Hinton, 2012.
Reproduced with permission.

10 个数字给出各
类别的响应分值

本章内容

- 6.1.1. 多层神经网络
- 6.1.2. 激活函数
- 6.1.3. 后向传播算法

神经网络

(前面) 线性响应函数:

$$f = Wx$$

$$x \in \mathbb{R}^D, W \in \mathbb{R}^{C \times D}$$

神经网络

(前面) 线性响应函数:

$$f = Wx$$

(现在) 2-层神经网络

$$f = W_2 \max(0, W_1 x)$$

$$W_2 \in \mathbb{R}^{C \times H} \quad W_1 \in \mathbb{R}^{H \times D} \quad x \in \mathbb{R}^D$$

(实际中每层有一个偏置)

神经网络

(前面) 线性响应函数:

$$f = Wx$$

(现在) 2-层神经网络
或 3-层神经网络

$$f = W_2 \max(0, W_1 x)$$

$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$

$$W_3 \in \mathbb{R}^{C \times H_2} \quad W_2 \in \mathbb{R}^{H_2 \times H_1} \quad W_1 \in \mathbb{R}^{H_1 \times D} \quad x \in \mathbb{R}^D$$

(实际中每层有一个偏置)

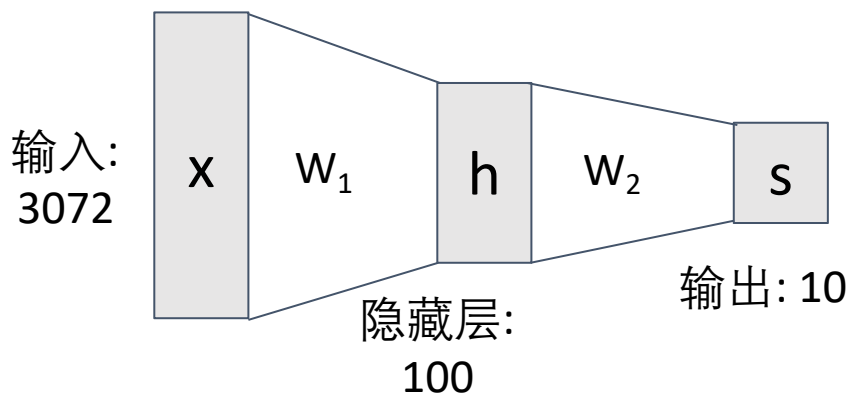
神经网络

(前面) 线性响应函数:

$$f = Wx$$

(现在) 2-层神经网络

$$f = W_2 \max(0, W_1 x)$$



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

神经网络

(前面) 线性响应函数:

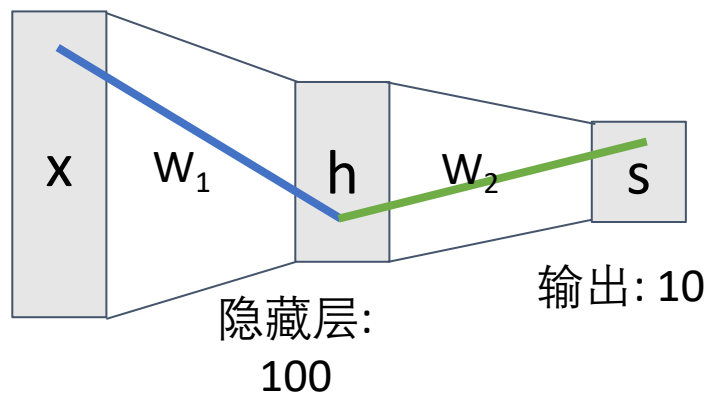
$$f = Wx$$

(现在) 2-层神经网络

$$f = W_2 \max(0, W_1 x)$$

W_1 元素 (i, j)
给出了 x_j 对 h_i
的影响

输入:
3072



W_2 元素 (i, j)
给出了 h_j 对
 s_i 影响

$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

神经网络

(前面) 线性响应函数:

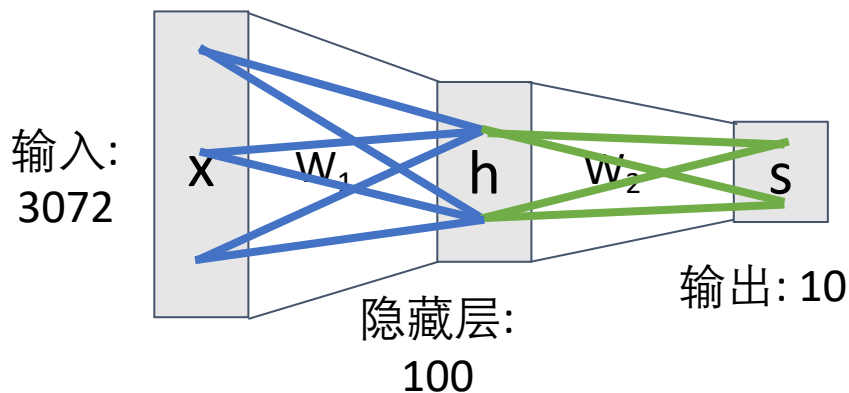
$$f = Wx$$

(现在) 2-层神经网络

$$f = W_2 \max(0, W_1 x)$$

W_1 元素 (i, j) 给出了 x_j 对 h_i 的影响

x 的所有元素影响 h 的所有元素



W_2 元素 (i, j) 给出了 h_j 对 s_i 影响

h 的所有元素影响 s 的所有元素

全连接神经网络
也称为“多层感知器”(MLP)

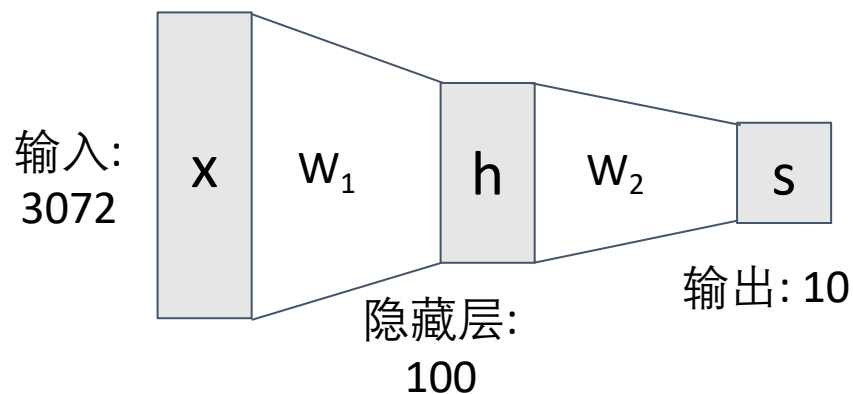
神经网络

线性分类器: 每类一个模板



(前面) 线性响应函数:

(现在) 2-层神经网络



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

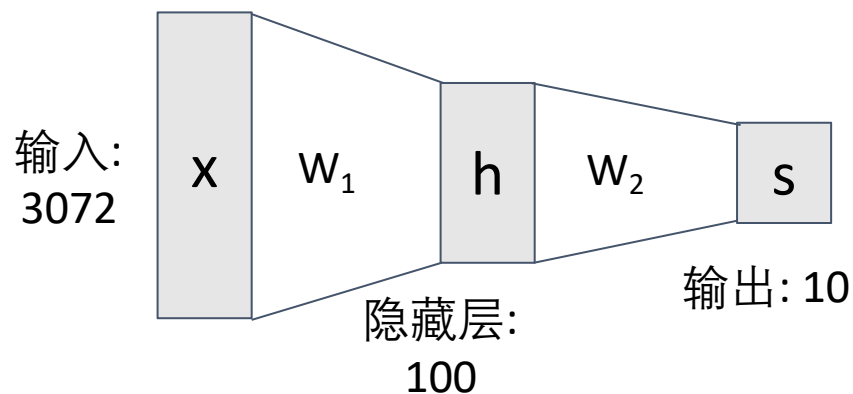
神经网络

神经网络: 第一层是模板;
第二层是第一层模板的组合



(前面) 线性响应函数:

(现在) 2-层神经网络



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

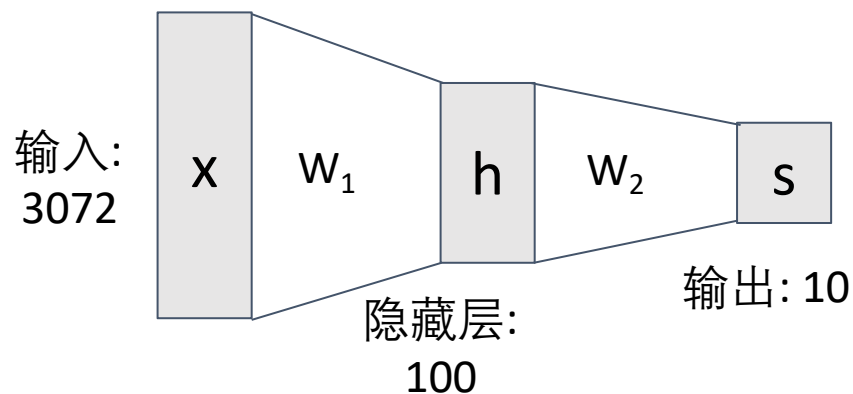
神经网络

能使用不同模板去建模图像的
多种模式!



(前面) 线性响应函数:

(现在) 2-层神经网络



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

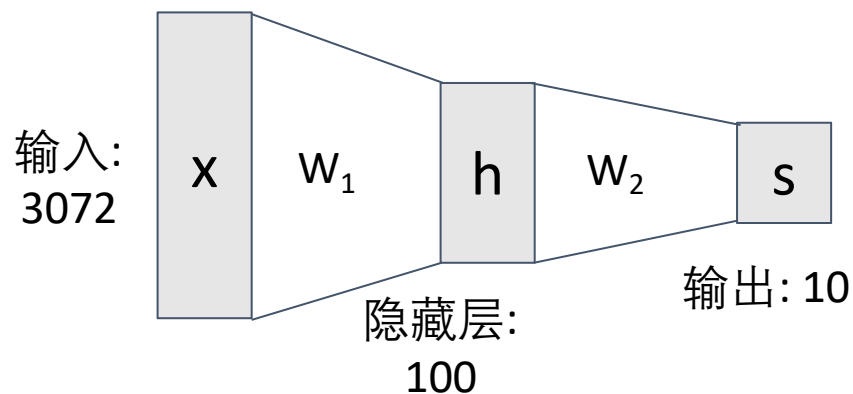
神经网络

“分布式表示”：大部分模板无法解释！



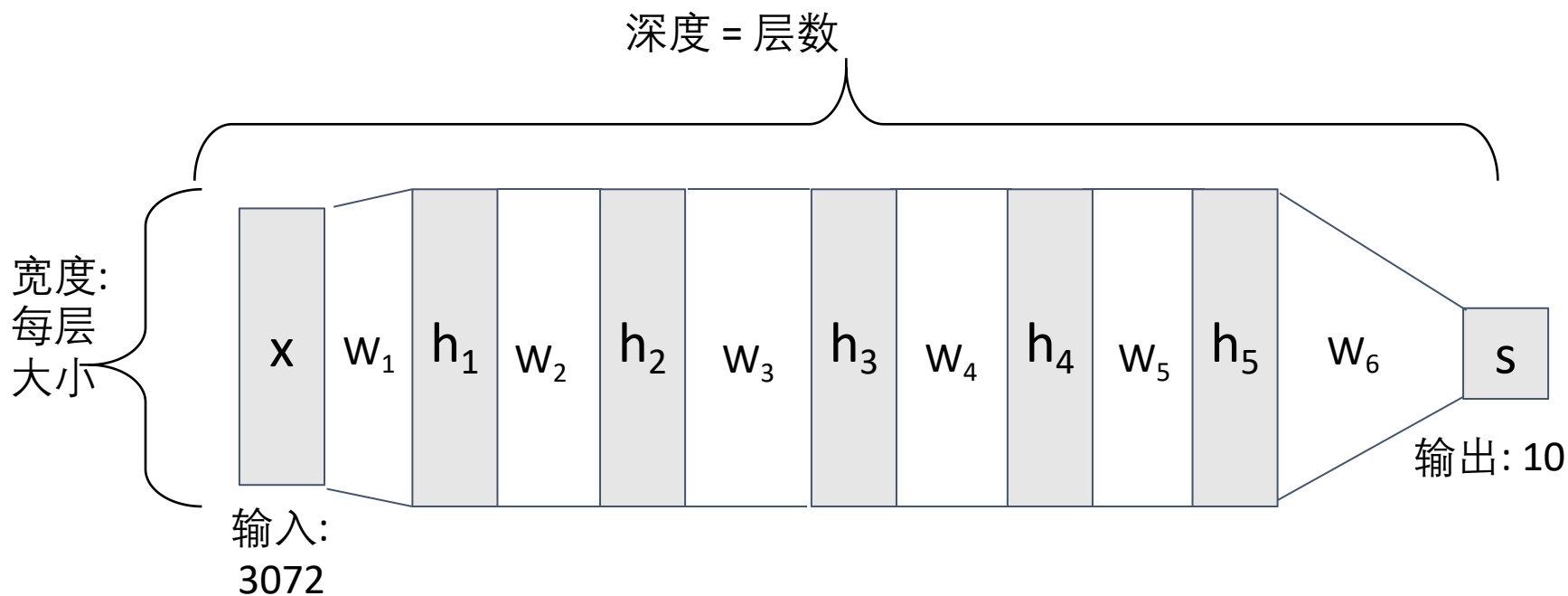
(前面) 线性响应函数:

(现在) 2-层神经网络



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

深度神经网络



$$s = W_6 \max(0, W_6 \max(0, W_5 \max(0, W_4 \max(0, W_3 \max(0, W_2 \max(0, W_1 x))))))$$

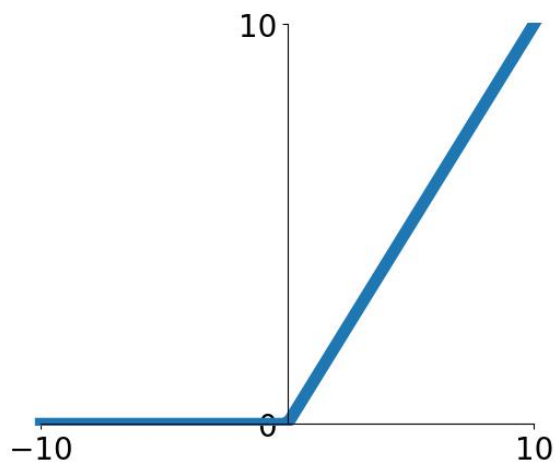
本章内容

- 6.1.1. 多层神经网络
- 6.1.2. 激活函数
- 6.1.3. 后向传播算法

激活函数

2-层神经网络

函数 $ReLU(z) = \max(0, z)$ 被称为
“修正线性单元(Rectified Linear Unit)”



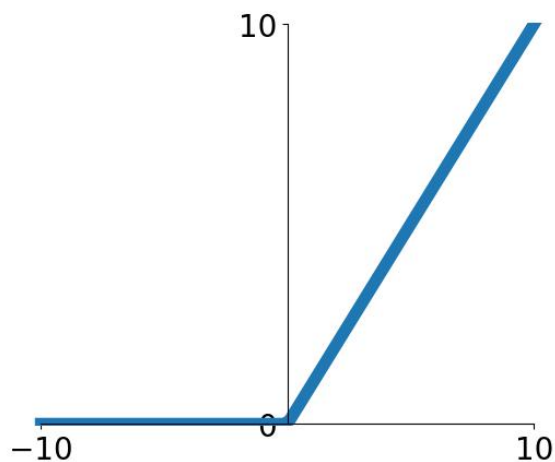
$$f = W_2 \max(0, W_1 x)$$

被称为神经网络的激活函数

激活函数

2-层神经网络

函数 $ReLU(z) = \max(0, z)$ 被称为“修正线性单元(Rectified Linear Unit)”



$$f = W_2 \max(0, W_1 x)$$

被称为神经网络的激活函数

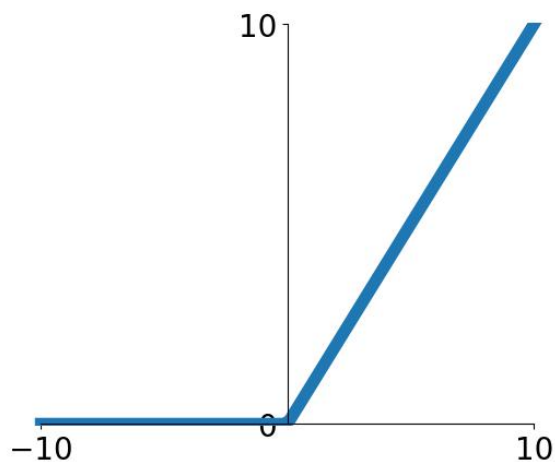
Q: 建立神经网络没有激活函数如何?

$$s = W_2 W_1 x$$

激活函数

2-层神经网络

函数 $ReLU(z) = \max(0, z)$ 被称为“修正线性单元(Rectified Linear Unit)”



$$f = W_2 \max(0, W_1 x)$$

被称为神经网络的激活函数

Q: 建立神经网络没有激活函数如何?

$$s = W_2 W_1 x$$

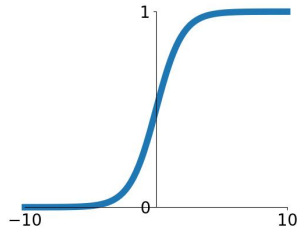
$$W_3 = W_2 W_1 \in \mathbb{R}^{C \times H} \quad s = W_3 x$$

A: 仍然是一个线性分类器!

激活函数

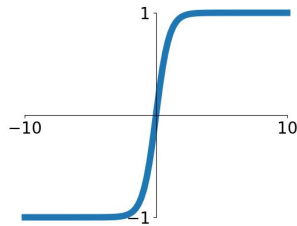
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



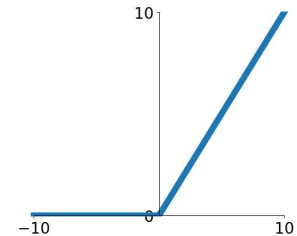
tanh

$$\tanh(x)$$



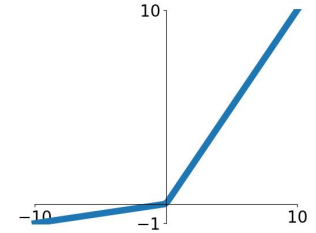
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

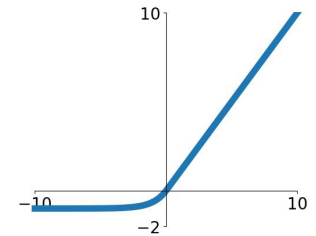


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

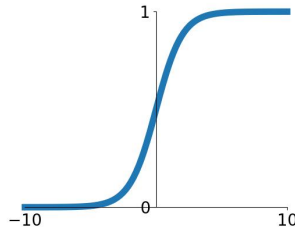
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



激活函数

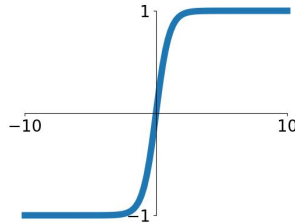
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



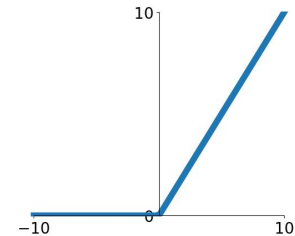
tanh

$$\tanh(x)$$



ReLU

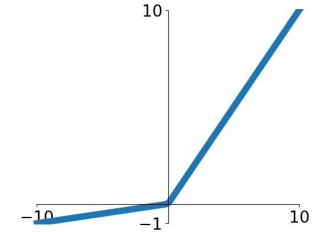
$$\max(0, x)$$



对大部分问题，ReLU 是很好的默认选择

Leaky ReLU

$$\max(0.1x, x)$$

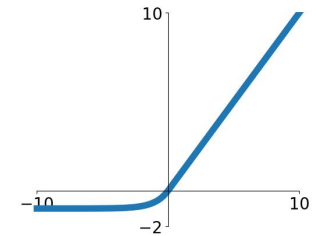


Maxout

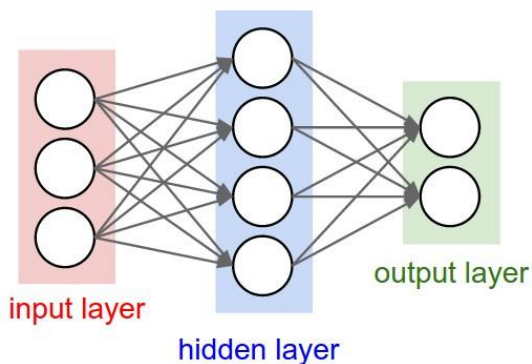
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



神经网络 <20 行!



初始权重和数据

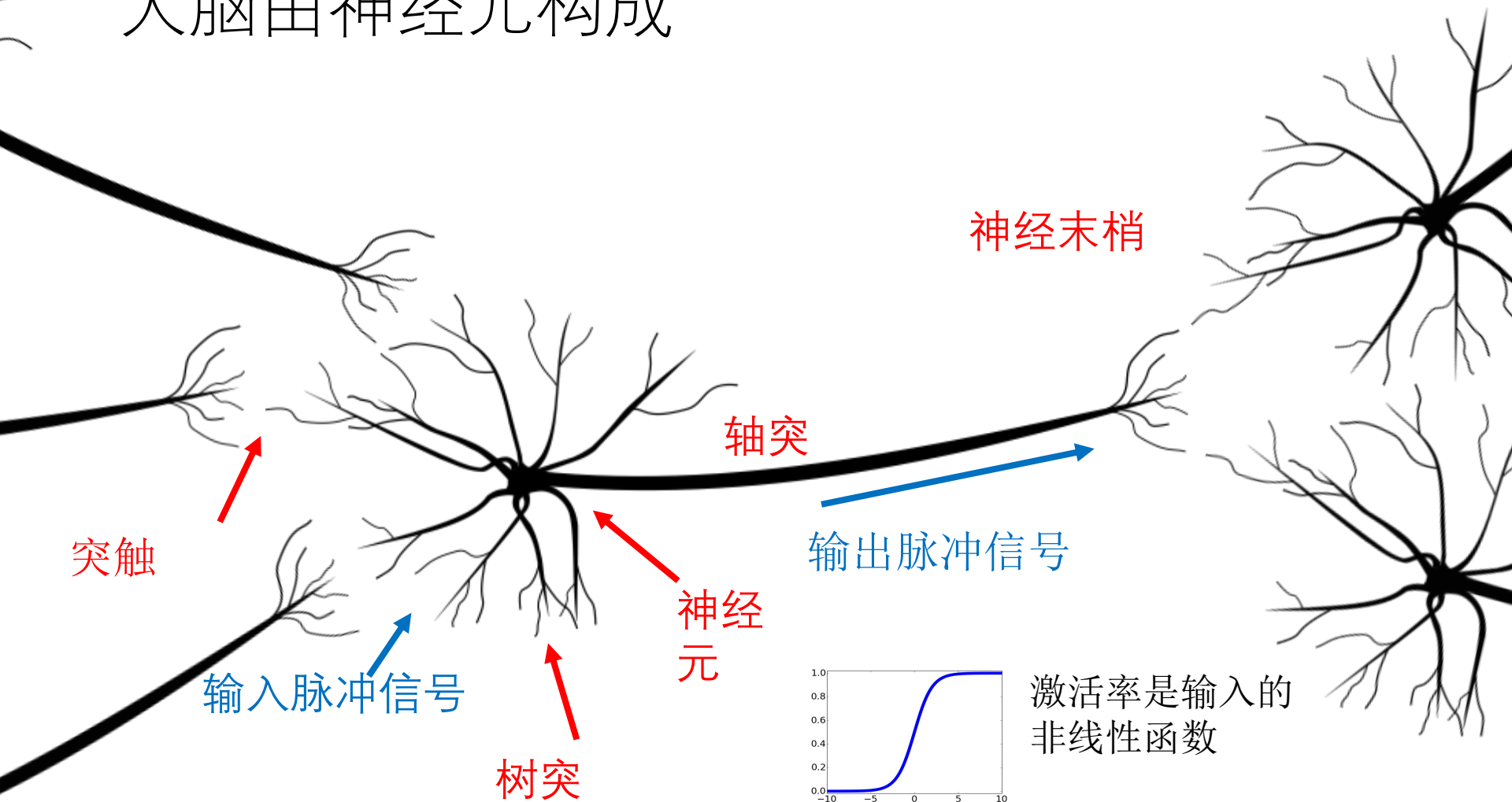
计算损失 (sigmoid 激活, L2 损失)

计算梯度

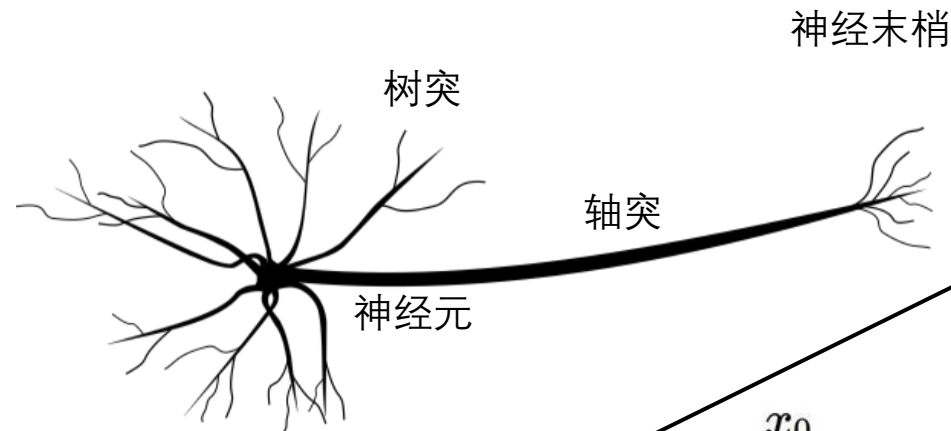
随机梯度速降 (SGD) 步骤

```
1 import numpy as np
2 from numpy.random import randn
3
4 N, Din, H, Dout = 64, 1000, 100, 10
5 x, y = randn(N, Din), randn(N, Dout)
6 w1, w2 = randn(Din, H), randn(H, Dout)
7 for t in range(10000):
8     h = 1.0 / (1.0 + np.exp(-x.dot(w1)))
9     y_pred = h.dot(w2)
10    loss = np.square(y_pred - y).sum()
11    dy_pred = 2.0 * (y_pred - y)
12    dw2 = h.T.dot(dy_pred)
13    dh = dy_pred.dot(w2.T)
14    dw1 = x.T.dot(dh * h * (1 - h))
15    w1 -= 1e-4 * dw1
16    w2 -= 1e-4 * dw2
```

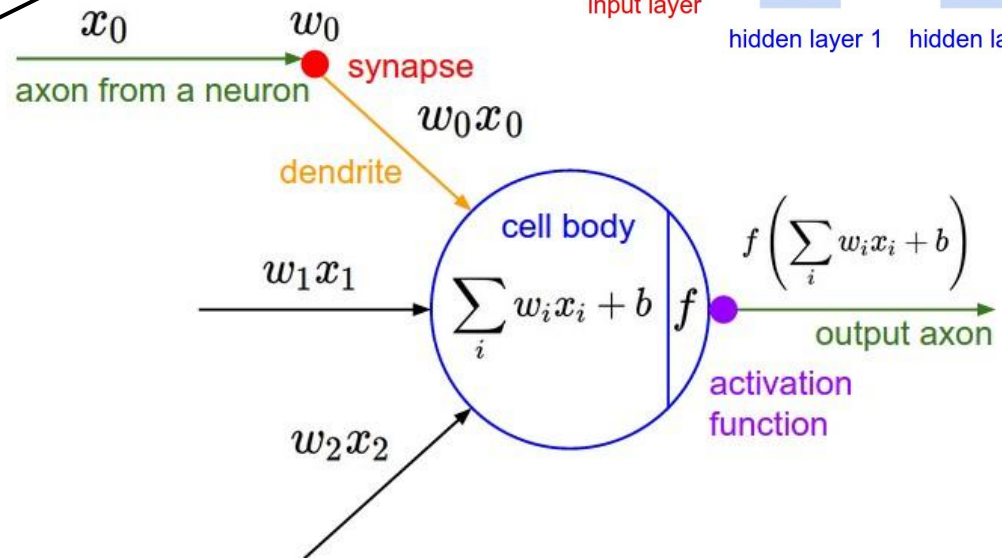
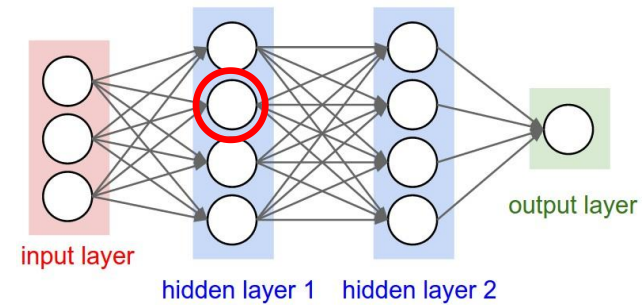
大脑由神经元构成



生物神经元

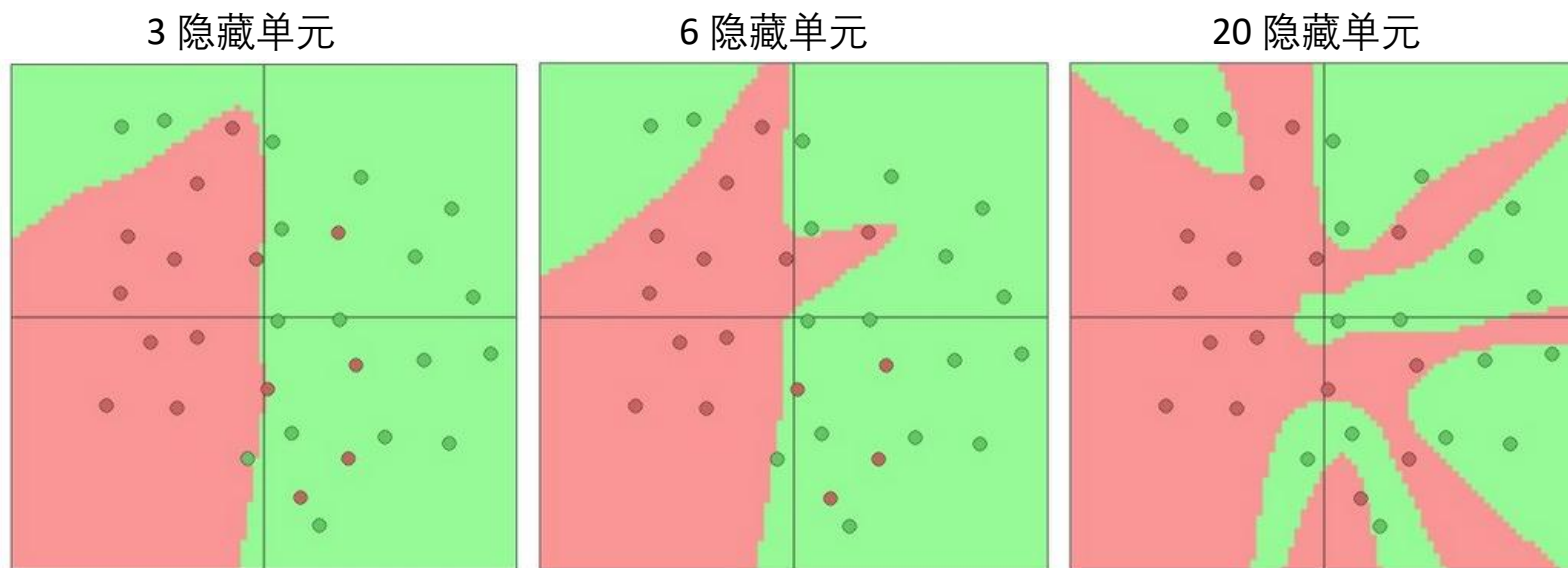


人工神经元



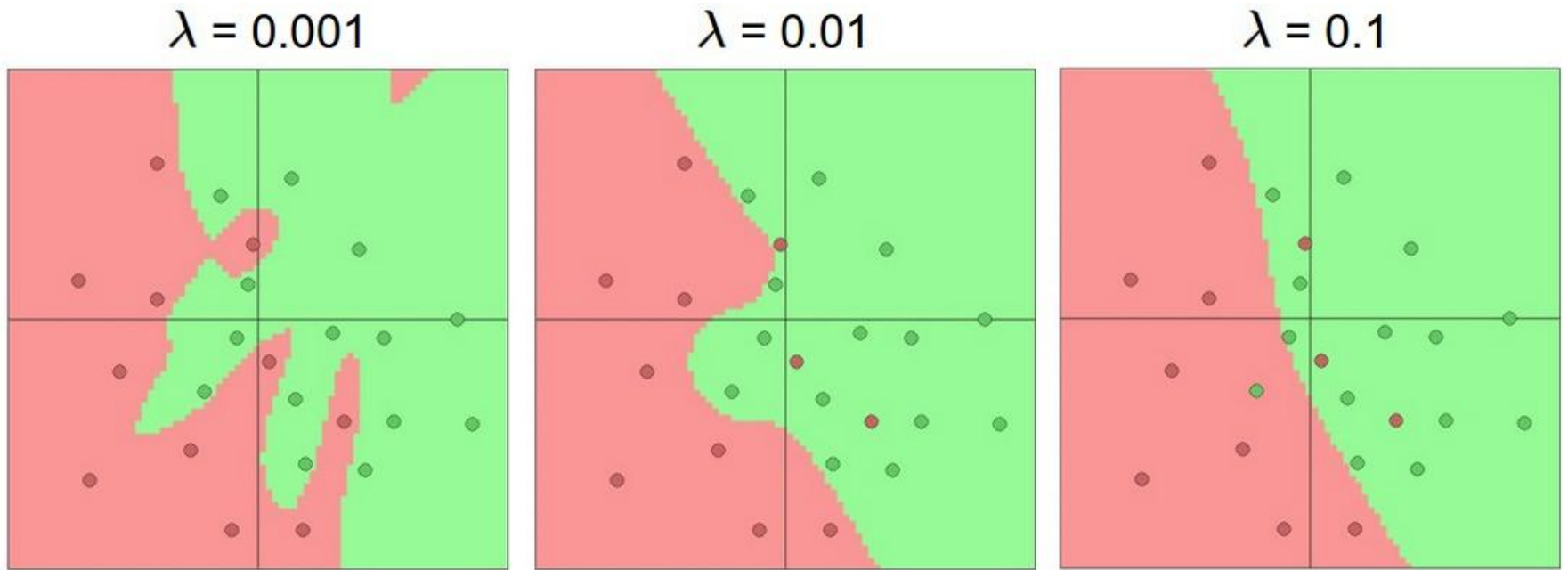
Neuron image by Felipe Peruchio
is licensed under [CC-BY 3.0](https://creativecommons.org/licenses/by/3.0/)

层数和大小设置



↑
更多隐藏单元 =
更大容量（非线性程度）

正则项



(Web demo with ConvNetJS:

<http://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html>)

本章内容

- 6.1.1. 多层神经网络
- 6.1.2. 激活函数
- 6.1.3. 后向传播算法

问题: 如何计算梯度?

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x) \quad \text{非线性响应函数}$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \quad \text{SVM 损失}$$

$$R(W) = \sum_k W_k^2 \quad \text{正则项}$$

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \lambda R(W_1) + \lambda R(W_2) \quad \text{总共损失: 数据损失 + 正则项}$$

如果我们能计算 $\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}$ 那么我们能学习 W_1 和 W_2

(不好)想法: 显示推导 $\nabla_W L$

$$s = f(x; W) = Wx$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$= \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1)$$

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \lambda \sum_k W_k^2$$

$$= \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2$$

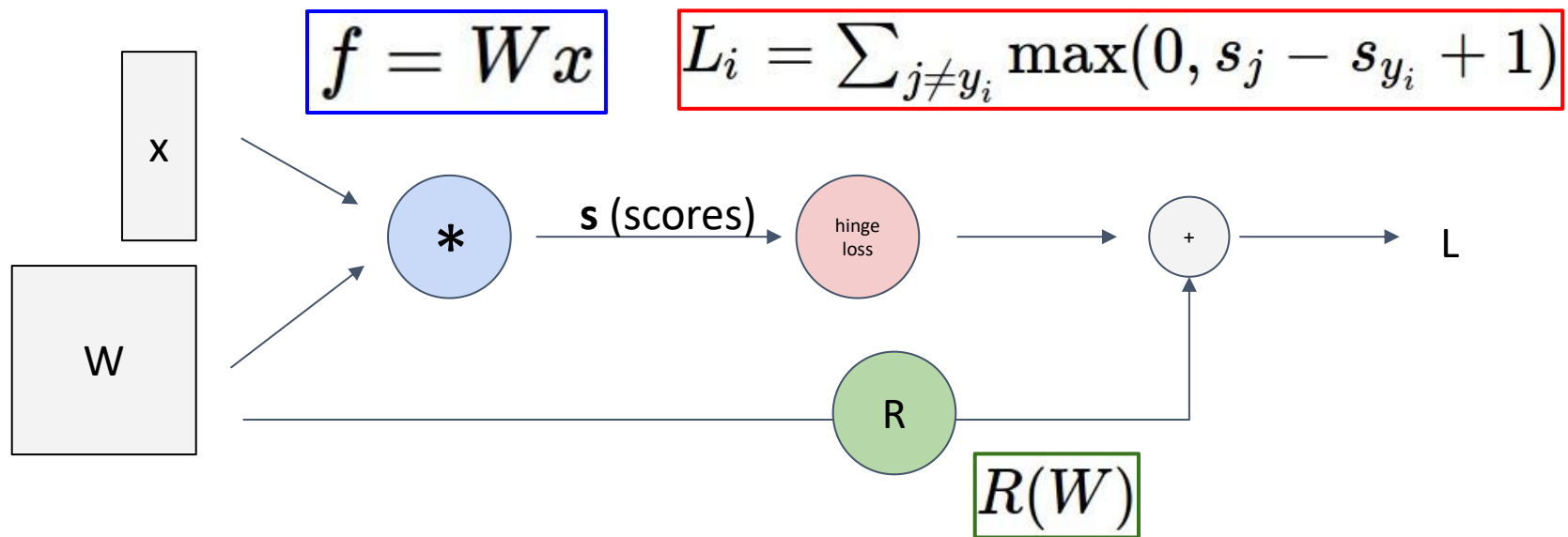
$$\nabla_W L = \nabla_W \left(\frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2 \right)$$

问题: 非常枯燥: 很多矩阵微积分

问题: 如何改变损失函数? E.g. 使用 softmax 替代 SVM? 需要重新推导!

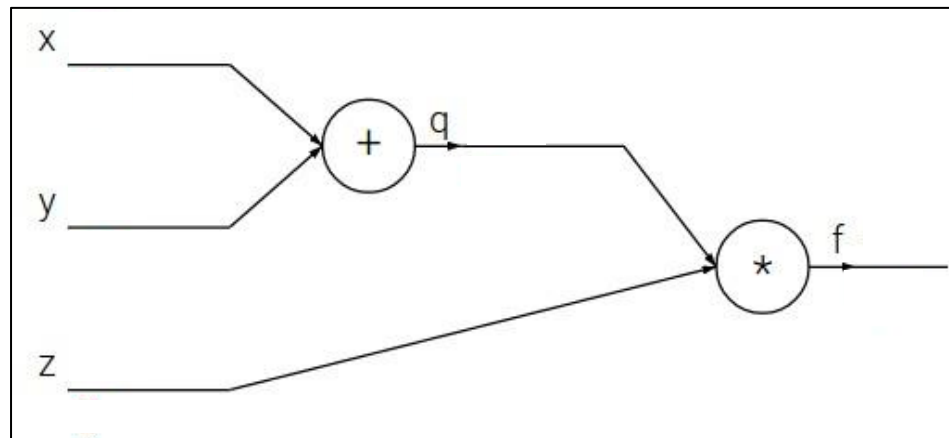
问题: 模型复杂时没法显示推导!

更好的方法: 计算图



后向传播算法:
举例

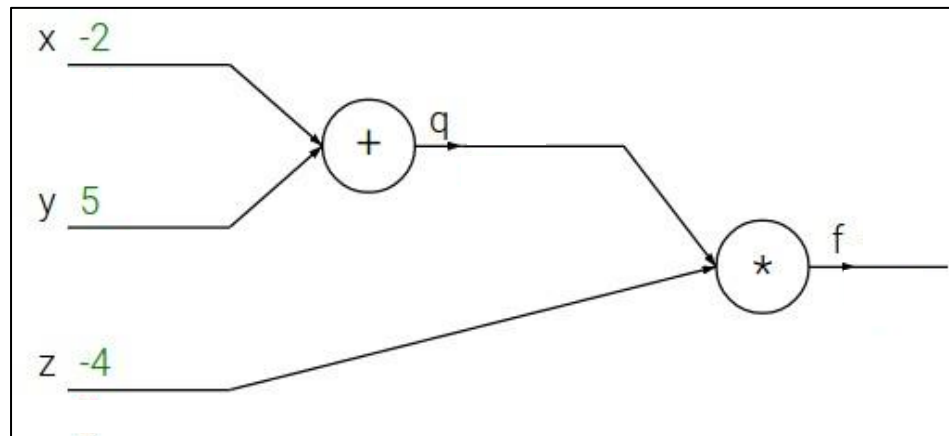
$$f(x, y, z) = (x + y)z$$



后向传播算法:
举例

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



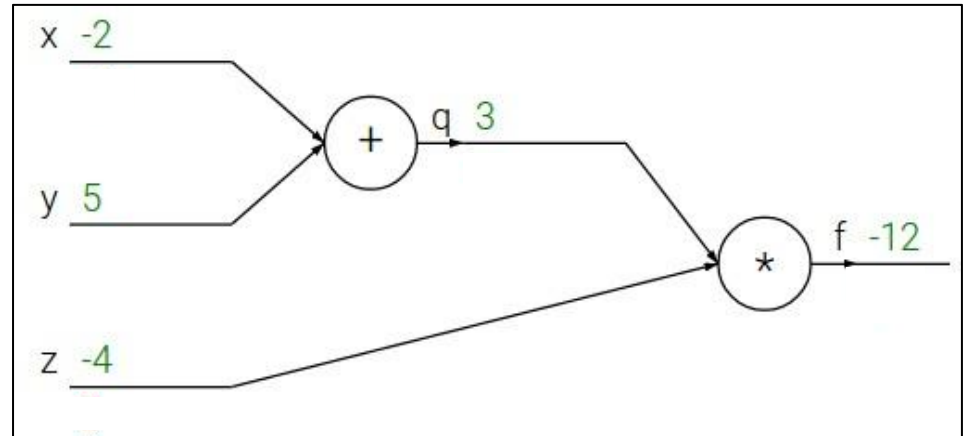
后向传播算法: 举例

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

1. 前向传递: 计算输出

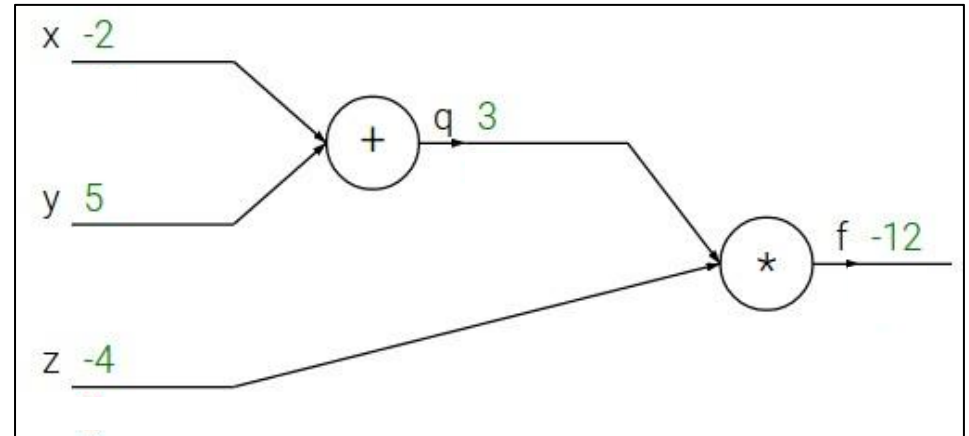
$$q = x + y \quad f = qz$$



后向传播算法: 举例

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



1. 前向传递: 计算输出

$$q = x + y \quad f = qz$$

2. 后向传递: 计算导数

想计算: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

后向传播算法: 举例

$$f(x, y, z) = (x + y)z$$

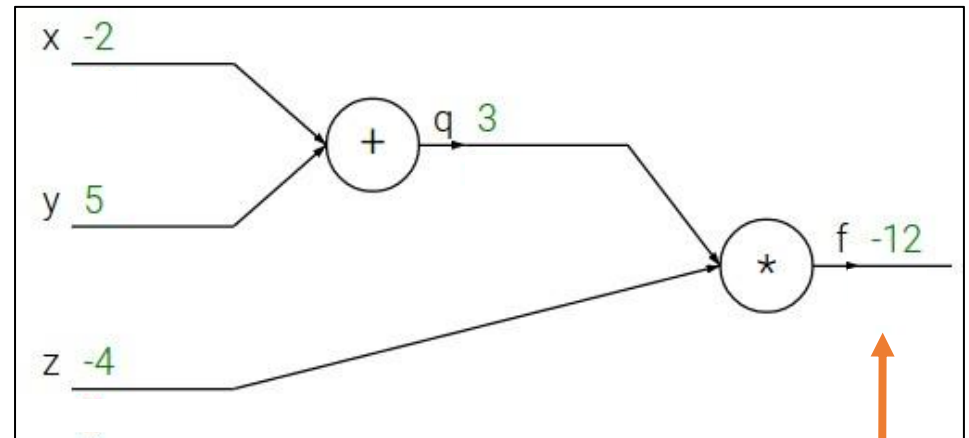
e.g. $x = -2, y = 5, z = -4$

1. 前向传递: 计算输出

$$q = x + y \quad f = qz$$

2. 后向传递: 计算导数

想计算: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial f}$$

后向传播算法: 举例

$$f(x, y, z) = (x + y)z$$

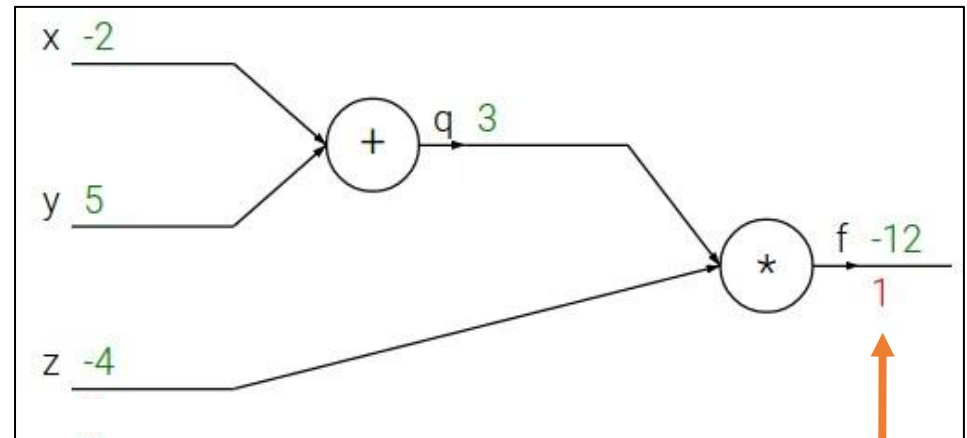
e.g. $x = -2, y = 5, z = -4$

1. 前向传递: 计算输出

$$q = x + y \quad f = qz$$

2. 后向传递: 计算导数

想计算: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial f}$$

后向传播算法: 举例

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

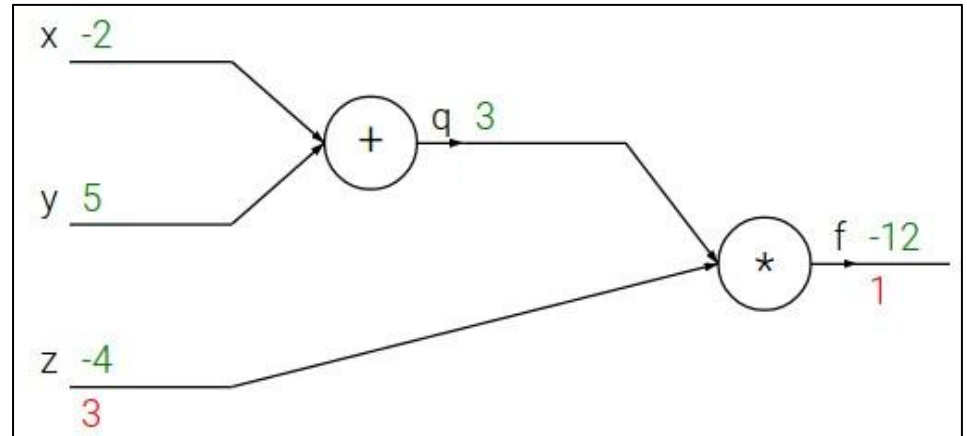
1. 前向传递: 计算输出

$$q = x + y$$

$$f = qz$$

2. 后向传递: 计算导数

想计算: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

后向传播算法: 举例

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

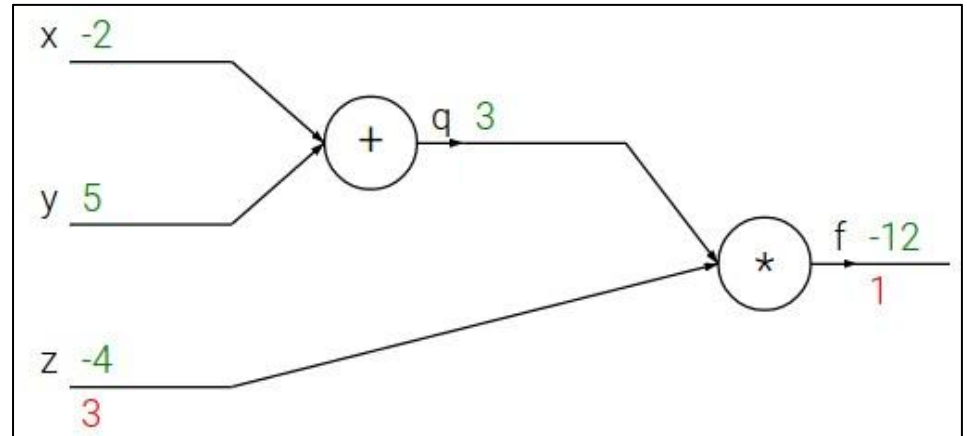
1. 前向传递: 计算输出

$$q = x + y$$

$$f = qz$$

2. 后向传递: 计算导数

想计算: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z} = q$$

后向传播算法: 举例

$$f(x, y, z) = (x + y)z$$

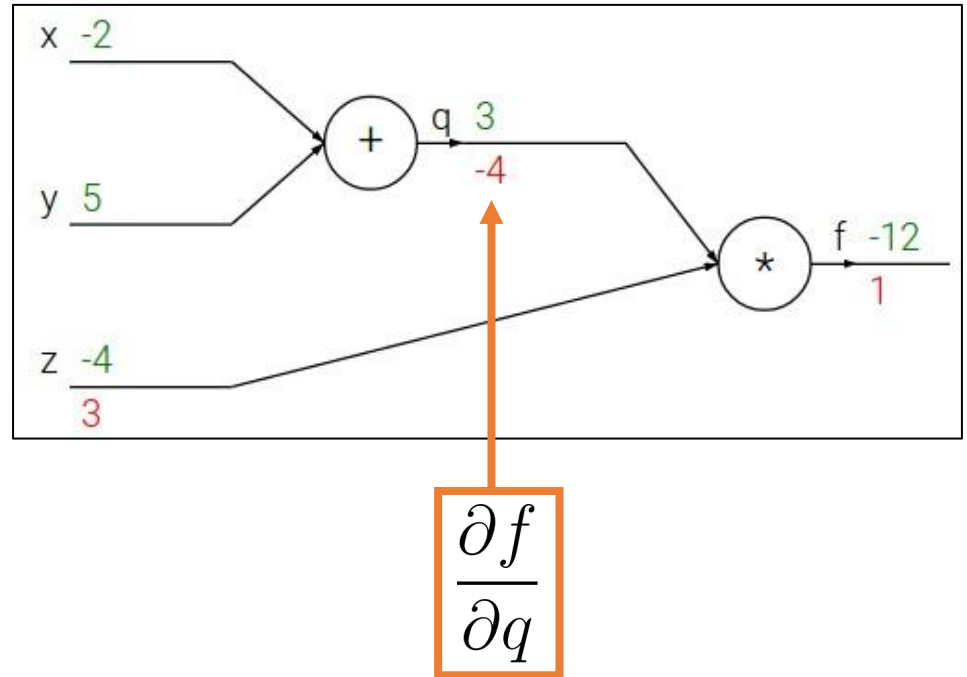
e.g. $x = -2, y = 5, z = -4$

1. 前向传递: 计算输出

$$q = x + y \quad f = qz$$

2. 后向传递: 计算导数

想计算: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



后向传播算法: 举例

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

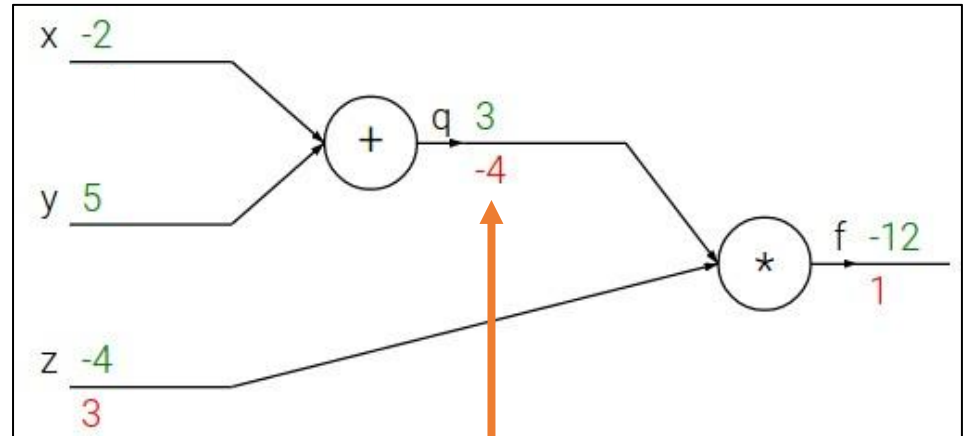
1. 前向传递: 计算输出

$$q = x + y$$

$$f = qz$$

2. 后向传递: 计算导数

想计算: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial q} = z$$

后向传播算法: 举例

$$f(x, y, z) = (x + y)z$$

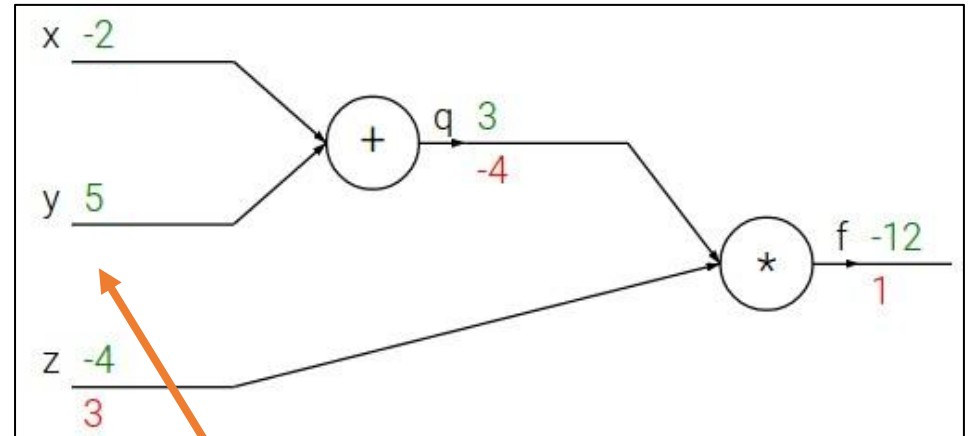
e.g. $x = -2, y = 5, z = -4$

1. 前向传递: 计算输出

$$q = x + y \quad f = qz$$

2. 后向传递: 计算导数

想计算: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial y}$$

后向传播算法: 举例

$$f(x, y, z) = (x + y)z$$

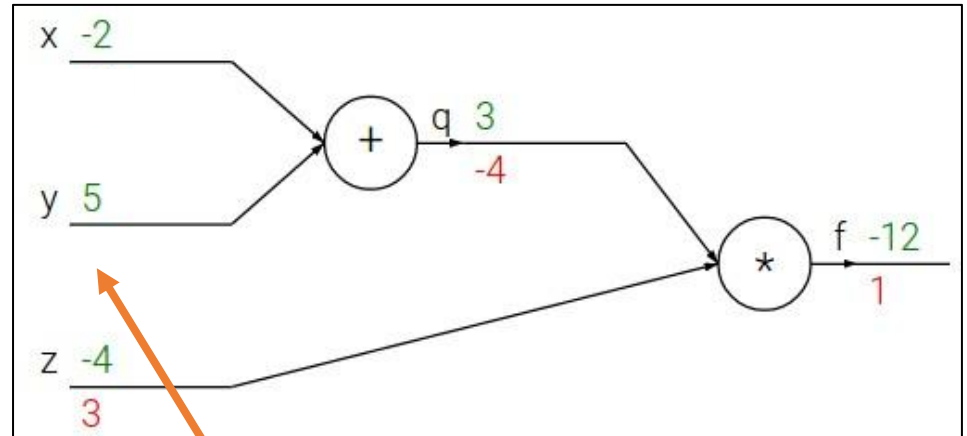
e.g. $x = -2, y = 5, z = -4$

1. 前向传递: 计算输出

$$q = x + y \quad f = qz$$

2. 后向传递: 计算导数

想计算: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



链式法则

$$\frac{\partial f}{\partial y} = \frac{\partial q}{\partial y} \frac{\partial f}{\partial q}$$

后向传播算法: 举例

$$f(x, y, z) = (x + y)z$$

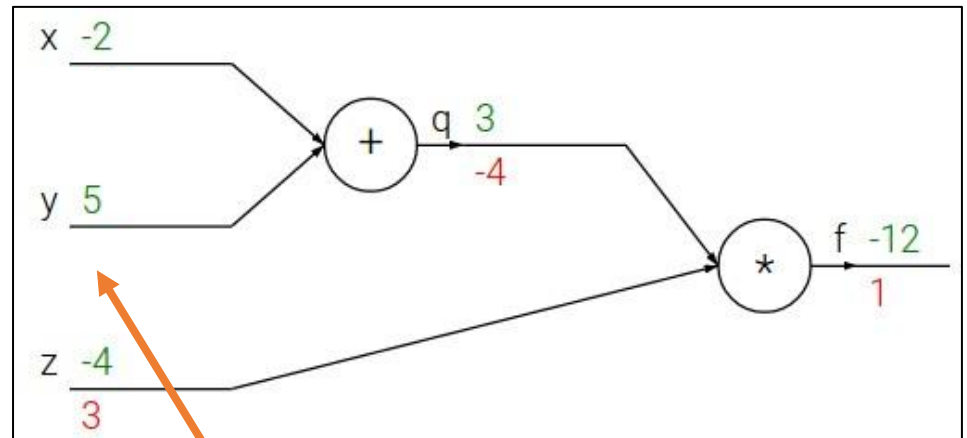
e.g. $x = -2, y = 5, z = -4$

1. 前向传递: 计算输出

$$q = x + y \quad f = qz$$

2. 后向传递: 计算导数

想计算: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



链式法则

$$\frac{\partial f}{\partial y} = \frac{\partial q}{\partial y} \frac{\partial f}{\partial q}$$

$$\frac{\partial q}{\partial y} = 1$$

下游梯度

局部梯度

上游梯度

后向传播算法: 举例

$$f(x, y, z) = (x + y)z$$

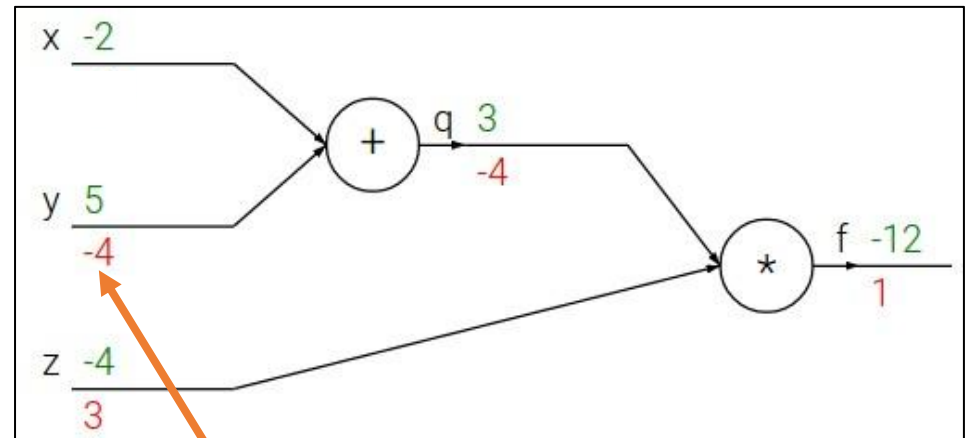
e.g. $x = -2, y = 5, z = -4$

1. 前向传递: 计算输出

$$q = x + y \quad f = qz$$

2. 后向传递: 计算导数

想计算: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



链式法则

$$\frac{\partial f}{\partial y} = \frac{\partial q}{\partial y} \frac{\partial f}{\partial q}$$

$$\frac{\partial q}{\partial y} = 1$$

下游梯度

局部梯度

上游梯度

后向传播算法: 举例

$$f(x, y, z) = (x + y)z$$

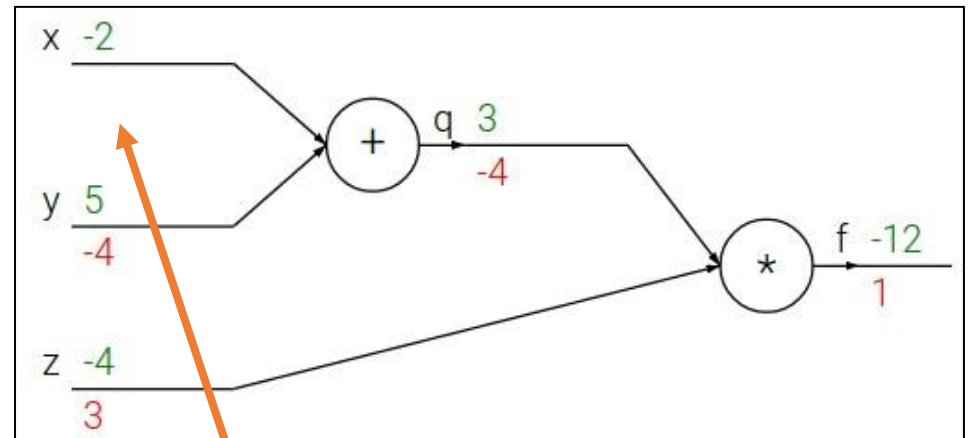
e.g. $x = -2, y = 5, z = -4$

1. 前向传递: 计算输出

$$q = x + y \quad f = qz$$

2. 后向传递: 计算导数

想计算: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



链式法则

$$\frac{\partial f}{\partial x} = \frac{\partial q}{\partial x} \frac{\partial f}{\partial q}$$

$$\frac{\partial q}{\partial x} = 1$$

下游梯度

局部梯度

上游梯度

后向传播算法: 举例

$$f(x, y, z) = (x + y)z$$

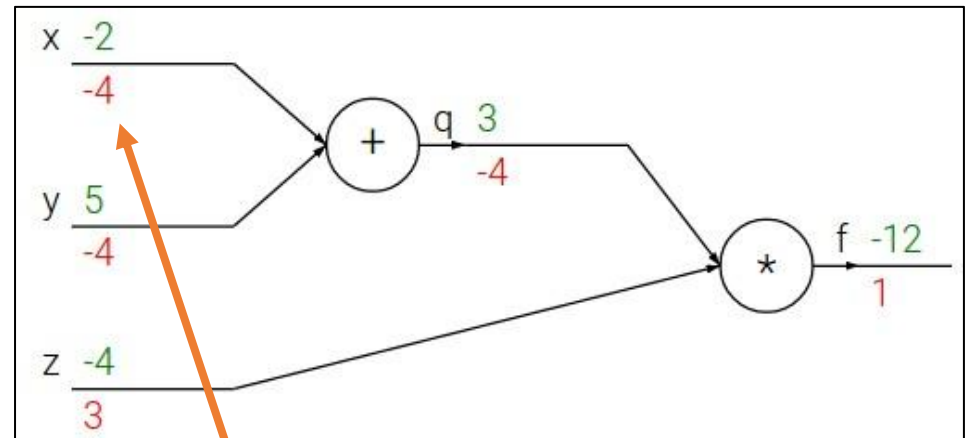
e.g. $x = -2, y = 5, z = -4$

1. 前向传递: 计算输出

$$q = x + y \quad f = qz$$

2. 后向传递: 计算导数

想计算: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



链式法则

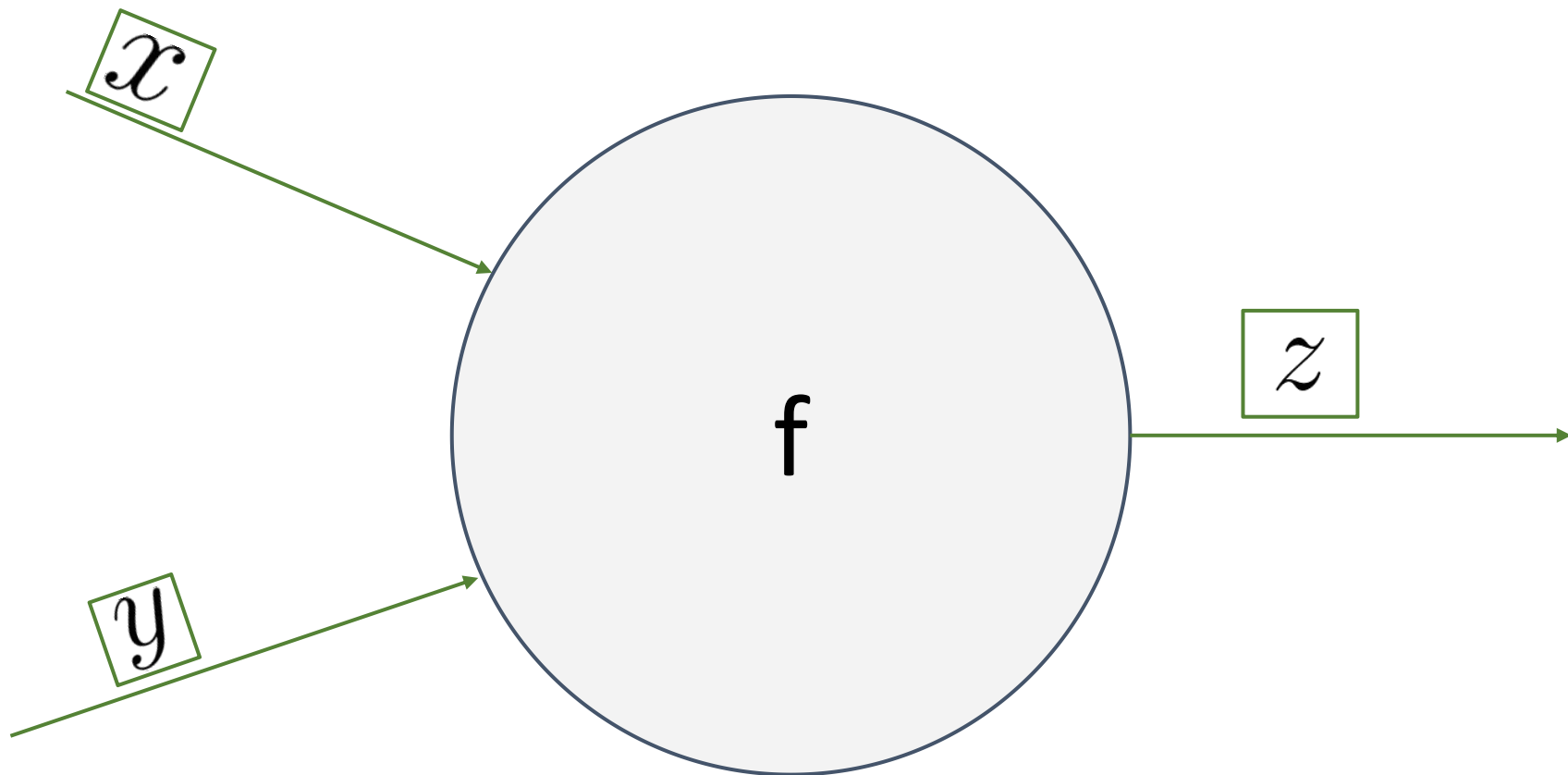
$$\frac{\partial f}{\partial x} = \frac{\partial q}{\partial x} \frac{\partial f}{\partial q}$$

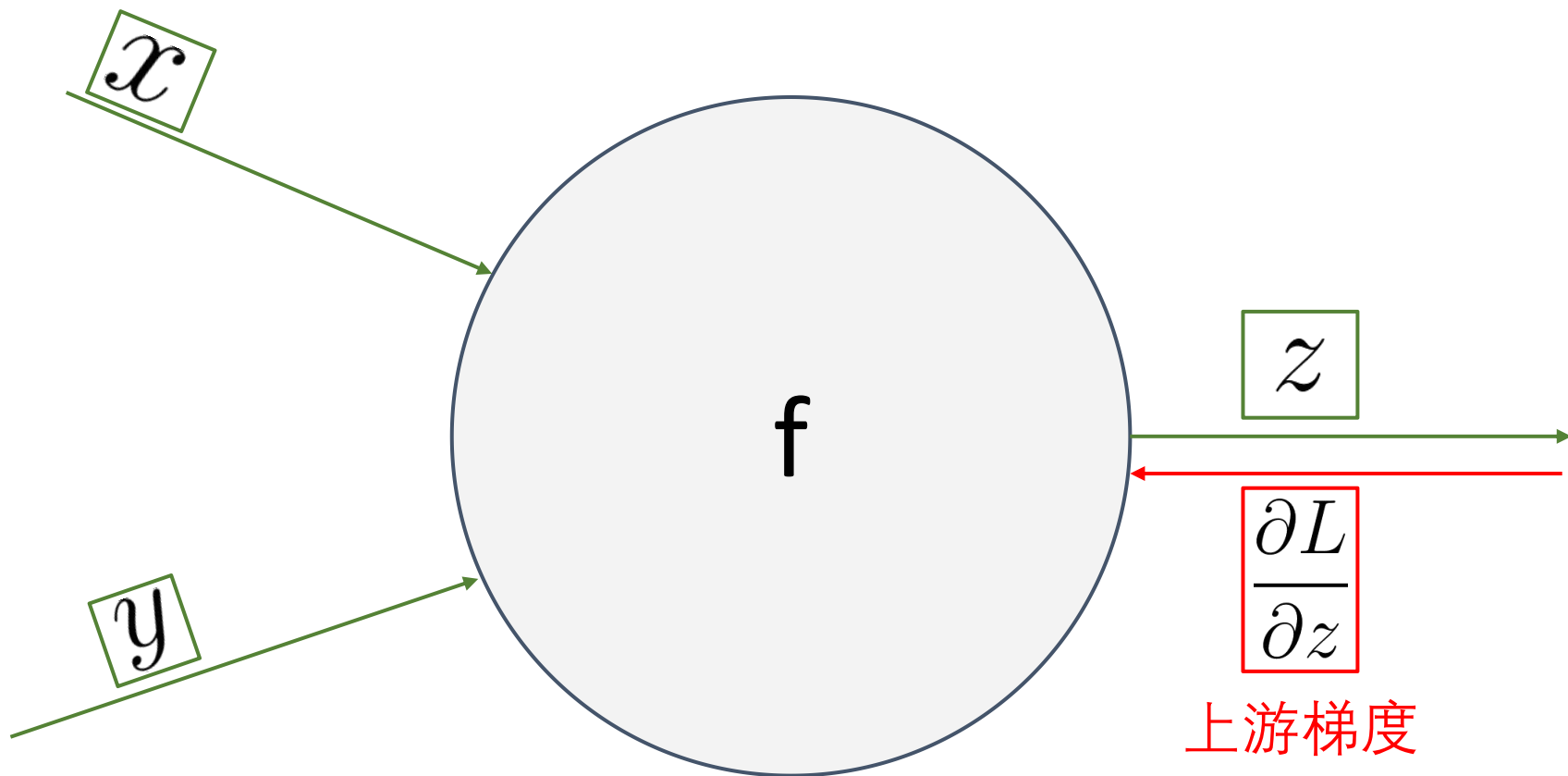
$$\frac{\partial q}{\partial x} = 1$$

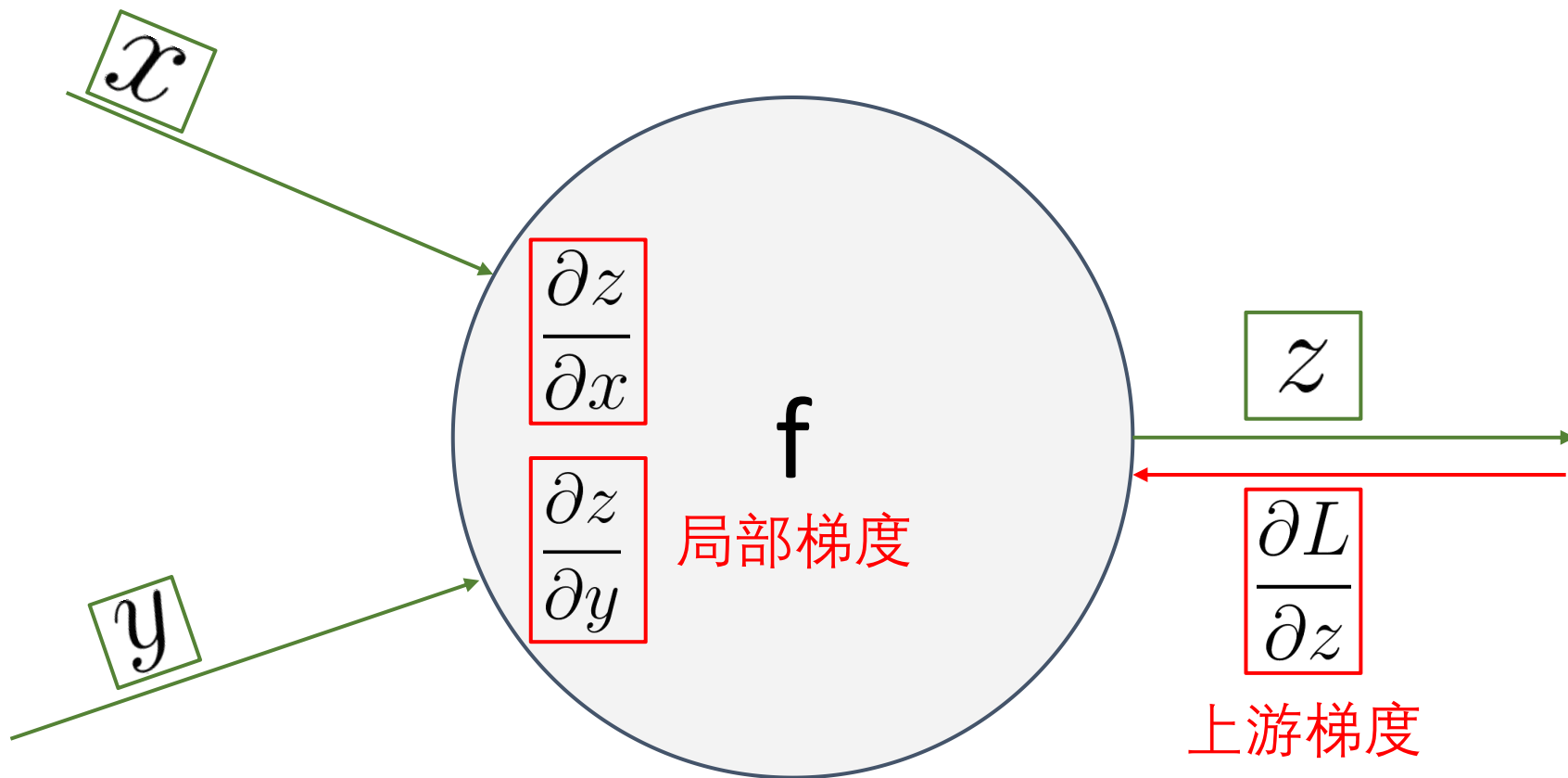
下游梯度

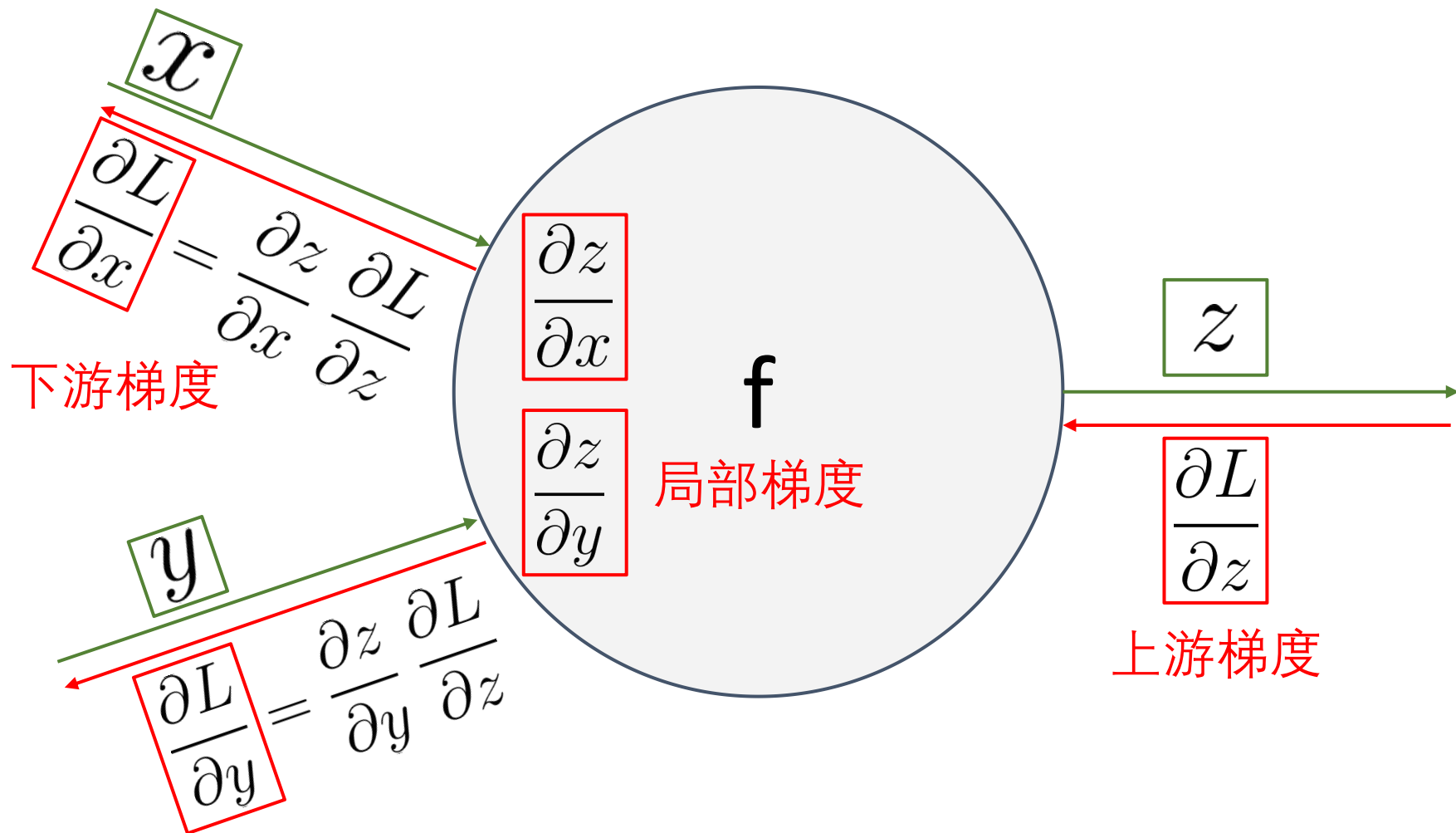
局部梯度

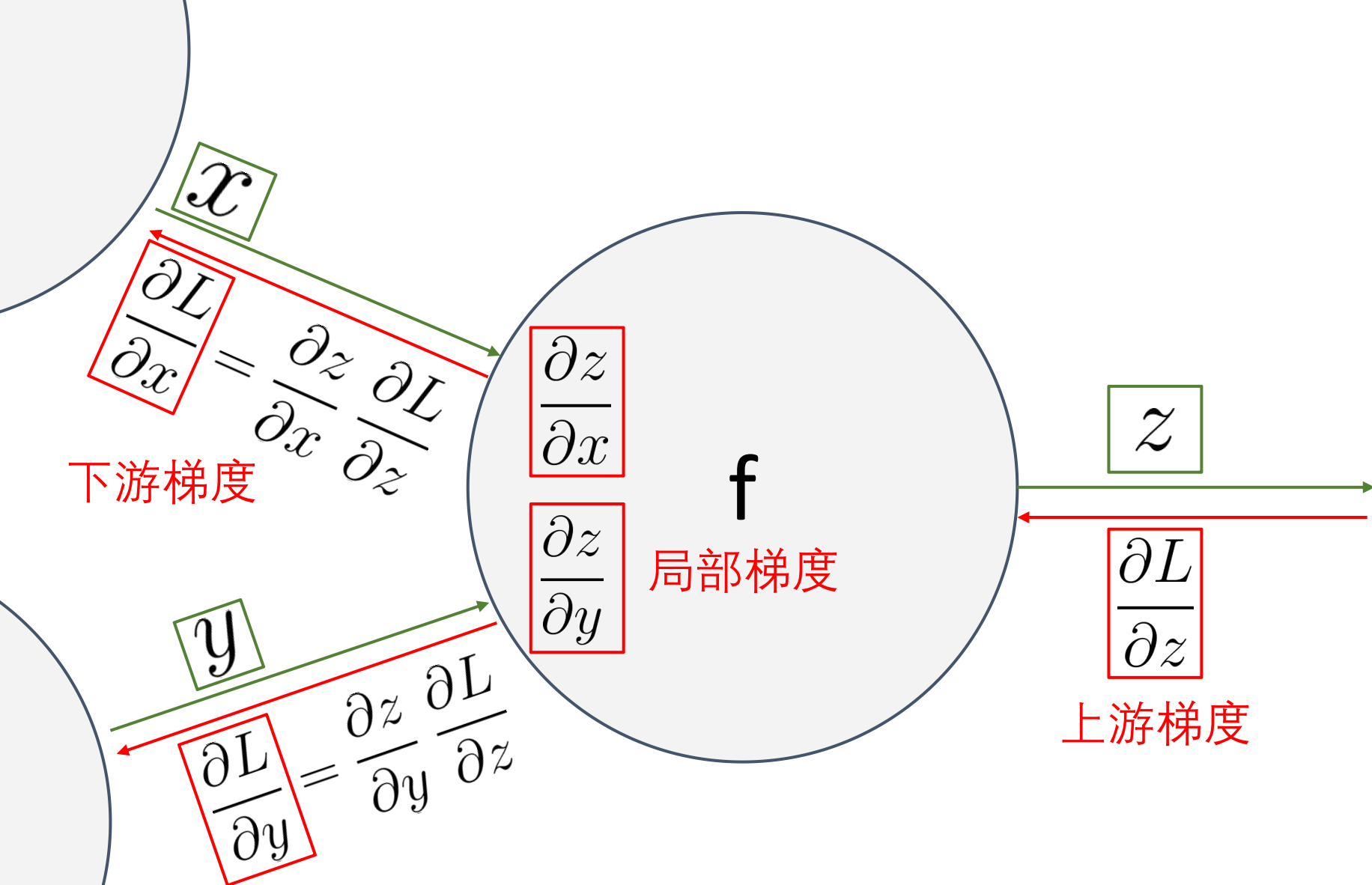
上游梯度





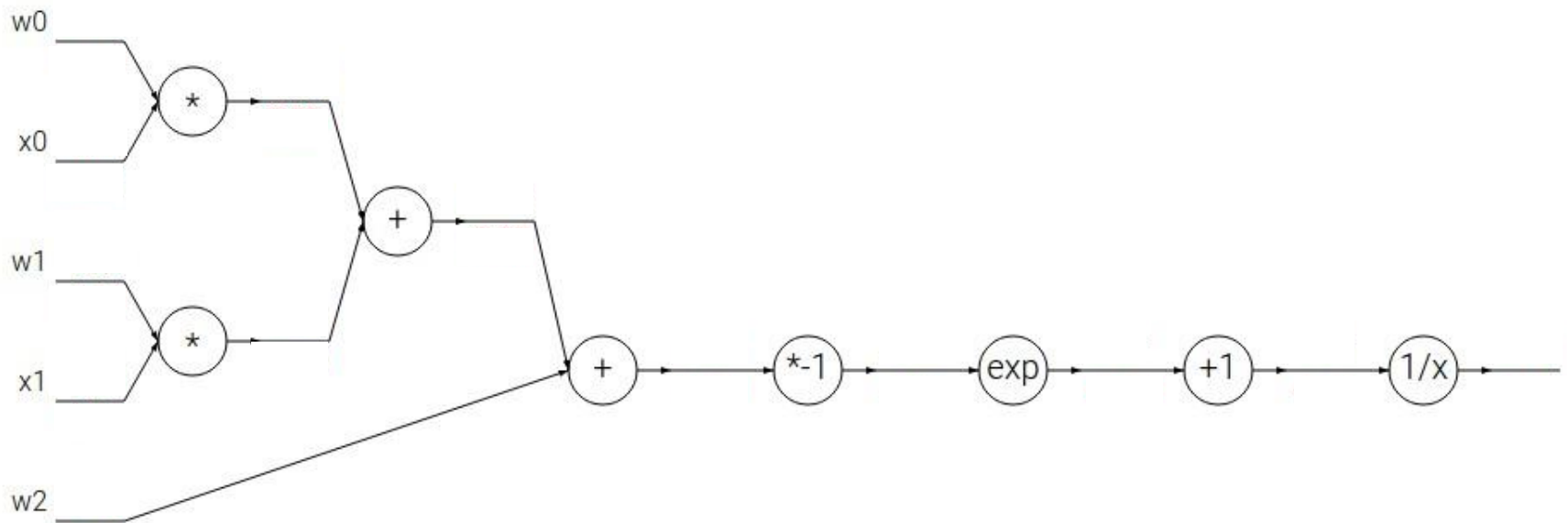






另一个示例

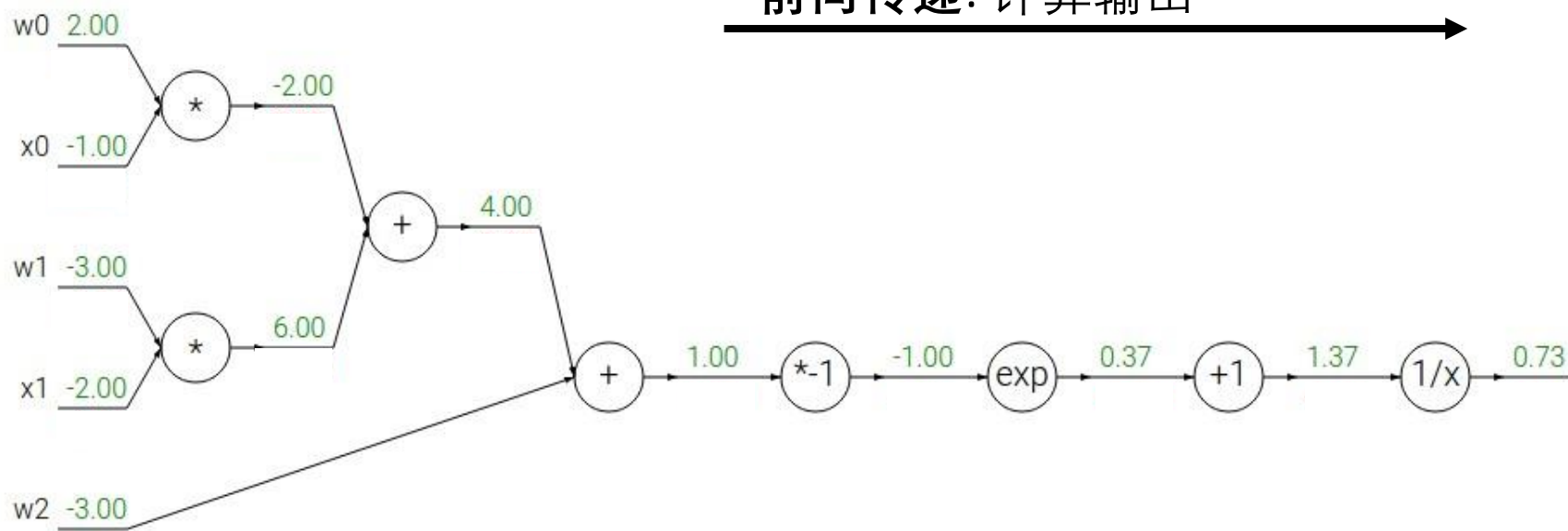
$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



另一个示例

$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

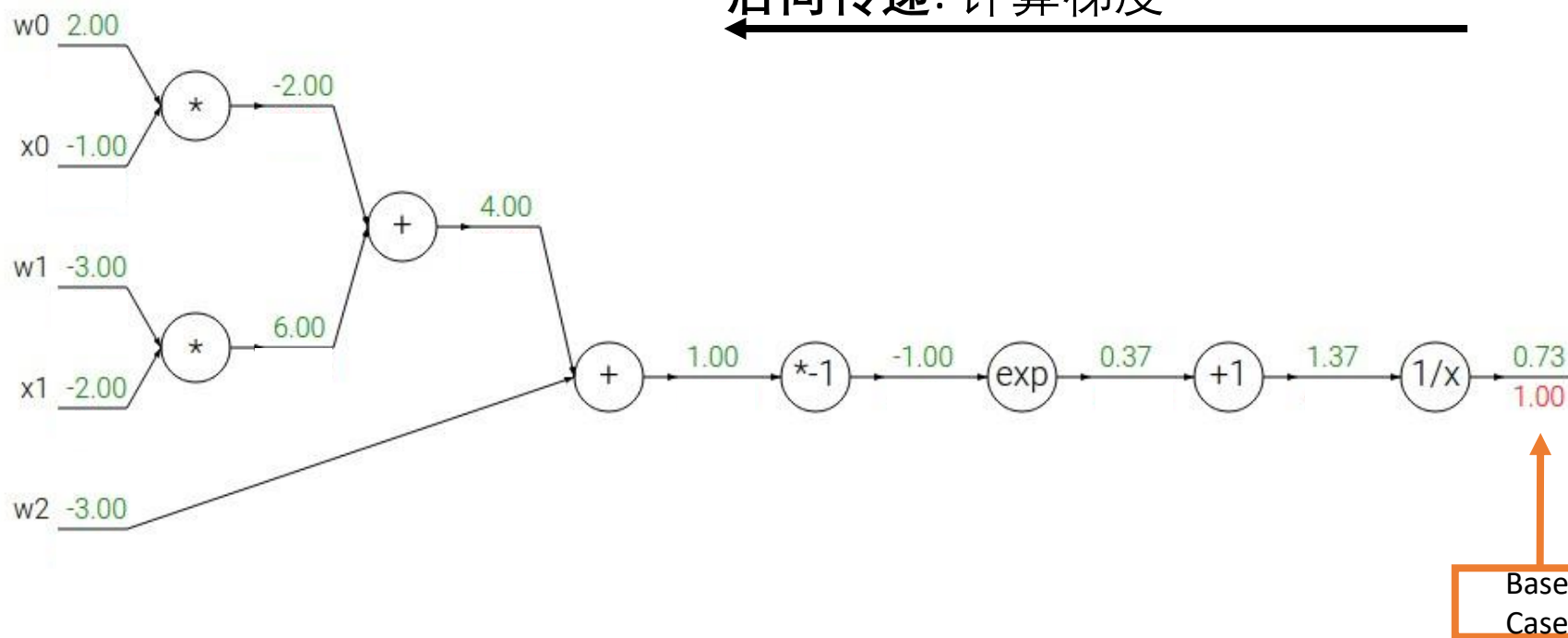
前向传递: 计算输出



另一个示例

$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

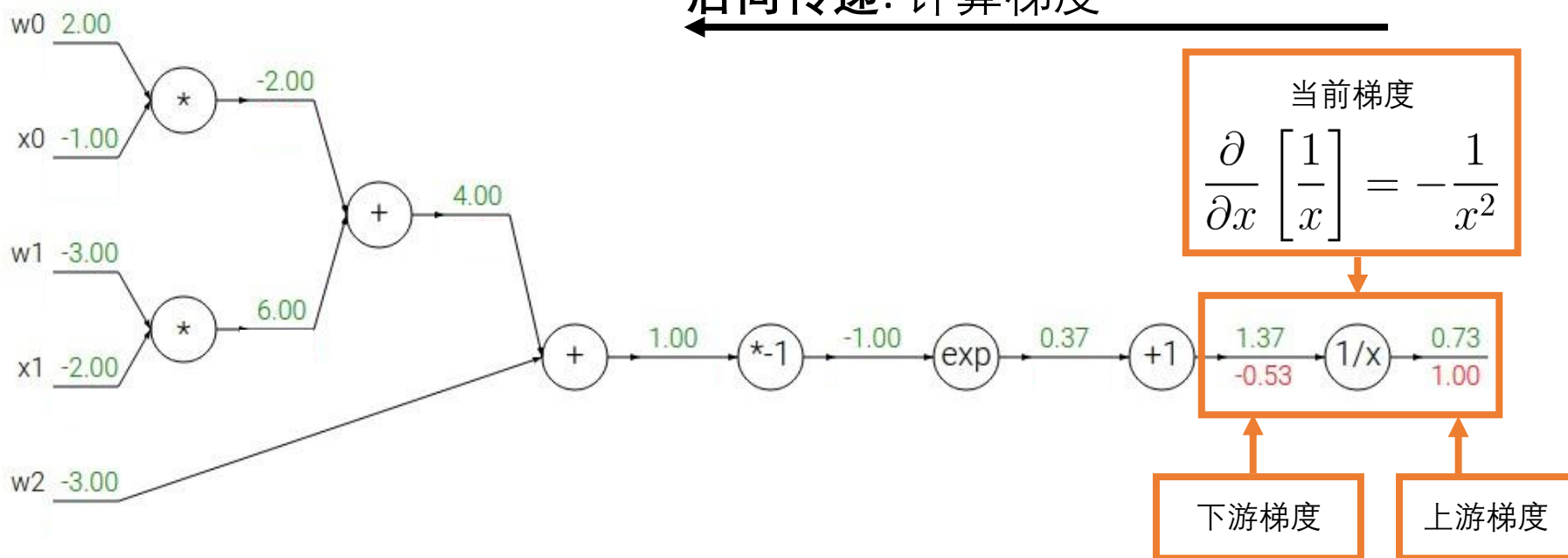
后向传递: 计算梯度



另一个示例

$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

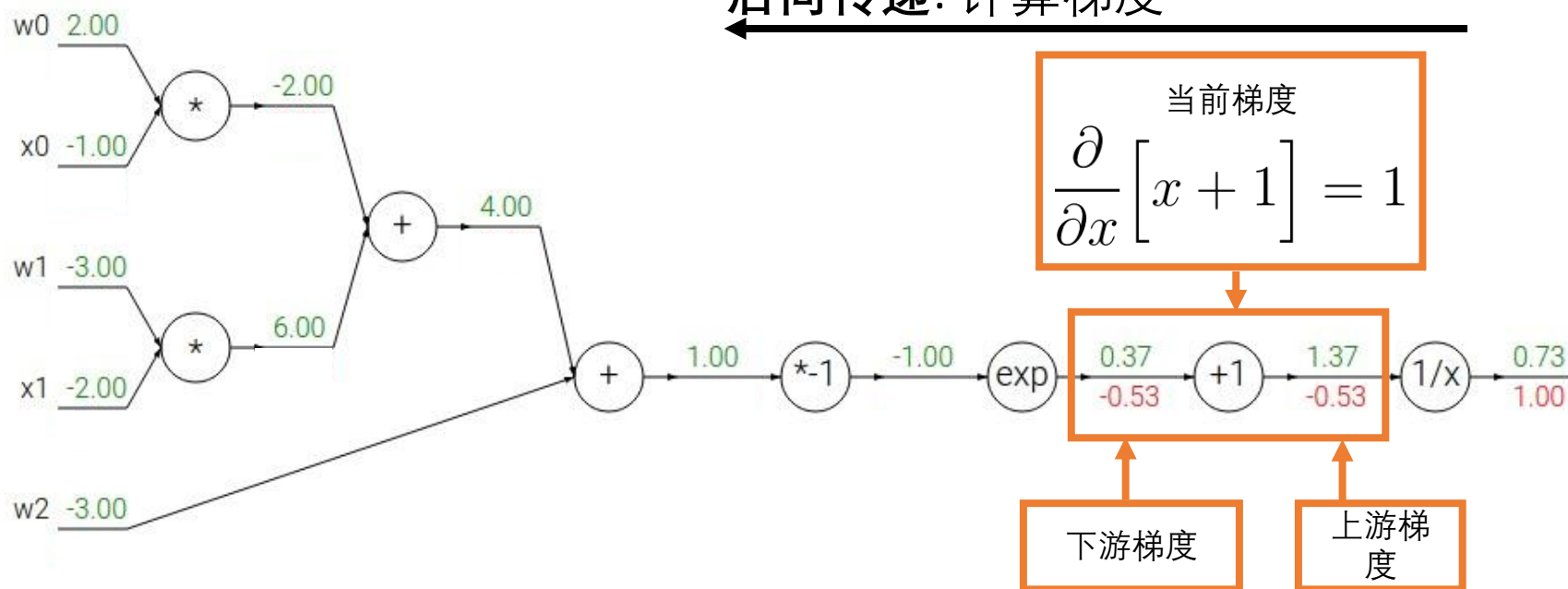
后向传递: 计算梯度



另一个示例

$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

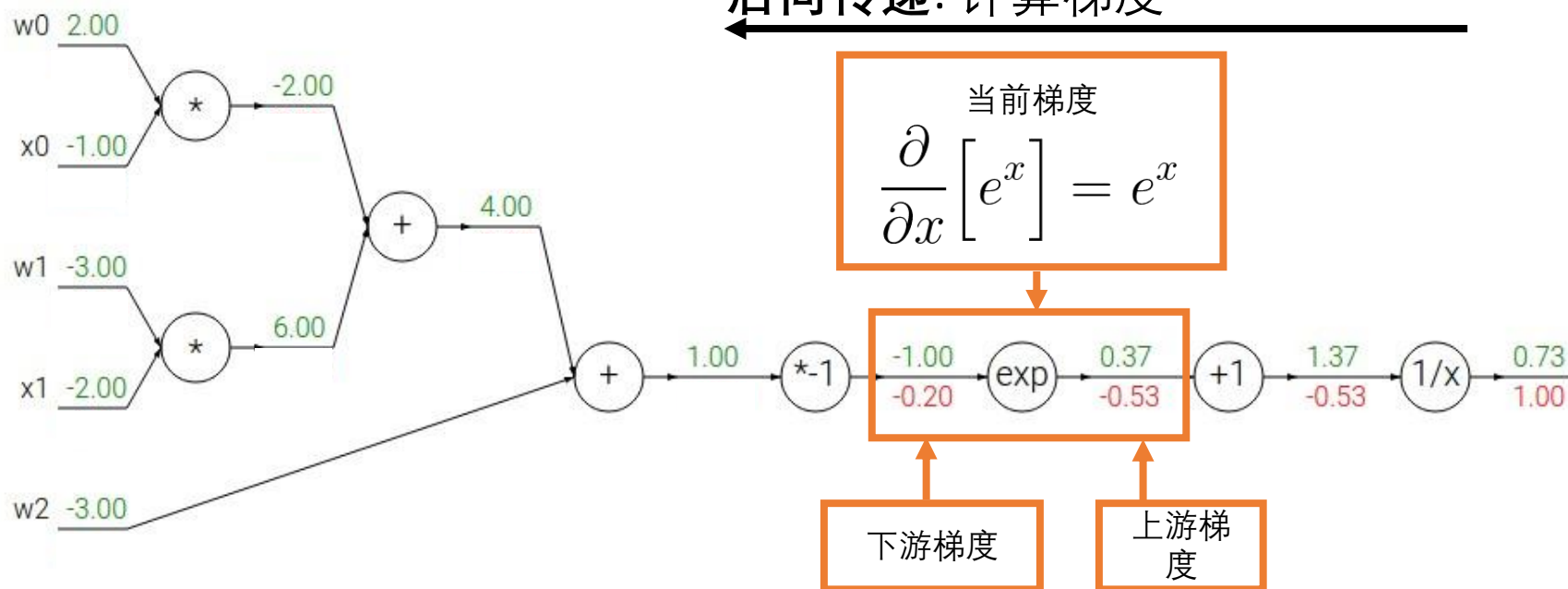
后向传递: 计算梯度



另一个示例

$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

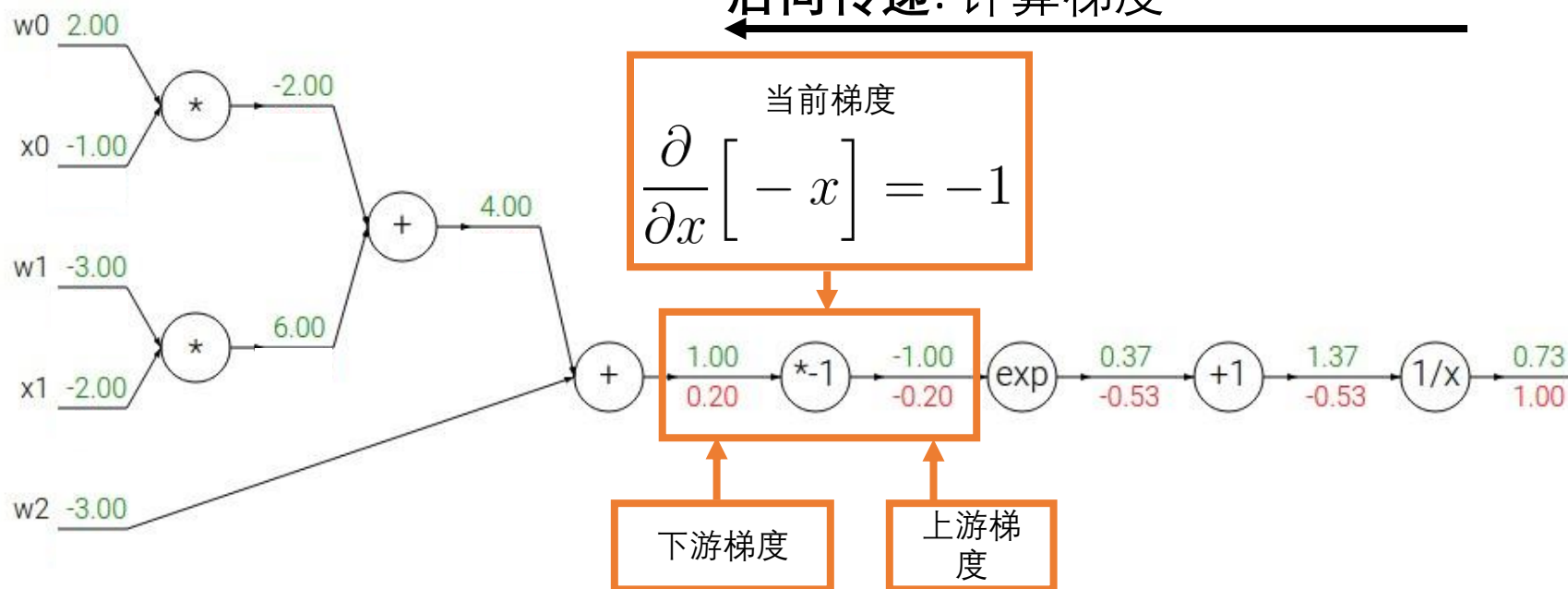
后向传递: 计算梯度



另一个示例

$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

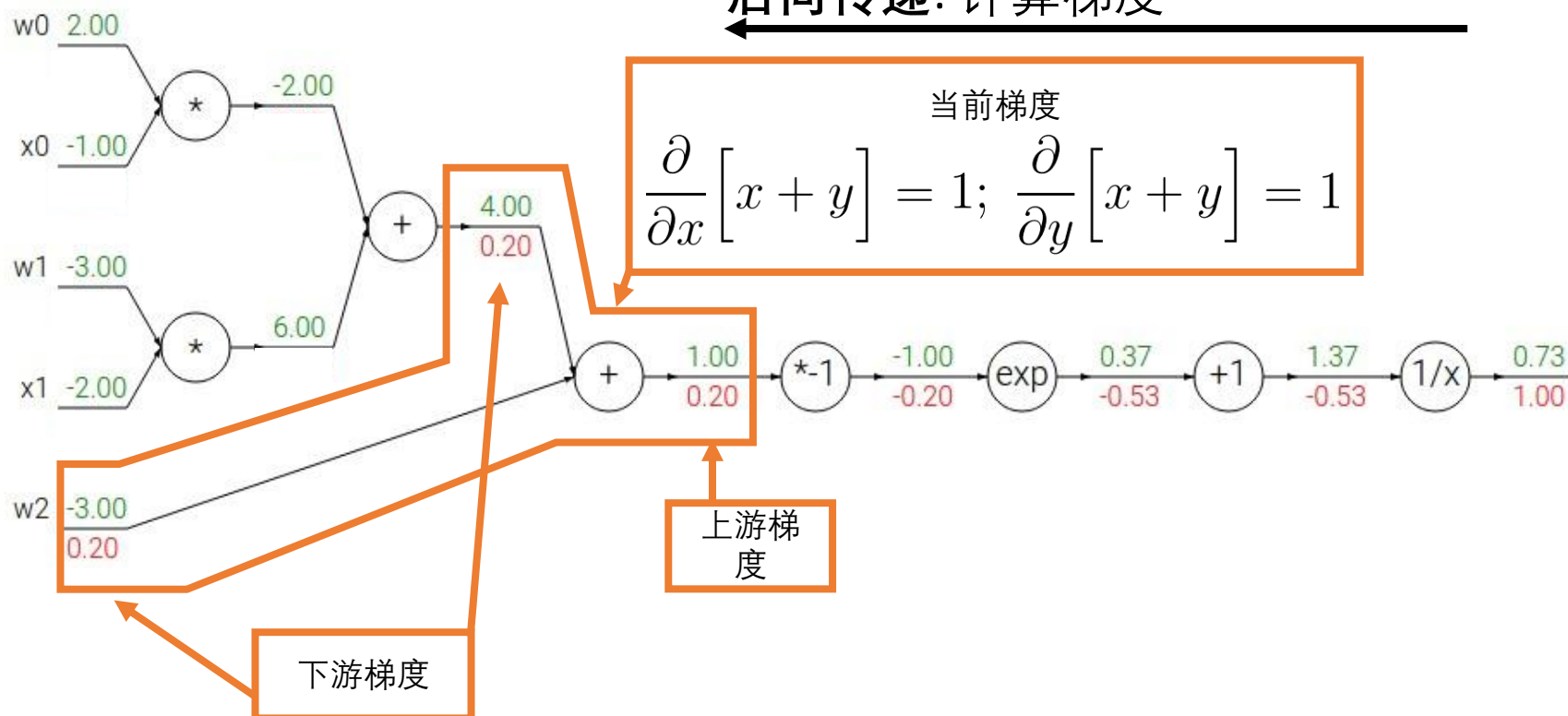
后向传递: 计算梯度



另一个示例

$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

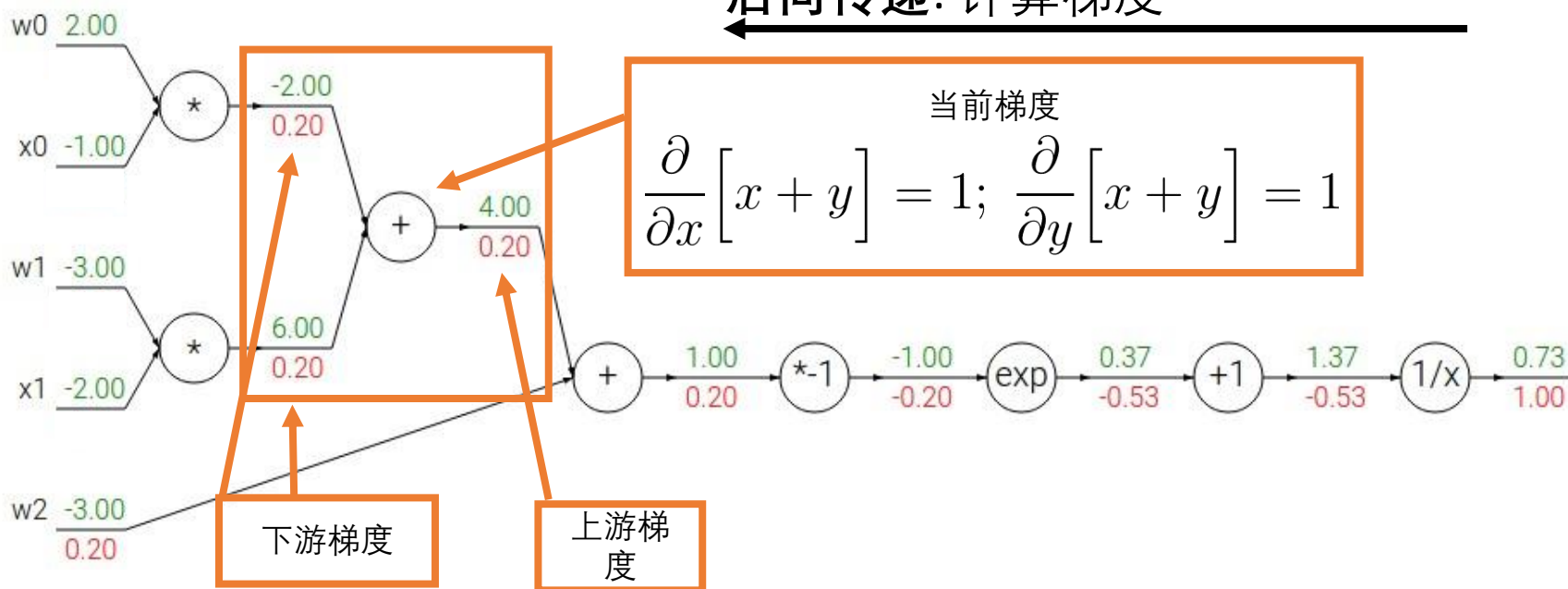
后向传递: 计算梯度



另一个示例

$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$

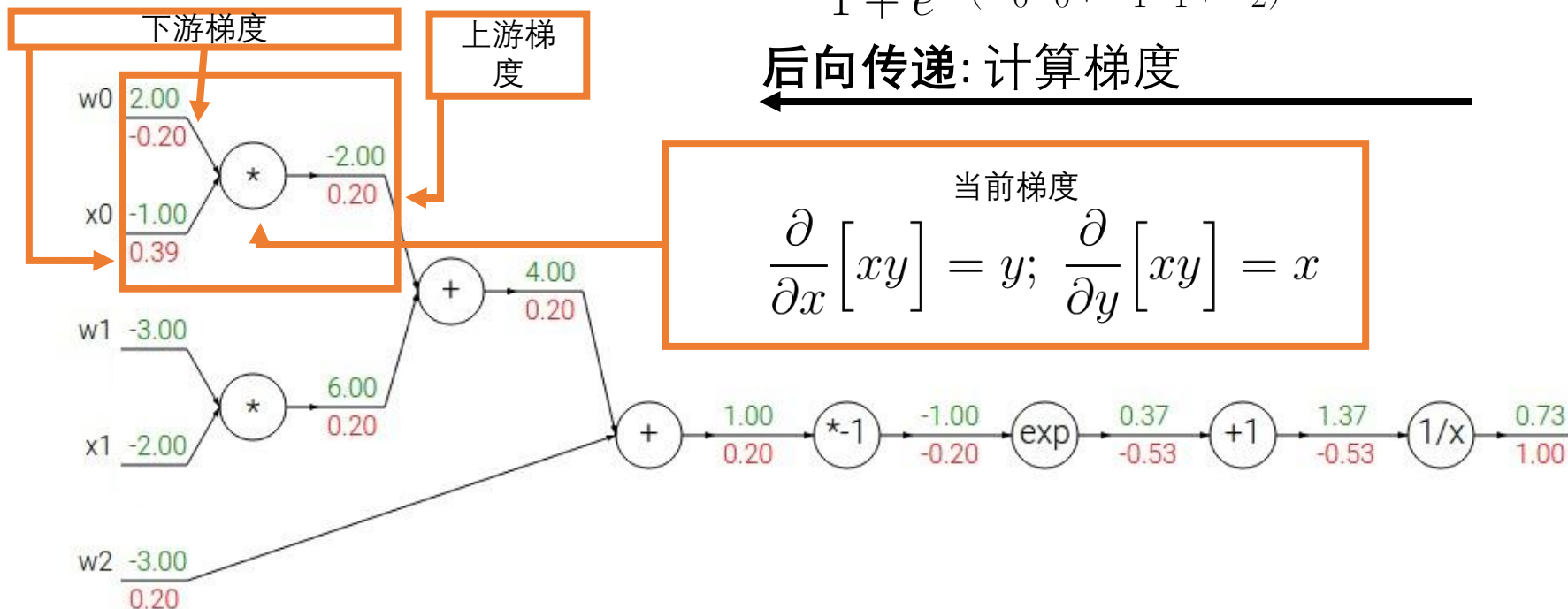
后向传递: 计算梯度



另一个示例

$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$

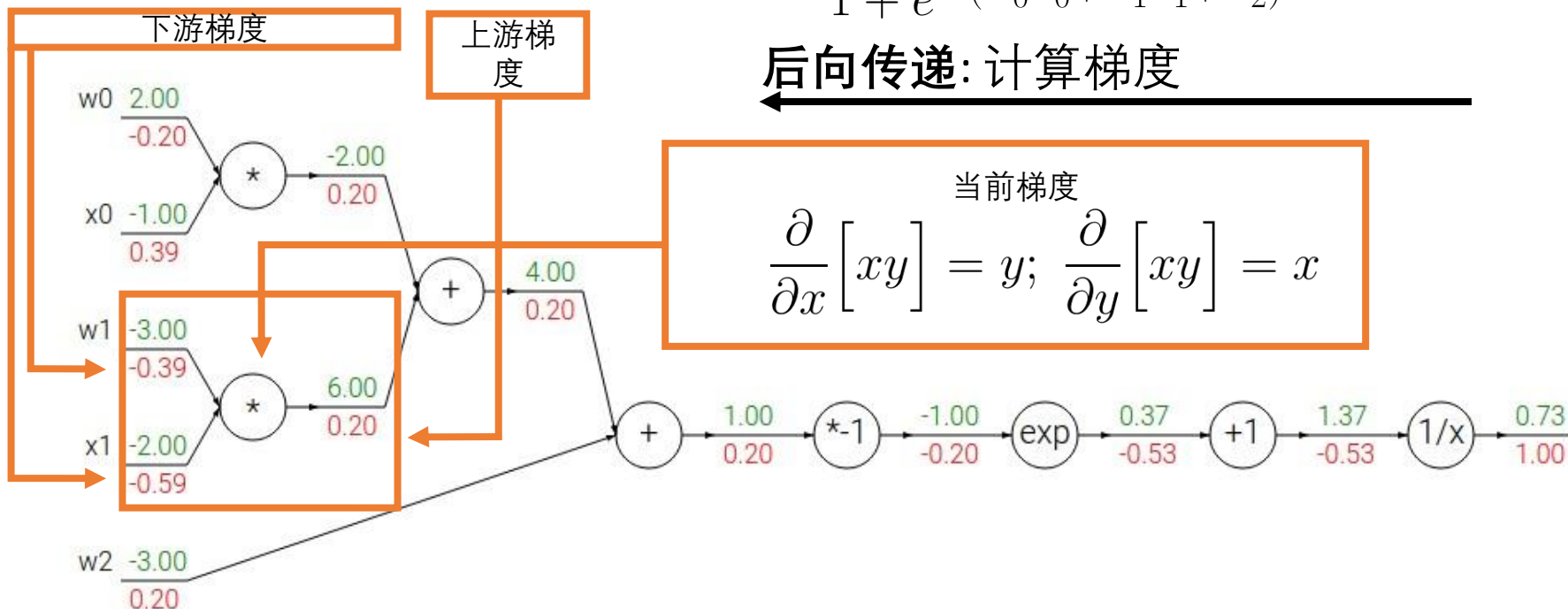
后向传递: 计算梯度



另一个示例

$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$

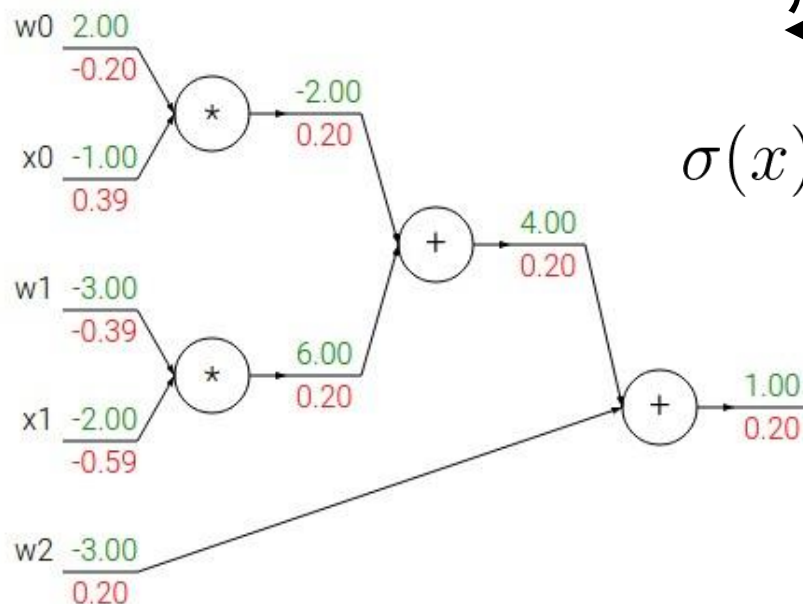
后向传递: 计算梯度



另一个示例

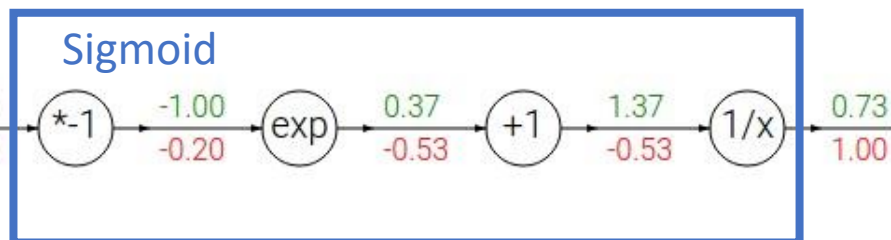
$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}} = \sigma(w_0x_0 + w_1x_1 + w_2)$$

后向传递: 计算梯度



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

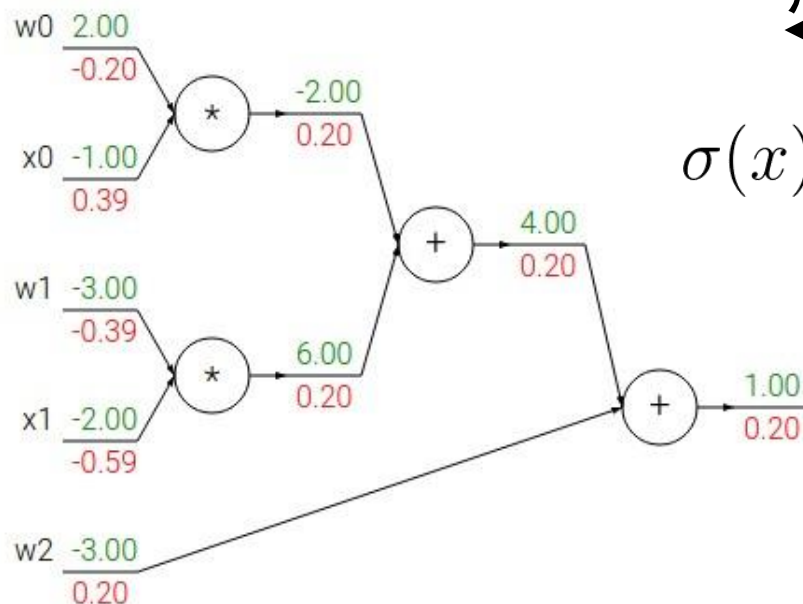
计算图不唯一: 可以使用简单梯度的基本单元



另一个示例

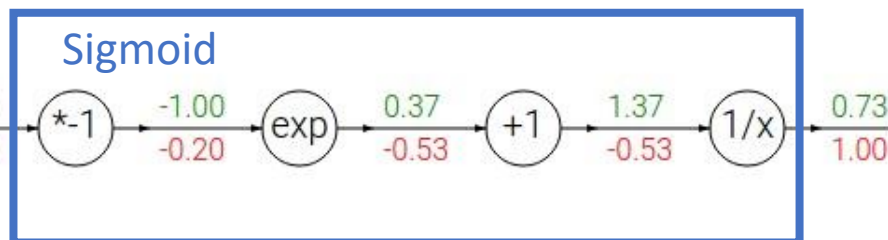
$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}} = \sigma(w_0x_0 + w_1x_1 + w_2)$$

后向传递: 计算梯度



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

计算图不唯一: 可以使用简单梯度的基本单元



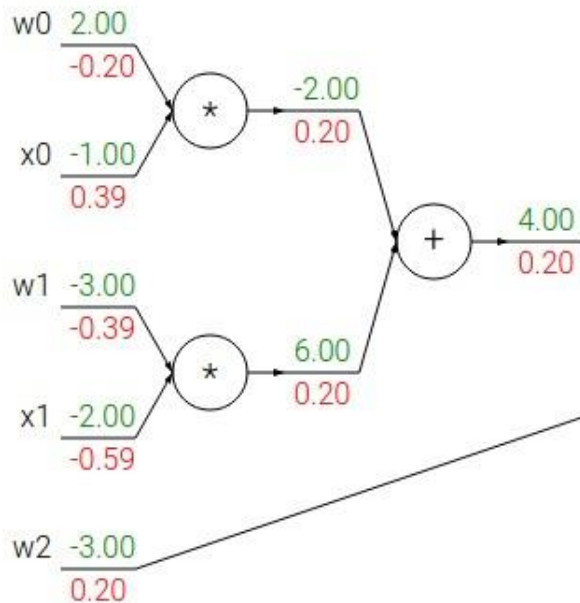
Sigmoid local gradient:

$$\frac{\partial}{\partial x} [\sigma(x)] = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x))\sigma(x)$$

另一个示例

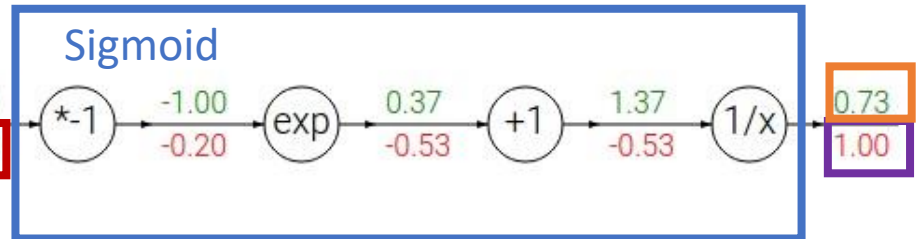
$$f(x, w) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}} = \sigma(w_0x_0 + w_1x_1 + w_2)$$

后向传递: 计算梯度



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

计算图不唯一: 可以使用简单梯度的基本单元



[下游] = [当前] * [上游]

$$= (1 - 0.73) * 0.73 * 1.0 = 0.2$$

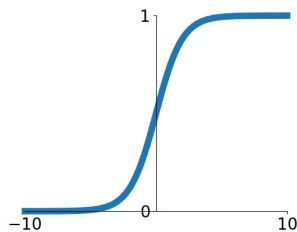
Sigmoid local gradient:

$$\frac{\partial}{\partial x} [\sigma(x)] = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x))\sigma(x)$$

其他激活函数的梯度?

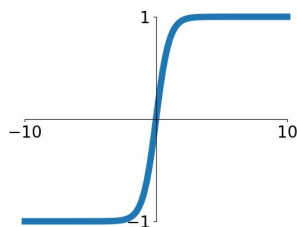
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



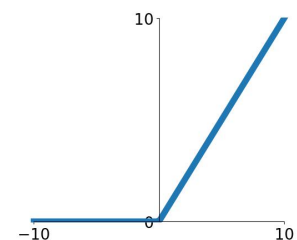
tanh

$$\tanh(x)$$



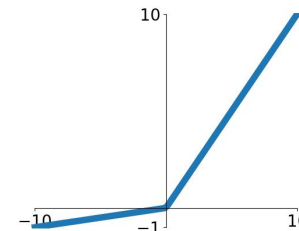
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

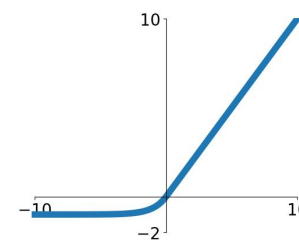


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

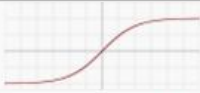
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



其他激活函数的梯度?

Sigmoid
当前梯度: $\frac{\partial}{\partial x} [\sigma(x)] = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x))\sigma(x)$

Name	Plot	Equation	Derivative
Tanh		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parameteric Rectified Linear Unit (PReLU) [2]		$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$

<https://towardsdatascience.com/activation-functions-neural-networks-1cbd9f8d91d6>

目前为止: 全是有关标量的后向传播

如果是矢量值函数会怎样?

回顾: 矢量导数

$$x \in \mathbb{R}, y \in \mathbb{R}$$

常规导数:

$$\frac{\partial y}{\partial x} \in \mathbb{R}$$

如果 x 变化一个微小
变量, y 如何改变?

回顾: 矢量导数

$$x \in \mathbb{R}, y \in \mathbb{R}$$

常规导数:

$$\frac{\partial y}{\partial x} \in \mathbb{R}$$

如果 x 变化一个微小变量, y 如何改变?

$$x \in \mathbb{R}^N, y \in \mathbb{R}$$

导数为梯度:

$$\frac{\partial y}{\partial x} \in \mathbb{R}^N \quad \left(\frac{\partial y}{\partial x} \right)_n = \frac{\partial y}{\partial x_n}$$

对于每一个 x , 如果其变化一个微小变量, y 如何变化?

回顾: 矢量导数

$$x \in \mathbb{R}, y \in \mathbb{R}$$

常规导数:

$$\frac{\partial y}{\partial x} \in \mathbb{R}$$

如果 x 变化一个微小变量, y 如何改变?

$$x \in \mathbb{R}^N, y \in \mathbb{R}$$

导数为梯度:

$$\frac{\partial y}{\partial x} \in \mathbb{R}^N \quad \left(\frac{\partial y}{\partial x} \right)_n = \frac{\partial y}{\partial x_n}$$

对于 x 中的每一个元素, 如果它变化一个微小变量, y 如何变化?

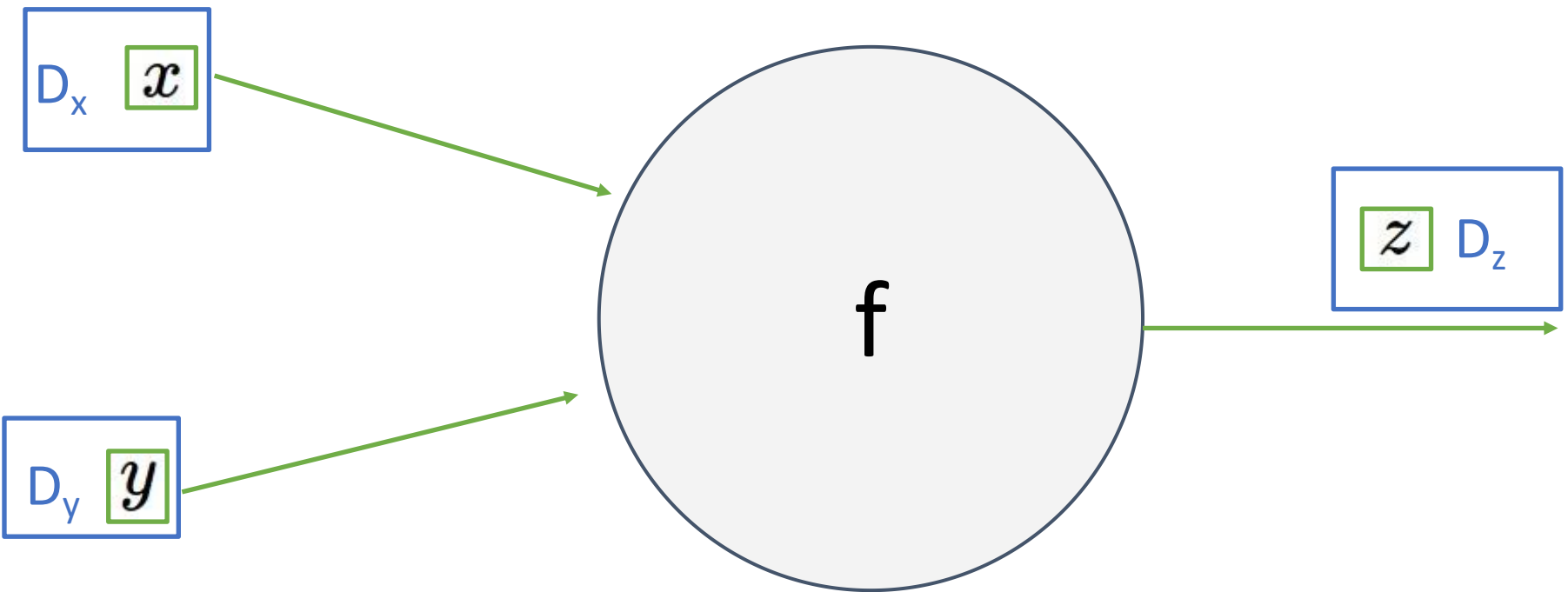
$$x \in \mathbb{R}^N, y \in \mathbb{R}^M$$

导数为 **Jacobian** 矩阵:

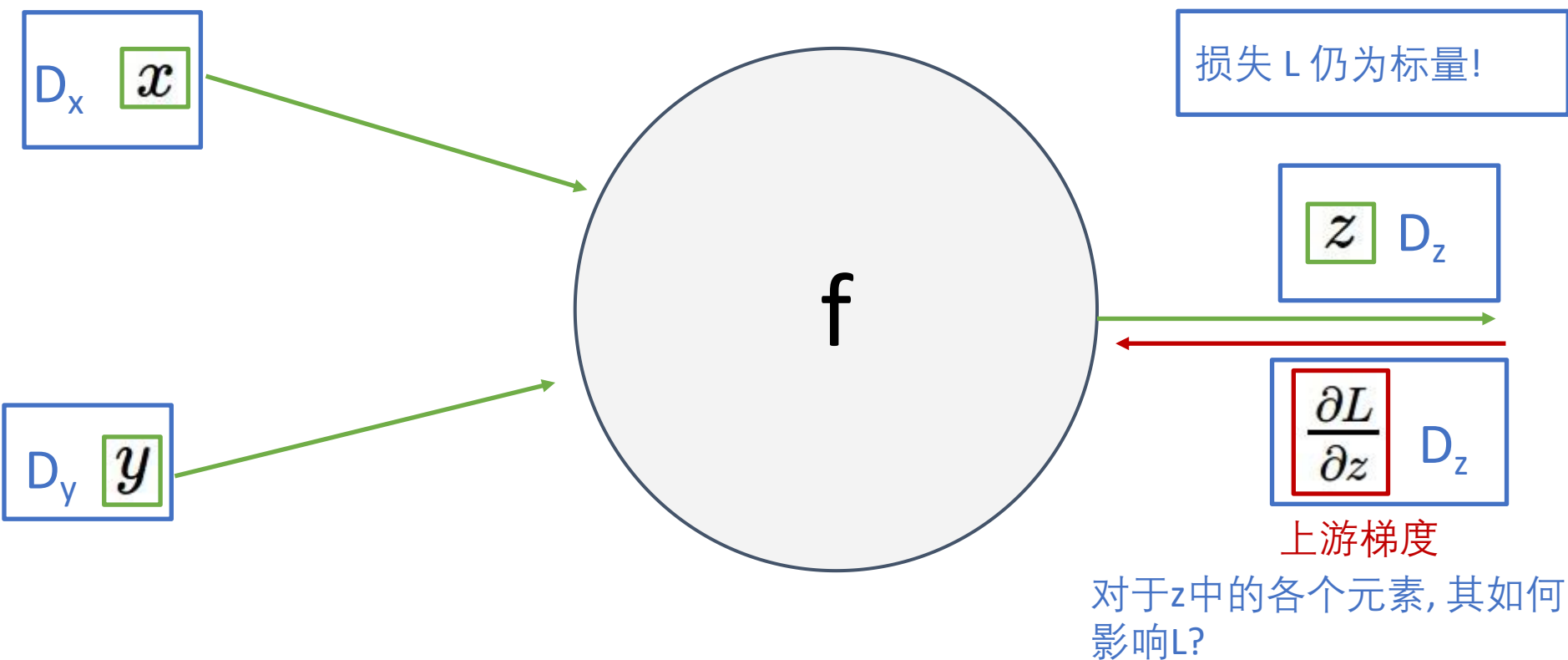
$$\frac{\partial y}{\partial x} \in \mathbb{R}^{N \times M} \quad \left(\frac{\partial y}{\partial x} \right)_{n,m} = \frac{\partial y_m}{\partial x_n}$$

对于 x 中的每一个元素, 如果其变化一个微小变量, y 中单个变量如何变化?

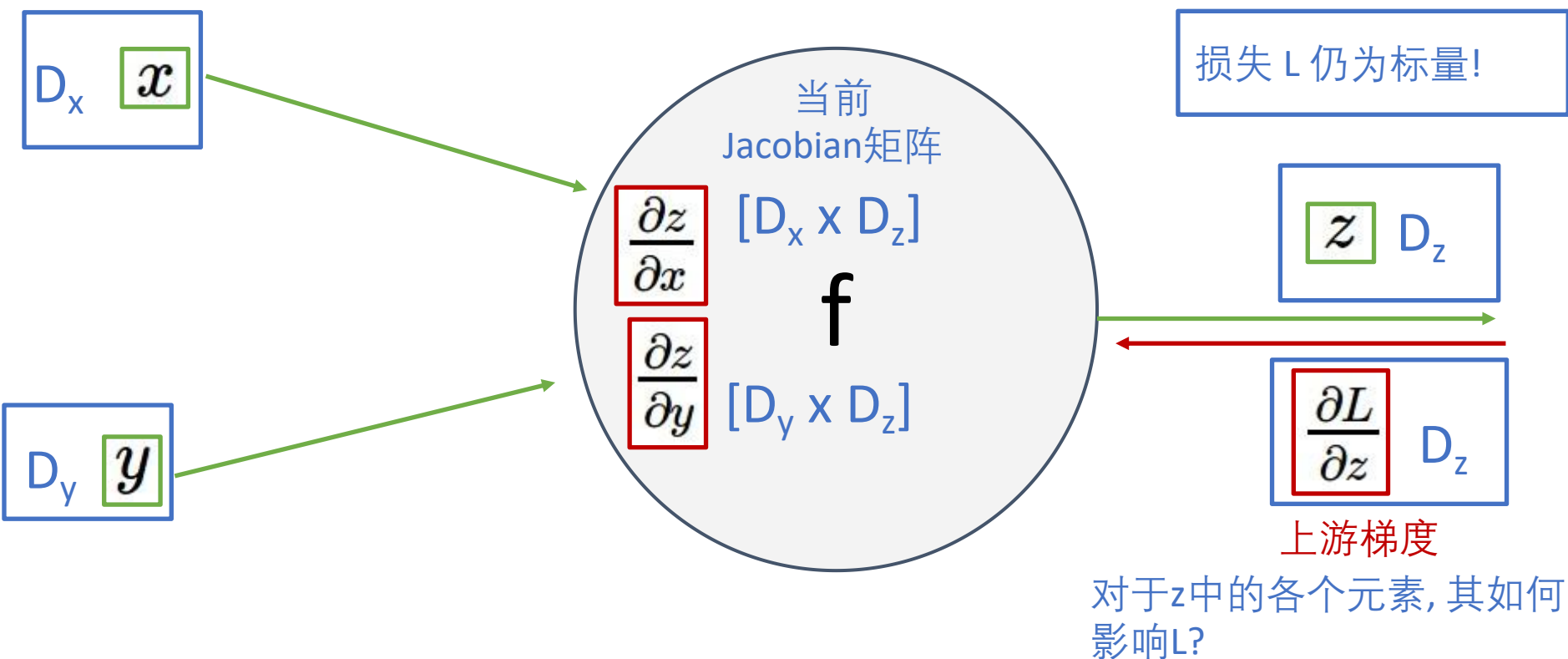
矢量函数后向传播



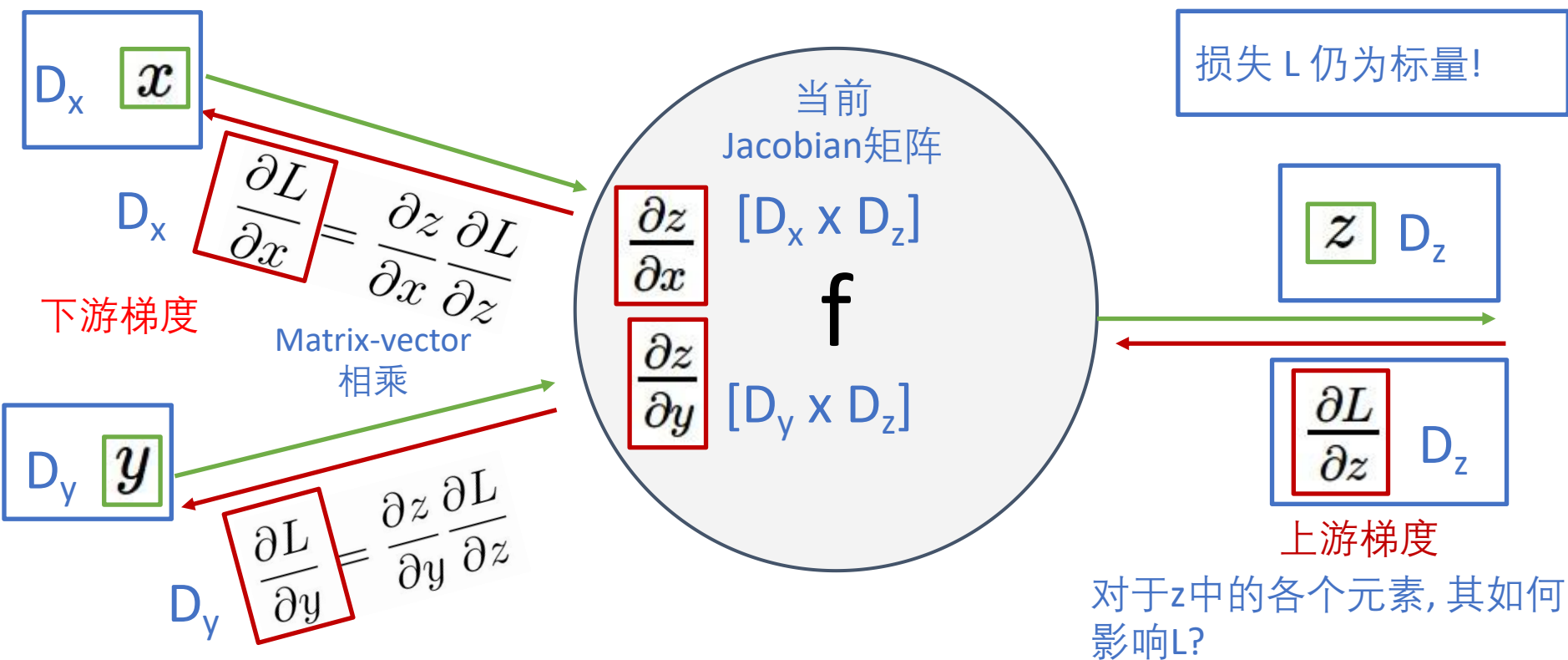
矢量函数后向传播



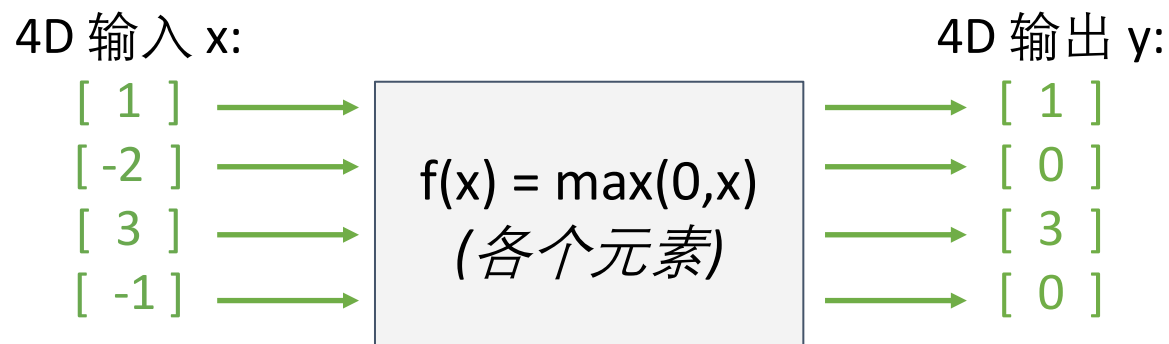
矢量函数后向传播



矢量函数后向传播



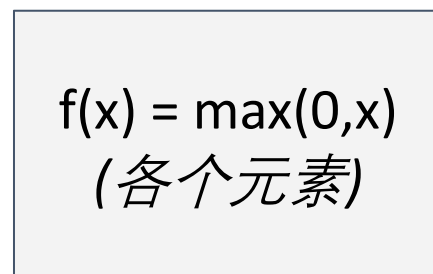
矢量函数后向传播



矢量函数后向传播

4D 输入 x:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$



4D 输出 y:

$\begin{bmatrix} 1 \\ 0 \\ 3 \\ 0 \end{bmatrix}$

4D dL/dy :

$\begin{bmatrix} 4 \\ -1 \\ 5 \\ 9 \end{bmatrix}$

上游梯度

矢量函数后向传播

4D 输入 x:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

$$f(x) = \max(0, x)$$

(各个元素)

4D 输出 y:

$\begin{bmatrix} 1 \\ 0 \\ 3 \\ 0 \end{bmatrix}$

Jacobian dy/dx

$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$

4D dL/dy :

$\begin{bmatrix} 4 \\ -1 \\ 5 \\ 9 \end{bmatrix}$

上游梯度

矢量函数后向传播

4D 输入 x:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

$$f(x) = \max(0, x)$$

(各个元素)

4D 输出 y:

$\begin{bmatrix} 1 \\ 0 \\ 3 \\ 0 \end{bmatrix}$

$[dy/dx] [dL/dy]$

$\begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 4 \end{bmatrix}$

$\begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} -1 \end{bmatrix}$

$\begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 5 \end{bmatrix}$

$\begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 9 \end{bmatrix}$

4D dL/dy:

$\begin{bmatrix} 4 \end{bmatrix}$

$\begin{bmatrix} -1 \end{bmatrix}$

$\begin{bmatrix} 5 \end{bmatrix}$

$\begin{bmatrix} 9 \end{bmatrix}$

上游梯度

矢量函数后向传播

4D 输入 x:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

$$f(x) = \max(0, x)$$

(各个元素)

4D 输出 y:

$\begin{bmatrix} 1 \\ 0 \\ 3 \\ 0 \end{bmatrix}$

4D dL/dx:

$\begin{bmatrix} 4 \\ 0 \\ 5 \\ 0 \end{bmatrix}$

$\begin{bmatrix} dy/dx & dL/dy \end{bmatrix}$

$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 4 \\ -1 \\ 5 \\ 9 \end{bmatrix}$

4D dL/dy:

$\begin{bmatrix} 4 \\ -1 \\ 5 \\ 9 \end{bmatrix}$

上游梯度

矢量函数后向传播

Jacobian是稀疏的: 非对角线元素都为0!
不显示计算矩阵成分; 计算单个元素相乘

4D 输入 x:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

$$f(x) = \max(0, x)$$

(各个元素)

4D 输出 y:

$\begin{bmatrix} 1 \\ 0 \\ 3 \\ 0 \end{bmatrix}$

4D dL/dx:

$\begin{bmatrix} 4 \\ 0 \\ 5 \\ 0 \end{bmatrix}$

$\begin{bmatrix} dy/dx & dL/dy \end{bmatrix}$

$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 4 \\ -1 \\ 5 \\ 9 \end{bmatrix}$

4D dL/dy:

$\begin{bmatrix} 4 \\ -1 \\ 5 \\ 9 \end{bmatrix}$

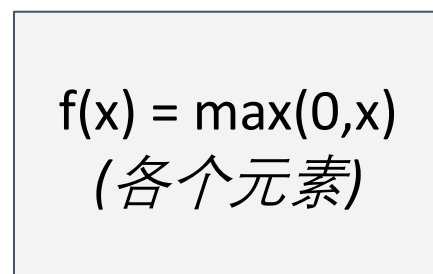
上游梯度

矢量函数后向传播

Jacobian是稀疏的: 非对角线元素都为0!
不显示计算矩阵成分; 计算单个元素相乘

4D 输入 x:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$



4D 输出 y:

$\begin{bmatrix} 1 \\ 0 \\ 3 \\ 0 \end{bmatrix}$

4D dL/dx:

$\begin{bmatrix} 4 \\ 0 \\ 5 \\ 0 \end{bmatrix}$

$[dy/dx] [dL/dy]$

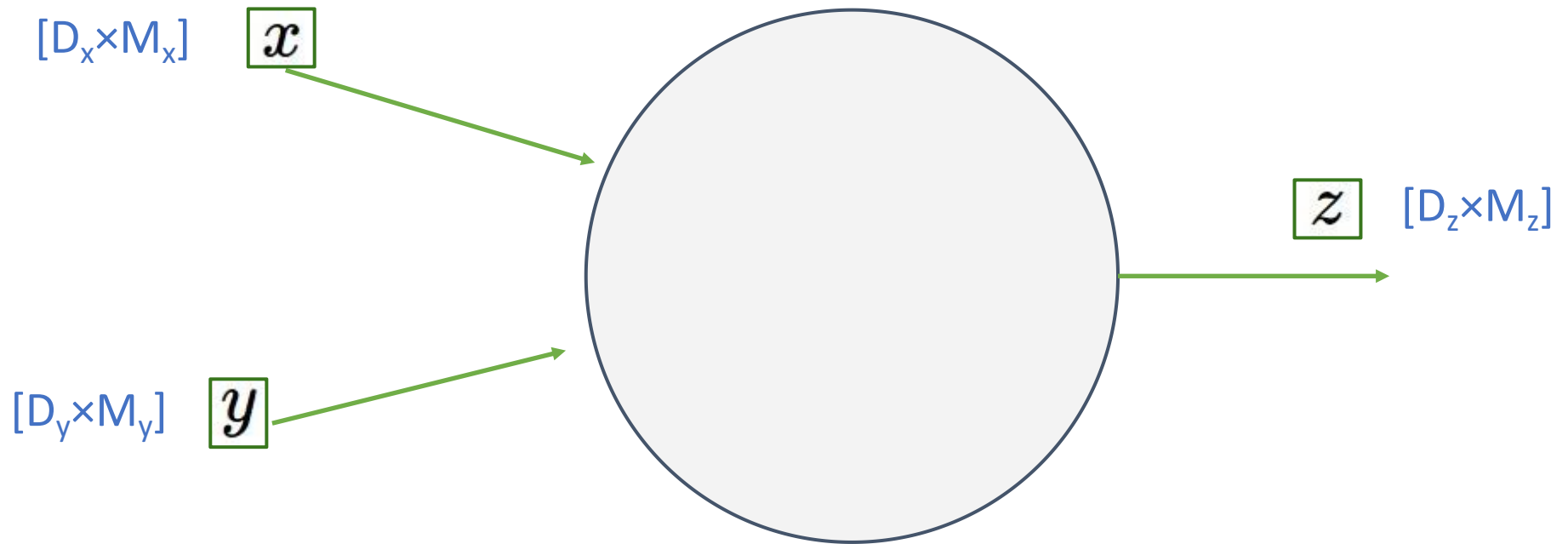
$$\left(\frac{\partial L}{\partial x}\right)_i = \begin{cases} \left(\frac{\partial L}{\partial y}\right)_i & \text{if } x_i > 0 \\ 0 & \text{otherwise} \end{cases}$$

4D dL/dy:

$\begin{bmatrix} 4 \\ -1 \\ 5 \\ 9 \end{bmatrix}$

上游梯度

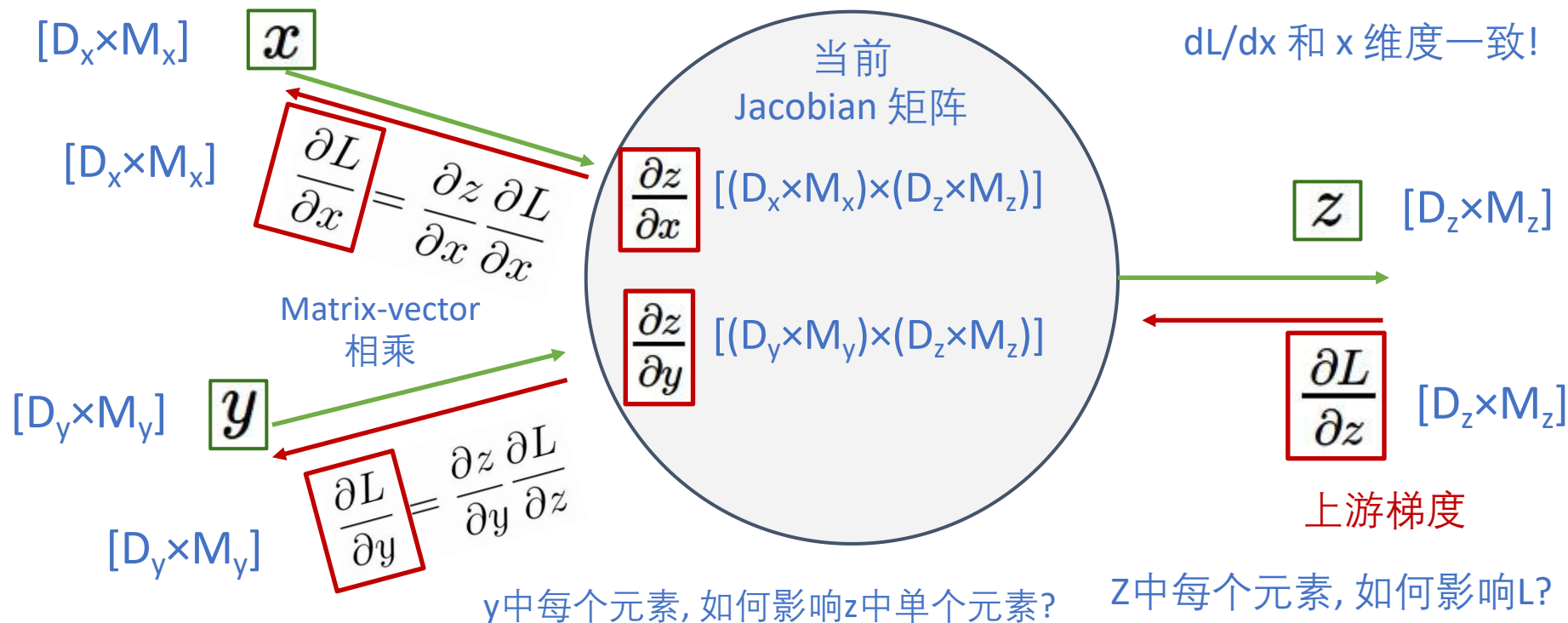
矩阵函数后向传播:



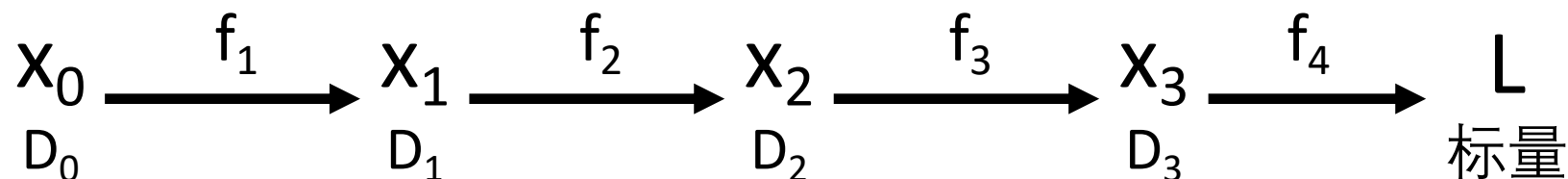
矩阵函数后向传播:

损失 L 仍为标量!

dL/dx 和 x 维度一致!



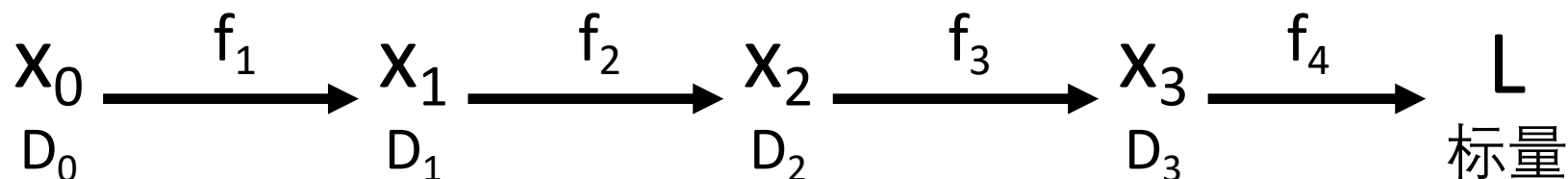
后向传播: 另一个角度



链式
法则

$$\frac{\partial L}{\partial x_0} = \left(\frac{\partial x_1}{\partial x_0} \right) \left(\frac{\partial x_2}{\partial x_1} \right) \left(\frac{\partial x_3}{\partial x_2} \right) \left(\frac{\partial L}{\partial x_3} \right)$$

后向传播: 另一个角度

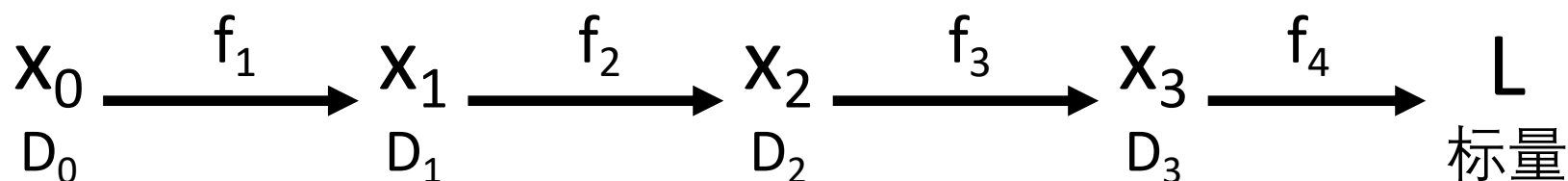


矩阵乘法具有结合性

链式法则

$$\frac{\partial L}{\partial x_0} = \underbrace{\left(\frac{\partial x_1}{\partial x_0} \right)}_{D_0 \times D_1} \underbrace{\left(\frac{\partial x_2}{\partial x_1} \right)}_{D_1 \times D_2} \underbrace{\left(\frac{\partial x_3}{\partial x_2} \right)}_{D_2 \times D_3} \underbrace{\left(\frac{\partial L}{\partial x_3} \right)}_{D_3}$$

微分逆向计算



矩阵-矩阵乘法从右至左依次计算

链式法则

$$\frac{\partial L}{\partial x_0} = \begin{pmatrix} \frac{\partial x_1}{\partial x_0} \\ \frac{\partial x_2}{\partial x_1} \end{pmatrix} \begin{pmatrix} \frac{\partial x_2}{\partial x_1} \\ \frac{\partial x_3}{\partial x_2} \end{pmatrix} \begin{pmatrix} \frac{\partial x_3}{\partial x_2} \\ \frac{\partial L}{\partial x_3} \end{pmatrix}$$

$D_0 \times D_1 \quad D_1 \times D_2 \quad D_2 \times D_3 \quad D_3$

A long horizontal arrow points from the rightmost term of the chain rule equation back towards the left, indicating the reverse direction of the backward pass.

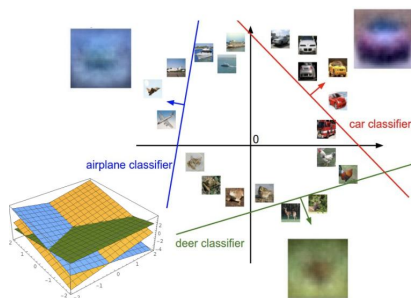
本章内容

- 6.1.神经网络
- 6.2.卷积神经网络
- 6.3.神经网络图像分类

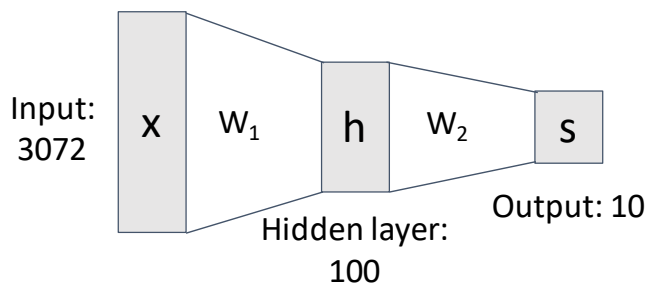
本章内容

- 6.2.1.卷积神经网络的组成
- 6.2.2.卷积层
- 6.2.3.池化与归一化
- 6.2.4.卷积神经网络的训练

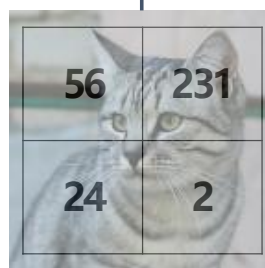
$$f(x, W) = Wx$$



$$f = W_2 \max(0, W_1 x)$$



Stretch pixels into column



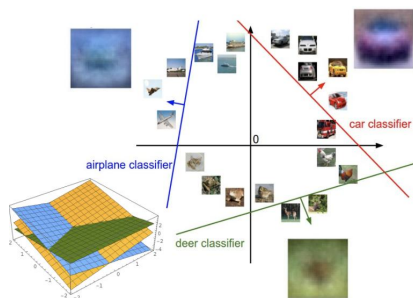
Input image
(2, 2)

问题: 目前为止分类器都未考虑图像的空域结构信息!

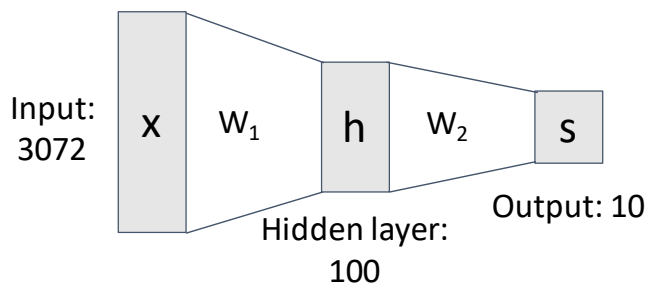
56
231
24
2

(4,)

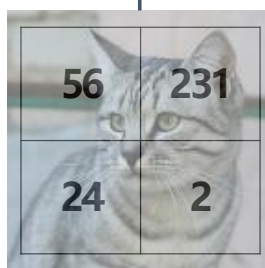
$$f(x, W) = Wx$$



$$f = W_2 \max(0, W_1 x)$$



Stretch pixels into column



Input image
(2, 2)

问题: 目前为止分类器都未考虑图像的空域结构信息!

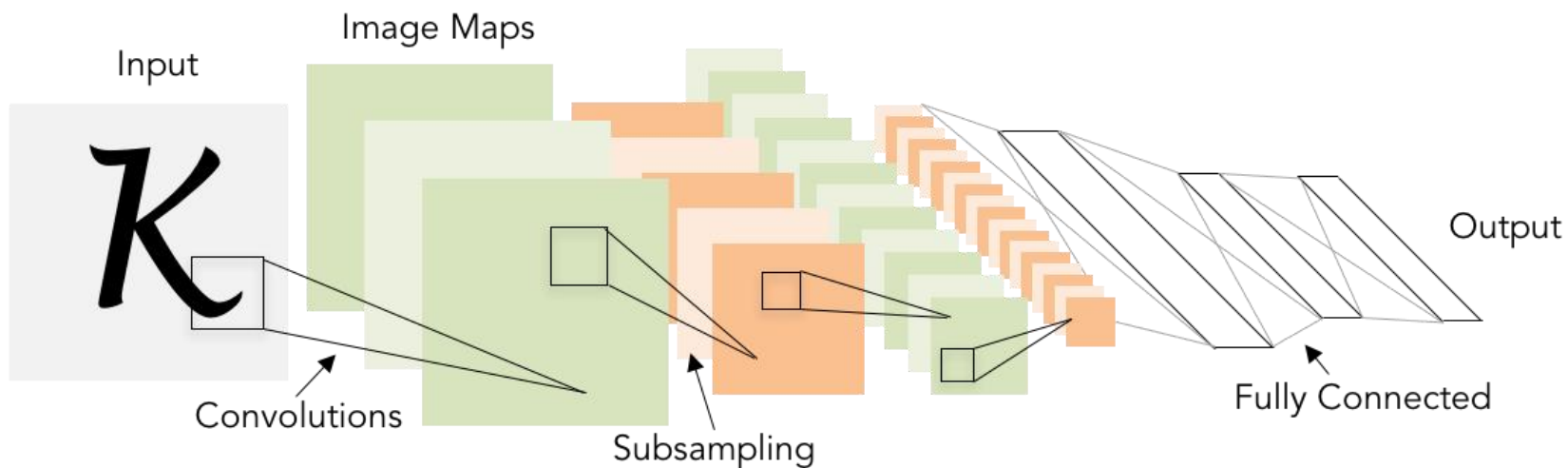
方案: 定义在图像局部区域处理的新计算节点!

56
231
24
2
(4,)

卷积网络LeNet-5

经典结构: [Conv, ReLU, Pool] x N, flatten, [FC, ReLU] x N, FC

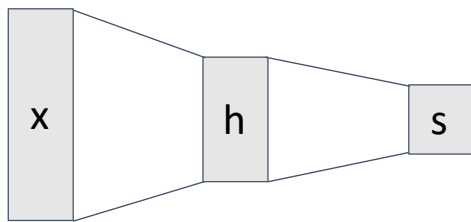
举例: LeNet-5



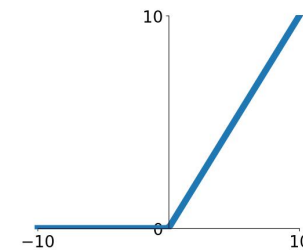
Lecun et al, "Gradient-based learning applied to document recognition", 1998

全连接网络的构成成分

全连接层

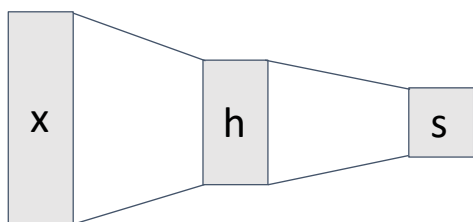


激活函数

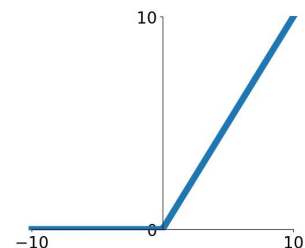


卷积神经网络的构成成分

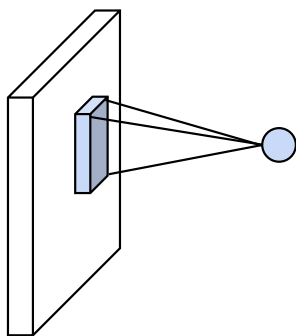
全连接层



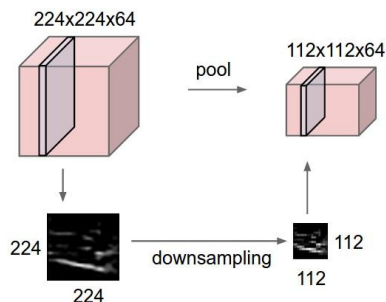
激活函数



卷积层



池化层



归一化

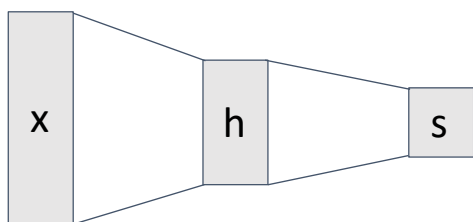
$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

本章内容

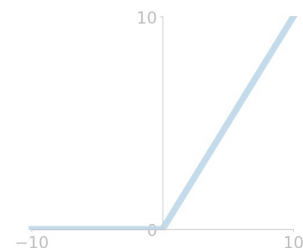
- 6.2.1.卷积神经网络的组成
- 6.2.2.卷积层
- 6.2.3.池化与归一化
- 6.2.4.卷积神经网络的训练

卷积网络的组成成分

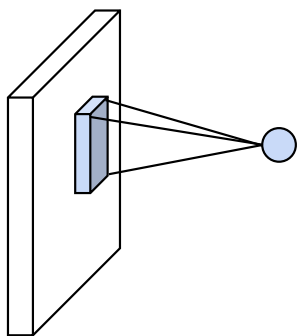
全连接层



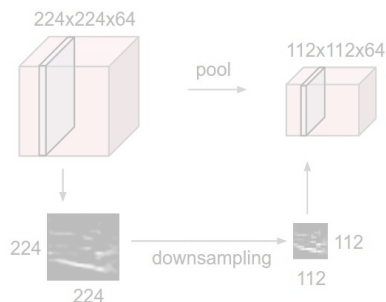
Activation Function



卷积层



Pooling Layers

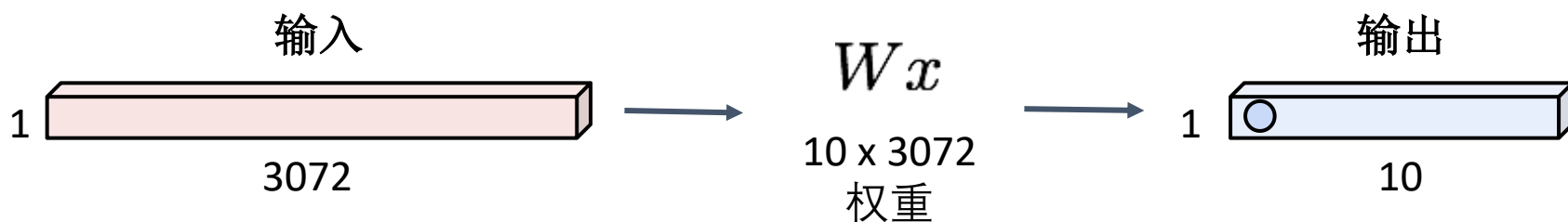


Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

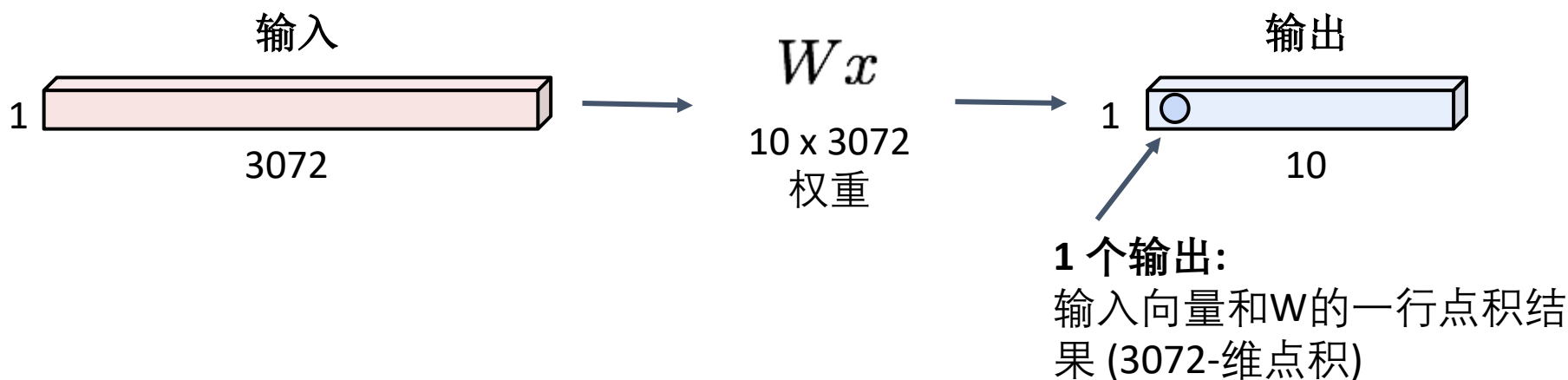
全连接层Fully-Connected Layer

32x32x3 图像 -> 向量化3072 x 1



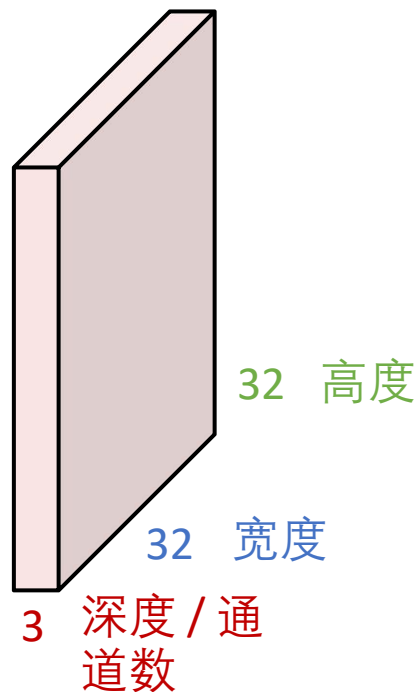
全连接层Fully-Connected Layer

32x32x3 图像 -> 向量化3072 x 1



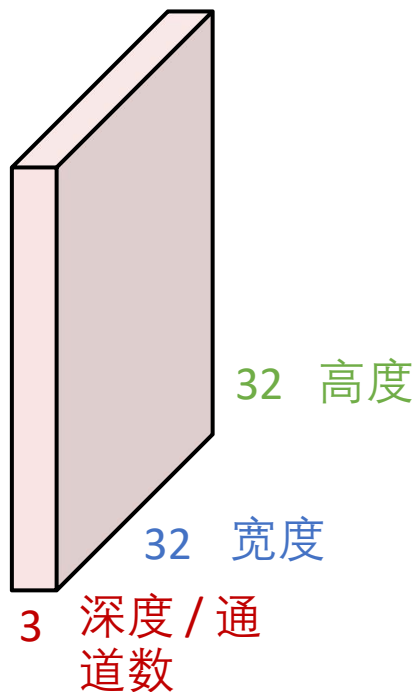
卷积层 Convolution Layer

3x32x32 图像: 保持空域结构



卷积层 Convolution Layer

3x32x32 图像

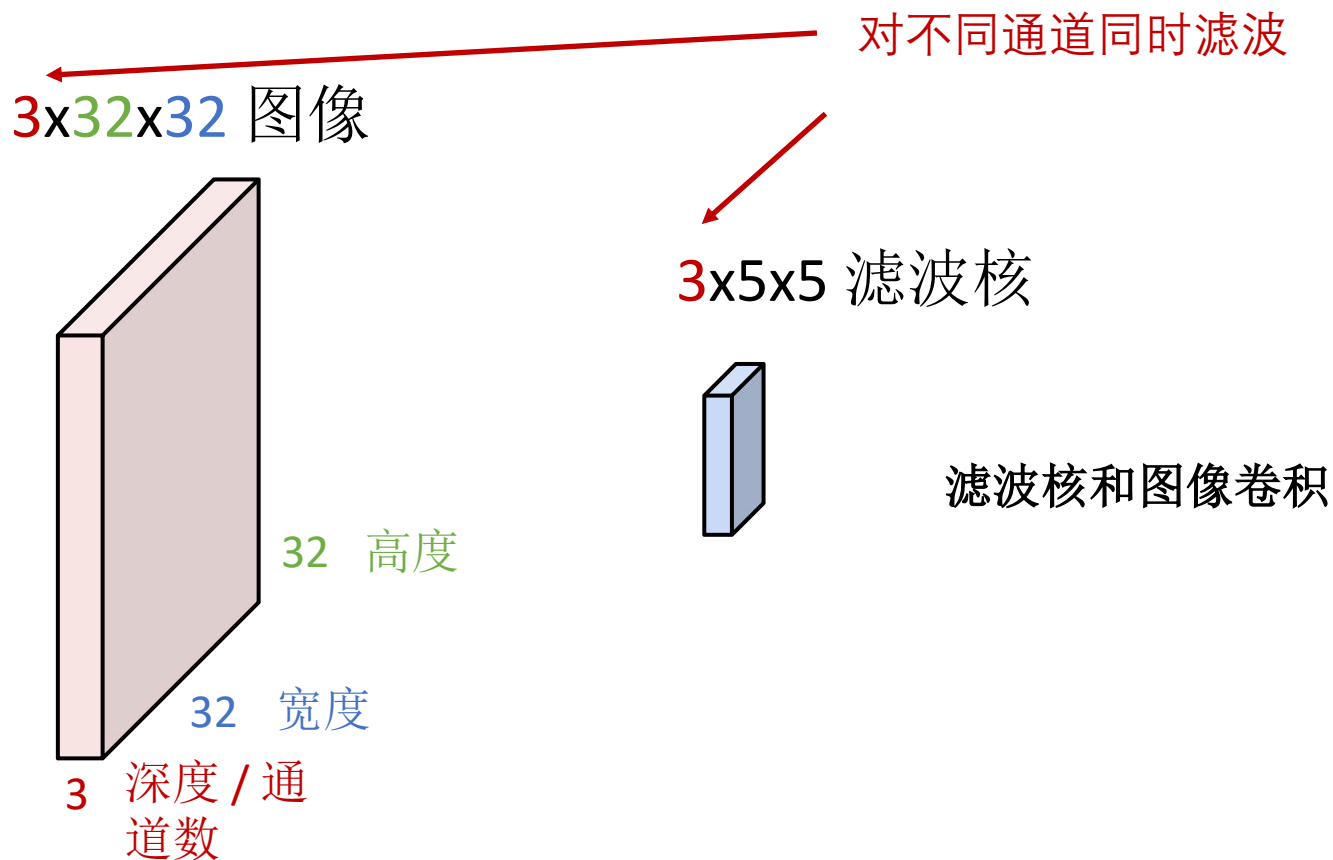


3x5x5 滤波核



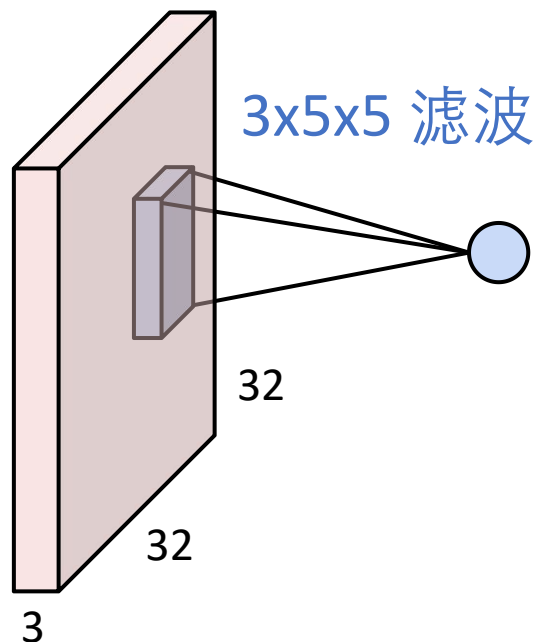
滤波核和图像卷积

卷积层Convolution Layer



卷积层 Convolution Layer

3x32x32 图像



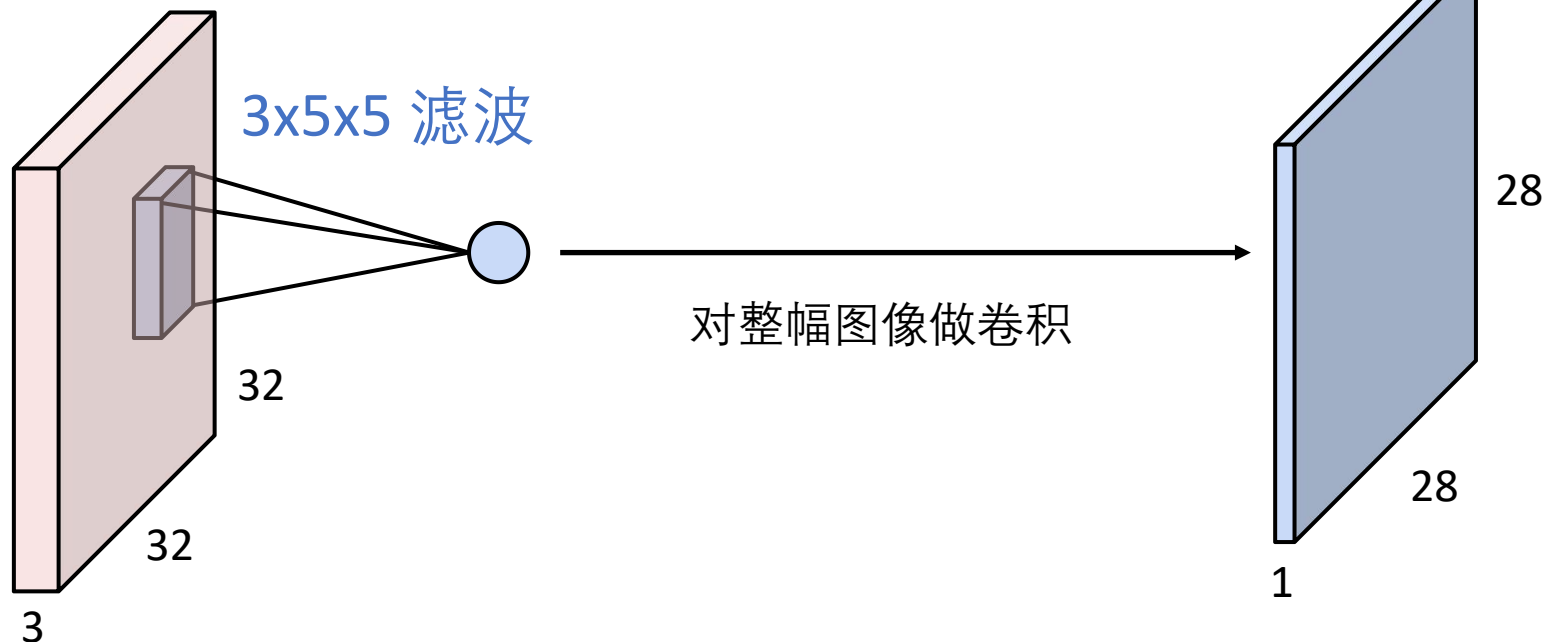
1 个输出:
输入向量和3x5x5滤波点积结果
(i.e. $3*5*5 = 75$ -维点积 + 偏置)

$$w^T x + b$$

卷积层 Convolution Layer

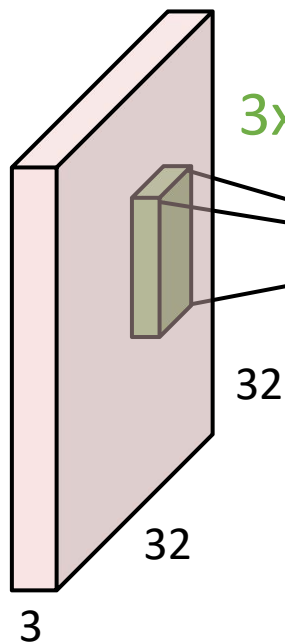
1x28x28 激活图

3x32x32 图像



卷积层Convolution Layer

3x32x32 图像



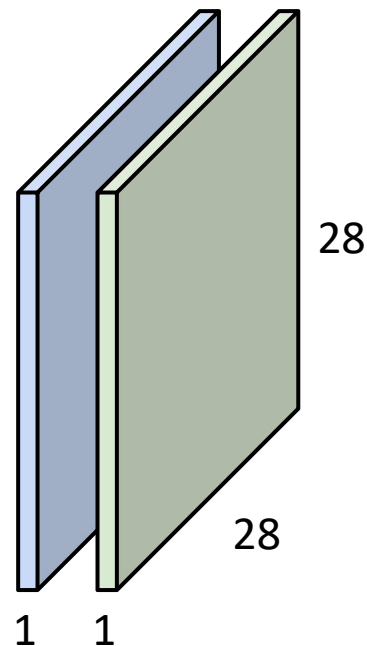
3x5x5 滤波

考虑第二个滤波核:



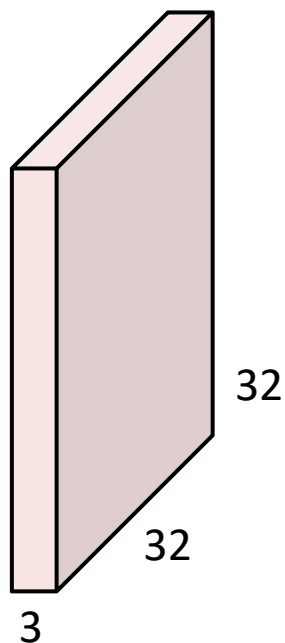
卷积

两个1x28x28
激活图



卷积层 Convolution Layer

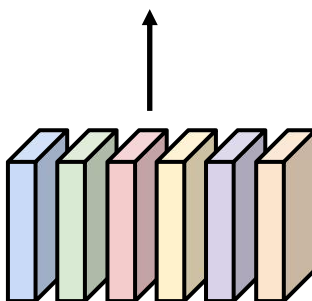
3x32x32 图像



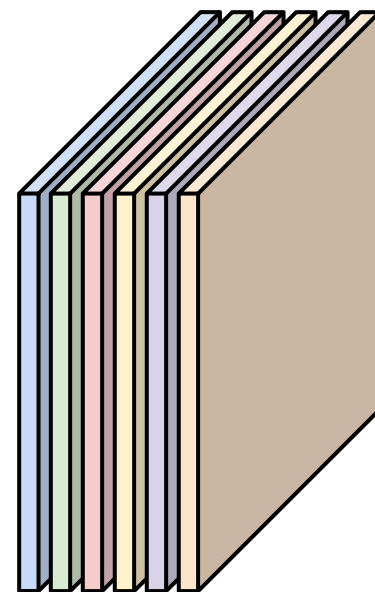
考虑 6 个滤波核,
每个 3x5x5

卷积层

6x3x5x5
滤波组



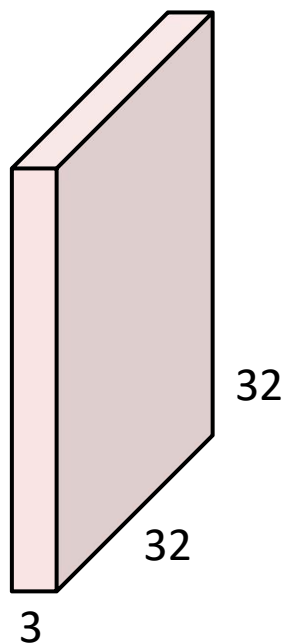
6 激活图,
每个 1x28x28



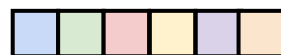
组合得到输出图像
6x28x28

卷积层 Convolution Layer

3x32x32 图像



6-dim 偏置向量:

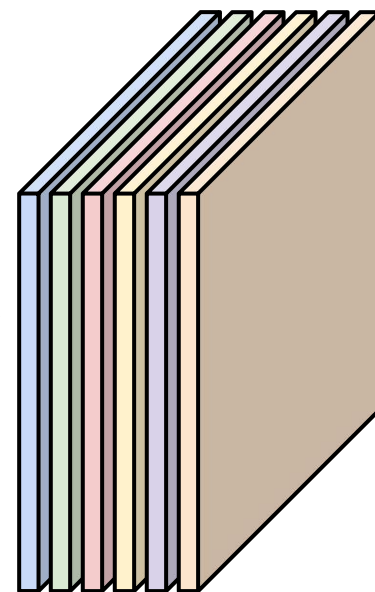


卷积层

6x3x5x5
滤波组



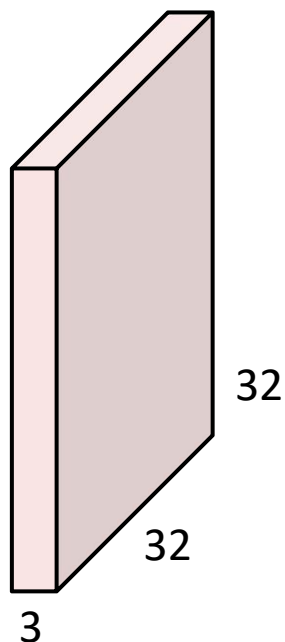
6 激活图,
每个 1x28x28



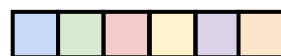
组合得到输出图像
6x28x28

卷积层Convolution Layer

3x32x32 图像



6-dim 偏置向量

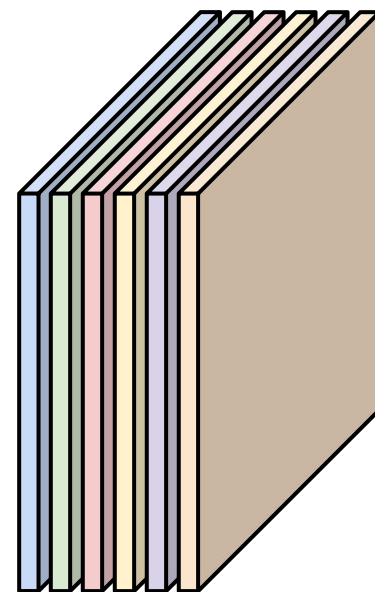


卷积层

6x3x5x5
滤波组

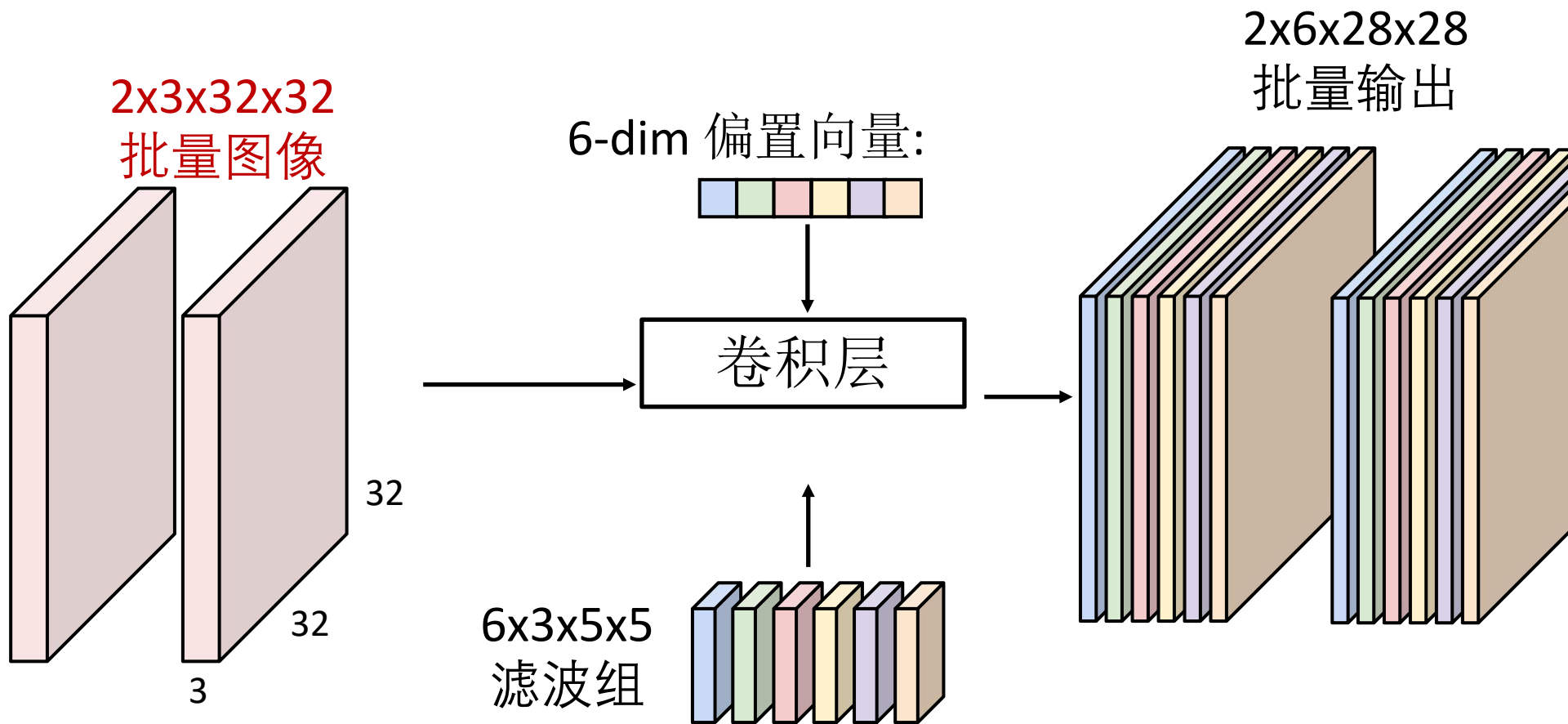


28x28 网格, 每一个
6-维向量

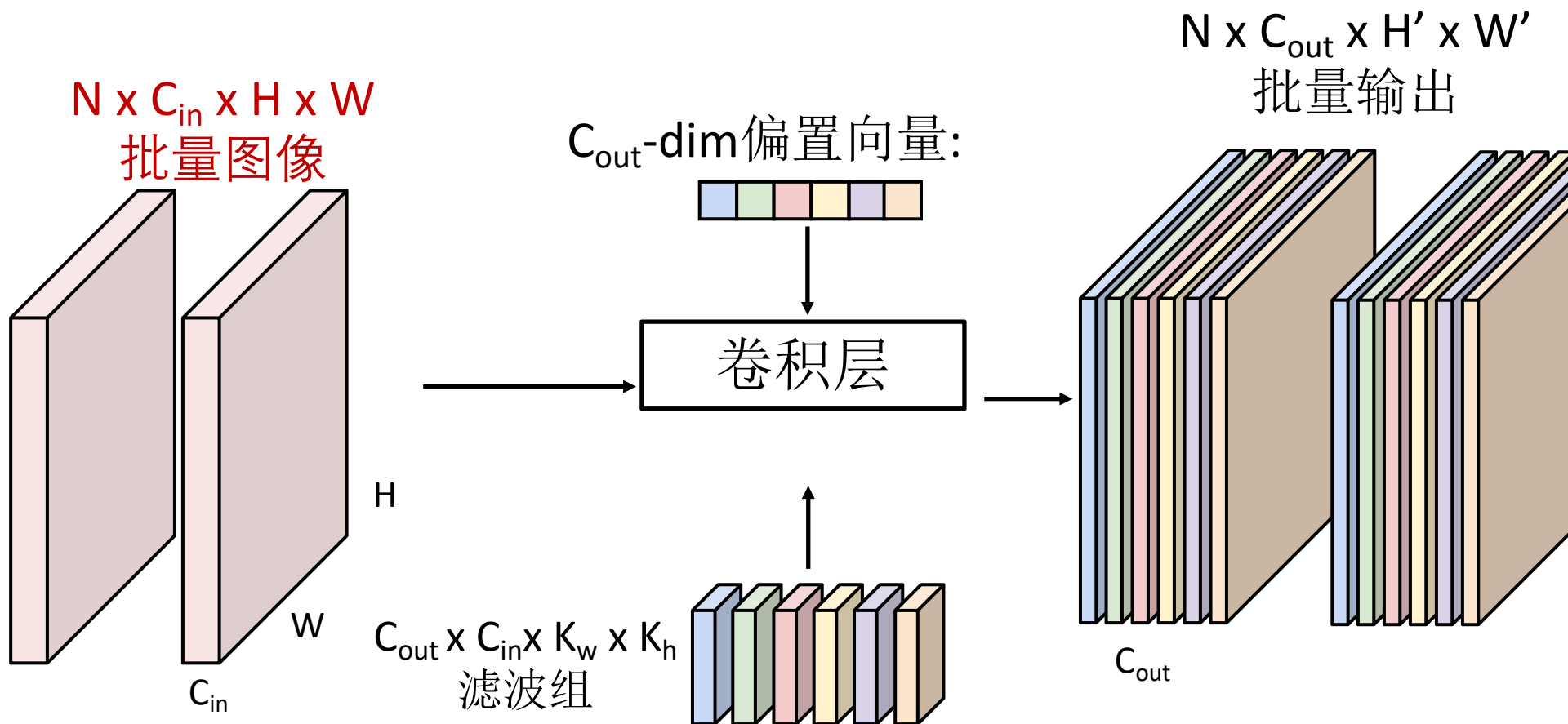


组合得到输出图像
6x28x28

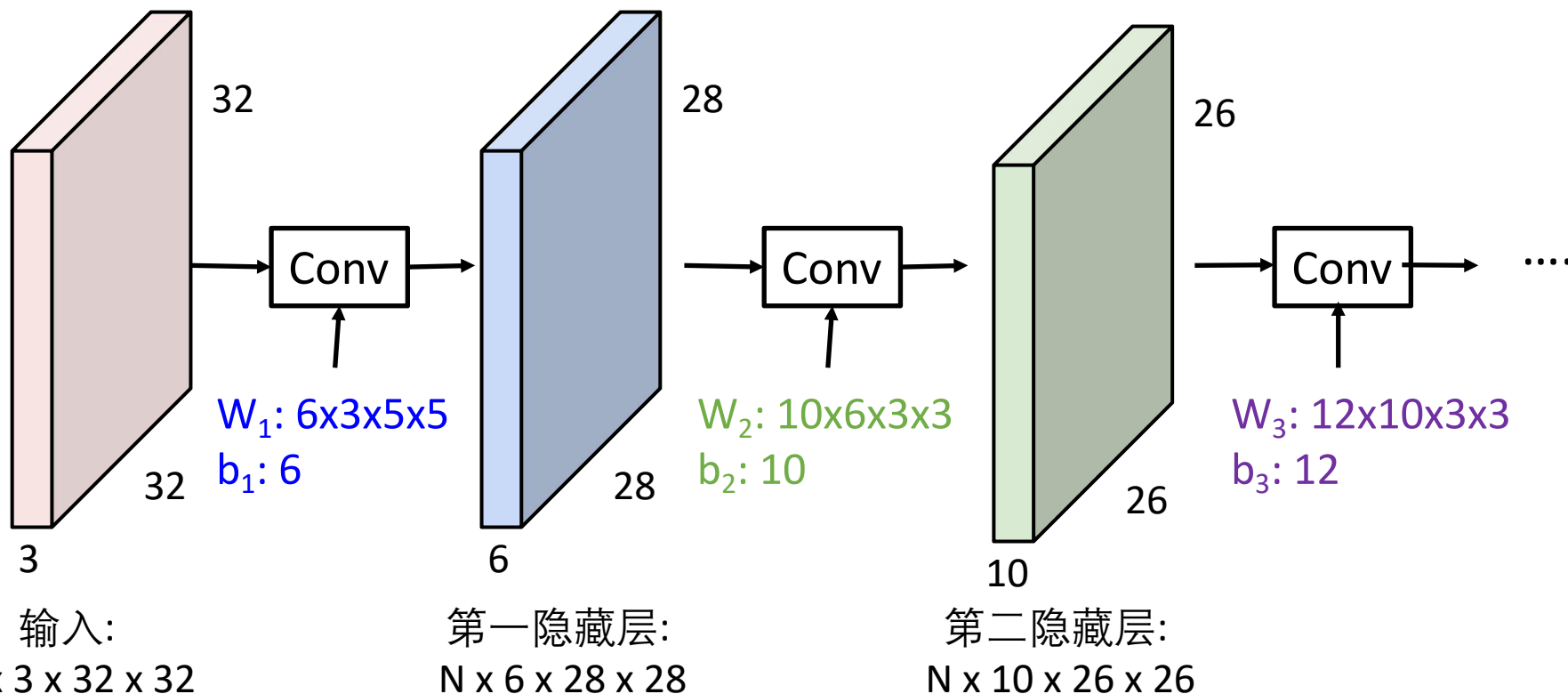
卷积层 Convolution Layer



卷积层 Convolution Layer



卷积层级联 Stacking Convolutions



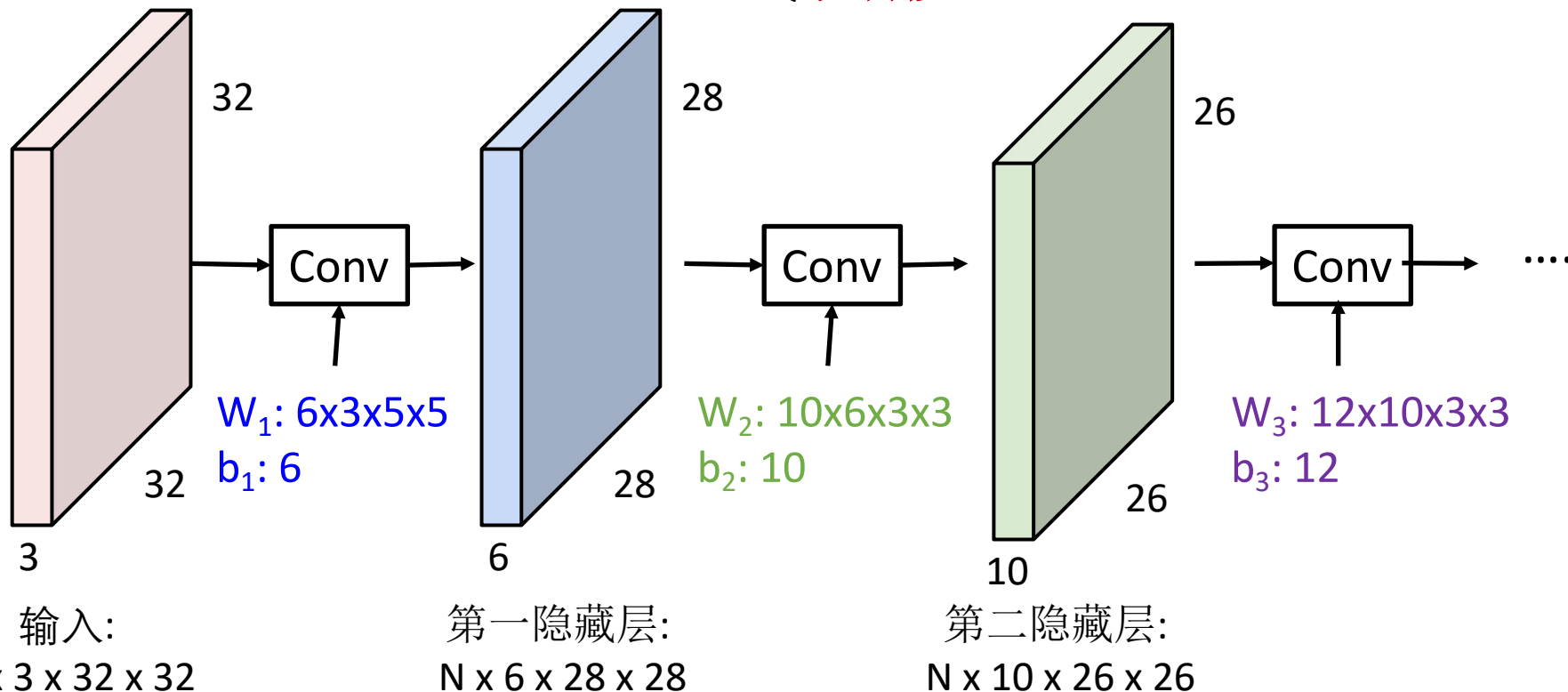
卷积层级联 Stacking Convolutions

Q: 多个卷积直接级联?

A: 一个卷积!

Q: 如何修正?

($y = W_2 W_1 x$ 等效为一个线性分类器)



卷积层级联Stacking Convolutions

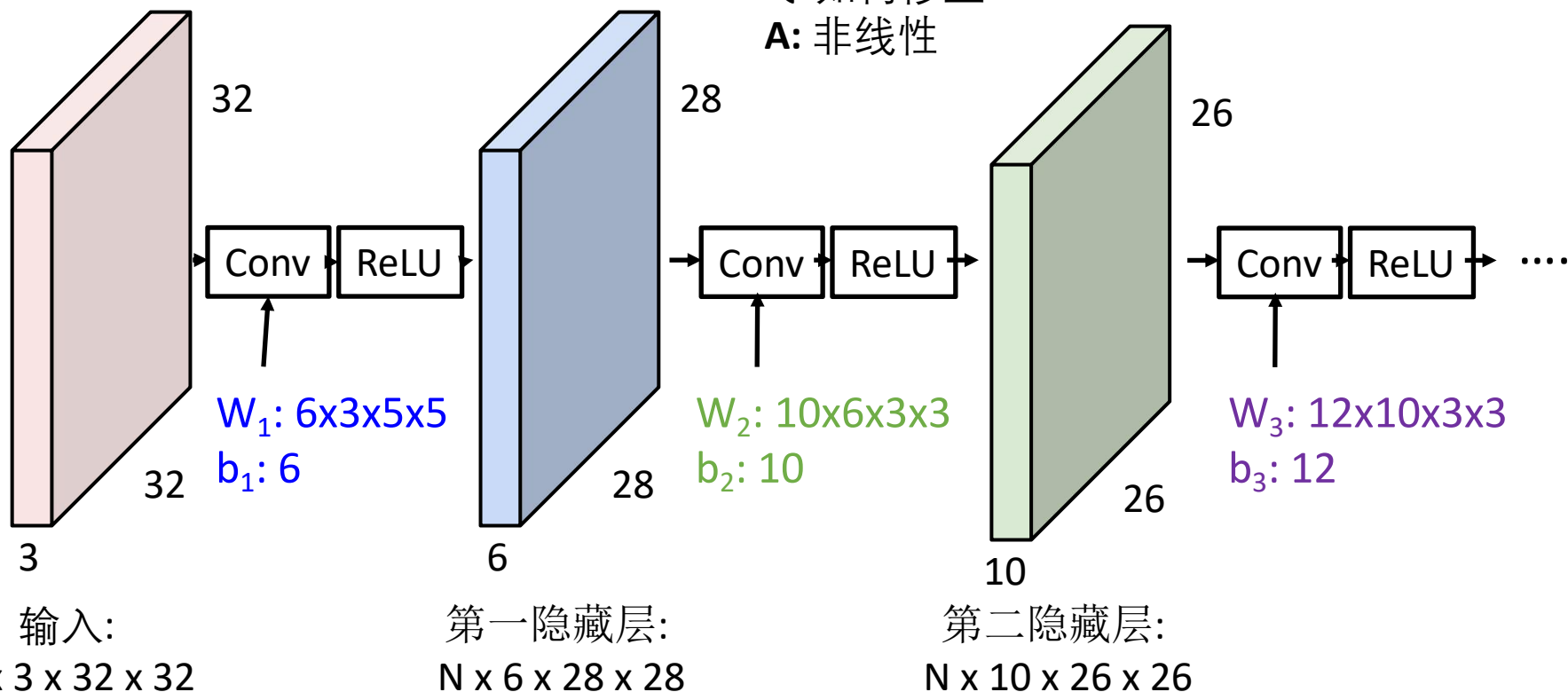
Q: 多个卷积直接级联?

A: 一个卷积!

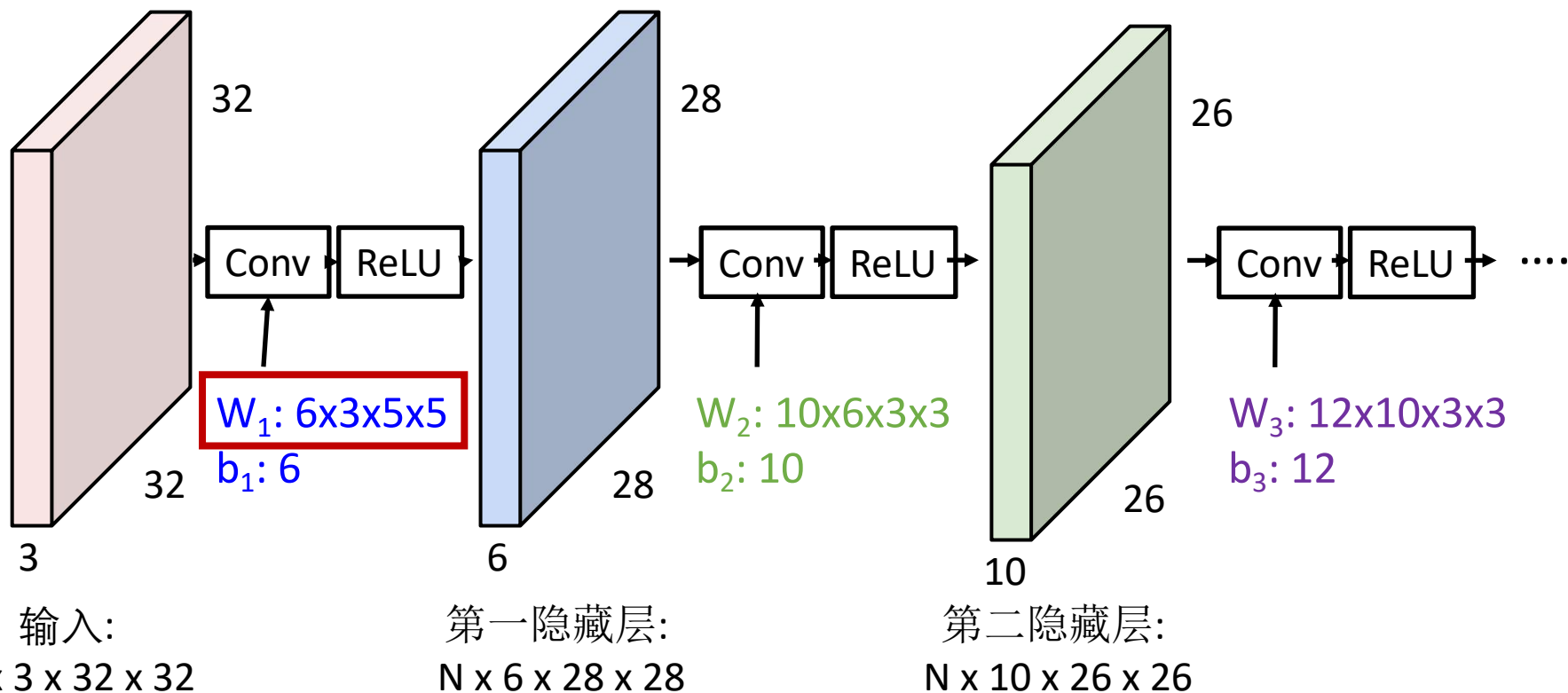
Q: 如何修正?

A: 非线性

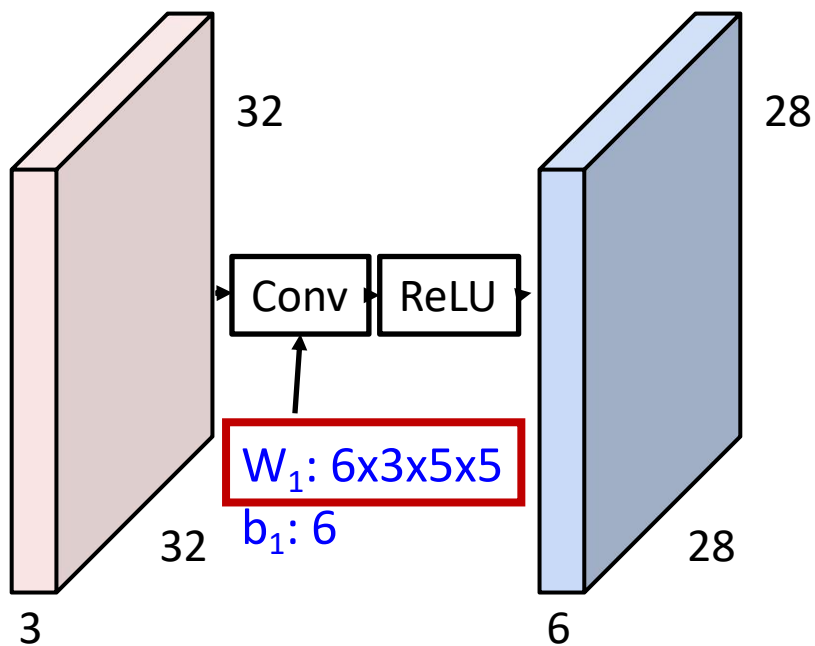
($y=W_2W_1x$ 等效为一个线性分类器)



卷积核可视化?



卷积核可视化?



输入:

$N \times 3 \times 32 \times 32$

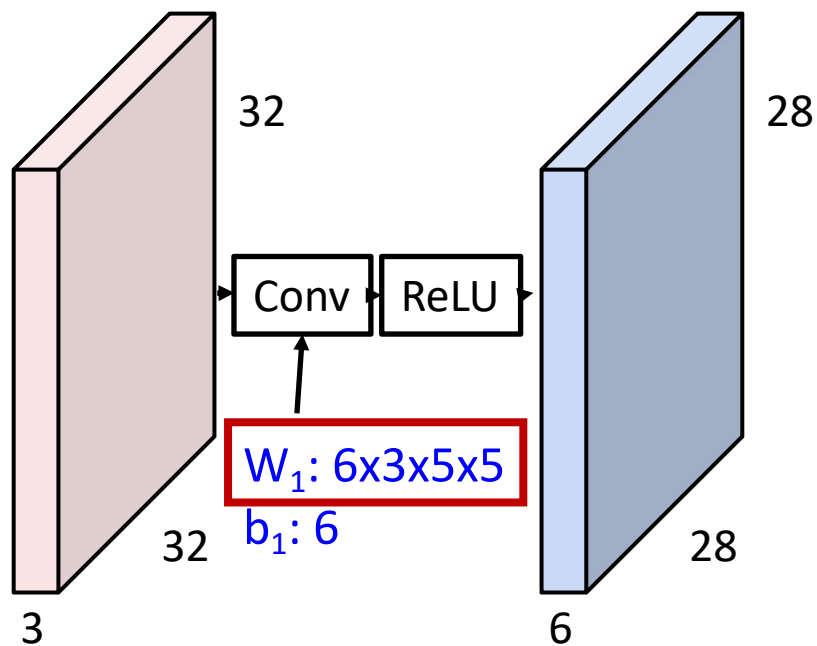
第一隐藏层:

$N \times 6 \times 28 \times 28$

线性分类器: 每个类别一个模板



卷积核可视化?



输入:

$N \times 3 \times 32 \times 32$

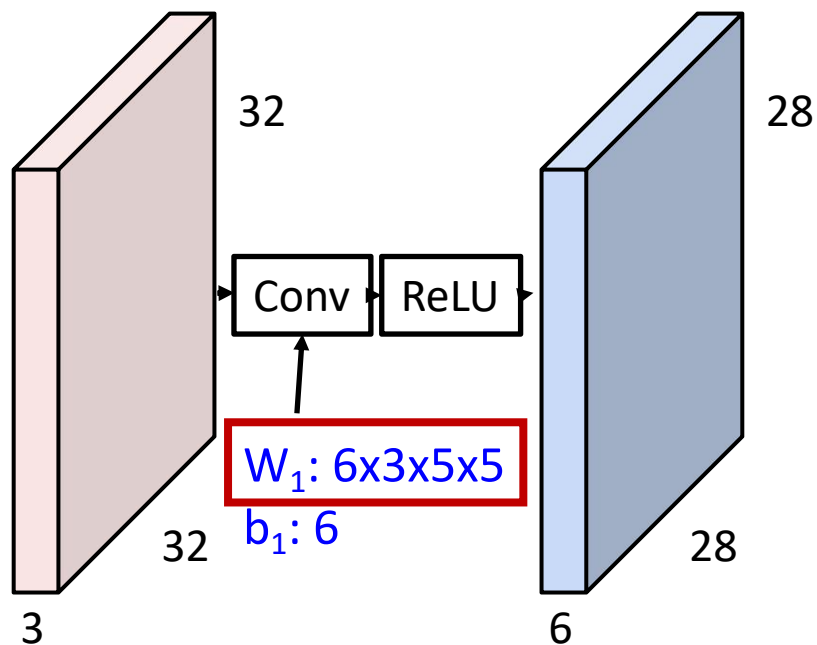
第一隐藏层:

$N \times 6 \times 28 \times 28$

MLP: 整个图像的特征模板



卷积核可视化?



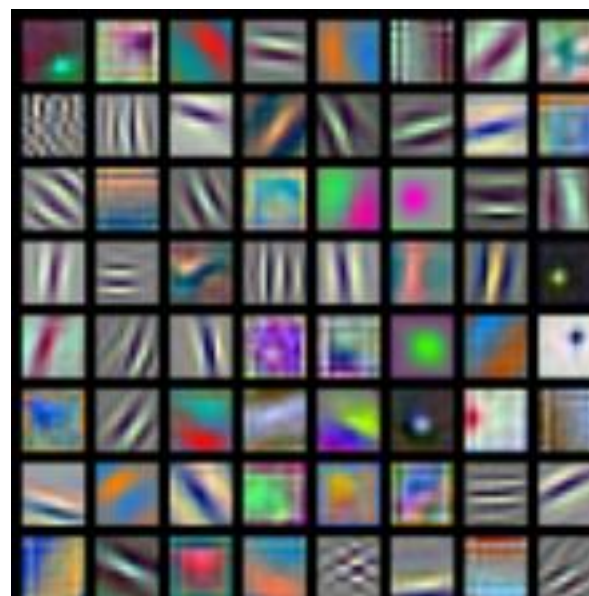
输入:

$N \times 3 \times 32 \times 32$

第一隐藏层:

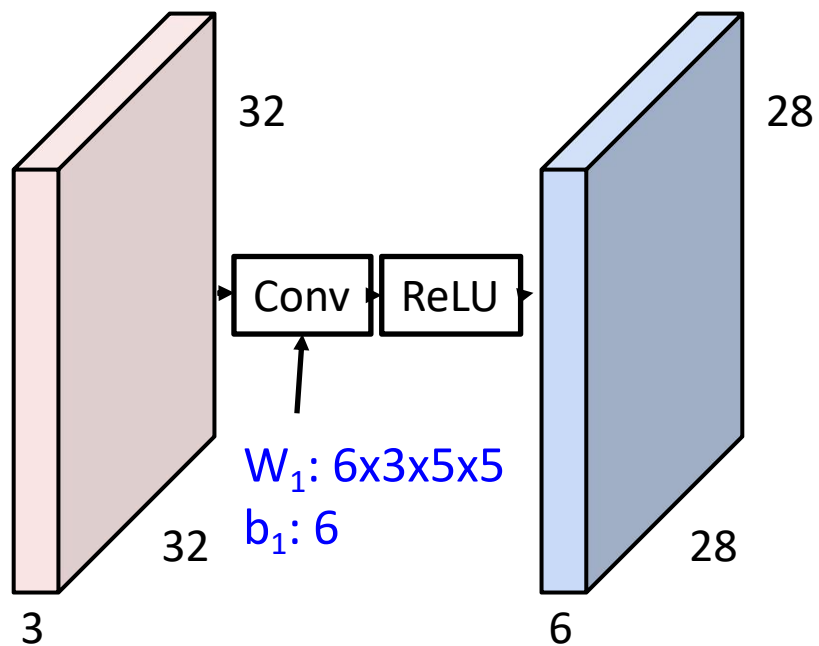
$N \times 6 \times 28 \times 28$

第一卷积层权重: 局部图像模板
(有方向的边缘, 相反的颜色)



AlexNet: 64 滤波组, 每个 $3 \times 11 \times 11$

空域维度处理



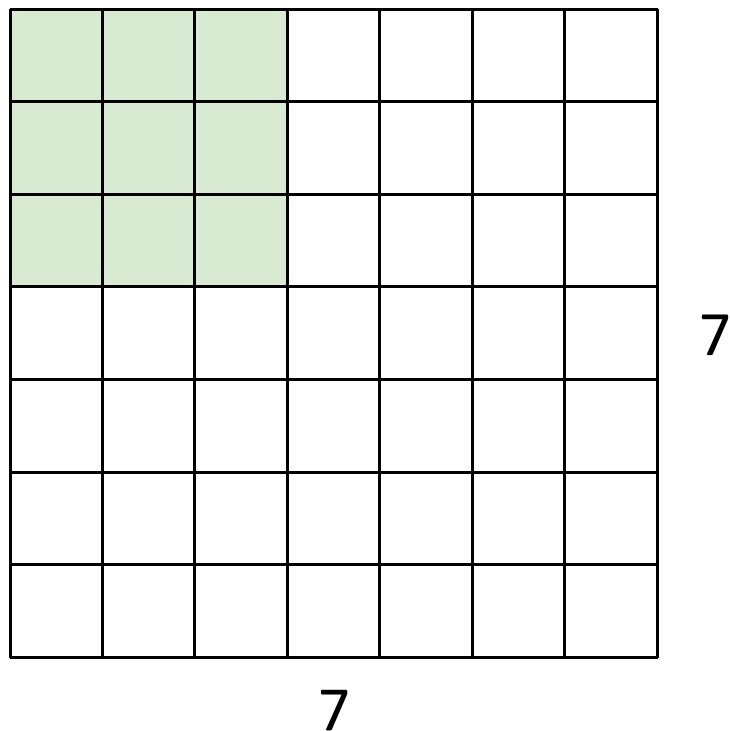
输入:

$N \times 3 \times 32 \times 32$

第一隐藏层:

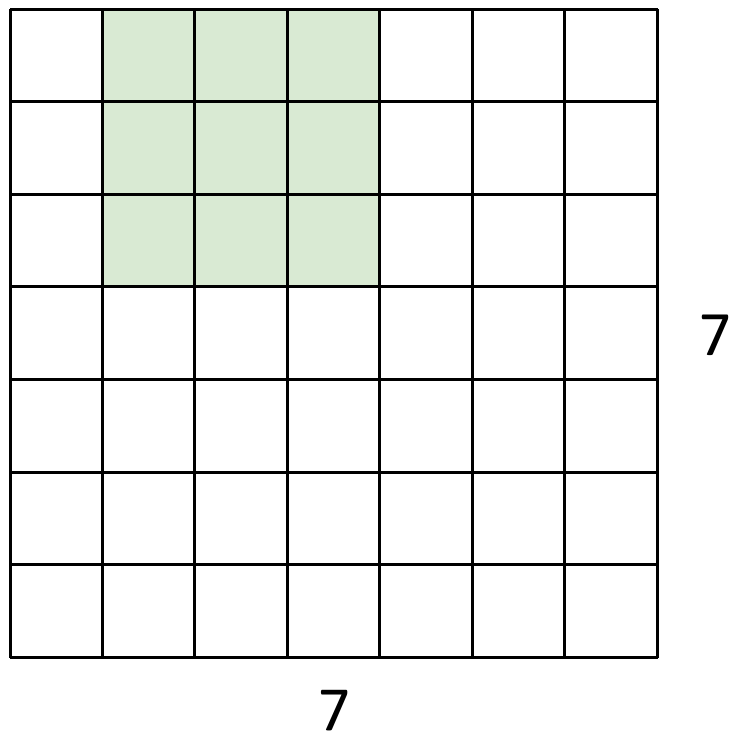
$N \times 6 \times 28 \times 28$

空域维度处理



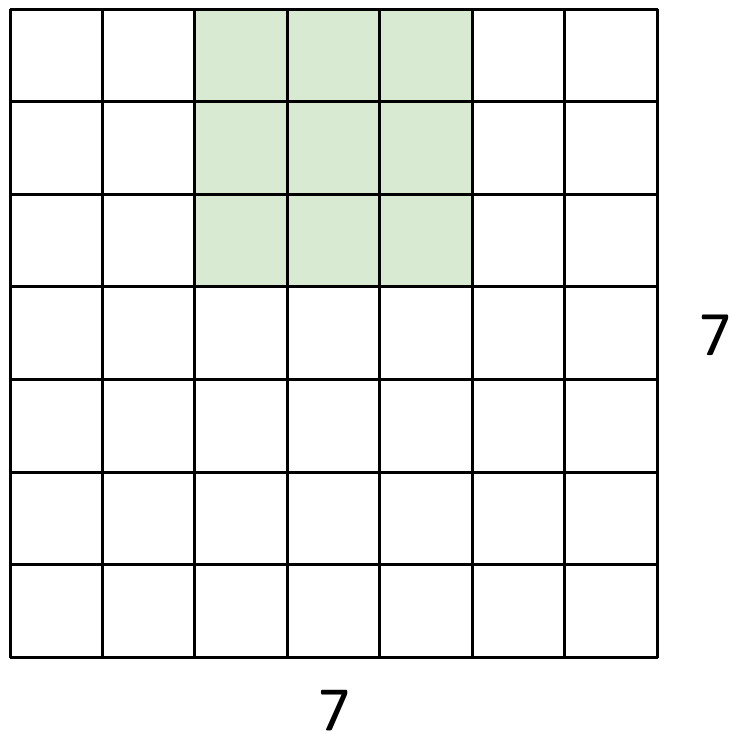
输入: 7x7
滤波: 3x3

空域维度处理



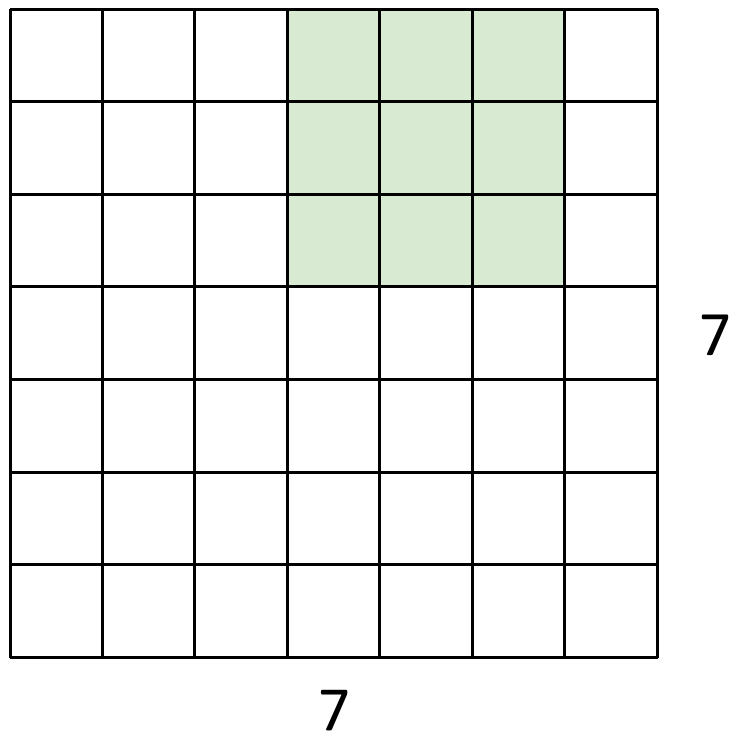
输入: 7x7
滤波: 3x3

空域维度处理



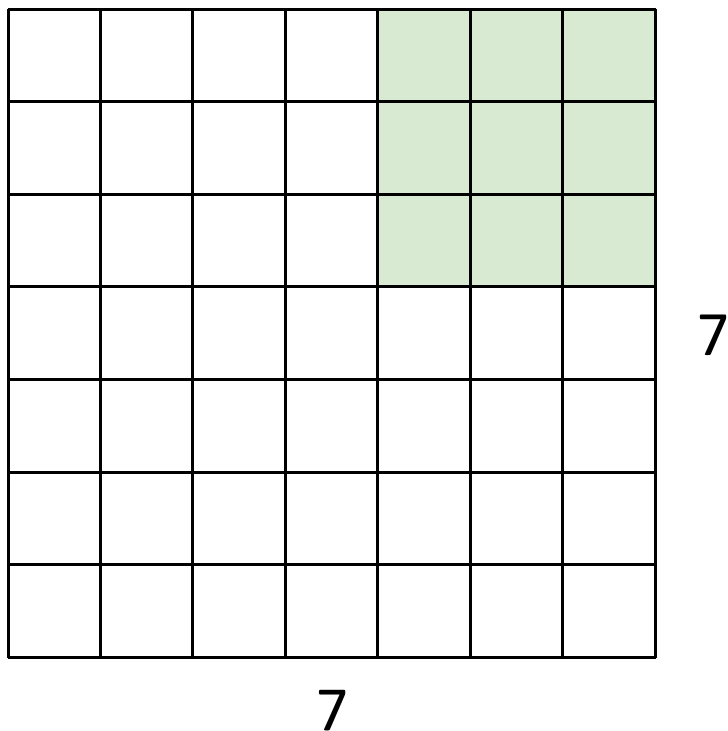
输入: 7x7
滤波: 3x3

空域维度处理



输入: 7x7
滤波: 3x3

空域维度处理

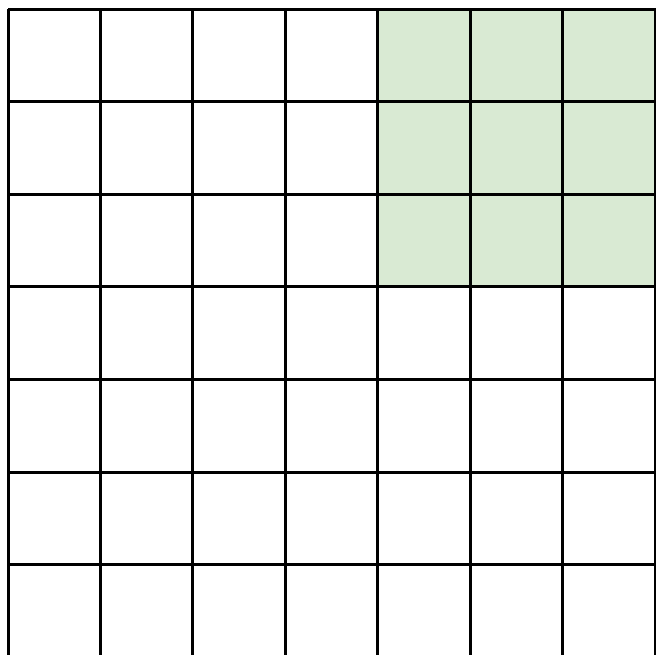


输入: 7x7

滤波: 3x3

输出: 5x5

空域维度处理



输入: 7×7

滤波: 3×3

输出: 5×5

通常:

输入: W

滤波: K

输出: $W - K + 1$

问题: 尺寸变小

空域维度处理

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

输入: 7x7

滤波: 3x3

输出: 5x5

通常:

问题: 尺寸变小

输入: W

滤波: K

输出: $W - K + 1$

方法: 填充

空域维度处理

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

输入: 7x7

滤波: 3x3

输出: 5x5

通常:

输入: W

滤波: K

填充: P

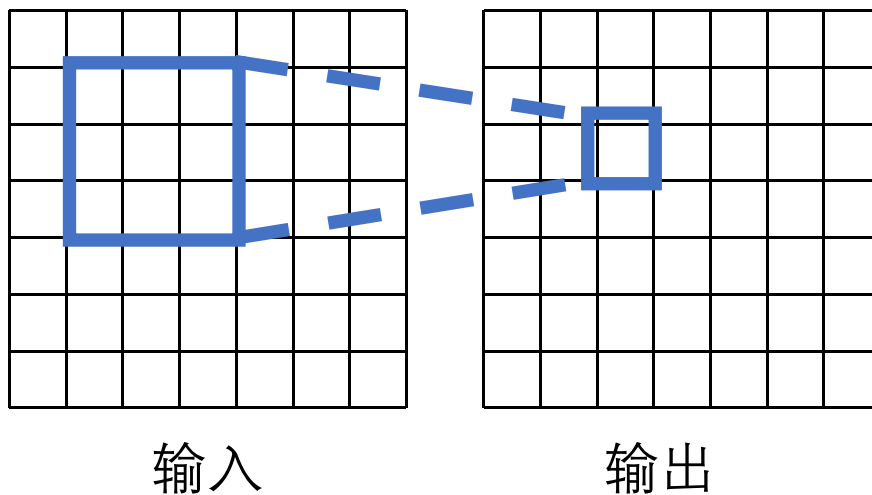
输出: $W - K + 1 + 2P$

通常:

设置 $P = (K - 1) / 2$
使得输出尺寸与输入尺寸一致

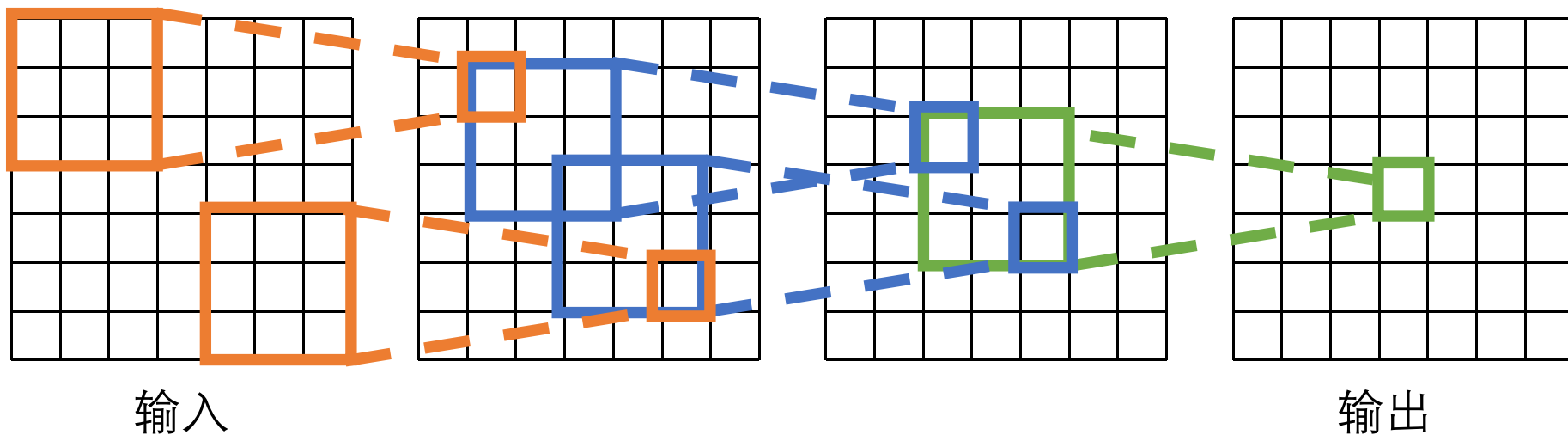
感受野

对于核大小为K的卷积, 输出中的每个元素依赖于输入 $K \times K$ **感受野** 区域



感受野

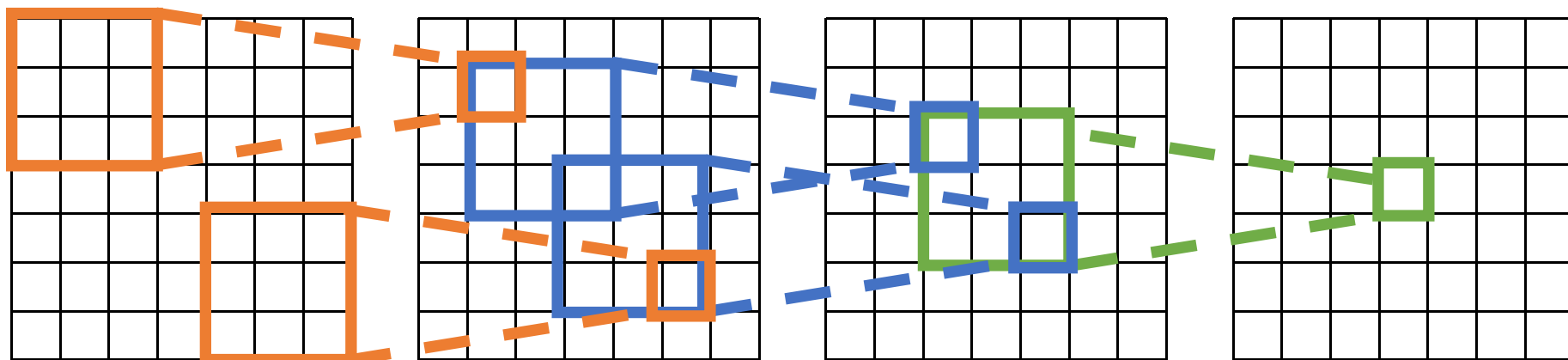
每个连续卷积加上 $K-1$ 到感受野之中
 L 层的感受野共 $1 + L * (K - 1)$



注意 - “输入感受野” vs “前一层感受野”!

感受野

每个连续卷积加上 $K-1$ 到感受野之中
 L 层的感受野共 $1 + L * (K - 1)$



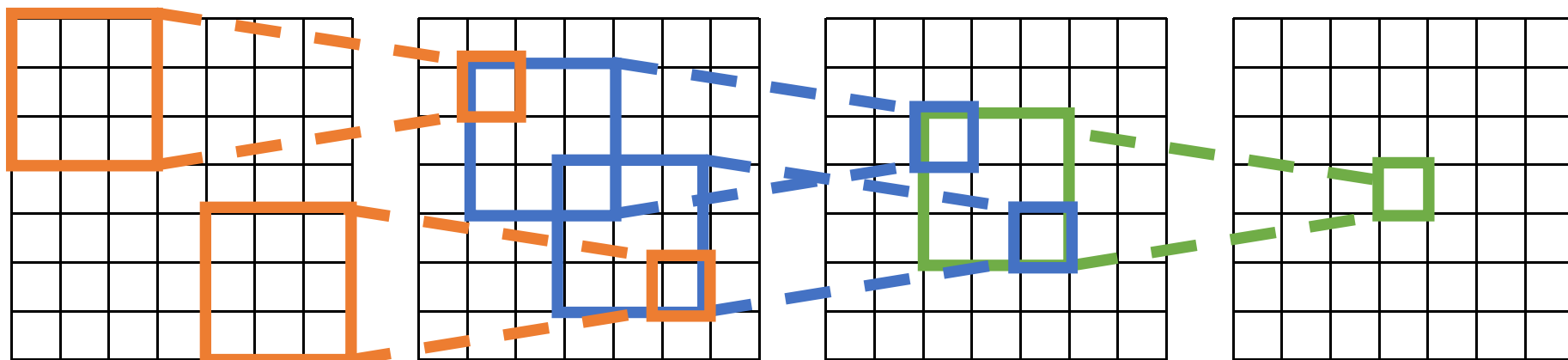
输入

问题: 对于大图像, 需要多层使得每个输出能够“看到”整个图像

输出

感受野

每个连续卷积加上 $K-1$ 到感受野之中
 L 层的感受野共 $1 + L * (K-1)$



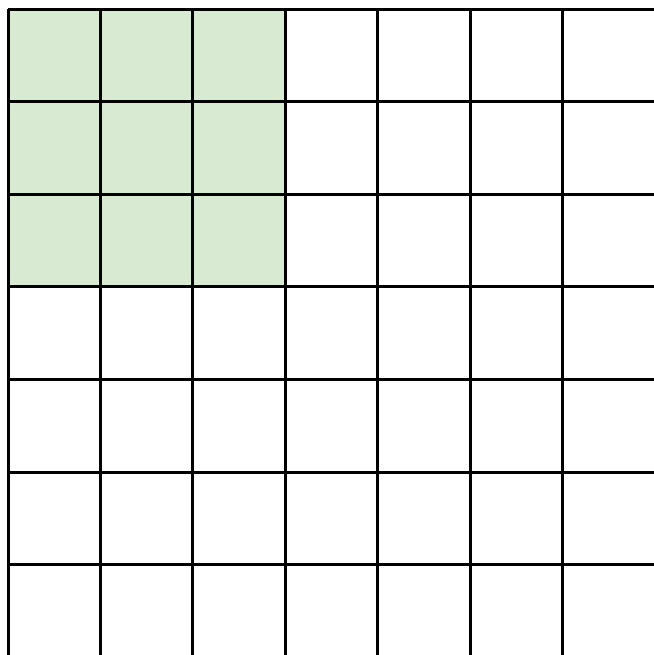
输入

问题: 对于大图像, 需要多层使得每个输出能够“看到”整个图像

方案: 下采样

输出

卷积步长

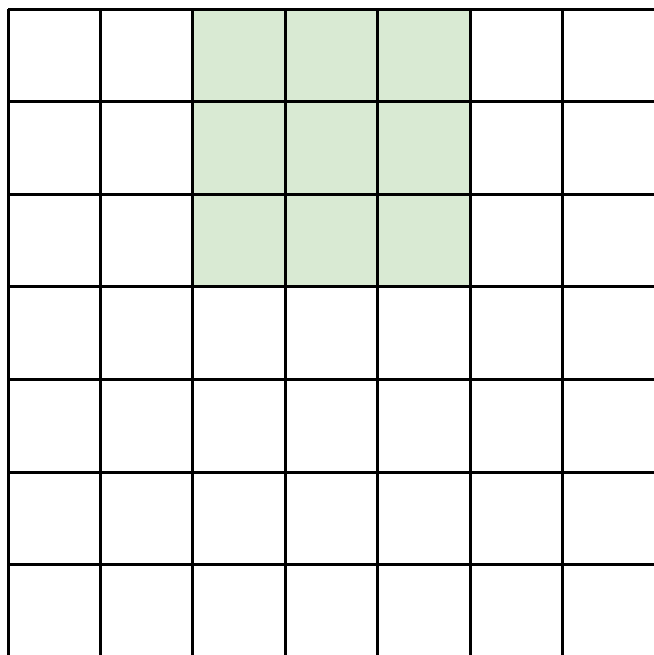


输入: 7x7

滤波: 3x3

步长: 2

卷积步长

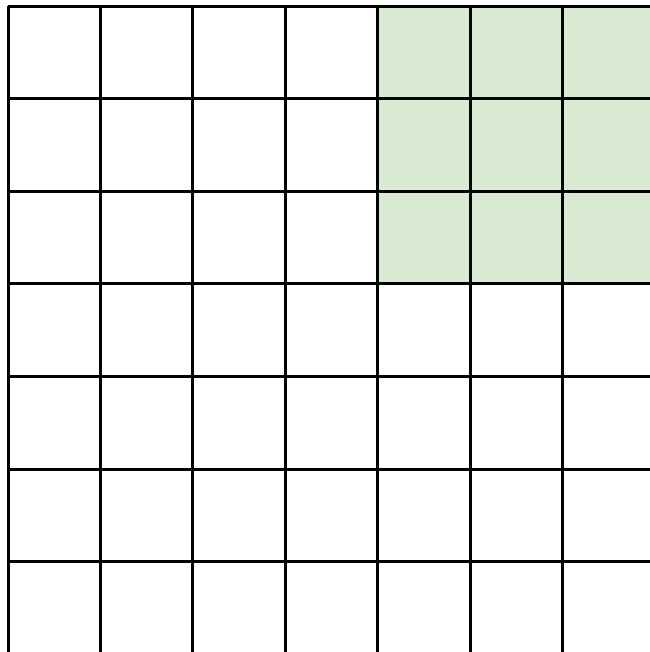


输入: 7x7

滤波: 3x3

步长: 2

卷积步长



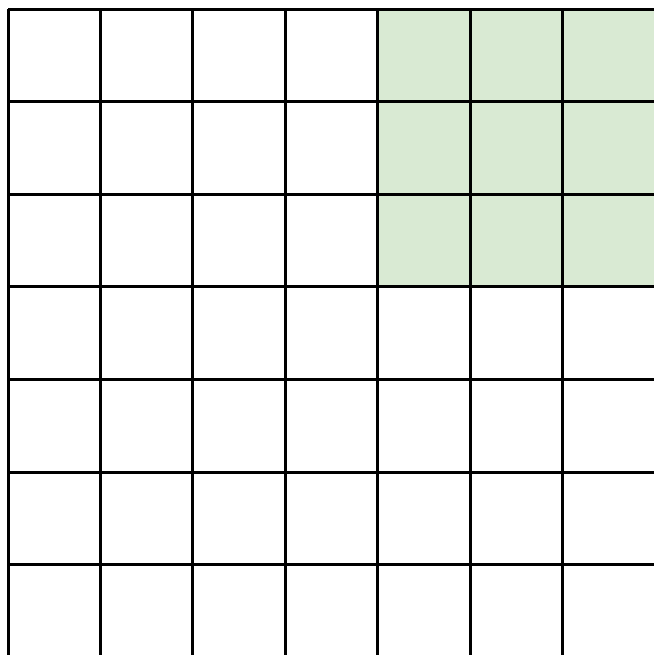
输入: 7×7

滤波: 3×3

步长: 2

输出: 3×3

卷积步长



输入: 7x7

滤波: 3x3

步长: 2

输出: 3x3

通常:

输入: W

滤波: K

填充: P

步长: S

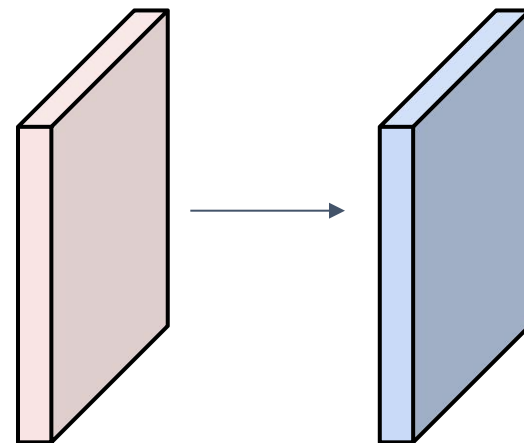
输出: $(W - K + 2P) / S + 1$

卷积举例

输入: $3 \times 32 \times 32$

10 5×5 滤波组 步长 1, 填充 2

输出大小: ?



卷积举例

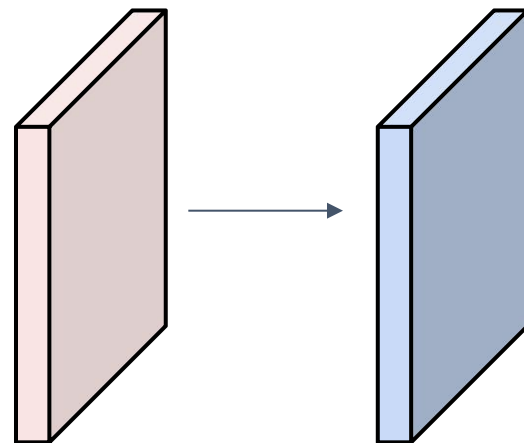
输入: 3 x 32 x 32

10 5x5 滤波组步长1, 填充 2

输出大小:

$(32 + 2 * 2 - 5) / 1 + 1 = 32$, 所以

10 x 32 x 32



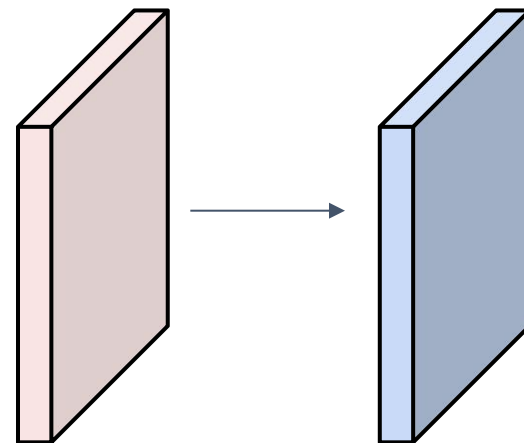
卷积举例

输入: $3 \times 32 \times 32$

10 5×5 滤波组步长1, 填充2

输出大小: $10 \times 32 \times 32$

学习参数规模: ?



卷积举例

输入: **3** x 32 x 32

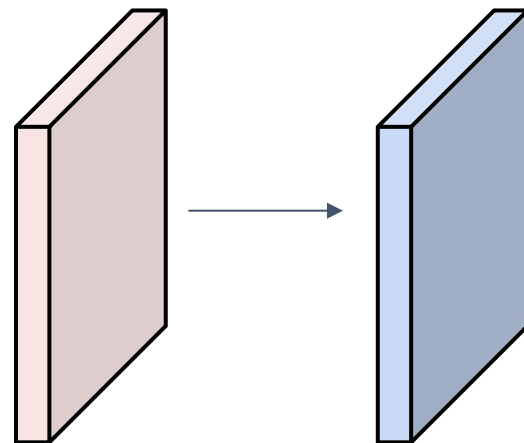
10 **5x5** 滤波组步长1, 填充2

输出大小: 10 x 32 x 32

学习参数规模: **760**

每个滤波核参数: **3*****5*****5** + 1 (偏置) = **76**

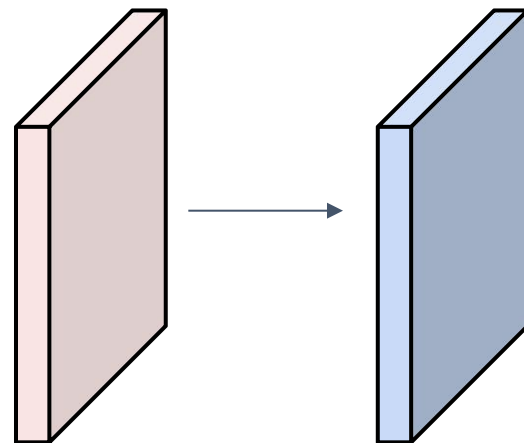
10 个滤波核, 所以总共 **10** * **76** = **760**



卷积举例

输入大小: $3 \times 32 \times 32$
10 5×5 滤波组步长1, 填充2

输出大小: $10 \times 32 \times 32$
学习参数规模: 760
乘法次数: ?



卷积举例

输入: **3** x 32 x 32

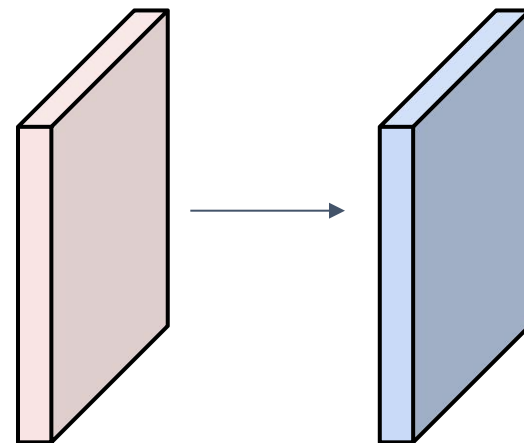
10 **5x5** 滤波组步长1, 填充2

输出大小: **10 x 32 x 32**

学习参数规模: 760

乘法次数: **768,000**

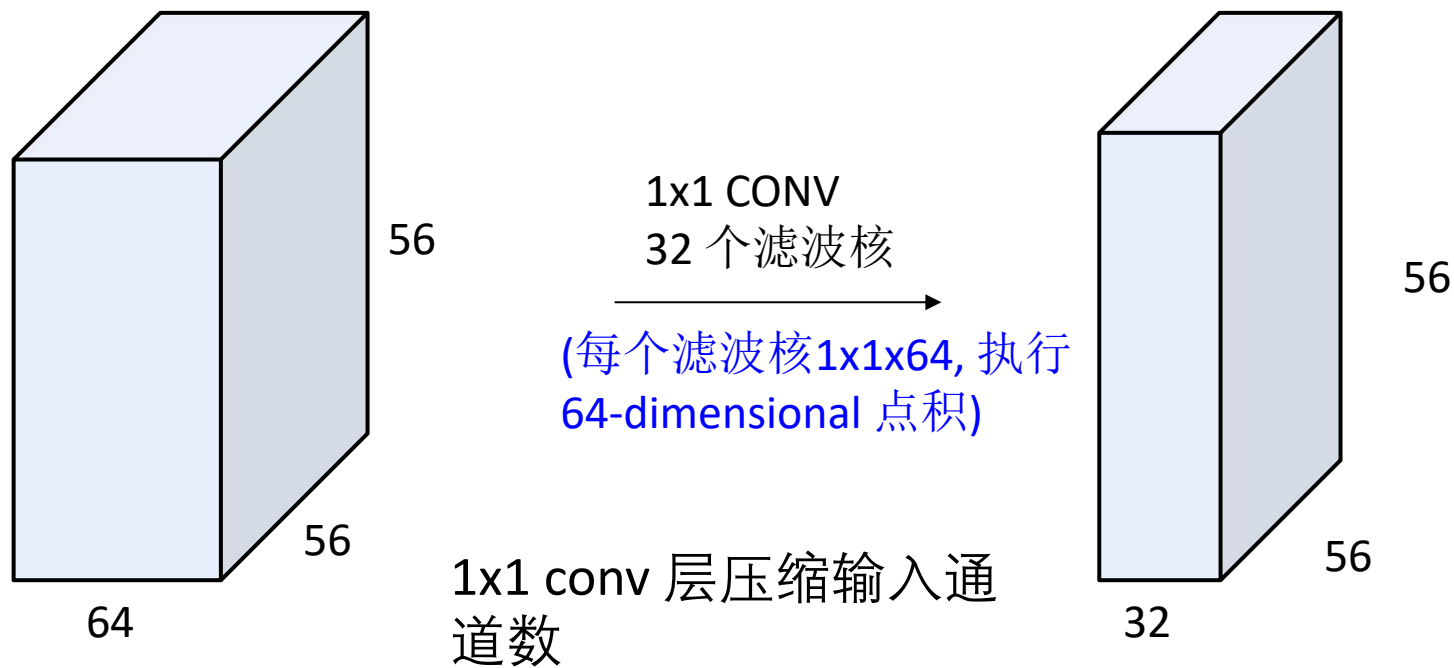
10*32*32 = 10,240 输出; 每个输出是两个 **3x5x5** 张量 (75 个元素) 的点积; 总计 = $75 * 10240 = 768K$



举例: 1x1 卷积



举例: 1x1 卷积



Lin et al, "Network in Network", ICLR 2014

卷积总结

输入: $C_{in} \times H \times W$

超参数:

- 核大小: $K_H \times K_W$
- 核数量: C_{out}
- 填充: P
- 步长: S

权重矩阵: $C_{out} \times C_{in} \times K_H \times K_W$ C_{out} 个

大小为 $C_{in} \times K_H \times K_W$ 滤波核

偏置向量: C_{out}

输出大小: $C_{out} \times H' \times W'$ 这里:

- $H' = (H - K + 2P) / S + 1$
- $W' = (W - K + 2P) / S + 1$

卷积总结

输入: $C_{in} \times H \times W$

超参数:

- 核大小: $K_H \times K_W$
- 核数量: C_{out}
- 填充: P
- 步长: S

权重矩阵: $C_{out} \times C_{in} \times K_H \times K_W$ 个
大小为 $C_{in} \times K_H \times K_W$ 滤波核

偏置向量: C_{out}

输出大小: $C_{out} \times H' \times W'$ 这里:

- $H' = (H - K + 2P) / S + 1$
- $W' = (W - K + 2P) / S + 1$

常用设置:

$K_H = K_W$ (等尺寸滤波核)

$P = (K - 1) / 2$ ("相同尺寸" 填充)

$C_{in}, C_{out} = 32, 64, 128, 256$ (2的幂次方)

$K = 3, P = 1, S = 1$ (3x3 conv)

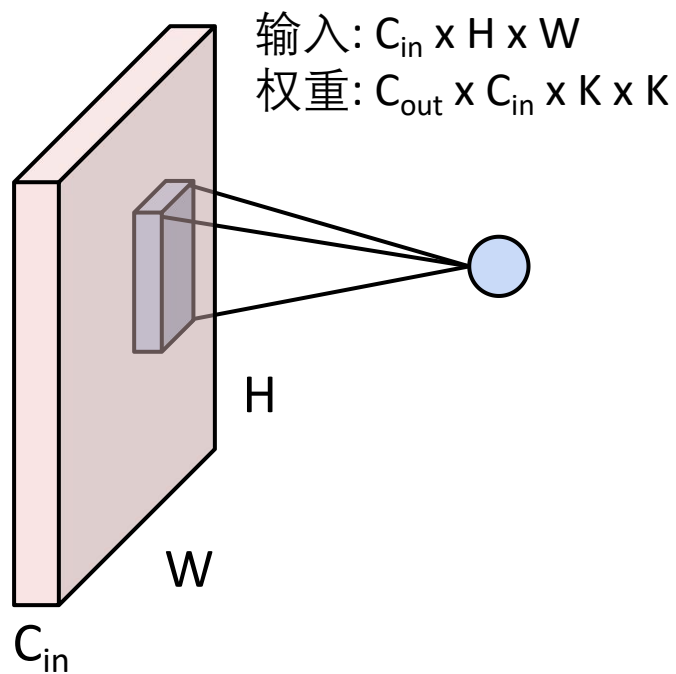
$K = 5, P = 2, S = 1$ (5x5 conv)

$K = 1, P = 0, S = 1$ (1x1 conv)

$K = 3, P = 1, S = 2$ (2倍下采样)

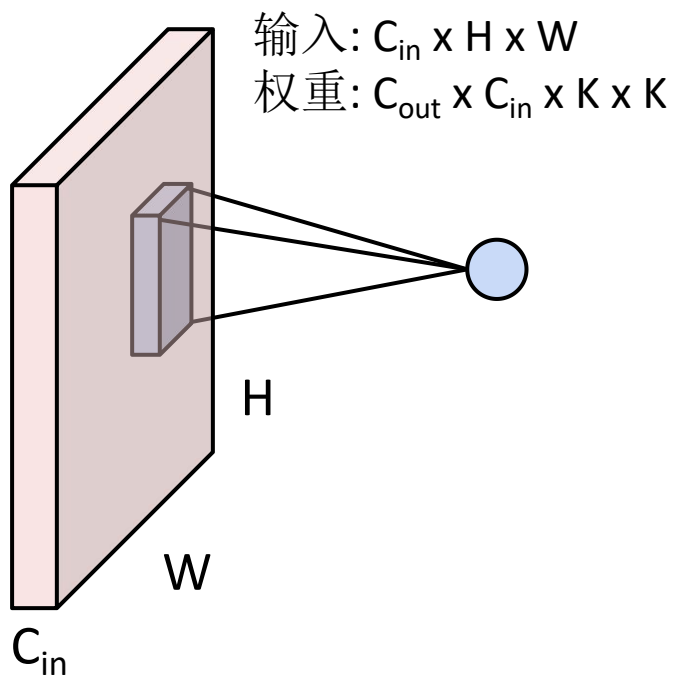
不同类型卷积

目前: 2D 卷积



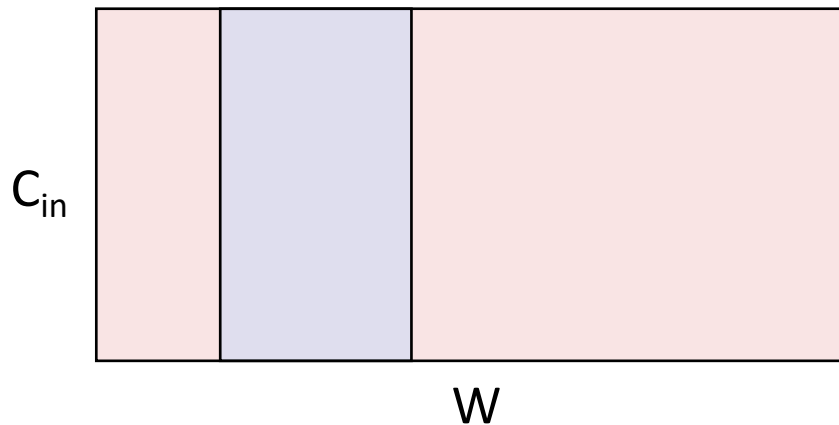
不同类型卷积

目前: 2D 卷积



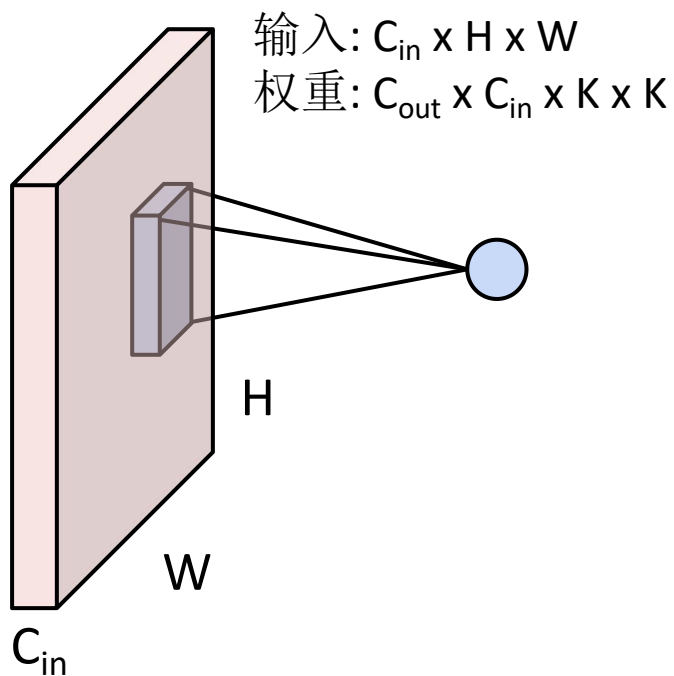
1D 卷积

输入: $C_{in} \times W$
权重: $C_{out} \times C_{in} \times K$



不同类型卷积

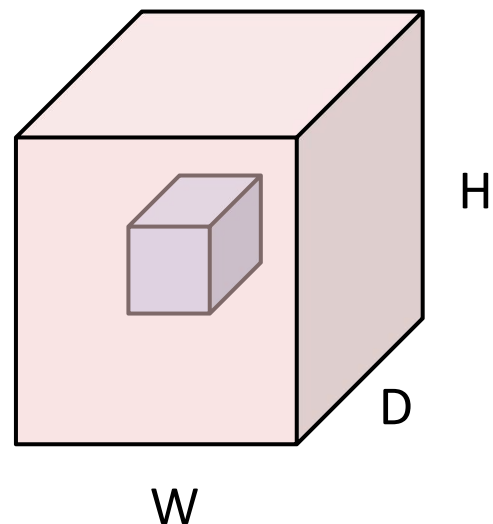
目前: 2D 卷积



C_{in} -dim

3D 卷积

输入: $C_{in} \times H \times W \times D$
权重: $C_{out} \times C_{in} \times K \times K \times K$



Mindspore 卷积层 Convolution Layer

mindspore.nn.Conv2d

```
class mindspore.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, pad_mode="same", padding=0, dilation=1, group=1, has_bias=False, weight_init="normal", bias_init="zeros", data_format="NCHW")
```

[\[源代码\]](#)

二维卷积层。

对输入Tensor计算二维卷积，该Tensor的常见shape为 $(N, C_{in}, H_{in}, W_{in})$ ，其中 N 为batch size， C_{in} 为空间维度， H_{in}, W_{in} 分别为特征层的高度和宽度。对于每个batch中的Tensor，其shape为 (C_{in}, H_{in}, W_{in}) ，公式定义如下：

$$\text{out}(N_i, C_{\text{out}j}) = \text{bias}(C_{\text{out}j}) + \sum_{k=0}^{C_{in}-1} \text{ccor}(\text{weight}(C_{\text{out}j}, k), X(N_i, k))$$

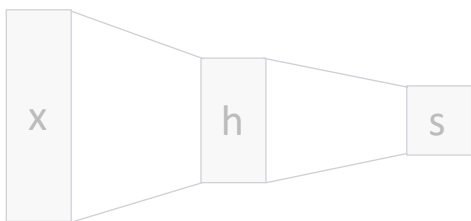
其中， ccor 为 [cross-correlation](#)， C_{in} 为输入空间维度， $\text{out}j$ 对应输出的第 j 个空间维度， j 的范围在 $[0, C_{out} - 1]$ 内。 $\text{weight}(C_{\text{out}j}, k)$ 的卷积核切片，其中 $\text{kernel_size}[0]$ 和 $\text{kernel_size}[1]$ 分别是卷积核的高度和宽度。 bias 为偏置参数， X 为输入Tensor。此时，输入Tensor对应的 data_format 为“NCHW”，完整卷积核的shape为 $(C_{out}, C_{in}/\text{group}, \text{kernel_size}[0], \text{kernel_size}[1])$ ，其中 group 是在空间维度上分割输入 x 的组数。如果输入Tensor对应的 data_format 为“NHWC”，完整卷积核的shape则为 $(C_{out}, \text{kernel_size}[0], \text{kernel_size}[1], C_{in}/\text{group})$ 。详细介绍请参考论文 [Gradient Based Learning Applied to Document Recognition](#)。

本章内容

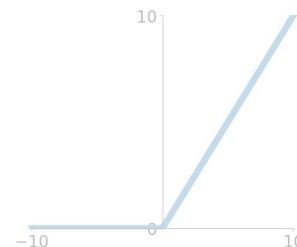
- 6.2.1.卷积神经网络的组成
- 6.2.2.卷积层
- 6.2.3.池化与归一化
- 6.2.4.卷积神经网络的训练

卷积网络的成分

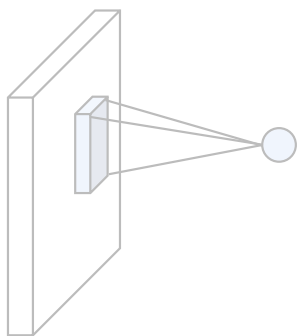
Fully-Connected Layers



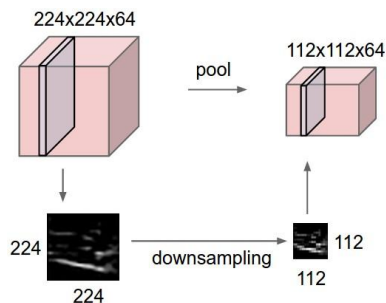
Activation Function



Convolution Layers



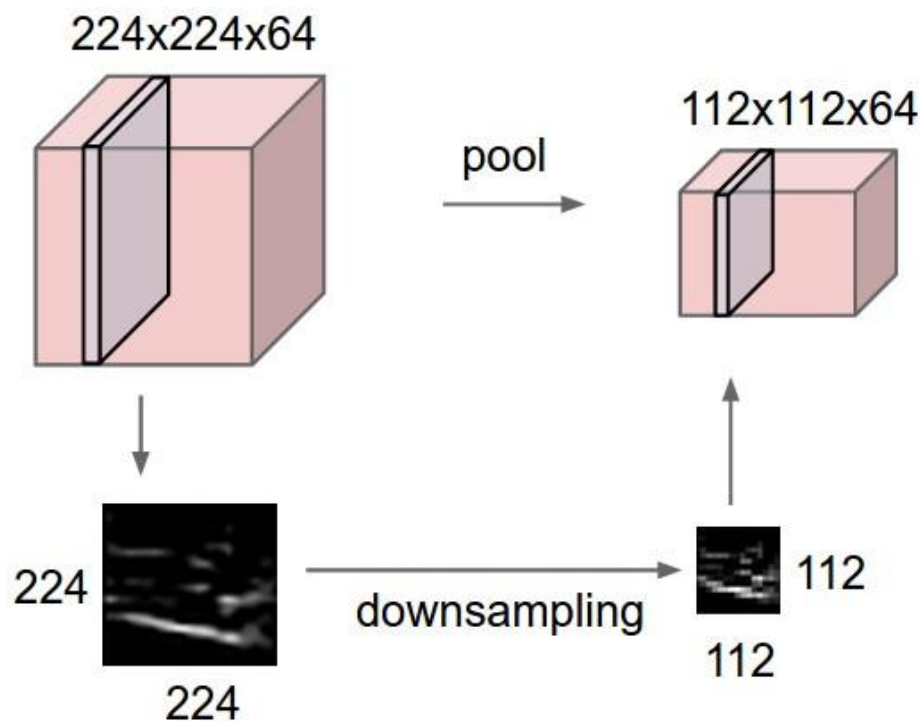
池化层



Normalization

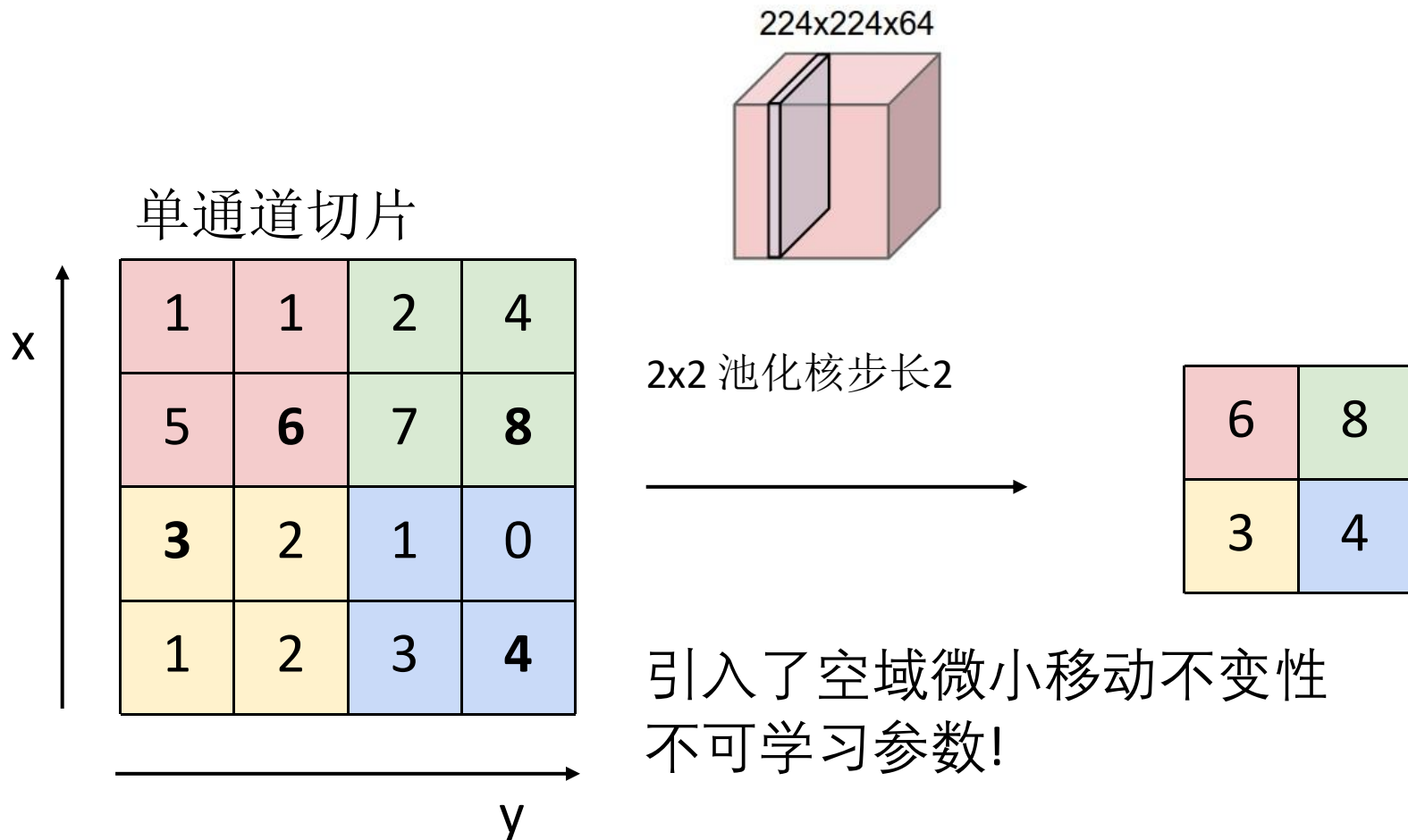
$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

池化层: 另一种下采样的方法



超参数:
内核尺寸
步长
池化函数

最大池化



池化总结

输入: $C \times H \times W$

超参数:

- 内核尺寸: K
- 步长: S
- 池化函数(max, avg)

输出: $C \times H' \times W'$ 这里

- $H' = (H - K) / S + 1$
- $W' = (W - K) / S + 1$

可学习的参数: None!

常用设置:

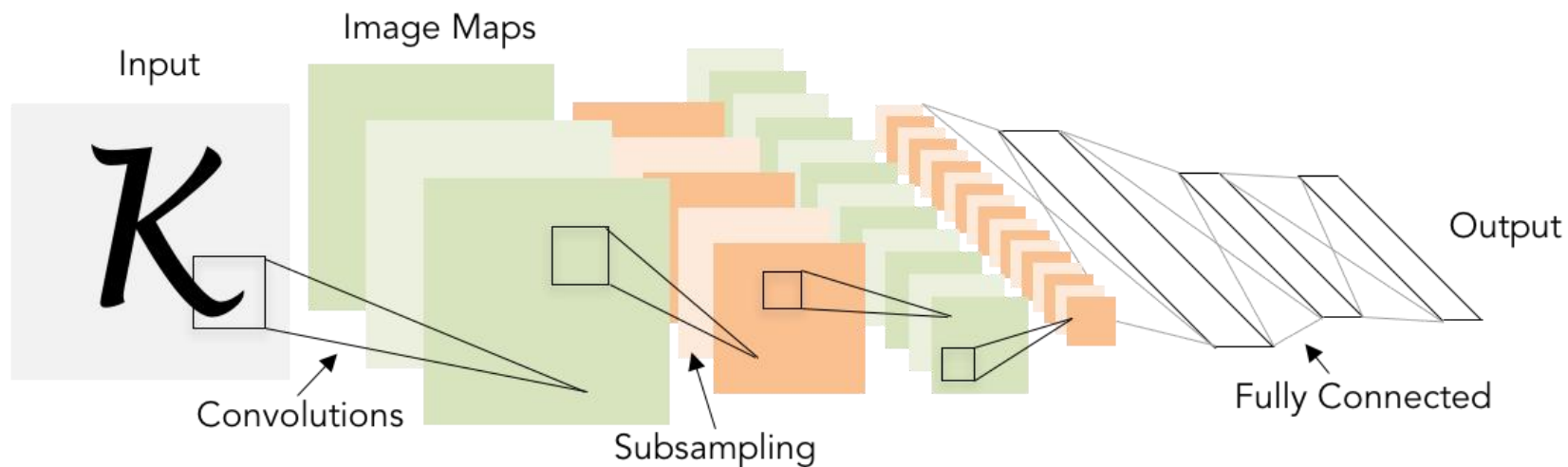
max, $K = 2, S = 2$

max, $K = 3, S = 2$ (AlexNet)

卷积网络LeNet-5

经典结构: [Conv, ReLU, Pool] x N, flatten, [FC, ReLU] x N, FC

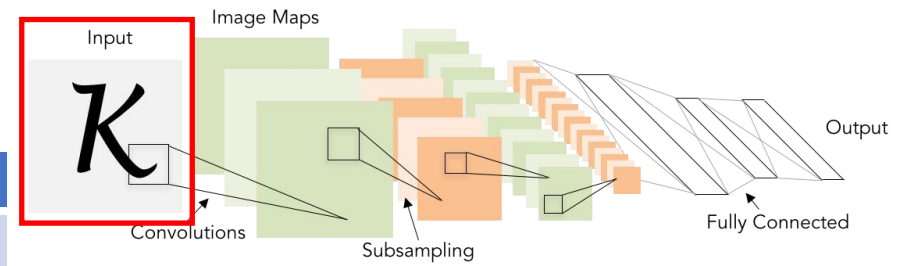
举例: LeNet-5



Lecun et al, "Gradient-based learning applied to document recognition", 1998

举例: LeNet-5

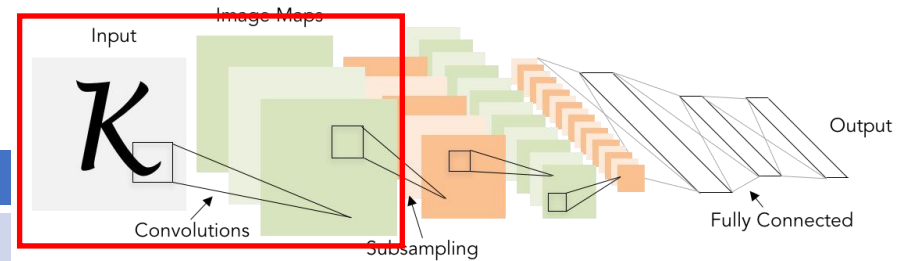
Layer	Output Size	Weight Size
Input	1 x 28 x 28	



Lecun et al, "Gradient-based learning applied to document recognition", 1998

举例: LeNet-5*

Layer	Output Size	Weight Size
Input	1 x 28 x 28	
Conv ($C_{out}=20^*$, $K=5$, $P=2$, $S=1$)	20 x 28 x 28	20 x 1 x 5 x 5
ReLU**	20 x 28 x 28	

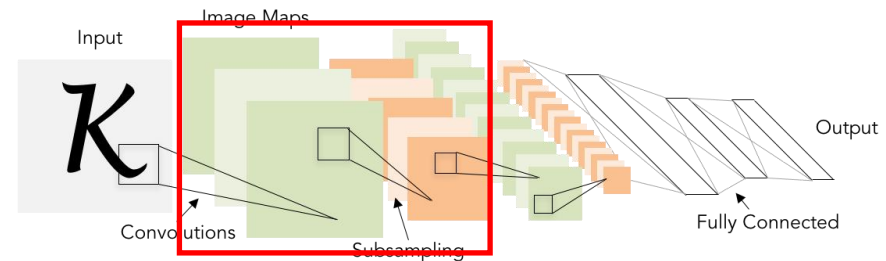


- * Original paper: $C_{out} = 6$
- ** Original paper: sigmoid

Lecun et al, "Gradient-based learning applied to document recognition", 1998

举例: LeNet-5

Layer	Output Size	Weight Size
Input	1 x 28 x 28	
Conv ($C_{out}=20$, $K=5$, $P=2$, $S=1$)	20 x 28 x 28	20 x 1 x 5 x 5
ReLU	20 x 28 x 28	
MaxPool($K=2$, $S=2$)*	20 x 14 x 14	

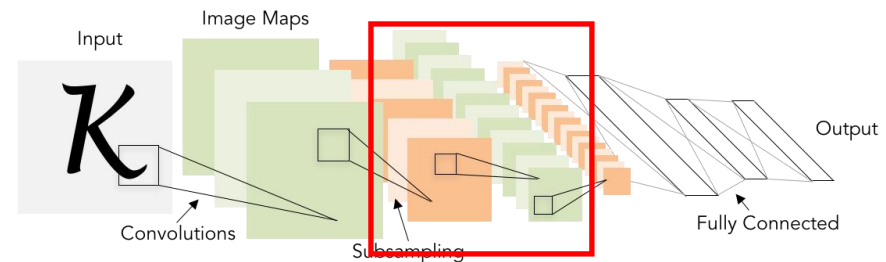


* 2x2 strided convolution

Lecun et al, "Gradient-based learning applied to document recognition", 1998

举例: LeNet-5

Layer	Output Size	Weight Size
Input	1 x 28 x 28	
Conv ($C_{out}=20$, $K=5$, $P=2$, $S=1$)	20 x 28 x 28	20 x 1 x 5 x 5
ReLU	20 x 28 x 28	
MaxPool($K=2$, $S=2$)	20 x 14 x 14	
Conv ($C_{out}=50^*$, $K=5$, $P=2$, $S=1$)	50 x 14 x 14	50 x 20 x 5 x 5
ReLU**	50 x 14 x 14	



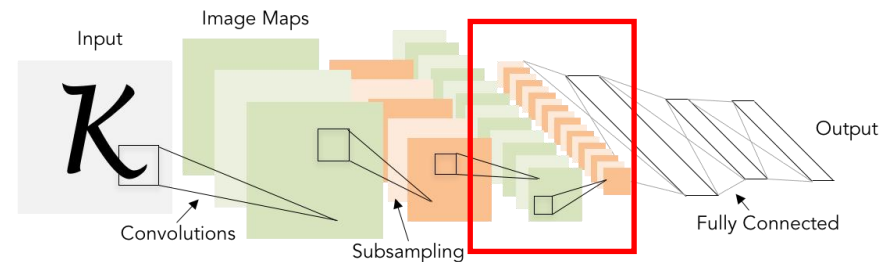
* Original paper: $C_{out} = 16$, grouped convolutions

** Original paper: sigmoid

Lecun et al, "Gradient-based learning applied to document recognition", 1998

举例: LeNet-5

Layer	Output Size	Weight Size
Input	1 x 28 x 28	
Conv ($C_{out}=20$, $K=5$, $P=2$, $S=1$)	20 x 28 x 28	20 x 1 x 5 x 5
ReLU	20 x 28 x 28	
MaxPool($K=2$, $S=2$)	20 x 14 x 14	
Conv ($C_{out}=50$, $K=5$, $P=2$, $S=1$)	50 x 14 x 14	50 x 20 x 5 x 5
ReLU	50 x 14 x 14	
MaxPool($K=2$, $S=2$)*	50 x 7 x 7	

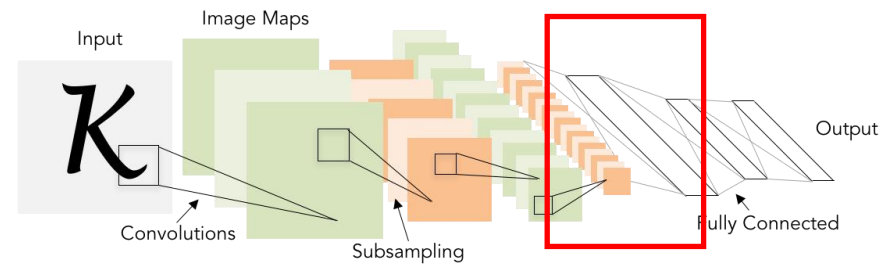


* 2x2 strided convolution

Lecun et al, "Gradient-based learning applied to document recognition", 1998

举例: LeNet-5

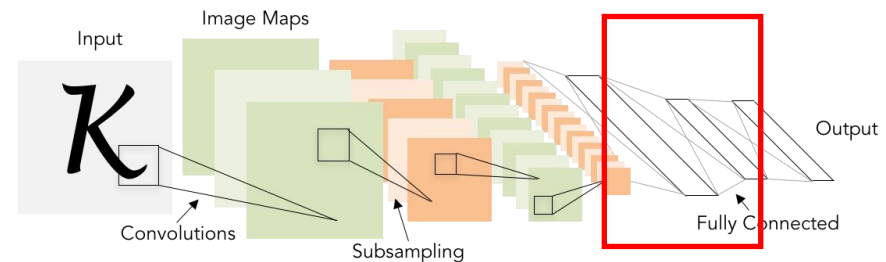
Layer	Output Size	Weight Size
Input	1 x 28 x 28	
Conv ($C_{out}=20$, $K=5$, $P=2$, $S=1$)	20 x 28 x 28	20 x 1 x 5 x 5
ReLU	20 x 28 x 28	
MaxPool($K=2$, $S=2$)	20 x 14 x 14	
Conv ($C_{out}=50$, $K=5$, $P=2$, $S=1$)	50 x 14 x 14	50 x 20 x 5 x 5
ReLU	50 x 14 x 14	
MaxPool($K=2$, $S=2$)	50 x 7 x 7	
Flatten	2450	



Lecun et al, "Gradient-based learning applied to document recognition", 1998

举例: LeNet-5

Layer	Output Size	Weight Size
Input	1 x 28 x 28	
Conv ($C_{out}=20$, $K=5$, $P=2$, $S=1$)	20 x 28 x 28	20 x 1 x 5 x 5
ReLU	20 x 28 x 28	
MaxPool($K=2$, $S=2$)	20 x 14 x 14	
Conv ($C_{out}=50$, $K=5$, $P=2$, $S=1$)	50 x 14 x 14	50 x 20 x 5 x 5
ReLU	50 x 14 x 14	
MaxPool($K=2$, $S=2$)	50 x 7 x 7	
Flatten	2450	
Linear (2450 -> 500)	500	2450 x 500
ReLU*	500	

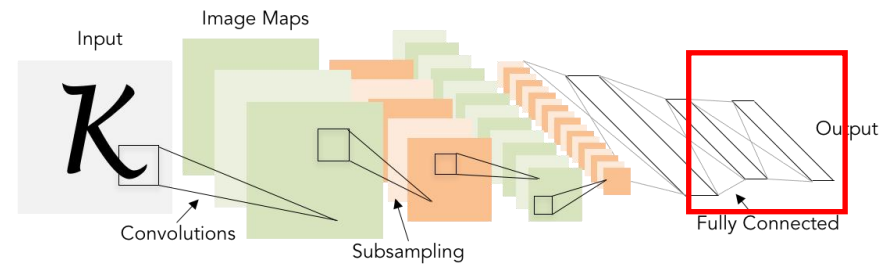


* Original paper has different 1x1 convolutions, sigmoid non-linearities

Lecun et al, "Gradient-based learning applied to document recognition", 1998

举例: LeNet-5*

Layer	Output Size	Weight Size
Input	1 x 28 x 28	
Conv ($C_{out}=20$, $K=5$, $P=2$, $S=1$)	20 x 28 x 28	20 x 1 x 5 x 5
ReLU	20 x 28 x 28	
MaxPool($K=2$, $S=2$)	20 x 14 x 14	
Conv ($C_{out}=50$, $K=5$, $P=2$, $S=1$)	50 x 14 x 14	50 x 20 x 5 x 5
ReLU	50 x 14 x 14	
MaxPool($K=2$, $S=2$)	50 x 7 x 7	
Flatten	2450	
Linear (2450 -> 500)	500	2450 x 500
ReLU	500	
Linear (500 -> 10)*	10	500 x 10

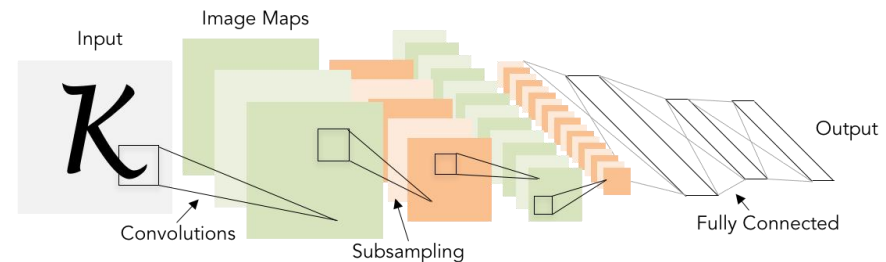


* Original paper uses RBF (radial basis function) kernels instead of a softmax

Lecun et al, "Gradient-based learning applied to document recognition", 1998

举例: LeNet-5

Layer	Output Size	Weight Size
Input	1 x 28 x 28	
Conv ($C_{out}=20$, $K=5$, $P=2$, $S=1$)	20 x 28 x 28	20 x 1 x 5 x 5
ReLU	20 x 28 x 28	
MaxPool($K=2$, $S=2$)	20 x 14 x 14	
Conv ($C_{out}=50$, $K=5$, $P=2$, $S=1$)	50 x 14 x 14	50 x 20 x 5 x 5
ReLU	50 x 14 x 14	
MaxPool($K=2$, $S=2$)	50 x 7 x 7	
Flatten	2450	
Linear (2450 -> 500)	500	2450 x 500
ReLU	500	
Linear (500 -> 10)	10	500 x 10



经过整个网络:

空域大小**递减**
(通过池化和带步长的卷积)

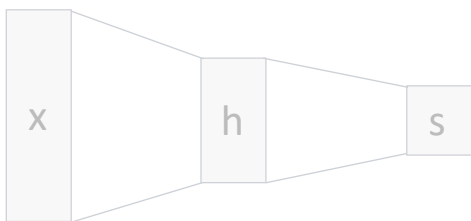
通道数**递增**
(整个“体积” 不变!)

Lecun et al, "Gradient-based learning applied to document recognition", 1998

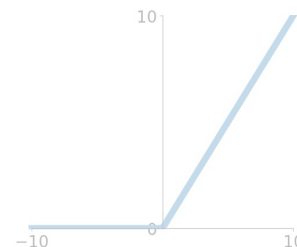
问题: 深度网络非常难于训练!

卷积网络的成分

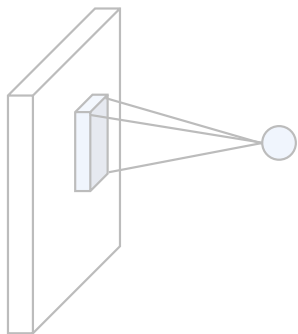
Fully-Connected Layers



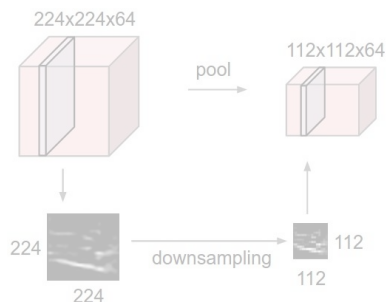
Activation Function



Convolution Layers



Pooling Layers



归一化

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

批归一化

想法: “归一化” 一层的输出, 使得其具有零均值和单位方差. 为什么?

为什么? 帮助减少“内部变量变化”, 改进优化

按以下公式归一化批:

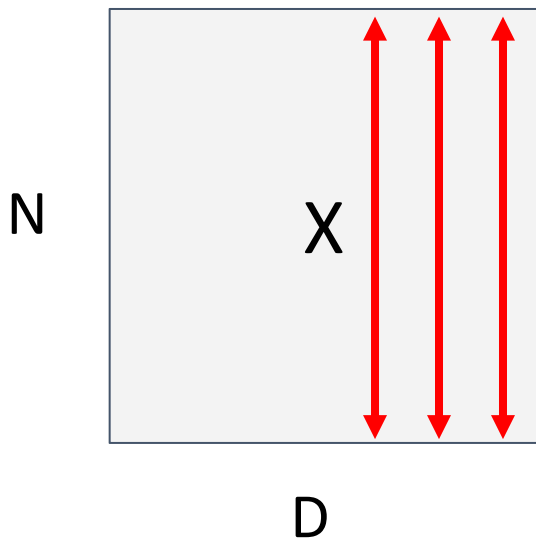
$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

这是一个 **可导函数**, 所以是后向传播的一个可操作节点!

Ioffe and Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift”, ICML 2015

批归一化

输入: $x : N \times D$



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad \text{每通道均值, 大小是 } D$$

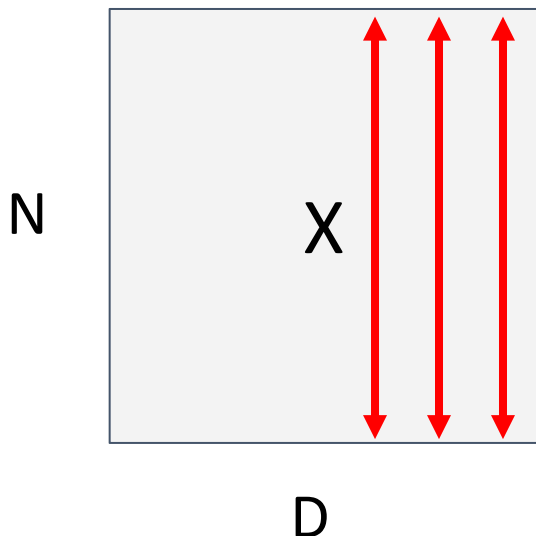
$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad \text{每通道 std, 大小是 } D$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}} \quad \text{归一化 } x, \text{ 大小 } N \times D$$

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

批归一化

输入: $x : N \times D$



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad \begin{array}{l} \text{每通道} \\ \text{均值, 大小是 } D \end{array}$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad \begin{array}{l} \text{每通道} \\ \text{std, 大小是 } D \end{array}$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}} \quad \begin{array}{l} \text{归一化 } x, \\ \text{大小 } N \times D \end{array}$$

问题: 零均值, 单位方差限制太强?

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

批归一化

输入: $x : N \times D$

可学习的尺度和平移:

$$\gamma, \beta : D$$

学习 $\gamma = \sigma, \beta = \mu$ 可以实现恒等映射!

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad \begin{array}{l} \text{每通道} \\ \text{均值, 大小是 } D \end{array}$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad \begin{array}{l} \text{每通道} \\ \text{std, 大小是 } D \end{array}$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}} \quad \begin{array}{l} \text{归一化 } x, \\ \text{大小 } N \times D \end{array}$$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j \quad \begin{array}{l} \text{输出,} \\ \text{大小 } N \times D \end{array}$$

批归一化: 测试

问题: 估计依赖于minibatch; 测试过程没有分批!

输入: $x : N \times D$

可学习的尺度和平移:

$\gamma, \beta : D$

学习 $\gamma = \sigma, \beta = \mu$ 可以实现恒等映射!

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad \text{每通道均值, 大小是 } D$$
$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad \text{每通道std, 大小是 } D$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}} \quad \text{归一化 } x, \text{ 大小 } N \times D$$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j \quad \text{归一化 } x, \text{ 大小 } N \times D$$

批归一化: 测试时间

输入: $x : N \times D$

$\mu_j =$ (运行) 整个训练样本的均值 每通道均值, 大小是 D

可学习的尺度和平移:

$\gamma, \beta : D$

$\sigma_j^2 =$ (运行) 整个训练样本的方差 每通道 std, 大小是 D

学习 $\gamma = \sigma, \beta = \mu$ 可以实现恒等映射!

$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$ 归一化 x, 大小 N x D

$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$ 归一化 x, 大小 N x D

批归一化: 测试时间

输入: $x : N \times D$

$\mu_j =$ (运行) 整个训练样本的均值 每通道均值, 大小是 D

可学习的尺度和平移:

$\gamma, \beta : D$

$\sigma_j^2 =$ (运行) 整个训练样本的方差 每通道 std, 大小是 D

batchnorm 在测试过程中是一个线性操作!

可以与前一层 fully-connected or conv layer 混合

$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$ 归一化 x, 大小 N x D

$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$ 归一化 x, 大小 N x D

ConvNets的批归一化

fully-connected 网络的批归一化

$$\mathbf{x} : \mathbf{N} \times \mathbf{D}$$

归一化



$$\boldsymbol{\mu}, \boldsymbol{\sigma} : 1 \times \mathbf{D}$$

$$\boldsymbol{\gamma}, \boldsymbol{\beta} : 1 \times \mathbf{D}$$

$$\mathbf{y} = \boldsymbol{\gamma} (\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta}$$

Convolutional网络的批归一化
(空域 Batchnorm, BatchNorm2D)

$$\mathbf{x} : \mathbf{N} \times \mathbf{C} \times \mathbf{H} \times \mathbf{W}$$

归一化

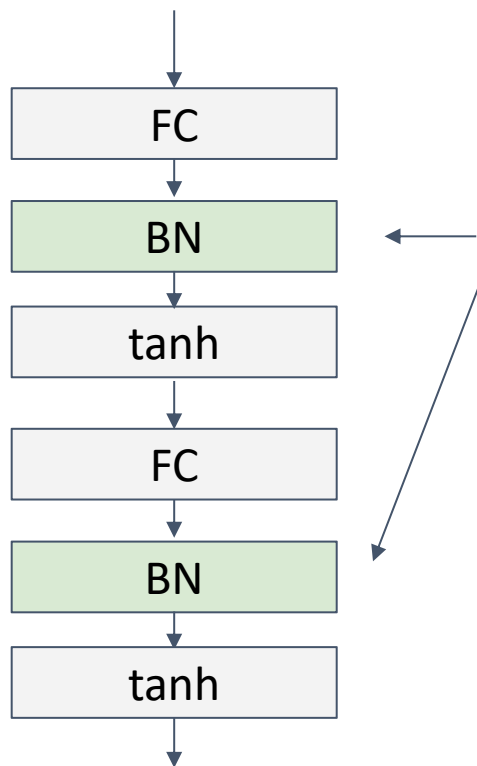


$$\boldsymbol{\mu}, \boldsymbol{\sigma} : 1 \times \mathbf{C} \times 1 \times 1$$

$$\boldsymbol{\gamma}, \boldsymbol{\beta} : 1 \times \mathbf{C} \times 1 \times 1$$

$$\mathbf{y} = \boldsymbol{\gamma} (\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta}$$

批归一化

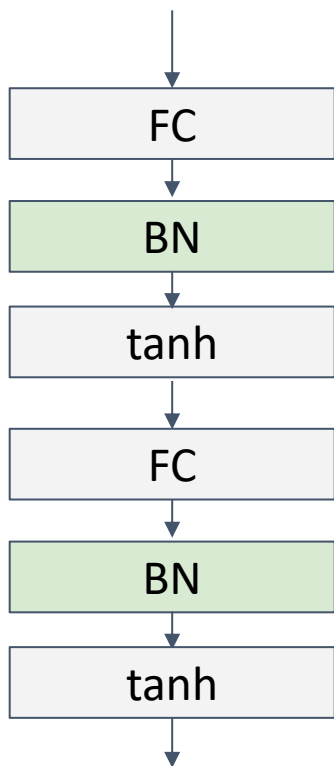


通常插在全连接层或卷积层之后，非线性层之前。

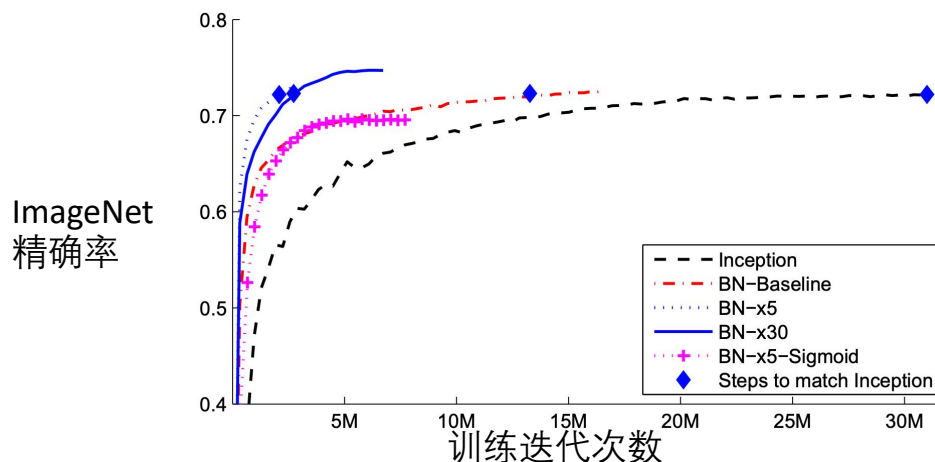
$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

批归一化

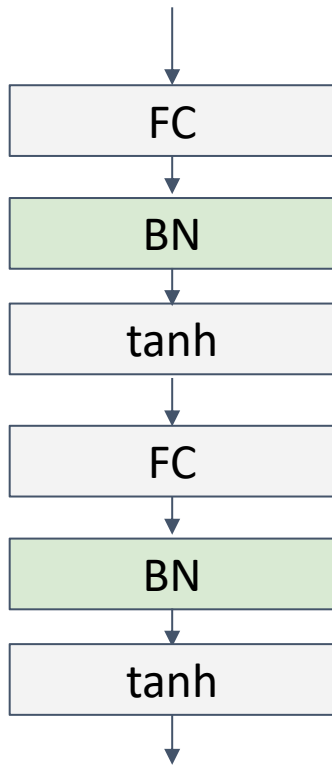


- 使得深度网络更容易训练!
- 允许高的学习速率, 快速收敛
- 网络初始化更鲁棒
- 类似于训练过程中的正则化
- 测试过程中没有开销: 与 conv 融为一体!



Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

批归一化



- 使得深度网络更容易训练!
- 允许高的学习速率, 快速收敛
- 网络初始化更鲁棒
- 类似于训练过程中的正则化
- 测试过程中没有开销: 与 conv 融为一体!
- 没有易于理解的理论
- 训练和测试过程中操作不一样: 是一个bugs操作!

Ioffe and Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift", ICML 2015

层归一化

fully-connected 网络的批归一化

$$\mathbf{x} : \mathbf{N} \times \mathbf{D}$$

归一化



$$\boldsymbol{\mu}, \boldsymbol{\sigma} : 1 \times \mathbf{D}$$

$$\boldsymbol{\gamma}, \boldsymbol{\beta} : 1 \times \mathbf{D}$$

$$\mathbf{y} = \boldsymbol{\gamma}(\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta}$$

fully-connected 网络的层归一化
训练和测试过程相同!
在RNNs, Transformers中使用

$$\mathbf{x} : \mathbf{N} \times \mathbf{D}$$

归一化



$$\boldsymbol{\mu}, \boldsymbol{\sigma} : \mathbf{N} \times 1$$

$$\boldsymbol{\gamma}, \boldsymbol{\beta} : 1 \times \mathbf{D}$$

$$\mathbf{y} = \boldsymbol{\gamma}(\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta}$$

Ba, Kiros, and Hinton, "Layer Normalization", arXiv 2016

实例归一化

convolutional网络的批
归一化

$\mathbf{x} : \mathbf{N} \times \mathbf{C} \times \mathbf{H} \times \mathbf{W}$

归一化



$\boldsymbol{\mu}, \boldsymbol{\sigma} : 1 \times \mathbf{C} \times 1 \times 1$

$\boldsymbol{\gamma}, \boldsymbol{\beta} : 1 \times \mathbf{C} \times 1 \times 1$

$$\mathbf{y} = \boldsymbol{\gamma} (\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta}$$

convolutional网络的实例归
一化
训练和测试过程相同!

$\mathbf{x} : \mathbf{N} \times \mathbf{C} \times \mathbf{H} \times \mathbf{W}$

归一化



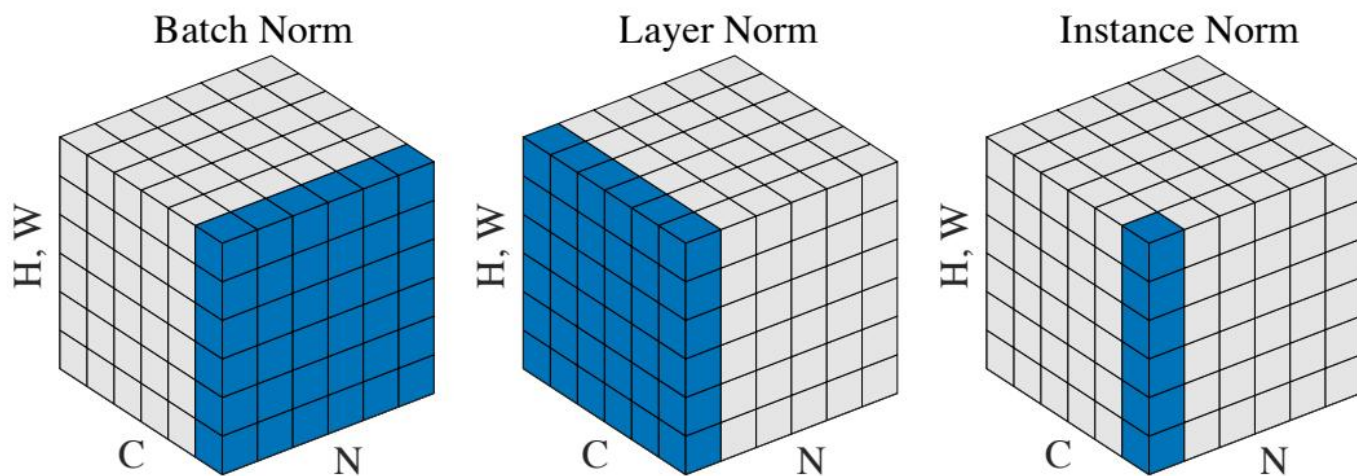
$\boldsymbol{\mu}, \boldsymbol{\sigma} : \mathbf{N} \times \mathbf{C} \times 1 \times 1$

$\boldsymbol{\gamma}, \boldsymbol{\beta} : 1 \times \mathbf{C} \times 1 \times 1$

$$\mathbf{y} = \boldsymbol{\gamma} (\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta}$$

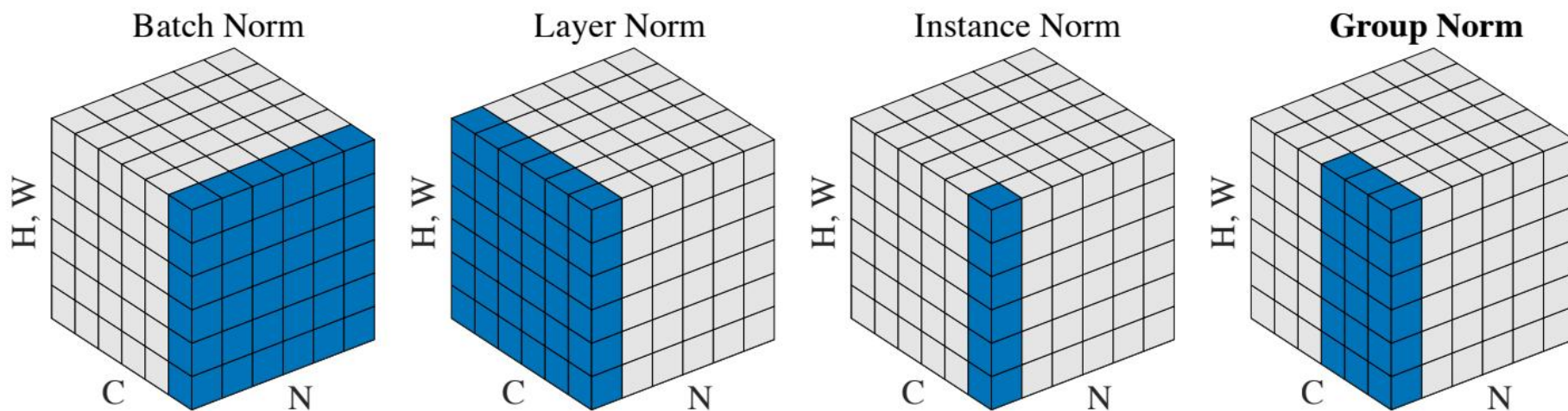
Ulyanov et al, Improved Texture Networks: Maximizing Quality and Diversity in Feed-forward Stylization and Texture Synthesis, CVPR 2017

归一化层的比较



Wu and He, "Group Normalization", ECCV 2018

组归一化



Wu and He, "Group Normalization", ECCV 2018

本章内容

- 6.2.1.卷积神经网络的组成
- 6.2.2.卷积层
- 6.2.3.池化与归一化
- 6.2.4.卷积神经网络的训练

总括

1. 直接设置

激活函数, 数据预处理, 权重初始化, 正则化

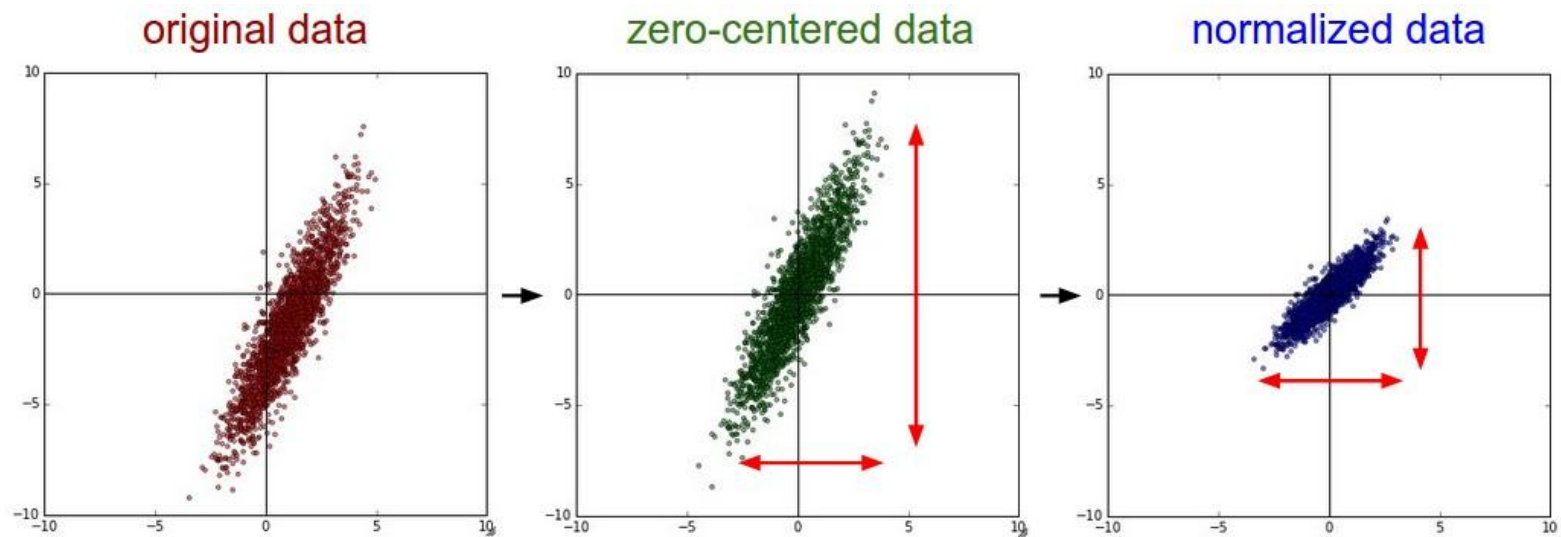
2. 训练过程中

学习速率计划; 训练批次大小; 超参数

3. 训练后设置

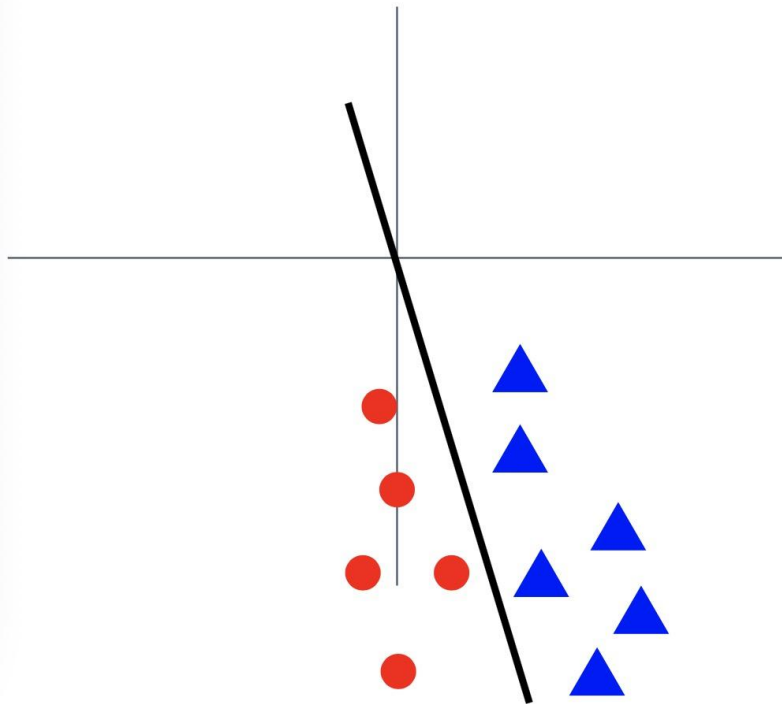
模型集成, 迁移学习

数据预处理

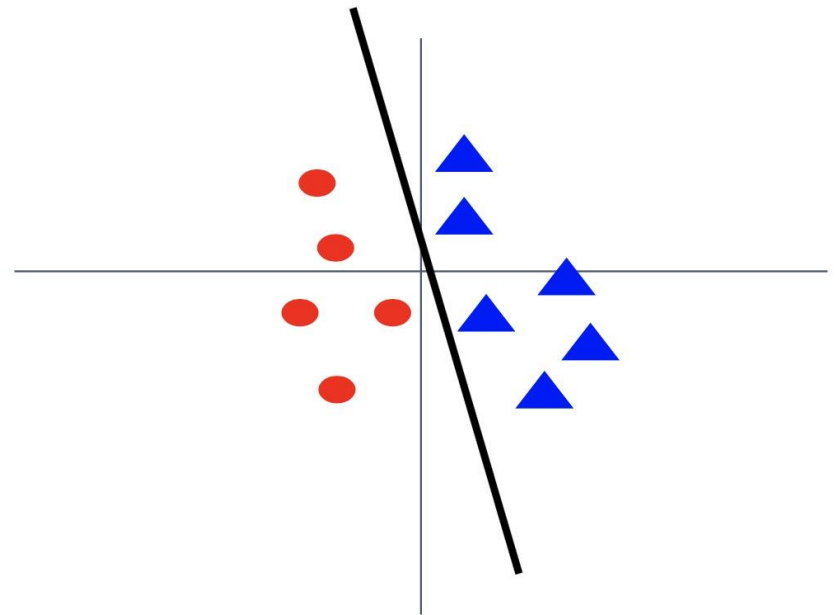


数据预处理

归一化之前：损失对权重矩阵变化敏感；难于优化



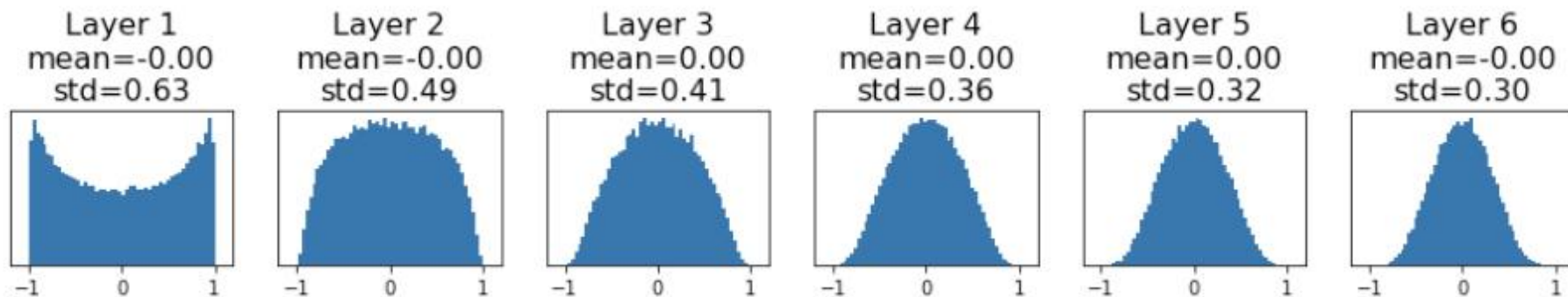
归一化之后：对权重矩阵变化敏感程度降低；易于优化



权重初始化

```
dims = [4096] * 7          "Xavier" 初始化:
hs = []                    std = 1/sqrt(Din)
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) / np.sqrt(Din)
    x = np.tanh(x.dot(W))
    hs.append(x)
```

$1/\sqrt{D_{in}}$ 目的: 使得初始情况下每层的权重分布相当!



Glorot and Bengio, "Understanding the difficulty of training deep feedforward neural networks", AISTAT 2010

权重初始化

输出方差 = 输入方差 推导

“Xavier” 初始化:
std = $1/\sqrt{D_{in}}$

$$y = Wx$$

$$y_i = \sum_{j=1}^{D_{in}} x_j w_j$$

$$\text{Var}(y_i) = D_{in} * \text{Var}(x_i w_i)$$

$$= D_{in} * (E[x_i^2]E[w_i^2] - E[x_i]^2 E[w_i]^2)$$

$$= D_{in} * \text{Var}(x_i) * \text{Var}(w_i)$$

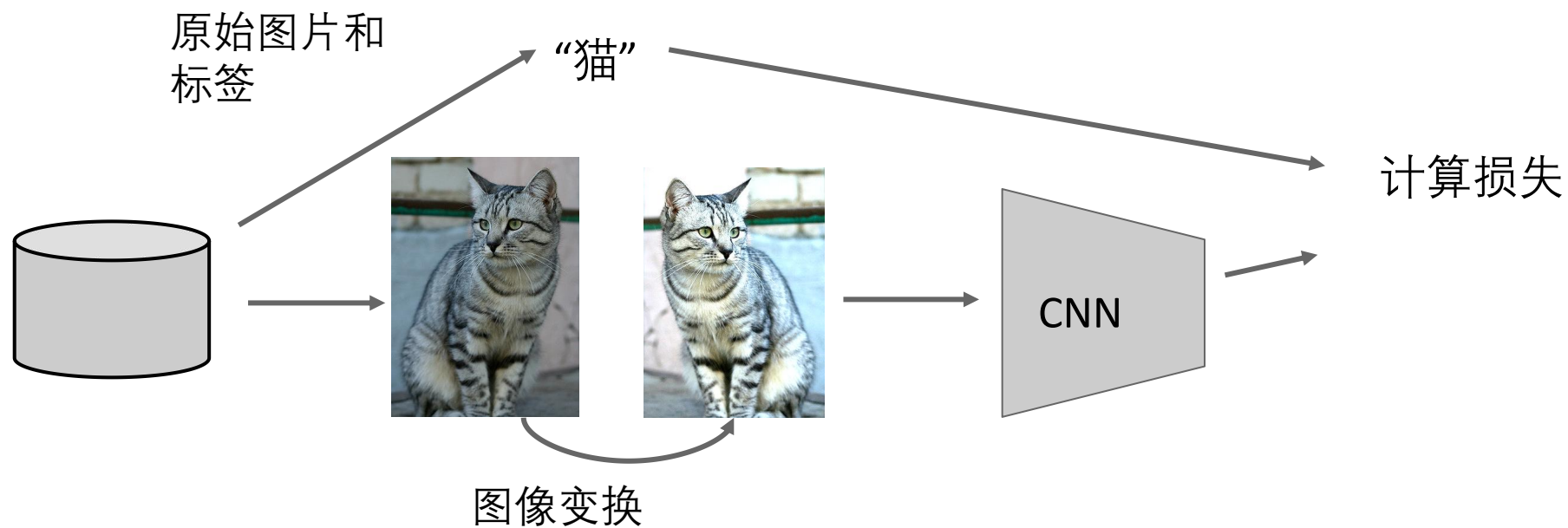
[Assume x, w are iid]

[Assume x, w independent]

[Assume x, w are zero-mean]

If $\text{Var}(w_i) = 1/D_{in}$ then $\text{Var}(y_i) = \text{Var}(x_i)$

数据增强

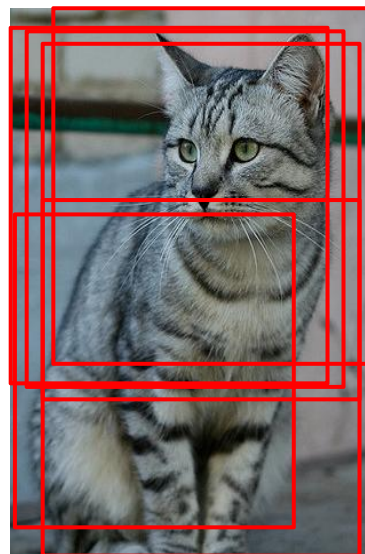


数据增强: 随机裁剪和缩放

训练: 随机裁剪和缩放

ResNet:

1. 在 $[256, 480]$ 中随机选择 L 个数
2. 调整训练图像尺寸, 短边 = L
3. 随机 224×224 块抽样



正则化

训练: 增加随机性

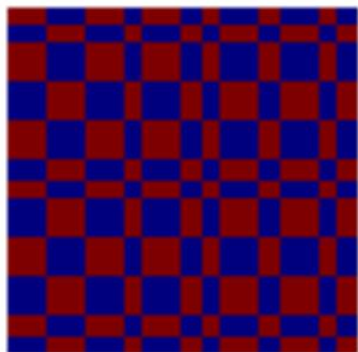
测试: 边缘化随机性

举例:

批归一化

数据增强

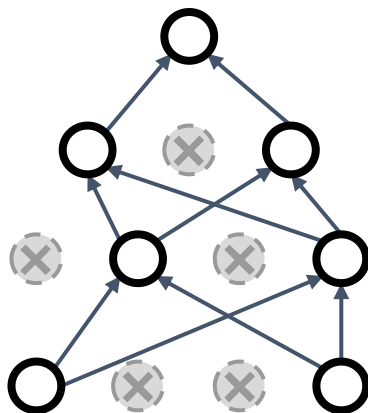
分数池化



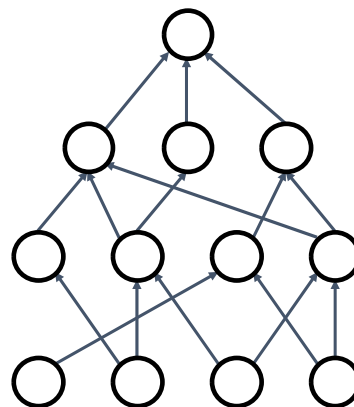
Cutout



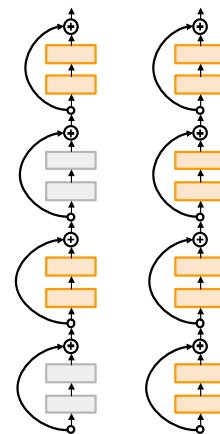
Dropout



DropConnect



随机深度



Mixup



(旧形式) 正则化: 损失函数增加正则项

$$L = \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, f(x_i; W)_j - f(x_i; W)_{y_i} + 1) + \lambda R(W)$$

常用:

L2 正则项

$$R(W) = \sum_k \sum_l W_{k,l}^2 \quad (\text{权重惩罚})$$

L1 正则

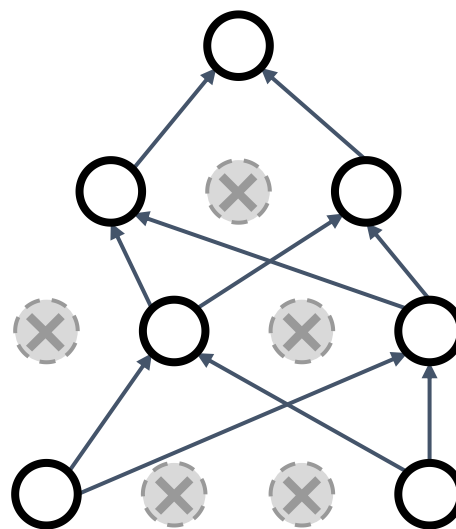
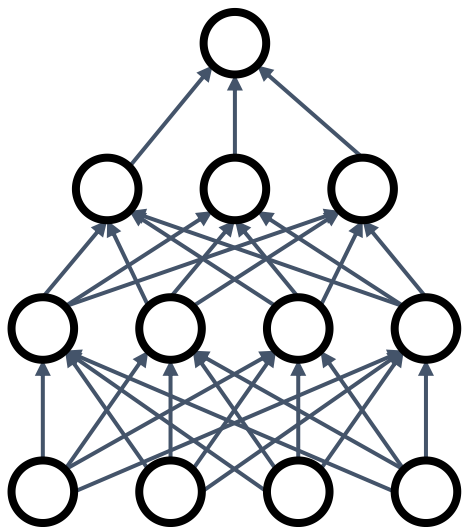
$$R(W) = \sum_k \sum_l |W_{k,l}|$$

弹性组合(L1 + L2)

$$R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$$

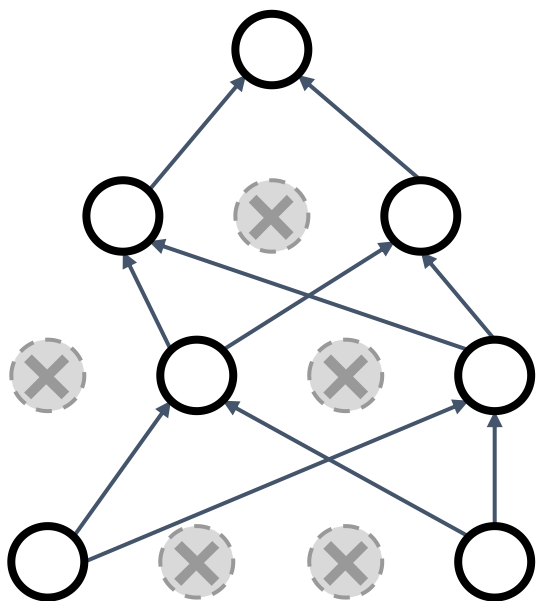
正则化: Dropout

在每个前向传递过程中，随机设置一些神经元的值为0
Dropout比例为超参数；通常设置为0.5



Srivastava et al, "Dropout: A simple way to prevent neural networks from overfitting", JMLR 2014

正则化: Dropout



另外一种解释:

Dropout 训练过程中多个模型的集成

每个节点二进制掩膜对应一个模型

一个有4096个输出单元的FC 层有

$2^{4096} \sim 10^{1233}$ 个可能的掩膜!

一共有 $\sim 10^{82}$ 个组合

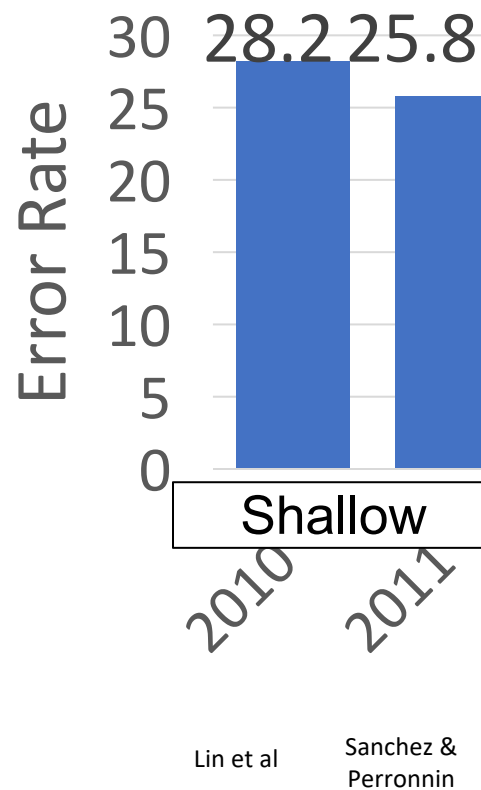
本章内容

- 6.1.神经网络
- 6.2.卷积神经网络
- 6.3.神经网络图像分类

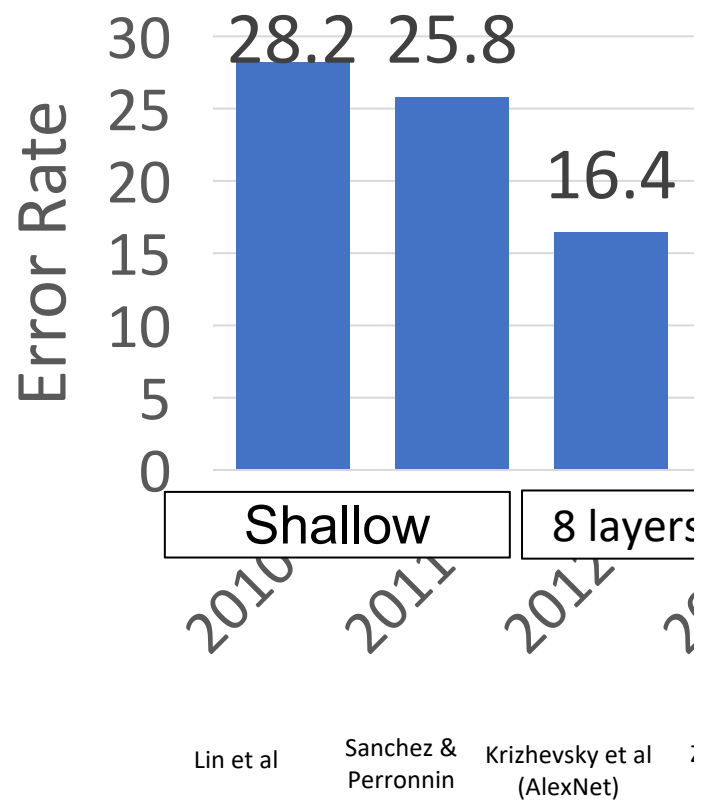
本章内容

- 6.3.1.ImageNet分类挑战
- 6.3.2.AlexNet
- 6.3.3.VGG
- 6.3.4.GoogleNet
- 6.3.5.Residual Network
- 6.3.6.不同网络比较及改进

ImageNet分类挑战



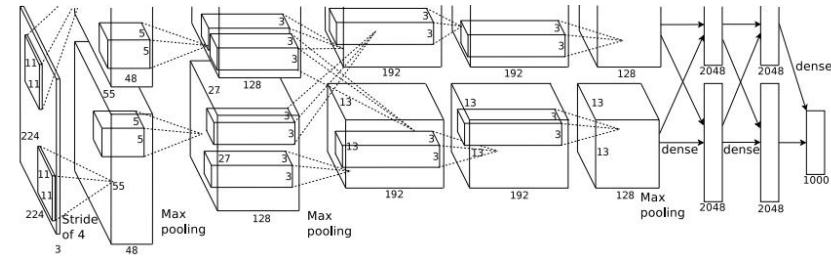
ImageNet分类挑战



本章内容

- 6.3.1.ImageNet分类挑战
- 6.3.2.AlexNet
- 6.3.3.VGG
- 6.3.4.GoogleNet
- 6.3.5.Residual Network
- 6.3.6.不同网络比较及改进

9.1 AlexNet



227 x 227 输入

5 卷积层

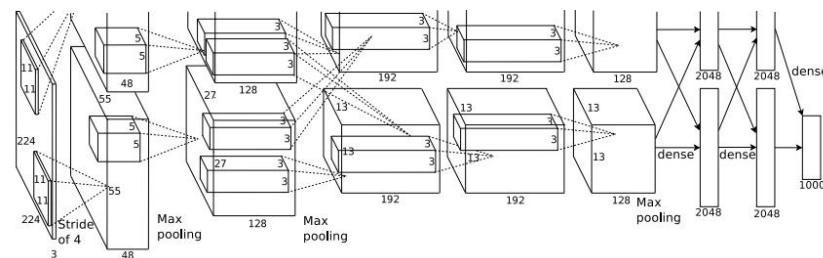
最大池化

3 全连接层

ReLU 激活

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet



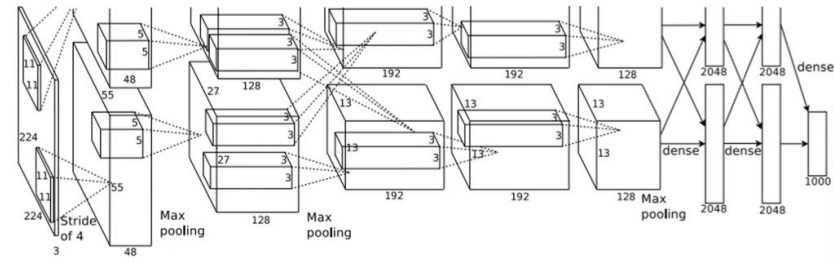
227 x 227 输入
5 卷积层
最大池化
3 全连接层
ReLU 激活

使用“局部响应归一化”；
目前已经不使用

使用两块GTX 580 GPUs 训练— 仅仅
3GB 存储空间! 模型分割到两块
GPUs

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

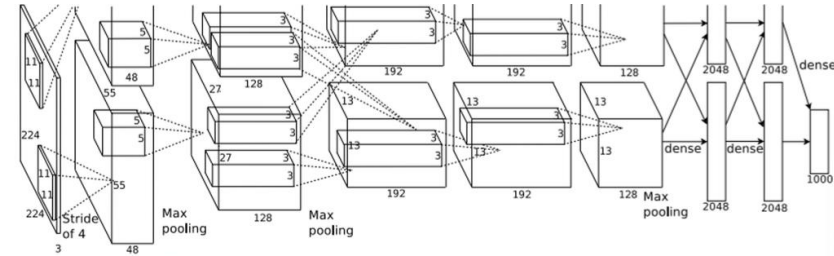
AlexNet



Layer	Input size		Layer				Output size	
	C	H / W	filters	kernel	stride	pad	C	H / W
conv1	3	227	64	11	4	2	?	

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet

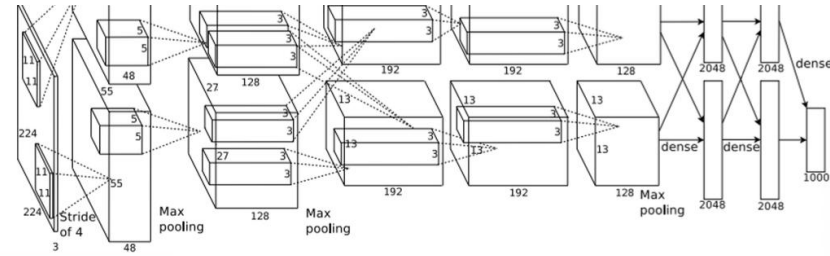


Layer	Input size		Layer				Output size	
	C	H / W	filters	kernel	stride	pad	C	H / W
conv1	3	227	64	11	4	2	64	?

Recall: Output channels = number of filters

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet

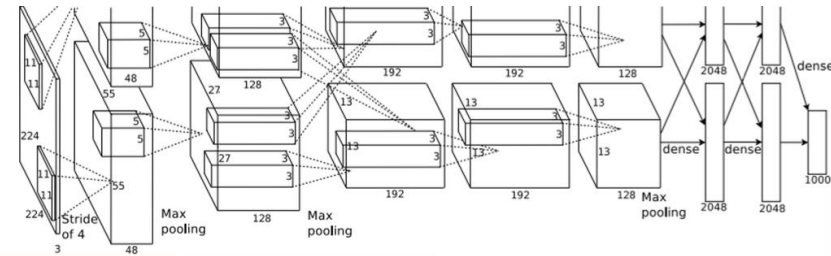


Layer	Input size		Layer				Output size	
	C	H / W	filters	kernel	stride	pad	C	H / W
conv1	3	227	64	11	4	2	64	56

$$\begin{aligned}
 \text{Recall: } W' &= (W - K + 2P) / S + 1 \\
 &= (227 - 11 + 2 \cdot 2) / 4 + 1 \\
 &= 220 / 4 + 1 = 56
 \end{aligned}$$

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

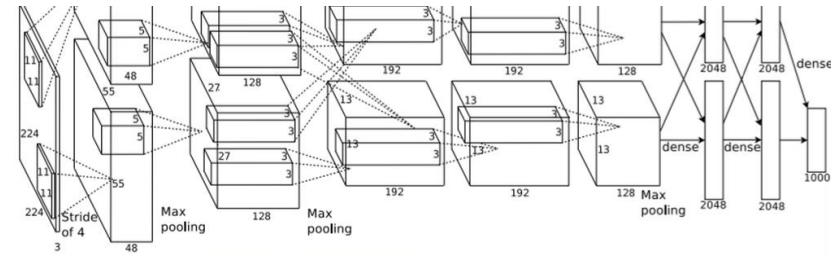
AlexNet



Layer	Input size		Layer				Output size		memory (KB)
	C	H / W	filters	kernel	stride	pad	C	H / W	
conv1	3	227	64	11	4	2	64	56	?

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet



Layer	Input size		Layer				Output size		
	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)
conv1	3	227	64	11	4	2	64	56	784

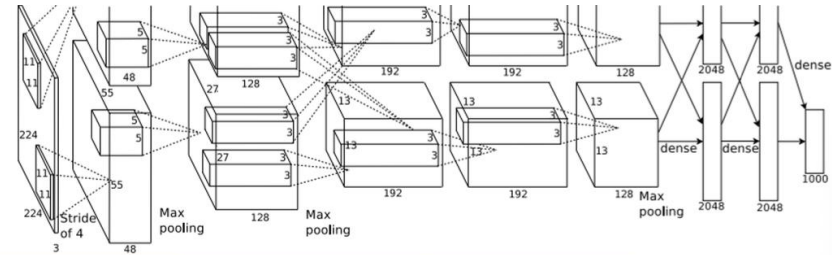
$$\begin{aligned}\text{Number of output elements} &= C * H' * W' \\ &= 64 * 56 * 56 = 200,704\end{aligned}$$

Bytes per element = 4 (for 32-bit floating point)

$$\begin{aligned}\text{KB} &= (\text{number of elements}) * (\text{bytes per elem}) / 1024 \\ &= 200704 * 4 / 1024 \\ &= \mathbf{784}\end{aligned}$$

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

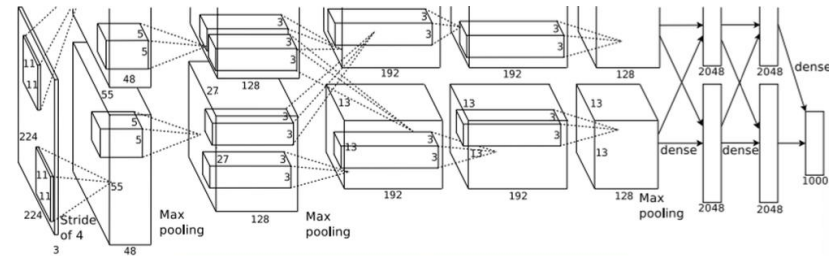
AlexNet



Layer	Input size			Layer				Output size			memory (KB)	params (k)
	C	H / W		filters	kernel	stride	pad	C	H / W			
conv1	3	227		64	11	4	2	64	56		784	?

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet



Layer	Input size		Layer				Output size			
	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)
conv1	3	227	64	11	4	2	64	56	784	23

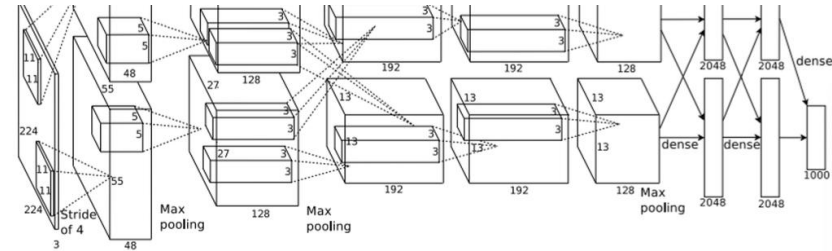
$$\begin{aligned}\text{Weight shape} &= C_{\text{out}} \times C_{\text{in}} \times K \times K \\ &= 64 \times 3 \times 11 \times 11\end{aligned}$$

$$\text{Bias shape} = C_{\text{out}} = 64$$

$$\begin{aligned}\text{Number of weights} &= 64 \times 3 \times 11 \times 11 + 64 \\ &= \mathbf{23,296}\end{aligned}$$

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

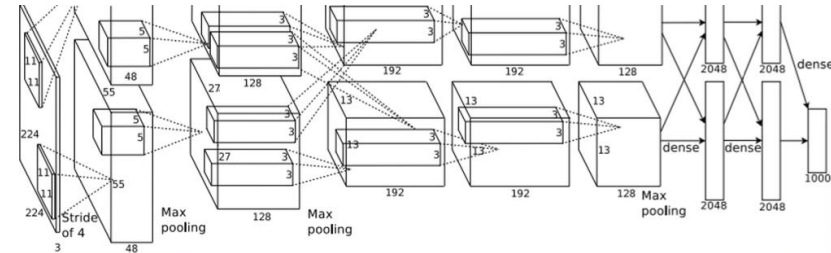
AlexNet



Layer	Input size		Layer				Output size			memory (KB)	params (k)	flop (M)
	C	H / W	filters	kernel	stride	pad	C	H / W				
conv1	3	227	64	11	4	2	64	56		784	23	?

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

AlexNet



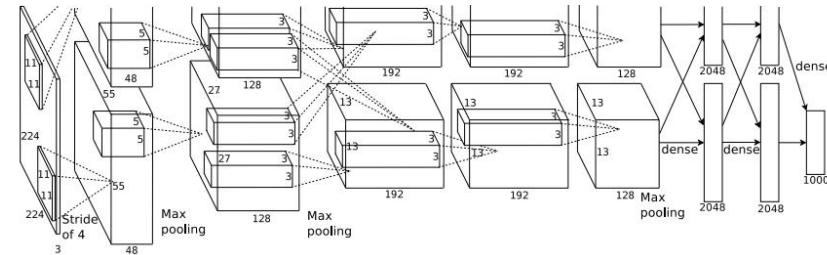
Layer	Input size		Layer				Output size				
	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)
conv1	3	227	64	11	4	2	64	56	784	23	73

Number of floating point operations (multiply+add)
 = (number of output elements) * (ops per output elem)
 = $(C_{out} \times H' \times W') * (C_{in} \times K \times K)$
 = $(64 * 56 * 56) * (3 * 11 * 11)$
 = $200,704 * 363$
 = **72,855,552**

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

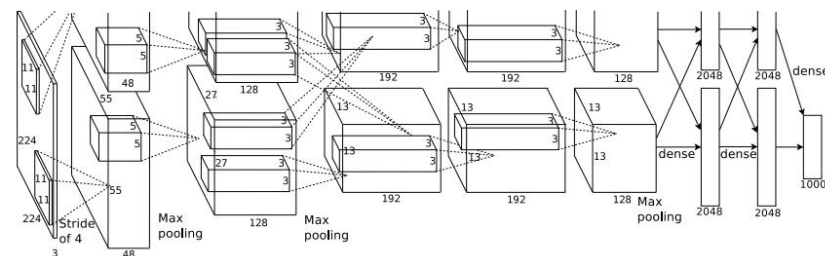
AlexNet

注意变化趋势!



	Input size		Layer				Output size				
Layer	C	H / W	filters	kernel	stride	pad	C	H / W	memory (KB)	params (k)	flop (M)
conv1	3	227	64	11	4	2	64	56	784	23	73
pool1	64	56		3	2	0	64	27	182	0	0
conv2	64	27	192	5	1	2	192	27	547	307	224
pool2	192	27		3	2	0	192	13	127	0	0
conv3	192	13	384	3	1	1	384	13	254	664	112
conv4	384	13	256	3	1	1	256	13	169	885	145
conv5	256	13	256	3	1	1	256	13	169	590	100
pool5	256	13		3	2	0	256	6	36	0	0
flatten	256	6					9216		36	0	0
fc6	9216		4096				4096		16	37,749	38
fc7	4096		4096				4096		16	16,777	17
fc8	4096		1000				1000		4	4,096	4

AlexNet

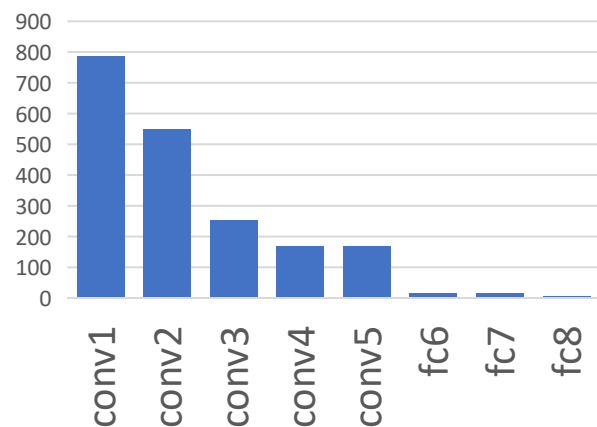


存储主要在卷积层

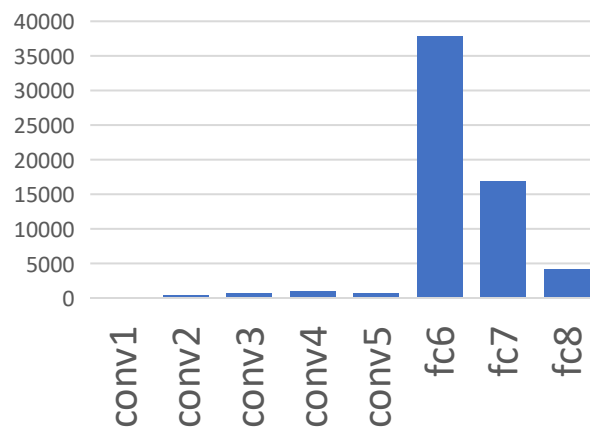
参数主要在全连接层

运算主要在卷积层

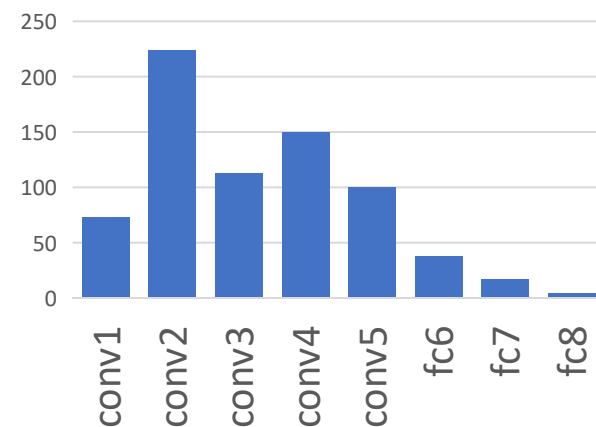
Memory (KB)



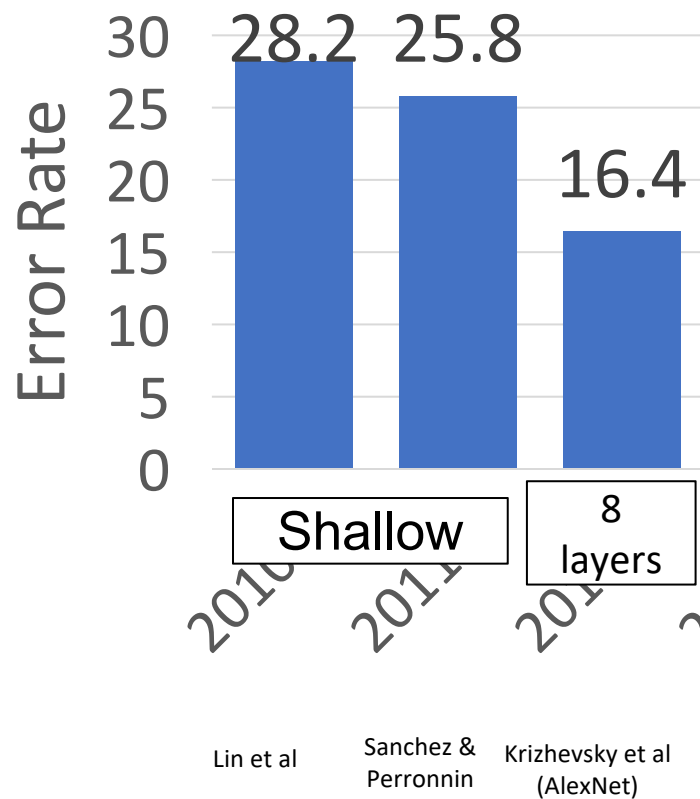
Params (K)



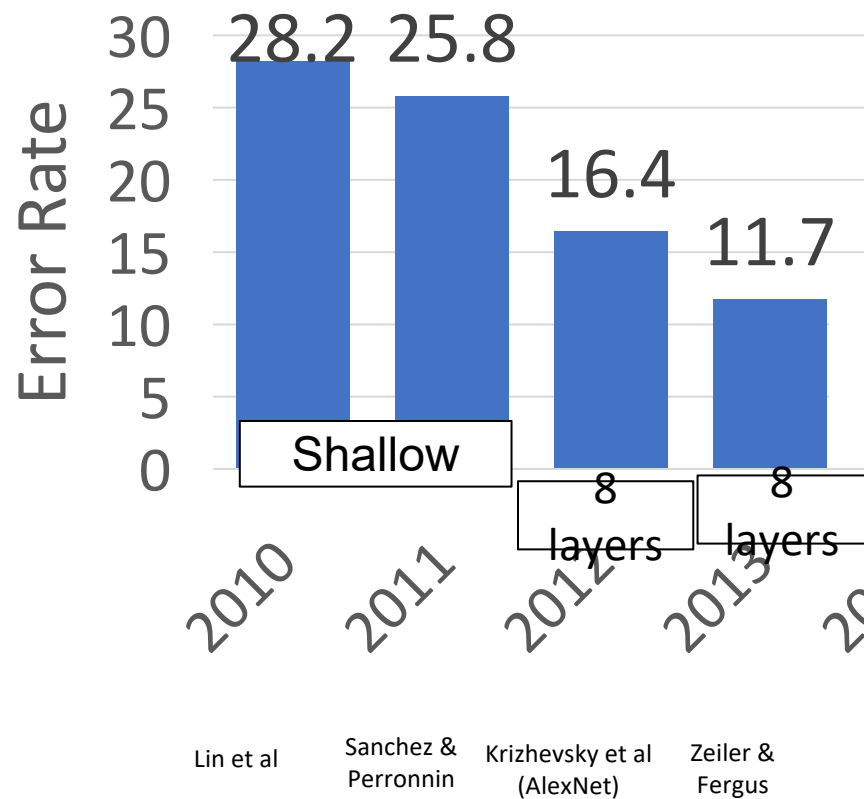
MFLOP



ImageNet分类挑战

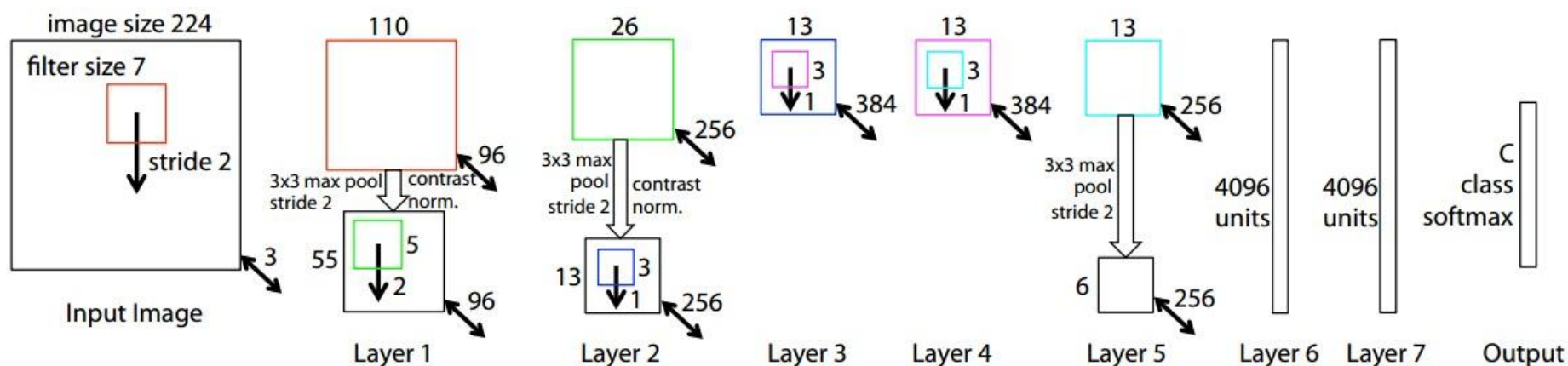


ImageNet分类挑战



ZFNet: 一个大型AlexNet

ImageNet top 5 错误: 16.4% -> 11.7%



AlexNet 但:

CONV1: 从 (11x11 stride 4) 修改为 (7x7 stride 2)

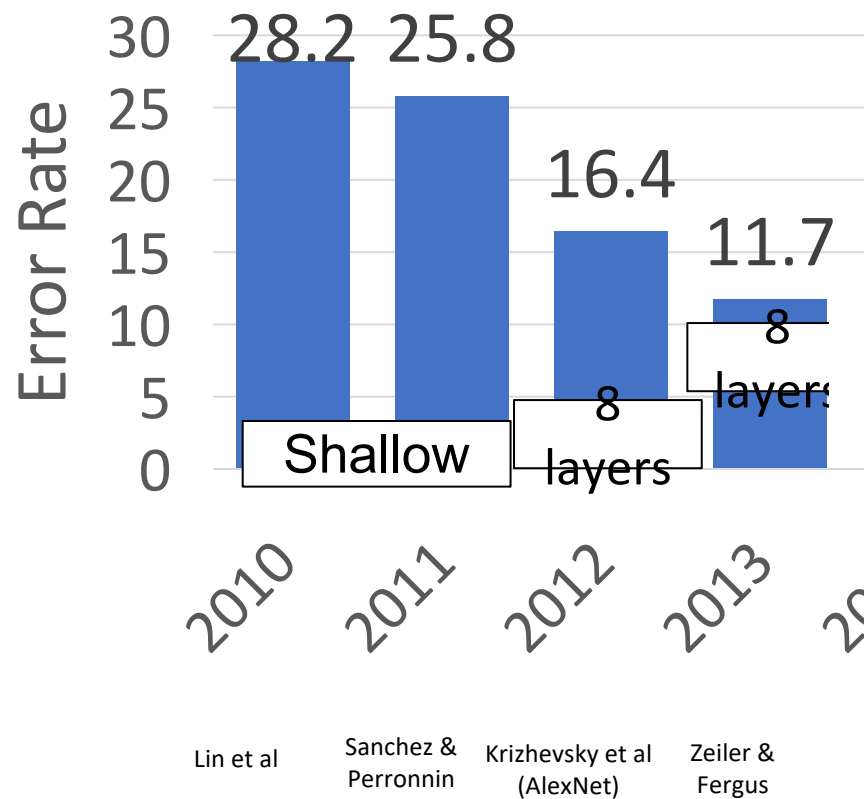
CONV3,4,5: 使用512, 1024, 512替换384, 384, 256滤波核

Zeiler and Fergus, "Visualizing and Understanding Convolutional Networks", ECCV 2014

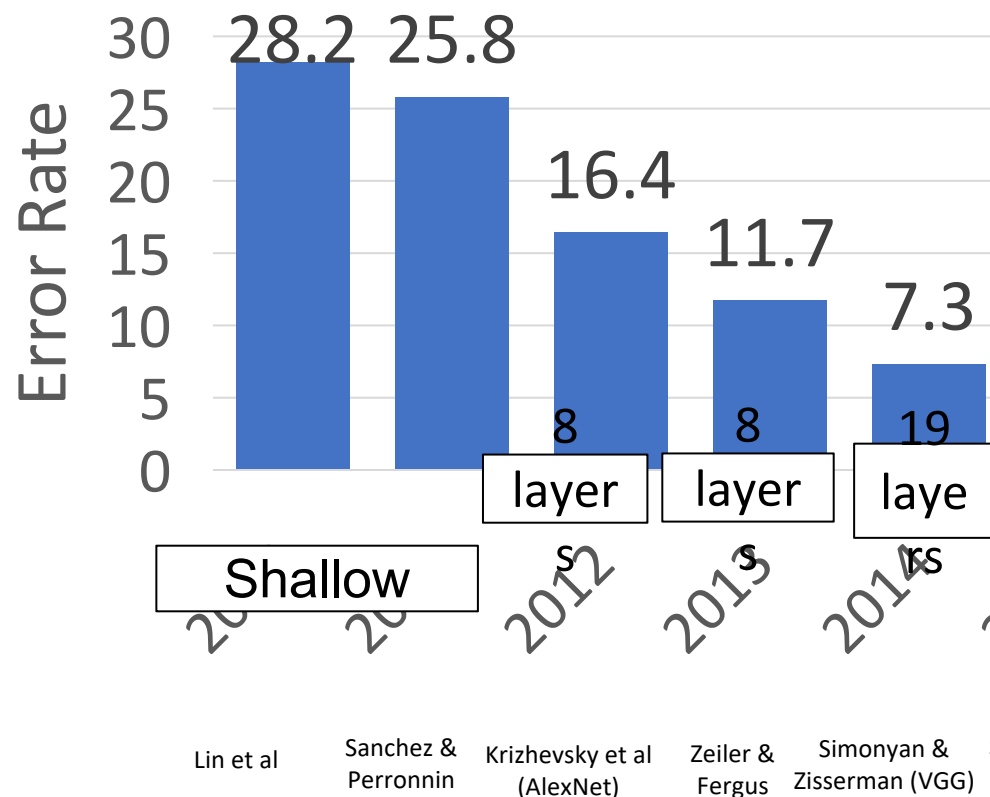
本章内容

- 6.3.1.ImageNet分类挑战
- 6.3.2.AlexNet
- 6.3.3.VGG
- 6.3.4.GoogleNet
- 6.3.5.Residual Network
- 6.3.6.不同网络比较及改进

ImageNet分类挑战



ImageNet分类挑战



VGG: 更深的网络

VGG 设计规则:

所有conv 使用 3x3 stride 1 pad 1

所有 max 池化使用 2x2 stride 2

池化后, 双倍输出通道

网络包含5个卷积段:

阶段 1: conv-conv-pool

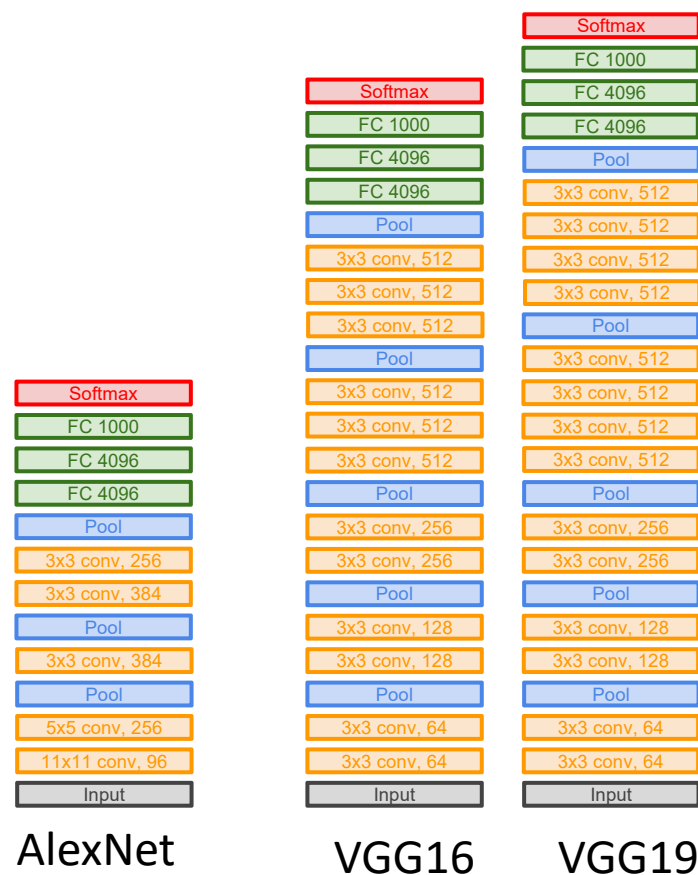
阶段 2: conv-conv-pool

阶段 3: conv-conv-pool

阶段 4: conv-conv-conv-[conv]-pool

阶段 5: conv-conv-conv-[conv]-pool

(VGG-19在阶段4和5有4conv)



Simonyan and Zissermann, "Very Deep Convolutional Networks for Large-Scale Image Recognition", ICLR 2015

VGG:更深的网络

VGG 设计规则:

所有conv 使用 3x3 stride 1 pad 1

所有 max 池化使用 2x2 stride 2

池化后, 双倍输出通道

选项1:

Conv(5x5, C -> C)

Params: $25C^2$

FLOPs: $25C^2HW$

选项2:

Conv(3x3, C -> C)

Conv(3x3, C -> C)

Params: $18C^2$

FLOPs: $18C^2HW$

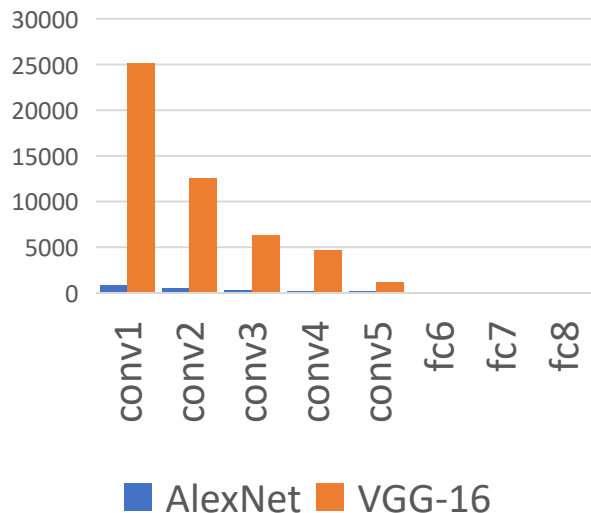
两个 3x3 conv 和一个 5x5 conv 有相同的感受野, 但参数和计算都变少



Simonyan and Zissermann, "Very Deep Convolutional Networks for Large-Scale Image Recognition", ICLR 2015

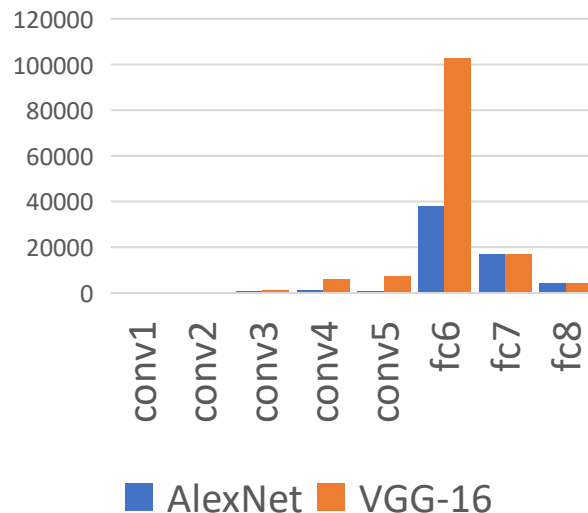
AlexNet vs VGG-16!

AlexNet vs VGG-16
(Memory, KB)



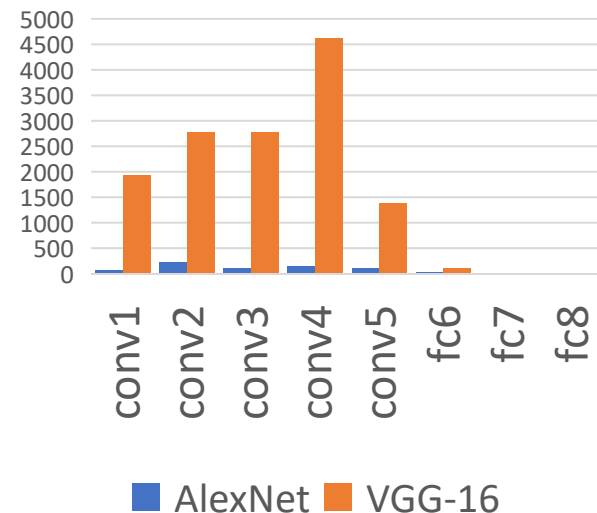
AlexNet 总共: 1.9 MB
VGG-16 总共: 48.6 MB (25x)

AlexNet vs VGG-16
(Params, M)



AlexNet 总共: 61M
VGG-16 总共: 138M (2.3x)

AlexNet vs VGG-16
(MFLOPs)

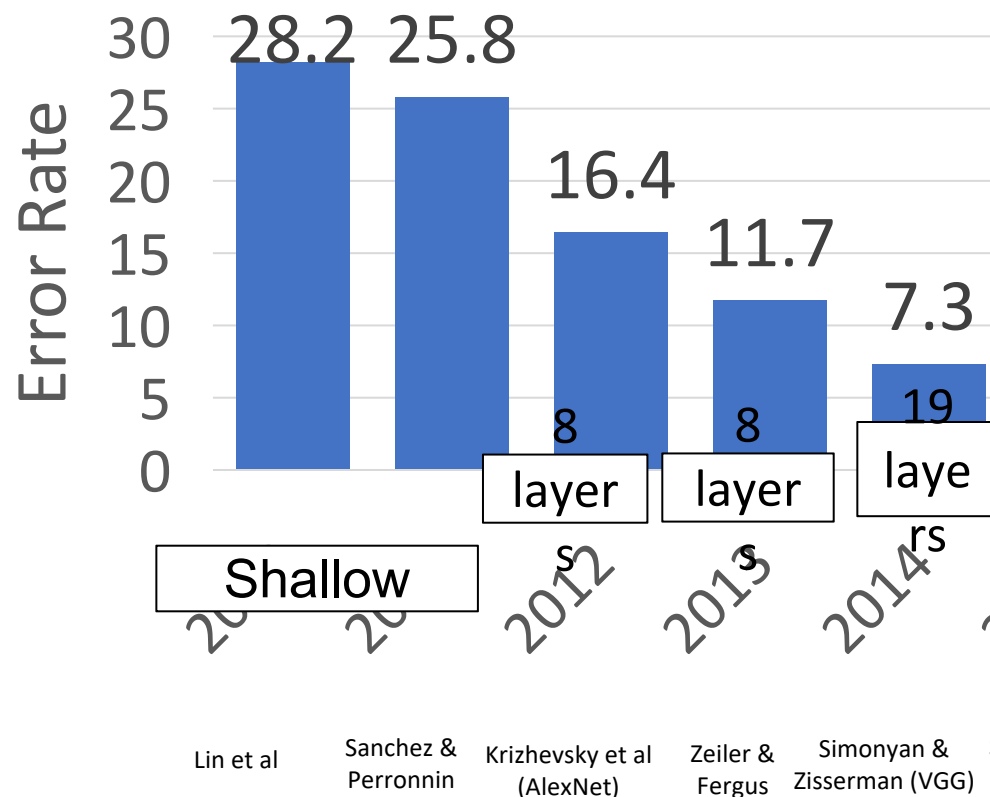


AlexNet 总共: 0.7 GFLOP
VGG-16 总共: 13.6 GFLOP (19.4x)

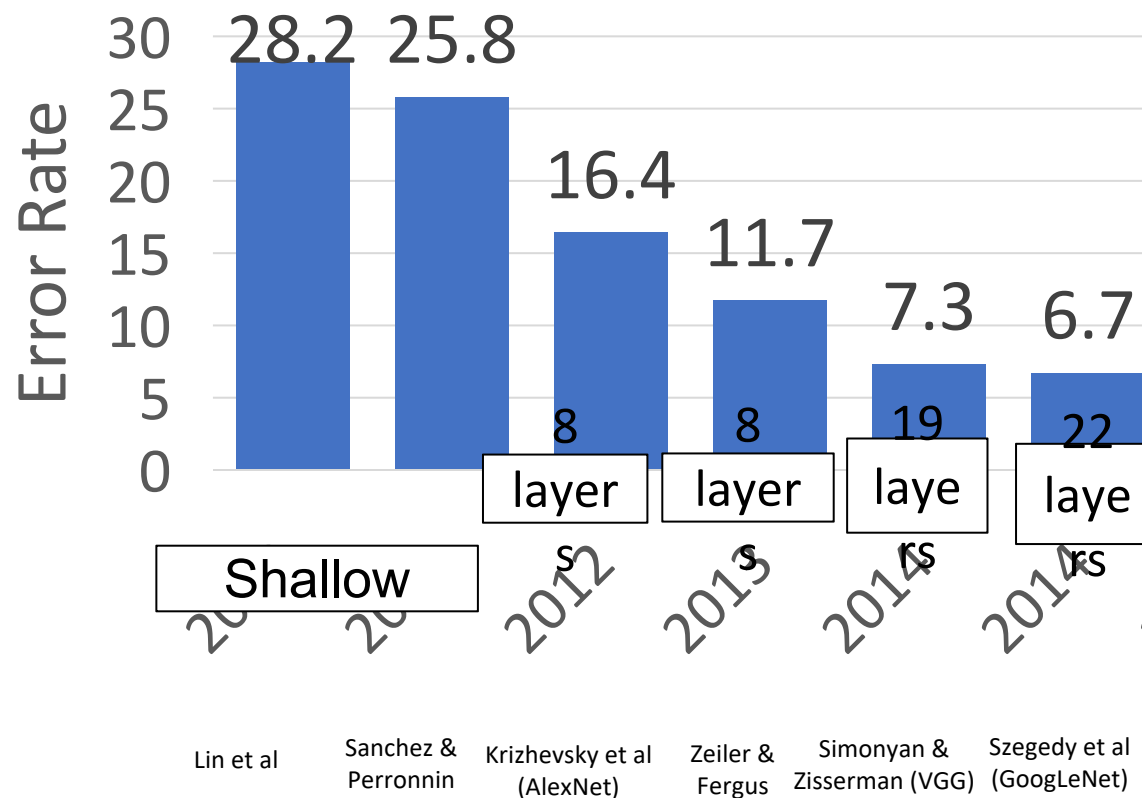
本章内容

- 6.3.1.ImageNet分类挑战
- 6.3.2.AlexNet
- 6.3.3.VGG
- 6.3.4.GoogleNet
- 6.3.5.Residual Network
- 6.3.6.不同网络比较及改进

ImageNet分类挑战

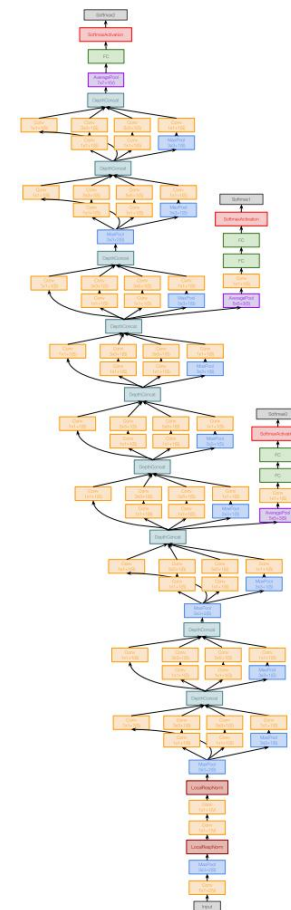


ImageNet分类挑战



9.3 GoogLeNet: 聚焦在效率提升

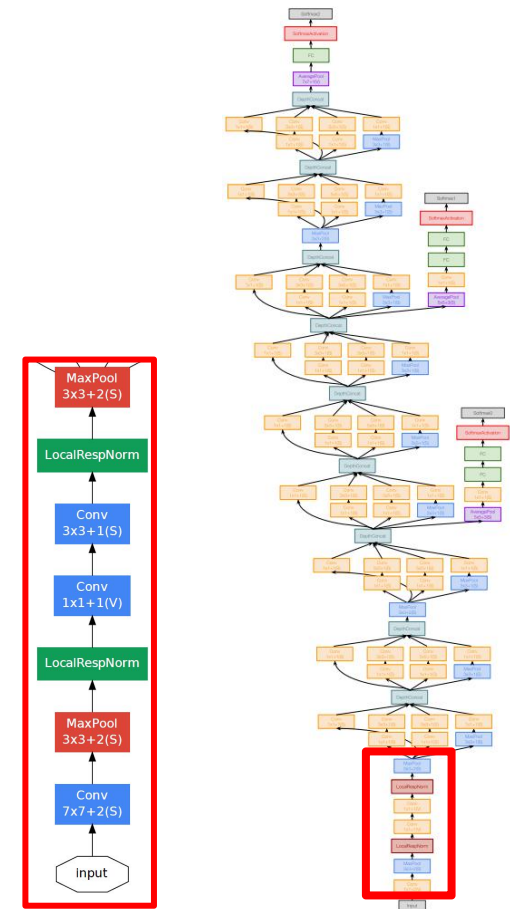
主要创新点在于效率提升: 减少参数规模, 存储空间, 计算



Szegedy et al, "Going deeper with convolutions", CVPR 2015

GoogLeNet: 主干

主干网络 起始位置快速下采样
(回想VGG-16: 大部分计算在起始层)



Szegedy et al, "Going deeper with convolutions", CVPR 2015

GoogLeNet: 主干

主干网络 起始位置快速下采样
(回想VGG-16: 大部分计算在起始层)

Layer	Input size		Layer				Output size		memory (KB)	params (K)	flop (M)
	C	H / W	filters	kernel	stride	pad	C	H/W			
conv	3	224	64	7	2	3	64	112	3136	9	118
max-pool	64	112		3	2	1	64	56	784	0	2
conv	64	56	64	1	1	0	64	56	784	4	13
conv	64	56	192	3	1	1	192	56	2352	111	347
max-pool	192	56		3	2	1	192	28	588	0	1

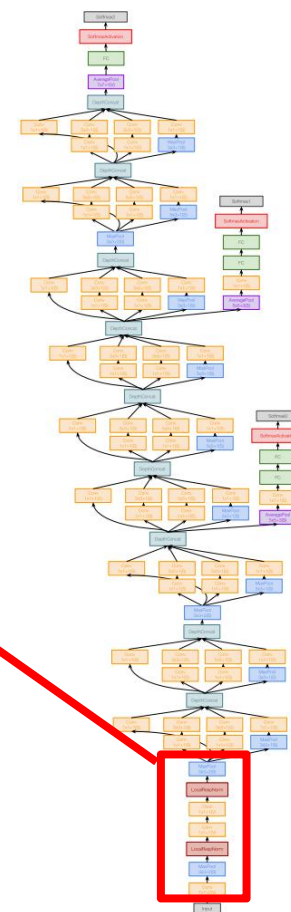
空间分辨率从224 到 28 :

Memory: 7.5 MB

Params: 124K

MFLOP: 418

Szegedy et al, "Going deeper with convolutions", CVPR 2015



GoogLeNet:主干

主干网络 起始位置快速下采样
(回想VGG-16: 大部分计算在起始层)

Layer	Input size		Layer				Output size		memory (KB)	params (K)	flop (M)
	C	H / W	filters	kerne	stride	pad	C	H/W			
conv	3	224	64	7	2	3	64	112	3136	9	118
max-pool	64	112		3	2	1	64	56	784	0	2
conv	64	56	64	1	1	0	64	56	784	4	13
conv	64	56	192	3	1	1	192	56	2352	111	347
max-pool	192	56		3	2	1	192	28	588	0	1

空间分辨率从224 到 28 :

Memory: 7.5 MB

Params: 124K

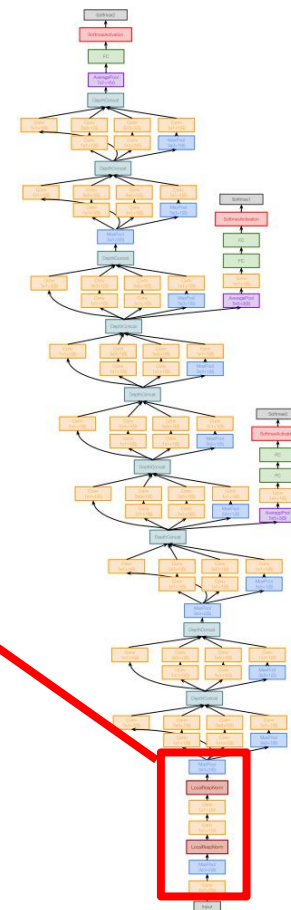
MFLOP: 418

与VGG-16比较:

Memory: 42.9 MB (5.7x)

Params: 1.1M (8.9x)

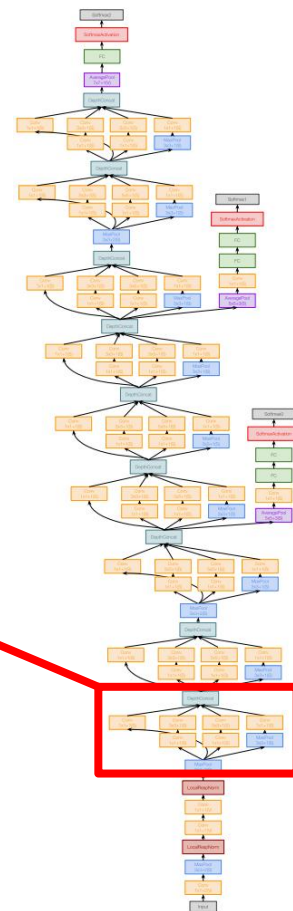
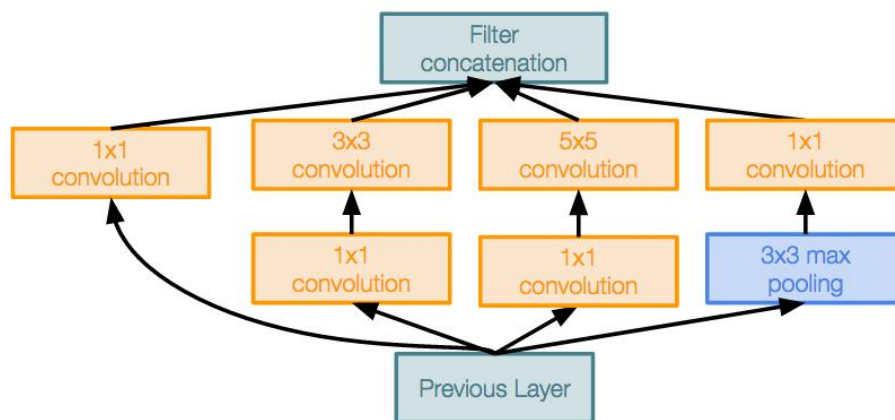
MFLOP: 7485 (17.8x)



GoogLeNet: Inception模块

Inception 模块
带并行分支的局部单元

整个网络中局部结构重复多次



Szegedy et al, "Going deeper with convolutions", CVPR 2015

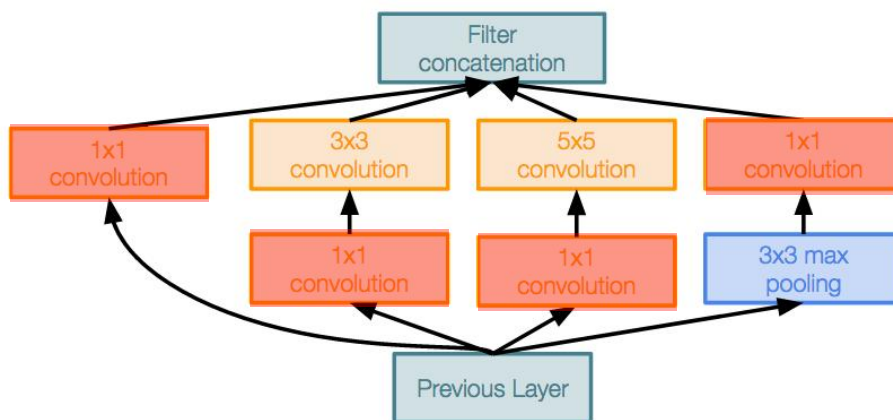
GoogLeNet: Inception模块

Inception 模块

带并行分支的局部单元

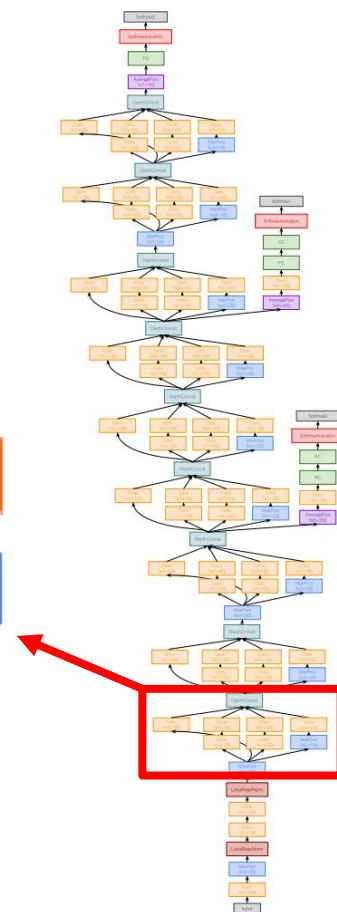
整个网络中局部结构重复多次

使用 1x1 “Bottleneck”
层减少输入conv之前
通道数



涉及到分离滤波器

Perona, “Deformable kernels for early vision”. PAMI 1995

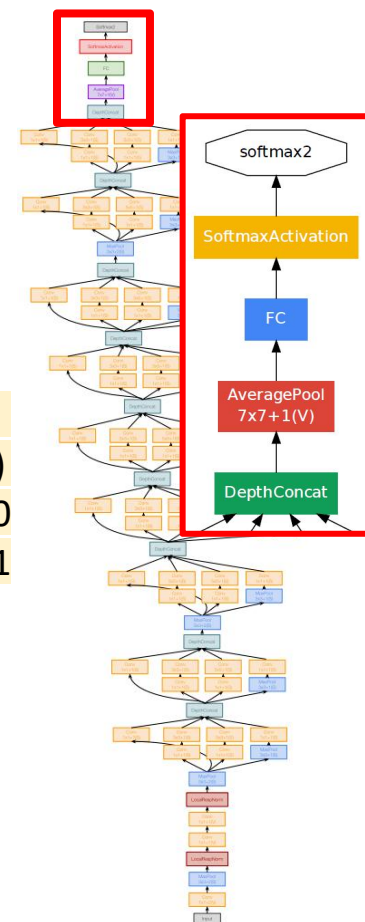


Szegedy et al, “Going deeper with convolutions”, CVPR 2015

GoogLeNet: 全局平均池化

末端没有全连接层! 使用**global average pooling** 坍塌空间尺寸, 最后有一个线性层计算输出类别响应
(回想VGG-16: 大部分参数在FC层!)

Layer	Input size		Layer				Output size		memory (KB)	params (k)	flop (M)
	C	H/W	filters	kernel	stride	pad	C	H/W			
avg-pool	1024	7		7	1	0	1024	1	4	0	0
fc	1024		1000				1000		0	1025	1



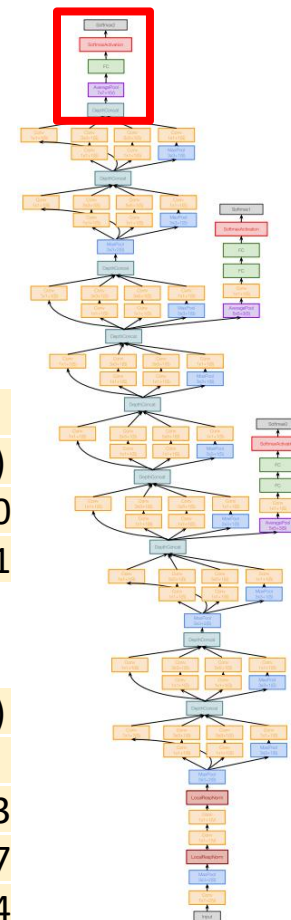
GoogLeNet:全局平均池化

末端没有全连接层! 使用**global average pooling** 坍塌空间尺寸, 最后有一个线性层计算输出类别响应
(回想VGG-16: 大部分参数在FC层!)

Layer	Input size		Layer				Output size		memory (KB)	params (k)	flop (M)
	C	H/W	filters	kernel	stride	pad	C	H/W			
avg-pool	1024	7		7	1	0	1024	1	4	0	0
fc	1024		1000				1000		0	1025	1

Compare with VGG-16:

Layer	C	H/W	filters	kernel	stride	pad	C	H/W	memory (KB)	params (K)	flop (M)
flatten	512	7					25088		98		
fc6	25088			4096			4096		16	102760	103
fc7	4096			4096			4096		16	16777	17
fc8	4096			1000			1000		4	4096	4

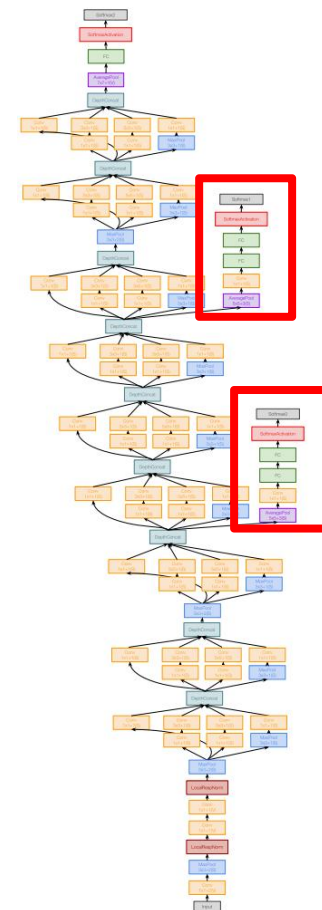


GoogLeNet: 辅助分类器

在末端使用损失函数进行训练性能不佳:
网络太深, 梯度传播失效

因此, 增加 “辅助分类层” 在网络的中间点

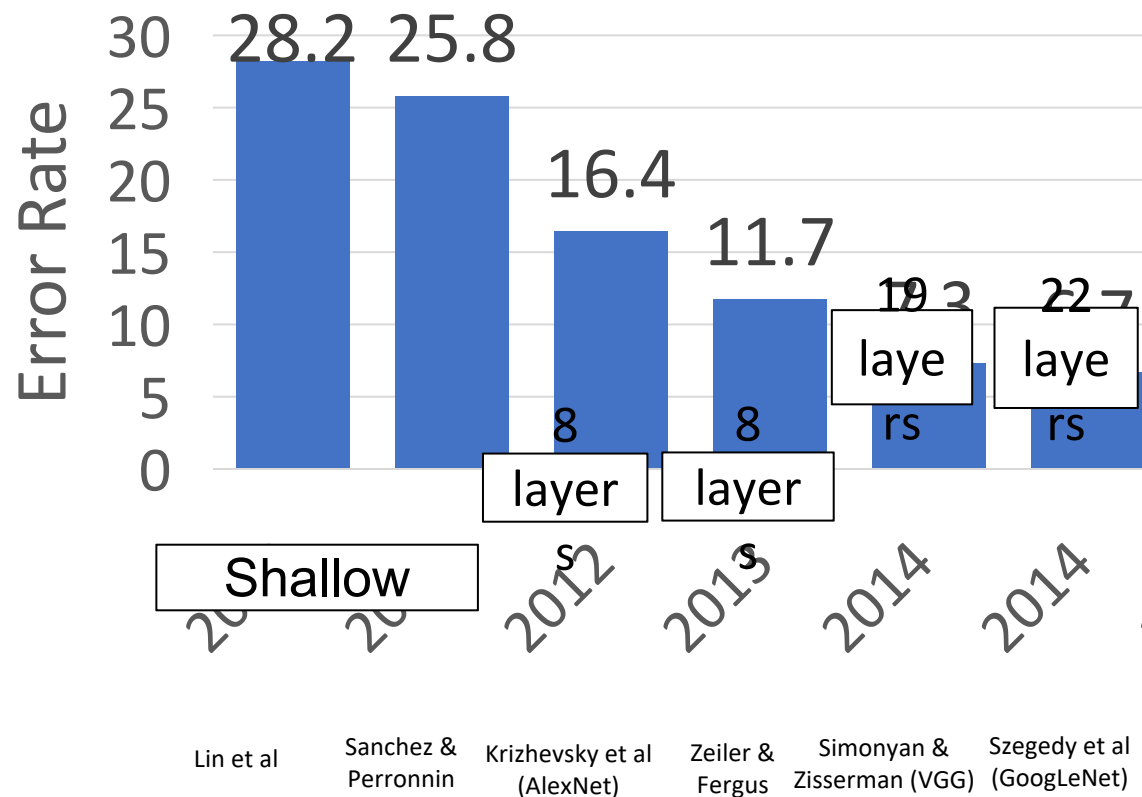
GoogLeNet 是在batch normalization (批归一化) 出现之前的网络!
有了BatchNorm 之后不需要在使用辅助分类层了



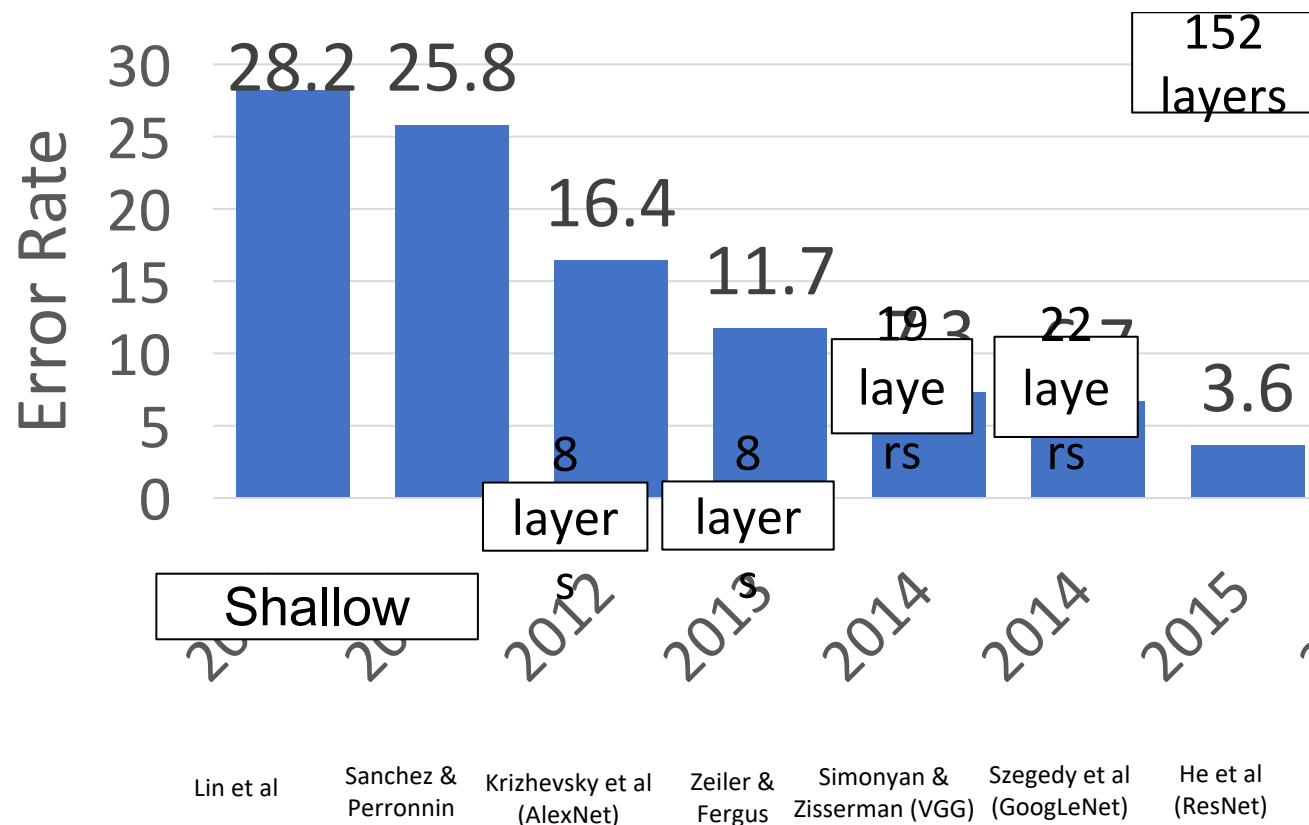
本章内容

- 6.3.1.ImageNet分类挑战
- 6.3.2.AlexNet
- 6.3.3.VGG
- 6.3.4.GoogleNet
- 6.3.5.Residual Network
- 6.3.6.不同网络比较及改进

ImageNet分类挑战



ImageNet分类挑战



残差网络

一旦我们有了Batch Normalization（批归一化），我们能训练10+层的网络. 网络层数更深会发生什么？

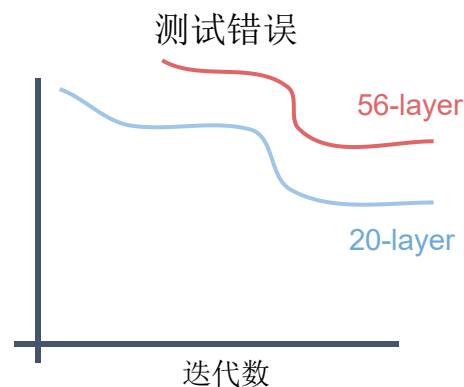
He et al, "Deep Residual Learning for Image Recognition", CVPR 2016

残差网络

一旦我们有了Batch Normalization（批归一化），我们能训练10+层的网络. 网络层数更深会发生什么？

深层模型性能不如浅层模型！

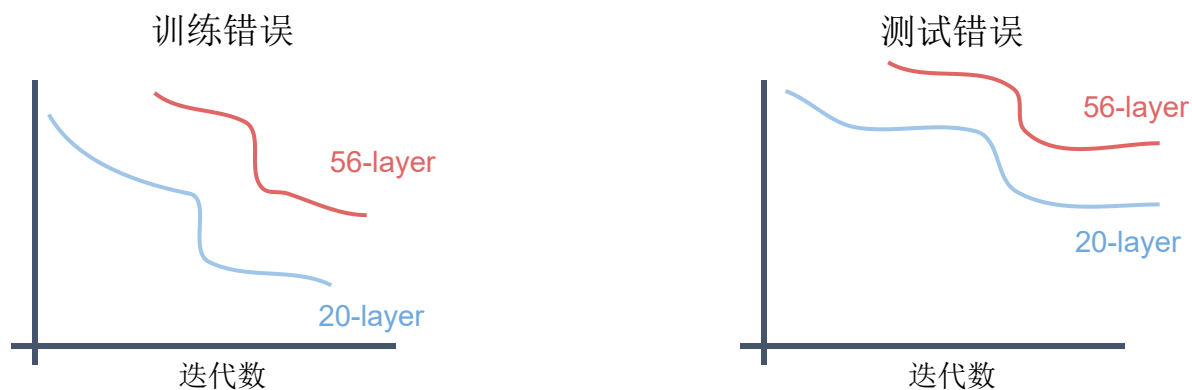
最初猜测:由于模型规模变大，深度模型会导致过拟合



He et al, "Deep Residual Learning for Image Recognition", CVPR 2016

残差网络

一旦我们有了Batch Normalization, 我们能训练10+层的网络. 网络层数更深会发生什么?



实际上深度模型是**欠拟合**的因为不管怎样其性能都不如浅层模型

He et al, "Deep Residual Learning for Image Recognition", CVPR 2016

残差网络

深层模型能模拟浅层模型: 直接复制浅层模型, 将额外增加的层设置为恒等映射

因此深层模型性能至少好于浅层模型

假设: 这是一个优化问题。深层模型是难于优化的, 实际中无法学习到恒等映射模拟浅层模型。

He et al, "Deep Residual Learning for Image Recognition", CVPR 2016

残差网络

深层模型能模拟浅层模型: 直接复制浅层模型, 将额外增加的层设置为恒等映射

因此深层模型性能至少好于浅层模型

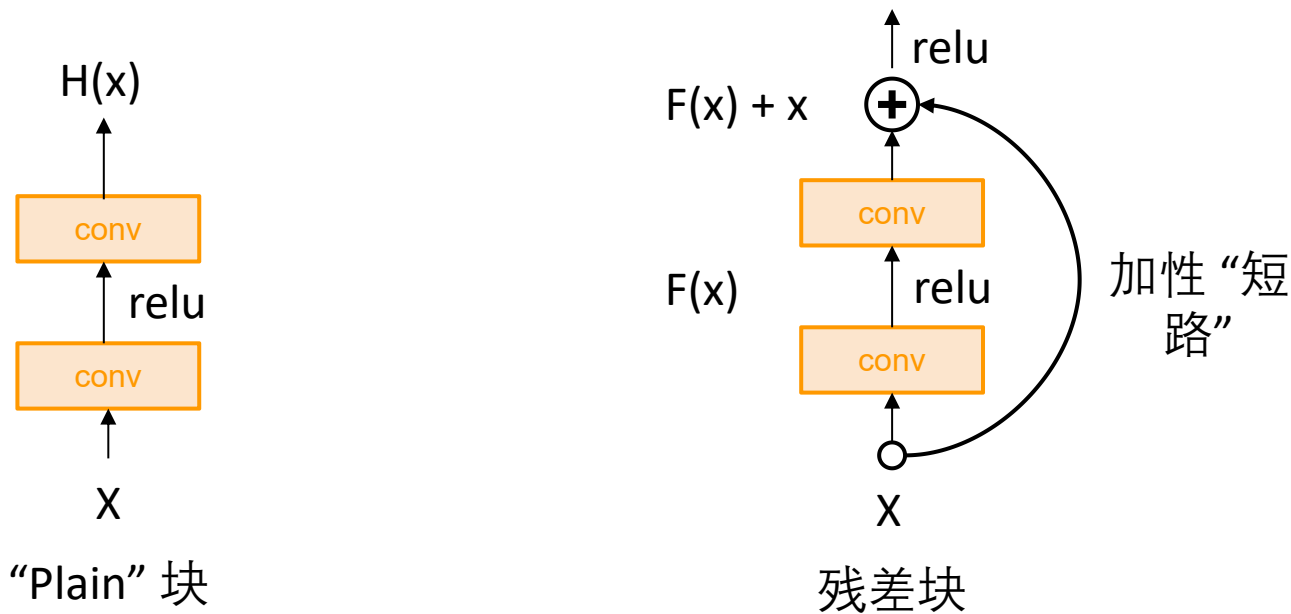
假设: 这是一个优化问题。深层模型是难于优化的, 实际中无法学习到恒等映射模拟浅层模型。

方案: 改变网络来使得额外的层学习恒等函数变得容易!

He et al, "Deep Residual Learning for Image Recognition", CVPR 2016

残差网络

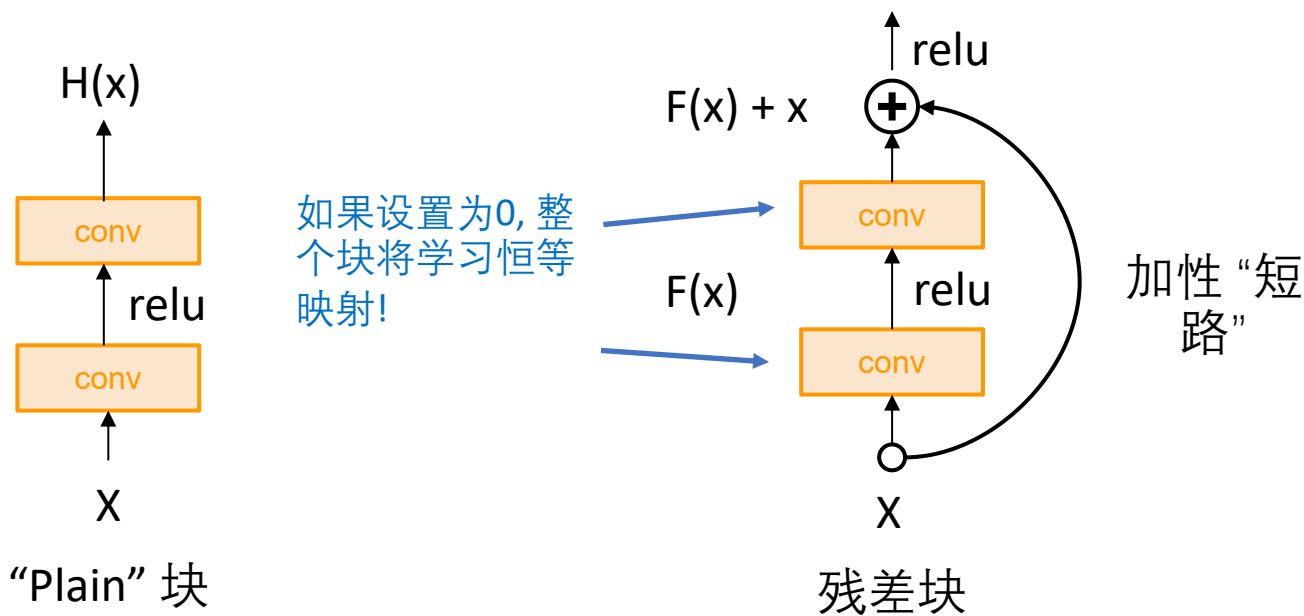
方案: 改变网络来使得额外的层学习恒等函数变得容易!



He et al, "Deep Residual Learning for Image Recognition", CVPR 2016

残差网络

方案: 改变网络来使得额外的层学习恒等函数变得容易!



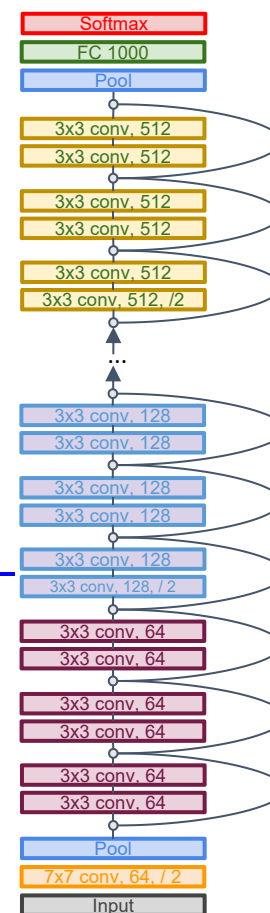
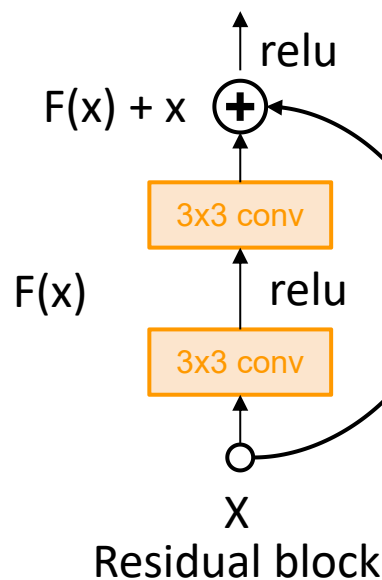
He et al, "Deep Residual Learning for Image Recognition", CVPR 2016

残差网络

残差网络由若干残差块堆叠而成

常规设计, 类似 VGG: 每一个残差块有两个 3x3 conv 块

网络被分为多个阶段: 每一阶段第一个块通过设置步长为2来降低两倍分辨率并加倍输出通道数

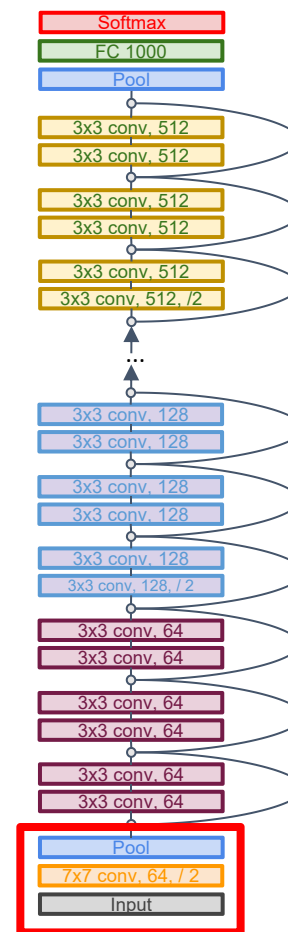


He et al, "Deep Residual Learning for Image Recognition", CVPR 2016

残差网络

使用和GoogleNet一样的主干网络，在残差块之前将输入下采样4倍

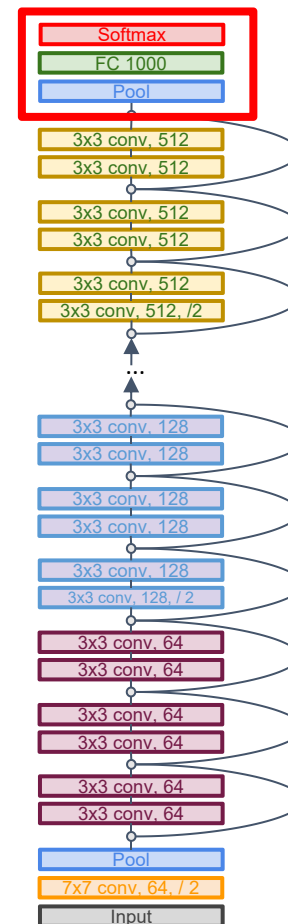
Layer	Input size		Layer				Output size		memory (KB)	params (k)	flop (M)
	C	H/W	filters	kernel	stride	pad	C	H/W			
conv	3	224	64	7	2	3	64	112	3136	9	118
max-pool	64	112		3	2	1	64	56	784	0	2



He et al, "Deep Residual Learning for Image Recognition", CVPR 2016

残差网络

类似 GoogLeNet, 没用大的fully-connected-layers: 使用 **global average pooling** (全局平均池化) 和一个线性层作为输出层替代



He et al, "Deep Residual Learning for Image Recognition", CVPR 2016

残差网络

ResNet-18:

Stem: 1 conv layer

Stage 1 (C=64): 2 res. block = 4 conv

Stage 2 (C=128): 2 res. block = 4 conv

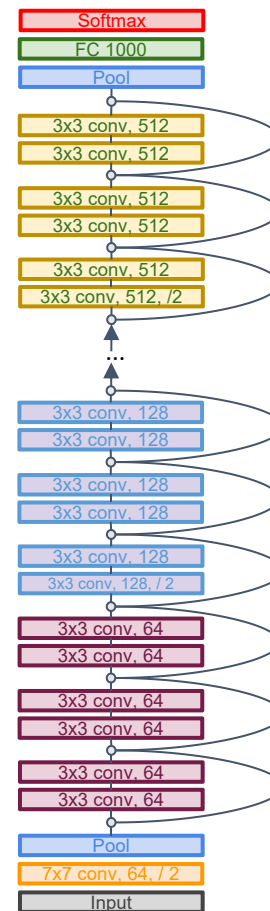
Stage 3 (C=256): 2 res. block = 4 conv

Stage 4 (C=512): 2 res. block = 4 conv

Linear

ImageNet top-5 错误率: 10.92

GFLOP: 1.8



He et al, "Deep Residual Learning for Image Recognition", CVPR 2016
Error rates are 224x224 single-crop testing, reported by [torchvision](#)

残差网络

ResNet-18:

Stem: 1 conv layer

Stage 1 (C=64): 2 res. block = 4 conv

Stage 2 (C=128): 2 res. block = 4 conv

Stage 3 (C=256): 2 res. block = 4 conv

Stage 4 (C=512): 2 res. block = 4 conv

Linear

ImageNet top-5 错误率: 10.92

GFLOP: 1.8

ResNet-34:

Stem: 1 conv layer

Stage 1: 3 res. block = 6 conv

Stage 2: 4 res. block = 8 conv

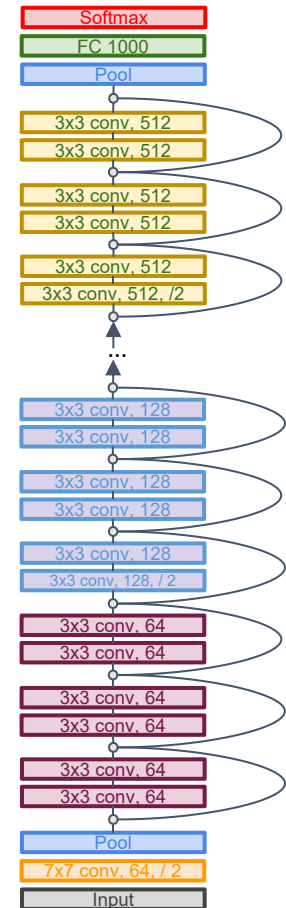
Stage 3: 6 res. block = 12 conv

Stage 4: 3 res. block = 6 conv

Linear

ImageNet top-5 错误率: 8.58

GFLOP: 3.6



He et al, "Deep Residual Learning for Image Recognition", CVPR 2016
Error rates are 224x224 single-crop testing, reported by [torchvision](#)

残差网络

ResNet-18:

Stem: 1 conv layer

Stage 1 (C=64): 2 res. block = 4 conv

Stage 2 (C=128): 2 res. block = 4 conv

Stage 3 (C=256): 2 res. block = 4 conv

Stage 4 (C=512): 2 res. block = 4 conv

Linear

ImageNet top-5 错误率: 10.92

GFLOP: 1.8

ResNet-34:

Stem: 1 conv layer

Stage 1: 3 res. block = 6 conv

Stage 2: 4 res. block = 8 conv

Stage 3: 6 res. block = 12 conv

Stage 4: 3 res. block = 6 conv

Linear

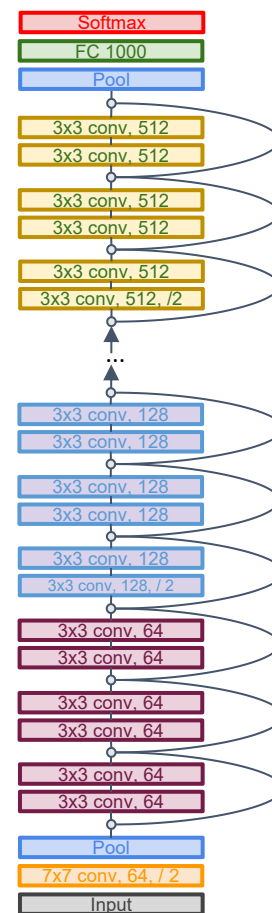
ImageNet top-5 错误率: 8.58

GFLOP: 3.6

VGG-16:

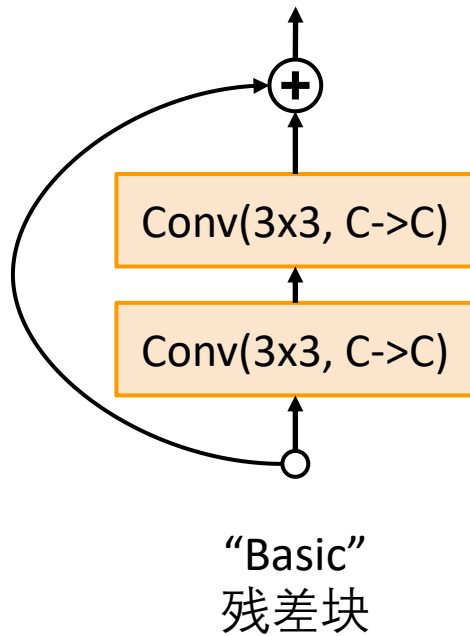
ImageNet top-5 错误率: 9.62

GFLOP: 13.6



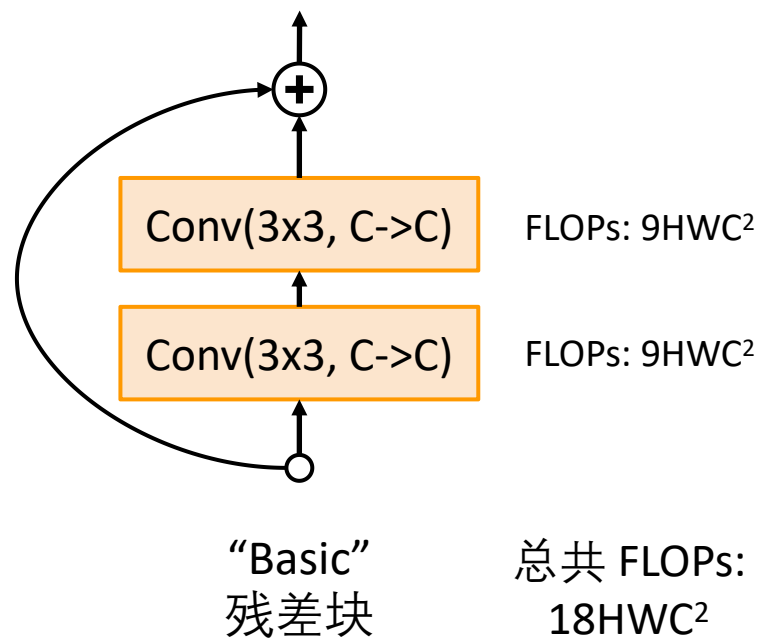
He et al, "Deep Residual Learning for Image Recognition", CVPR 2016
Error rates are 224x224 single-crop testing, reported by [torchvision](https://github.com/pytorch/vision)

残差网络: 基本块



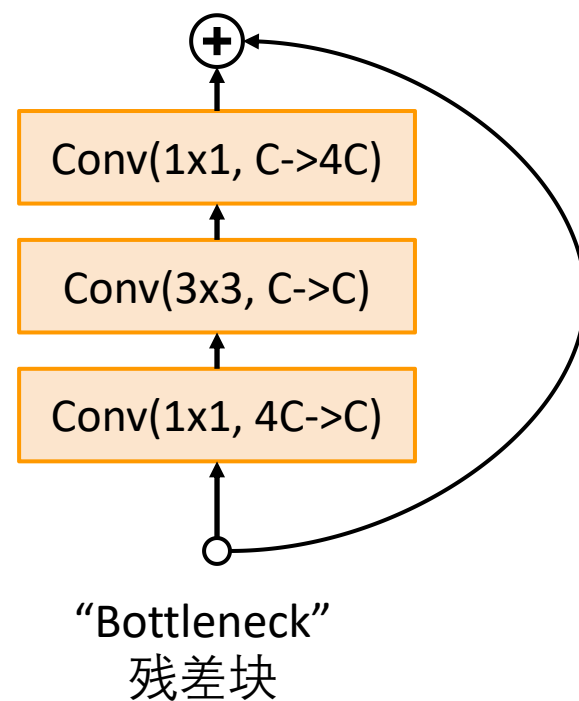
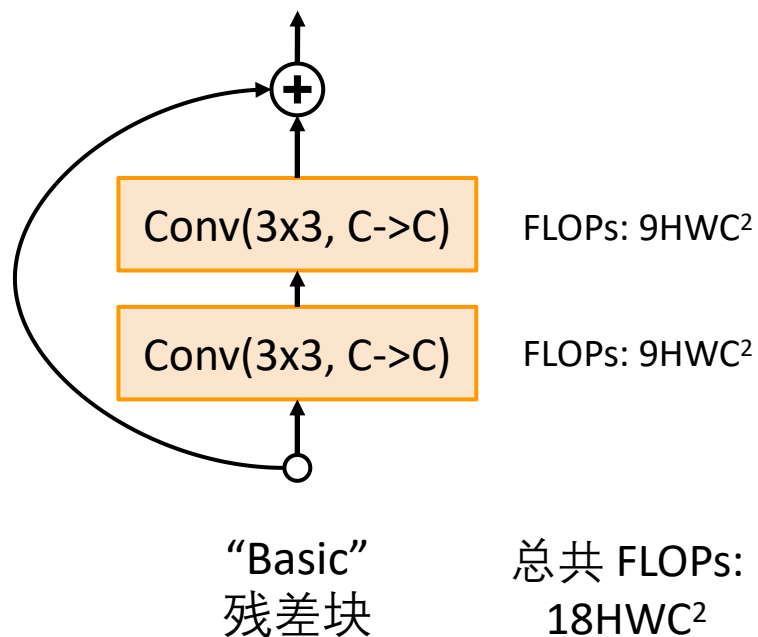
He et al, "Deep Residual Learning for Image Recognition", CVPR 2016

残差网络: 基本块



He et al, “Deep Residual Learning for Image Recognition”, CVPR 2016

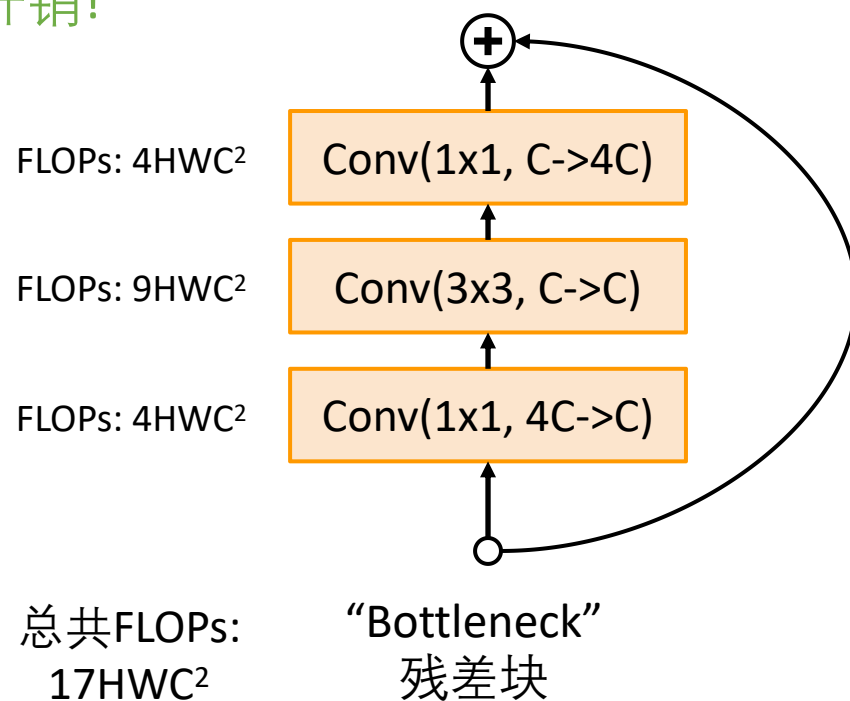
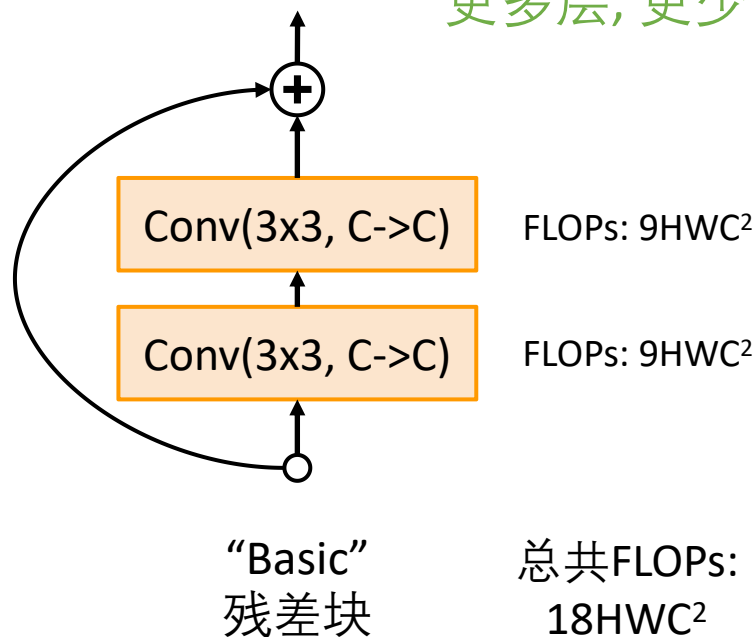
残差网络: Bottleneck 块



He et al, “Deep Residual Learning for Image Recognition”, CVPR 2016

残差网络: Bottleneck 块

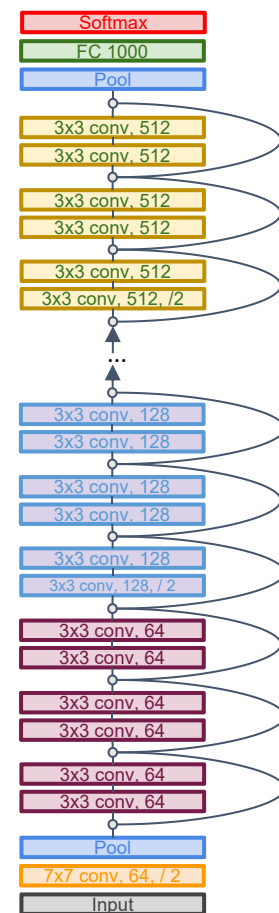
更多层, 更少的计算开销!



He et al, “Deep Residual Learning for Image Recognition”, CVPR 2016

残差网络

			Stage 1		Stage 2		Stage 3		Stage 4				
	Block type	Stem layers	Blocks	Layers	Blocks	Layers	Blocks	Layers	Blocks	Layers	FC layers	GFLOP	ImageNet top-5 error
ResNet-18	Basic	1	2	4	2	4	2	4	2	4	1	1.8	10.92
ResNet-34	Basic	1	3	6	4	8	6	12	3	6	1	3.6	8.58

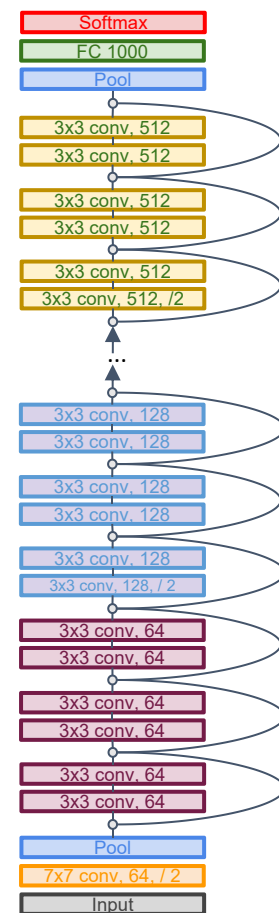


He et al, "Deep Residual Learning for Image Recognition", CVPR 2016
 Error rates are 224x224 single-crop testing, reported by [torchvision](https://github.com/pytorch/vision)

残差网络

ResNet-50 和ResNet-34一样, 但是将Basic 块替换为Bottleneck 块. 对于许多任务而言, 这是一个最好的基线结果!

			Stage 1		Stage 2		Stage 3		Stage 4				
	Block type	Stem layers	Blocks	Layers	Blocks	Layers	Blocks	Layers	Blocks	Layers	FC layers	GFLOP	ImageNet top-5 error
ResNet-18	Basic	1	2	4	2	4	2	4	2	4	1	1.8	10.92
ResNet-34	Basic	1	3	6	4	8	6	12	3	6	1	3.6	8.58
ResNet-50	Bottle	1	3	9	4	12	6	18	3	9	1	3.8	7.13

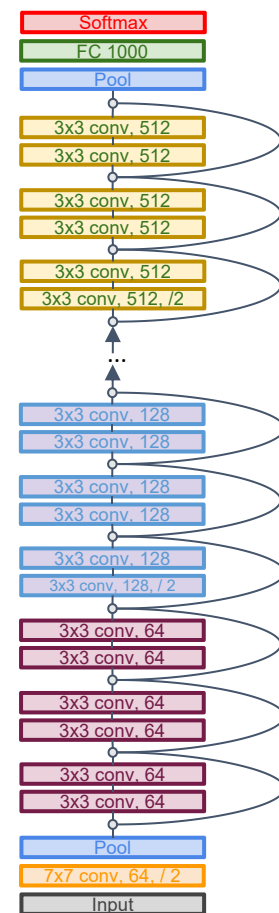


He et al, "Deep Residual Learning for Image Recognition", CVPR 2016
Error rates are 224x224 single-crop testing, reported by [torchvision](#)

残差网络

更深的 ResNet-101 和 ResNet-152 模型更精确, 但是计算量也相应增加

			Stage 1		Stage 2		Stage 3		Stage 4				
	Block type	Stem layers	Blocks	Layers	Blocks	Layers	Blocks	Layers	Blocks	Layers	FC layers	GFLOP	ImageNet top-5 error
ResNet-18	Basic	1	2	4	2	4	2	4	2	4	1	1.8	10.92
ResNet-34	Basic	1	3	6	4	8	6	12	3	6	1	3.6	8.58
ResNet-50	Bottle	1	3	9	4	12	6	18	3	9	1	3.8	7.13
ResNet-101	Bottle	1	3	9	4	12	23	69	3	9	1	7.6	6.44
ResNet-152	Bottle	1	3	9	8	24	36	108	3	9	1	11.3	5.94



He et al, "Deep Residual Learning for Image Recognition", CVPR 2016
Error rates are 224x224 single-crop testing, reported by [torchvision](https://github.com/pytorch/vision)

残差网络

- 能够训练非常深的网络
- 网络变深性能能够变好
- 在所有ILSVRC 和COCO 2015 竞赛横扫第一
- 今天仍然广泛应用!

MSRA @ ILSVRC & COCO 2015 Competitions

- **1st places in all five main tracks**

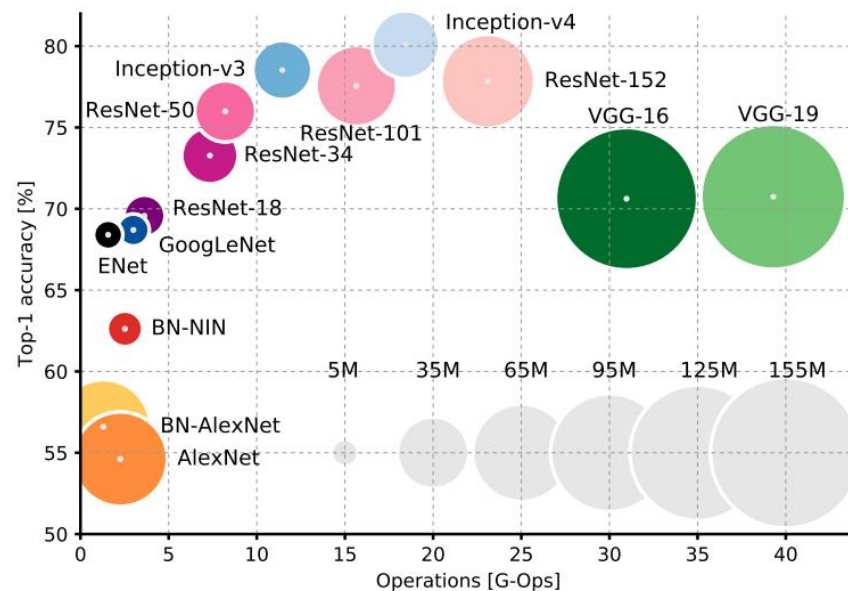
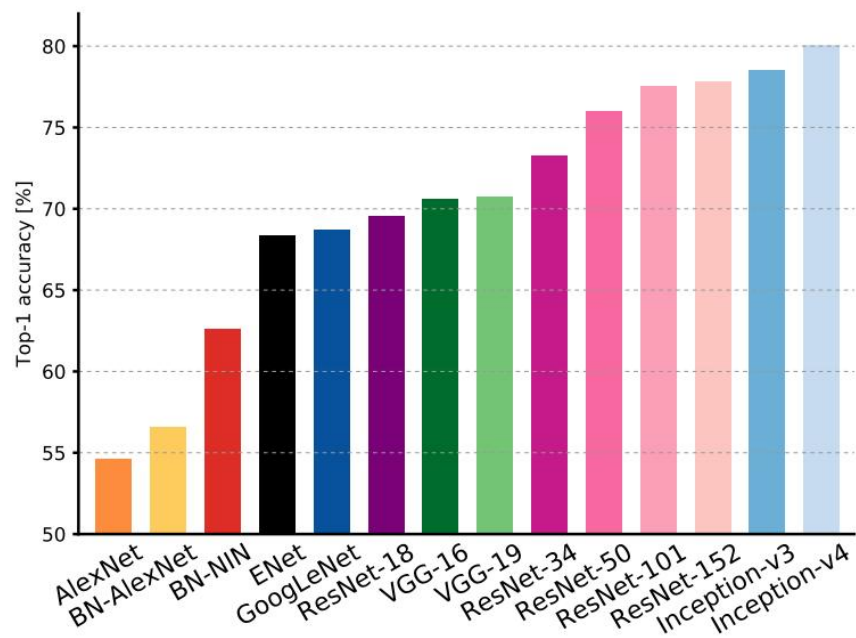
- ImageNet Classification: “Ultra-deep” (quote Yann) **152-layer** nets
- ImageNet Detection: **16%** better than 2nd
- ImageNet Localization: **27%** better than 2nd
- COCO Detection: **11%** better than 2nd
- COCO Segmentation: **12%** better than 2nd

He et al, “Deep Residual Learning for Image Recognition”, CVPR 2016

本章内容

- 6.3.1.ImageNet分类挑战
- 6.3.2.AlexNet
- 6.3.3.VGG
- 6.3.4.GoogleNet
- 6.3.5.Residual Network
- 6.3.6.不同网络比较及改进

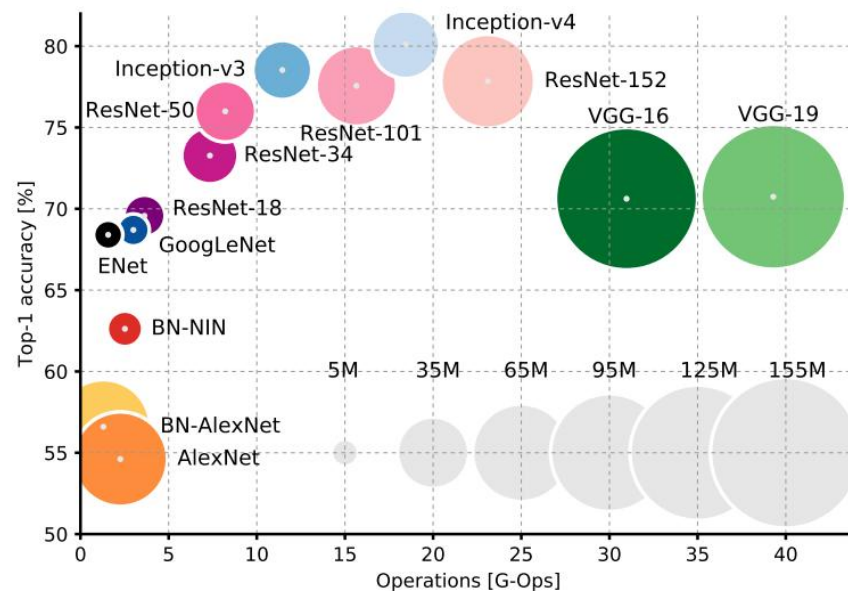
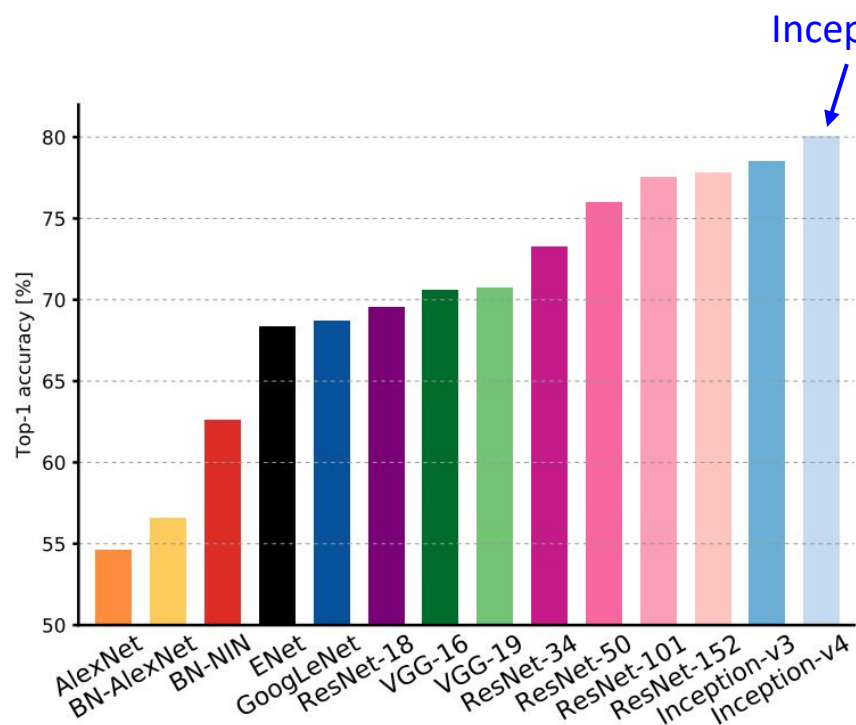
复杂度比较



BN: 批归一化版本

Canziani et al, "An analysis of deep neural network models for practical applications", 2017

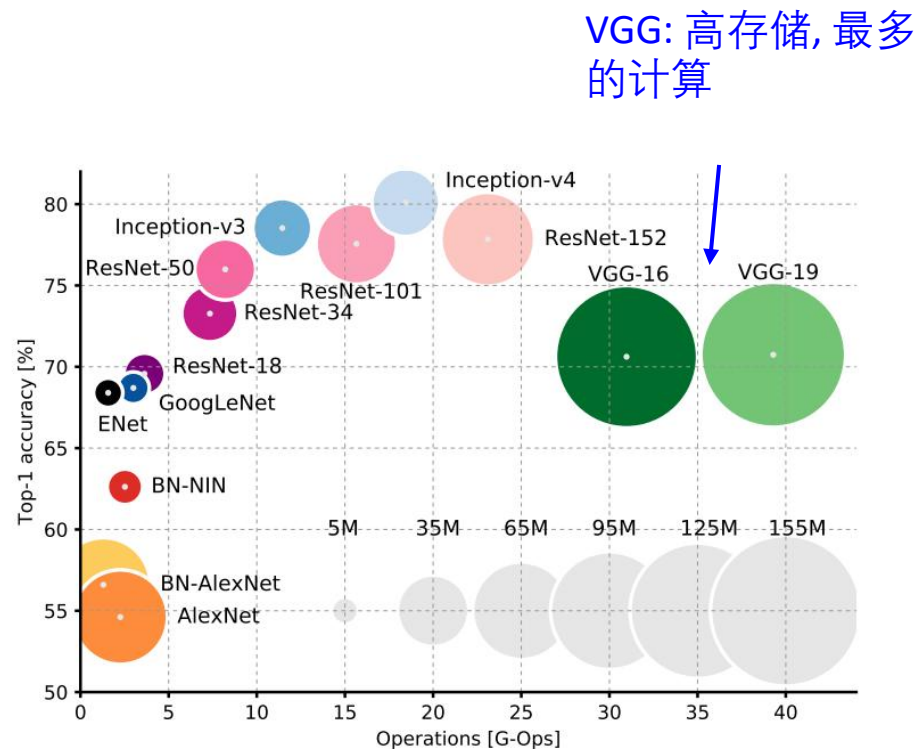
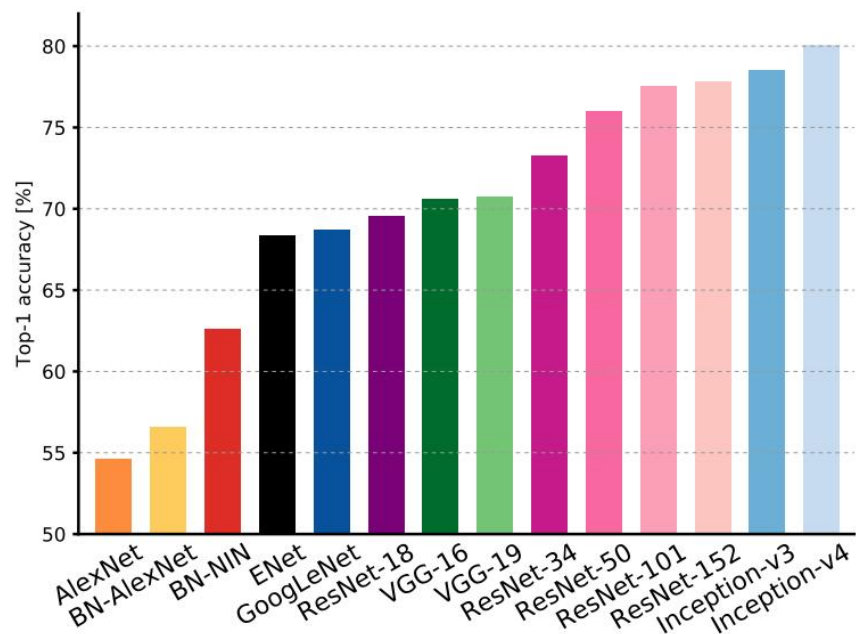
复杂度比较



BN:批归一化版本

Canziani et al, "An analysis of deep neural network models for practical applications", 2017

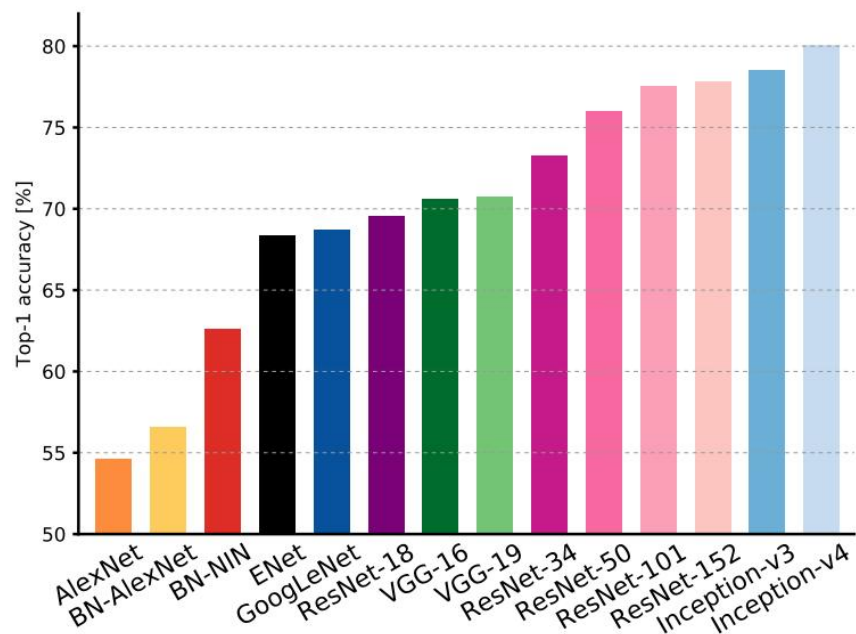
复杂度比较



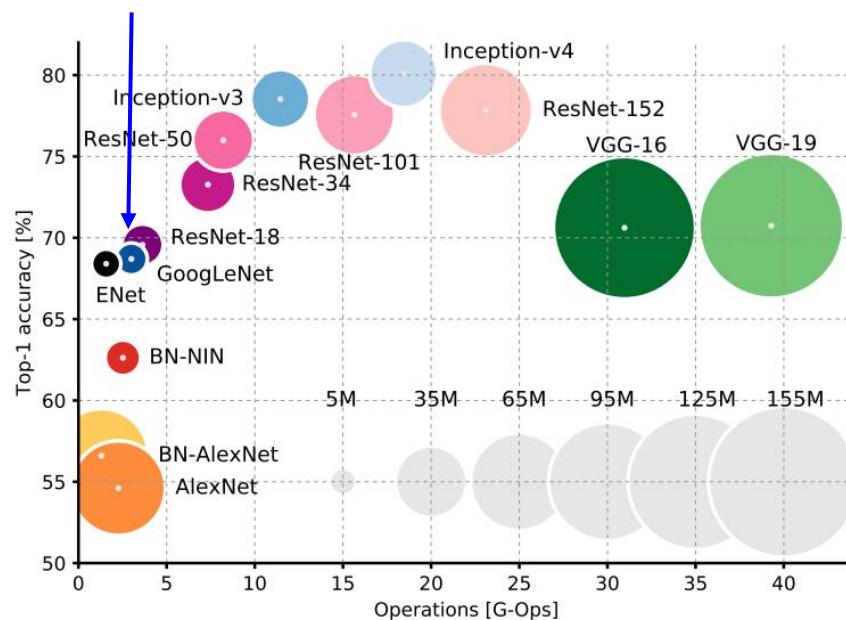
BN:批归一化版本

Canziani et al, "An analysis of deep neural network models for practical applications", 2017

复杂度比较



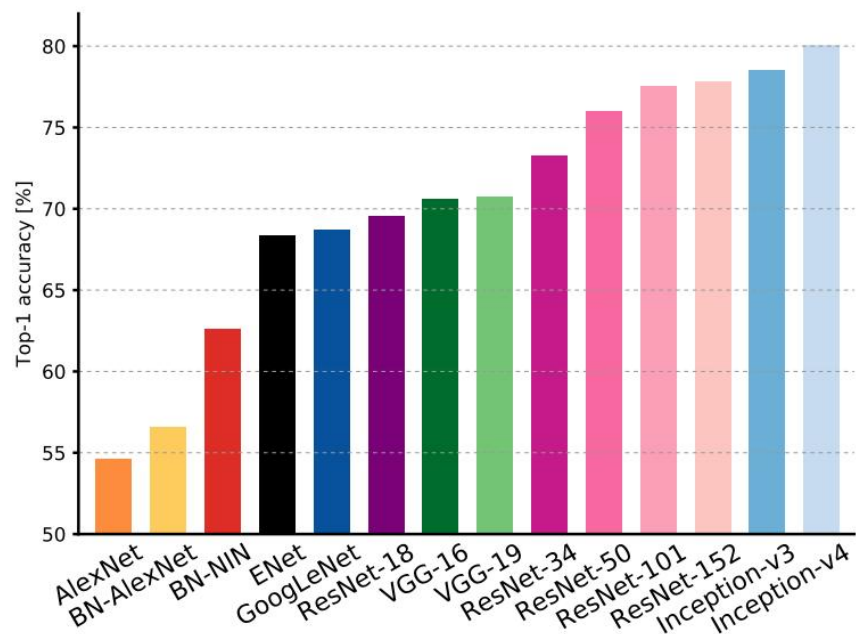
GoogLeNet:
非常高效!



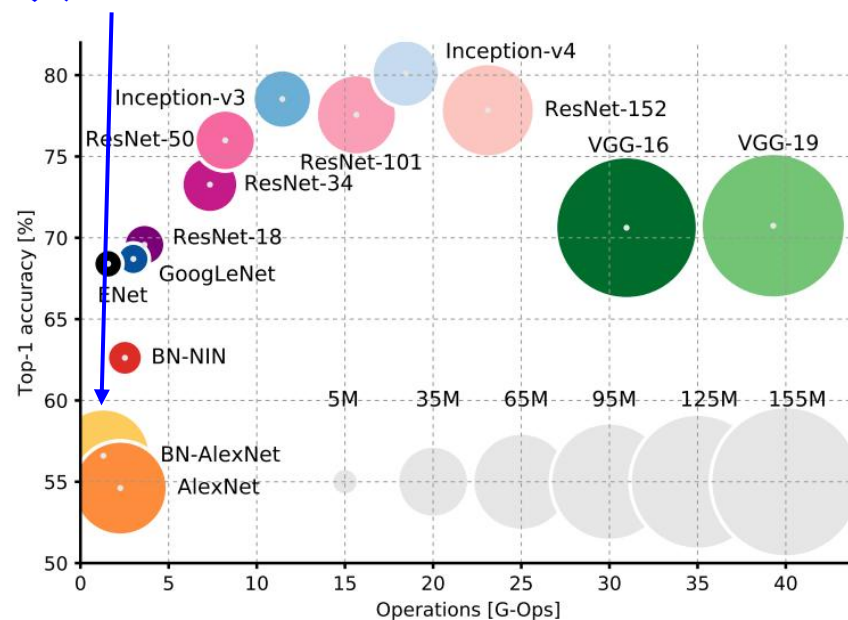
BN:批归一化版本

Canziani et al, "An analysis of deep neural network models for practical applications", 2017

复杂度比较



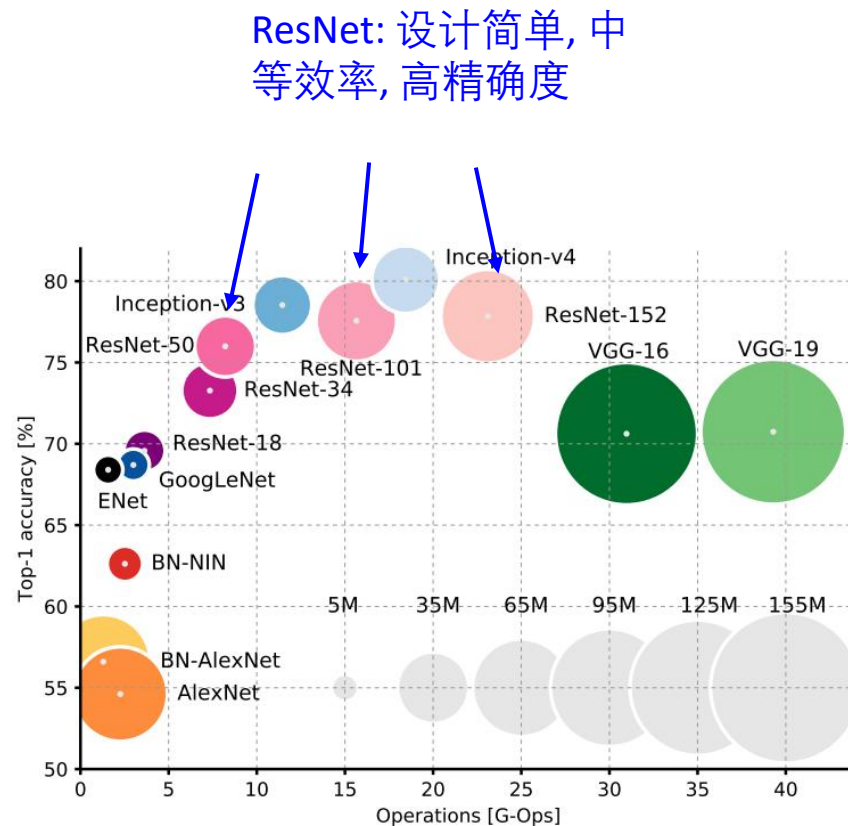
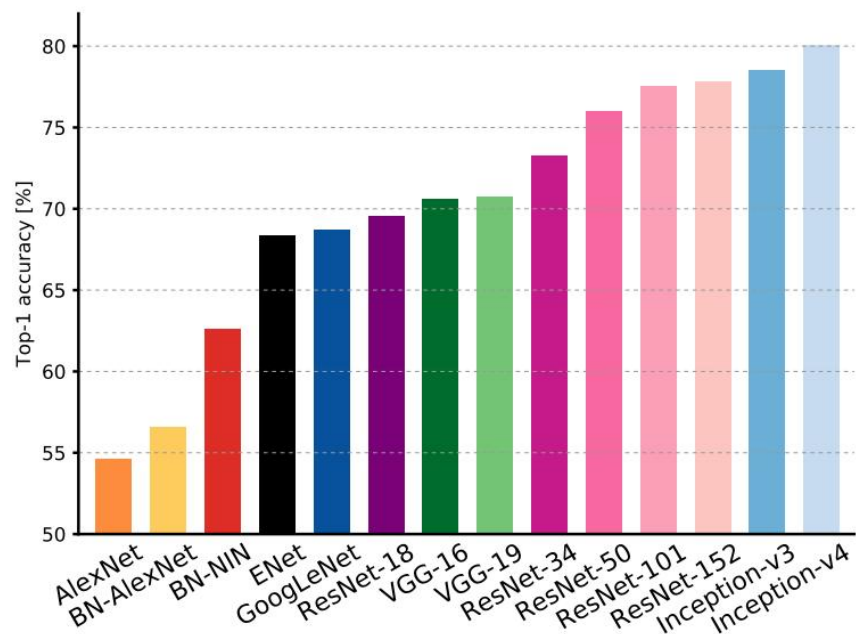
AlexNet: 低计算, 参数规模大



BN:批归一化版本

Canziani et al, "An analysis of deep neural network models for practical applications", 2017

复杂度比较



BN:批归一化版本

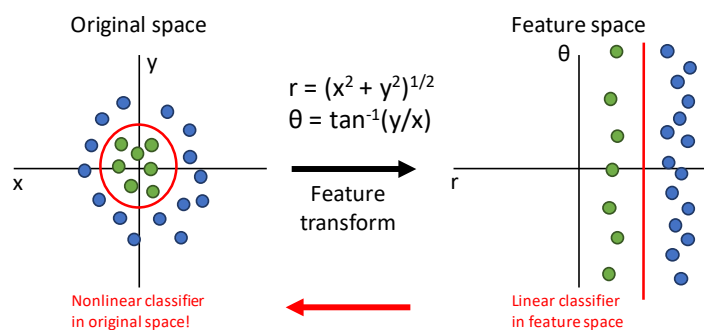
Canziani et al, "An analysis of deep neural network models for practical applications", 2017

本章小结

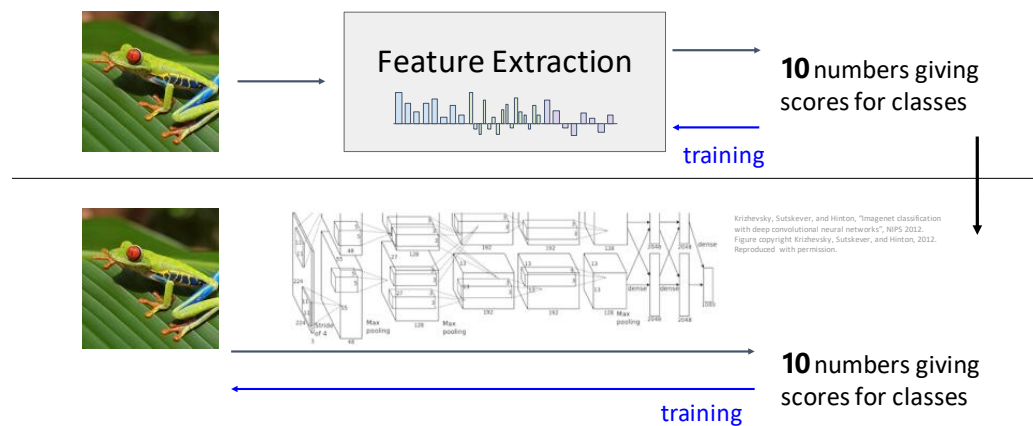
- 多层神经网络
 - 特征变换
 - 线性分类器级联
 - 神经网络的组成
- 激活函数
 - 神经元
 - Sigmoid激活函数
 - 线性修正单元
- 后向传播算法
 - 计算图
 - 梯度后向传播计算方法
- 卷积神经网络的成分
 - 卷积网络目的
 - 卷积网络组成
- 卷积层
 - 卷积参数
 - 1维卷积
 - 感受野
- 池化和归一化
 - 池化参数
 - 批归一化目的
 - 不同归一化操作

总结

特征变换+线性分类器允许非线性决策边界



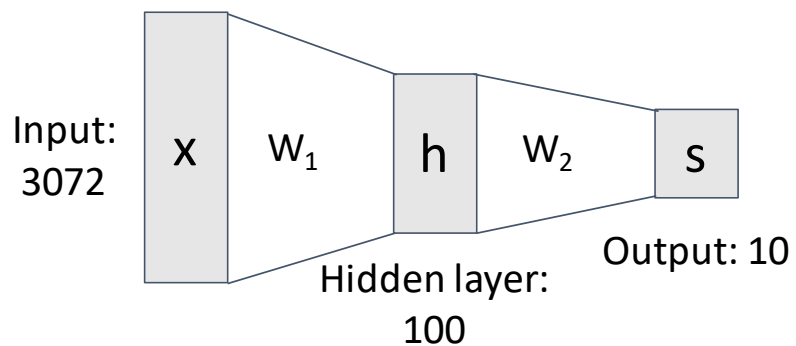
神经网络作为可学习的特征变换



总结

从线性分类器到
全连接层网络

$$f = W_2 \max(0, W_1 x)$$



线性分类器: 每类一个模板

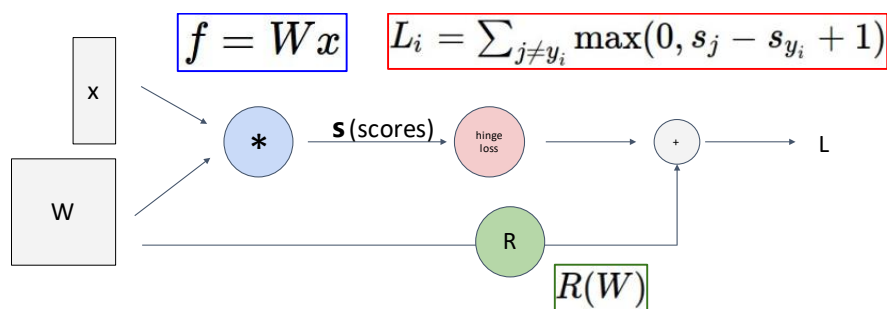


神经网络: 许多可重复模板

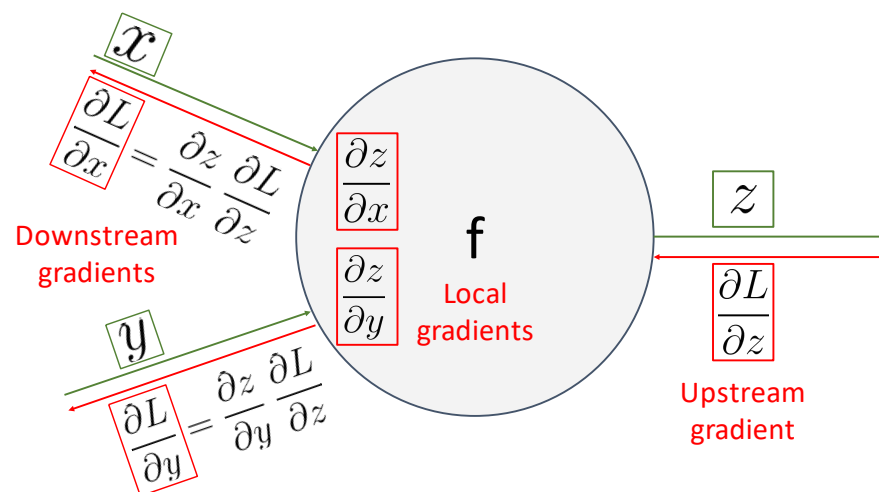


总结

计算图表示方法

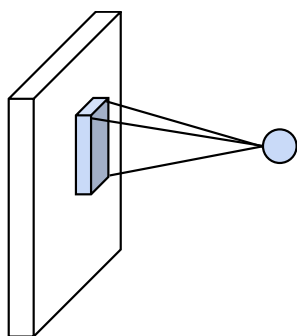


在后向传播过程中, 图中每一个节点接收上游梯度, 将其和当前梯度相乘得到下游梯度

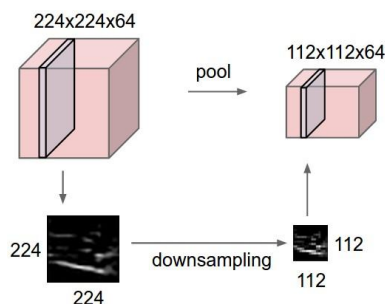


总结: 卷积网络的构成成分

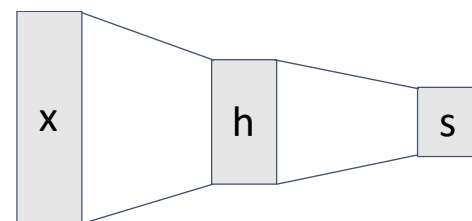
卷积层



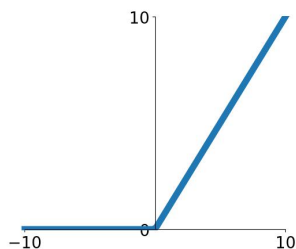
池化层



全连接层



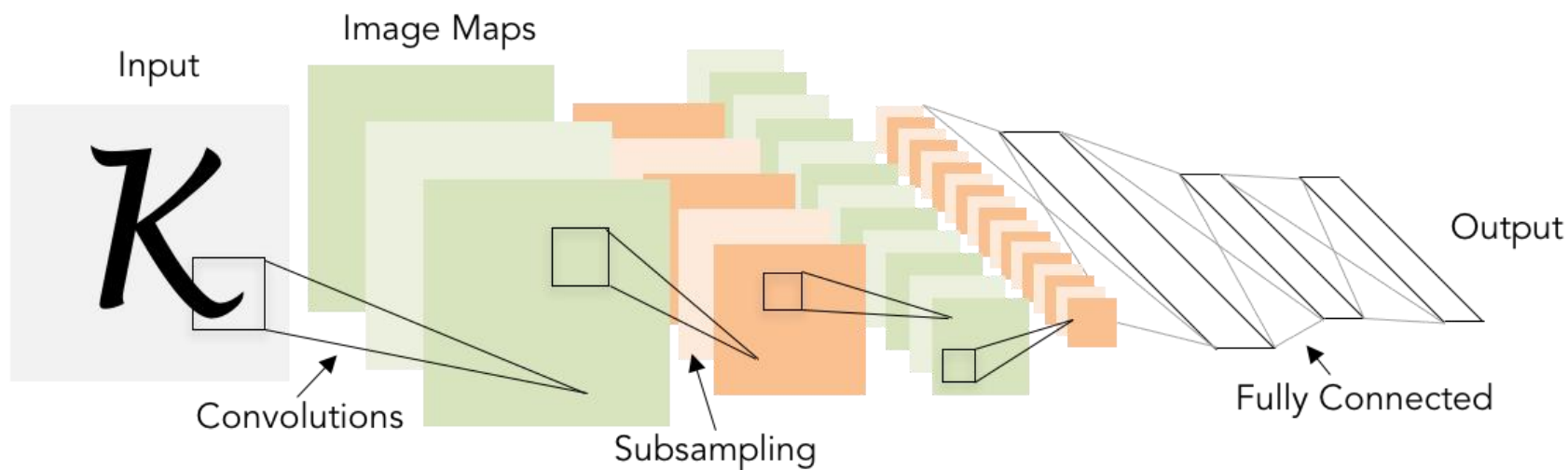
激活函数



归一化

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

总结: 卷积网络的构成



结束

下一次课

- 目标检测